UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Trabajo de Diploma - Proyecto Integrador

UAI Universidad Abierta Interamericana

Alumno: Gesualdo Antonini, Sofía.
Localización: Centro - 091
Comisión: 3B
Turno: Mañana
Documento: 46.956.657
Docente: Gamboa Jiménez, Leonel

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Índice:

G01. Propósito y Motivación
G02. Requisitos Funcionales
G03. Definiciones, Acrónimos y Abreviaciones
G04. Tabla de Roles
G05. Otros Requisitos
G06. Diagramas de Clases
G07. Diagrama Entidad Relación
N01. Especificación funcional por proceso de negocio
    N01.PN1.Gestión de ventas en tienda física
    N01.PN2.Gestión de préstamos y devolución de libros en tienda física
N02. Especificaciones de Casos de Uso por Proceso de Negocio
    N02.PN1.1 Diagrama de casos de uso general
    N02.PN1.2 CUN01 “Registrar libro en el carrito”
    N02.PN1.3 CUN02 “Seleccionar libro”
    N02.PN1.4 CUN03 “Cobrar venta”
    N02.PN1.5 CUN04 “Registrar cliente”
    N02.PN2.1 Diagrama de casos de uso general
    N02.PN2.2 CUN05 “Registrar préstamo”
    N02.PN2.3 CUN06 “Cobrar préstamo”
    N02.PN2.4 CUN07 “Seleccionar préstamo vigente”
    N02.PN2.5 CUN08 “Cobrar recargo”
    N02.PN2.6 CUN09 “Registrar devolución”
T01. Arquitectura
    T01.1. Diseño de la Arquitectura
    T01.2. Configuración de la Arquitectura en Visual Studio
    T01.3. Diagrama de Secuencia de Persistencia de Datos
    T01.4. Diagrama de Secuencia de Consulta de Datos
    T01.5. Esquema Jerárquico
    T01.6. GUI (Menú Principal)
T02. Usuarios
    T02.1. Diseño Crear Usuario
    T02.2. Diseño Login
        CU20: Administrar Error de Integridad
    T02.3. Diseño Desbloquear Usuario
    T02.4. Diseño Modificar Usuario
    T02.5. Diseño Cambiar Clave
    T02.6. Diseño Logout

- T02.7. Diseño Activar/Desactivar Usuario
- T03. Encriptación
- T04. Perfiles
  - T04.1. Crear Familia
  - T04.2. Eliminar Familia
  - T04.3. Crear Perfil
  - T04.4. Eliminar Perfil
  - T04.5. Agregar Permisos a Familia
  - T04.6. Quitar Permisos a Familia
  - T04.7. Agregar Permisos a Perfil
  - T04.8. Quitar Permisos a Perfil
  - T04.9. Agregar Familias a Perfil
  - T04.10. Quitar Familias a Perfil
- T05. Idiomas
- T06.1 Bitácora de Eventos
- T06.2 Bitácora de Cambios
- T07. Gestión de Respaldos
- T08. Dígito Verificador

G01. Propósito y Motivación

Bookly es un emprendimiento con sede en CABA orientado a la comercialización y el préstamo de libros nuevos y usados de distintos géneros y categorías, destinado a clientes que buscan adquirir sus propios ejemplares de manera presencial o que desean alquilar un ejemplar determinado por un corto período de tiempo. Su catálogo incluye novelas, cuentos, literatura, ciencia ficción, historia, libros académicos, entre otros. La empresa desarrolla sus actividades con el objetivo de facilitar el acceso a la lectura y ofrecer una amplia variedad de ejemplares de acuerdo con las necesidades de sus clientes, manteniendo una propuesta orientada a brindar una experiencia de compra y préstamo ágil y accesible. Su visión se encuentra orientada a consolidarse como una alternativa reconocida en la comercialización y el préstamo de libros, ampliando la variedad de ejemplares disponibles y mejorando continuamente la calidad de sus servicios.

Actualmente, Bookly presenta dificultades en la organización y el control de las operaciones comerciales y del inventario. La gestión manual de las ventas realizadas en la tienda suele generar inconsistencias con la información de los productos y demoras durante el proceso de compra. A su vez, la falta de un control centralizado del stock dificulta conocer con precisión la disponibilidad de los ejemplares, no permitiendo identificar aquellos que requieren reposición. También, la empresa posee inconvenientes en cuanto al alquiler de los ejemplares, debido a la falta de la trazabilidad de los libros que se prestaron y las fechas de préstamo y devolución. Estas dificultades pueden afectar tanto la organización interna como la experiencia de los clientes.

A partir del análisis de las actividades desarrolladas por Bookly, se identificaron como principales Procesos de Negocio la Venta de Libros en Tienda Física, el Préstamo y Devolución de Ejemplares y la Gestión de Inventario y Reposición. Estos procesos comprenden las operaciones relacionadas con la comercialización de los libros, el registro y seguimiento de los ejemplares prestados, y el control de la disponibilidad y administración de información de los mismos. Además, se considera la gestión de los datos de los clientes y de las operaciones realizadas, debido a que dicha información interviene en el desarrollo y seguimiento de los distintos procesos de la organización.

Frente a esta situación, Bookly requiere una solución informatizada que permita gestionar las ventas y los préstamos de los libros de manera organizada. La empresa requiere la consulta y manejo del catálogo, el registro de las ventas, la gestión de los préstamos, la administración de los datos de los clientes y su registro correspondiente en la base de datos, el procesamiento de los pagos y la emisión de comprobantes. En consecuencia, la solución

debe permitir actualizar automáticamente el stock, administrar la información y disponibilidad de los libros y registrar el ingreso de nuevos ejemplares o la reposición de ejemplares viejos.

La motivación principal para el desarrollo de este proyecto es mejorar la eficiencia y organización de las operaciones de Bookly mediante una solución tecnológica que centralice la información de las ventas y de los préstamos, y que genere un inventario consistente. Se busca reducir errores en el registro de las operaciones, agilizar los procesos y mantener actualizada la disponibilidad de los libros. Además, el sistema permitirá contar con información útil para detectar las necesidades de reposición y analizar el comportamiento de las operaciones, contribuyendo a una mejor toma de decisiones y a una experiencia de compra más eficiente.

G02. Requisitos Funcionales

PN1: Gestión de ventas en tienda física

Este requisito funcional permite gestionar la venta de libros en la tienda física. El proceso contempla la búsqueda de ejemplares, la selección de productos, el registro de los datos cliente, el procesamiento del pago, la emisión del comprobante y la actualización automática del stock, garantizando que la información de las ventas y la disponibilidad de los ejemplares se mantenga actualizada.

Las operaciones definidas dentro de este proceso de negocio son:

- Seleccionar libros (1 o varios libros).
- Registrar libros en el carrito de compra (1 o varios libros).
- Registrar datos del cliente.
- Cobrar la venta.
- Emitir la Factura (Operación automática).
- Actualizar el stock (Operación automática).

Dichas operaciones son descompuestas en los siguientes pasos lógicos secuenciales determinados por un analista funcional luego de un análisis detallado del proceso de negocio y de los actores involucrados:

1. El Cliente le indica al Vendedor el libro que desea comprar (título, autor/es, editorial). N veces.
2. El Vendedor selecciona del catálogo el libro solicitado (título, autor/es, editorial, precio). N veces
3. El Vendedor le comunica al Cliente el precio del libro solicitado. N veces.

4. Si el Cliente está conforme con el precio y desea comprar el libro, se lo indica al Vendedor.
5. El Vendedor registra el libro en el carrito de compras. N veces.
6. Si el Cliente ya no requiere más libros, se lo indica al Vendedor.
7. El Vendedor le solicita al Cliente su DNI para asociarlo a su carrito junto con los libros solicitados.
8. Si el Cliente no está registrado en la base de datos, el Vendedor lo registra con sus datos personales (DNI, Nombre, Apellido, Correo Electrónico, Teléfono, Dirección).
9. El Vendedor le indica al Cliente el total para proceder con el pago de los artículos.
10. El Cliente le entrega al Vendedor su tarjeta de Débito o Crédito (Banco, Número de la Tarjeta, Fecha de Vencimiento, Número de Seguridad) para pagar su compra.
11. El Vendedor utiliza la tarjeta recibida a través de la plataforma de pago para cobrar la venta.
12. Si se acepta la operación, la venta se confirma y se emite la factura correspondiente.
13. Si se rechaza la operación, la venta es revocada.
14. El Vendedor le devuelve al Cliente su Tarjeta Bancaria y una copia de la Factura de la compra (DNI, Teléfono Cliente, Fecha, Hora, Número de la Factura, Libros adquiridos con Precio, Total).
15. Se actualiza el Stock de productos. (Código del Producto, Cantidad).

Por medio de la presente tabla, se indican las especificaciones y descripciones de los colores utilizados para la descripción de procesos:

Color	Acción	Descripción
Fucsia	Actor que envía o recibe información	N/A
Celeste	Acción de envío de información.	Indica / Ingresa / Comunica / Pregunta
Amarillo	Actor que procesa información.	N/A
Verde	Acción de procesamiento de información.	Agrega / Solicita / Asocia / Recibe / Provee / Finaliza
Gris	Conjunto de datos involucrados.	N/A
Rojo	Caso con alternativa. Toma de decisión.	Si... Entonces…

PN2: Gestión de préstamos y devolución de libros en tienda física

Este proceso de negocio permite gestionar el préstamo y la devolución de libros en la tienda física. El proceso contempla la selección de ejemplares, la verificación de su disponibilidad, el registro de los datos del cliente, la generación y seguimiento del préstamo, la emisión del comprobante correspondiente y el registro de la devolución del ejemplar. De esta manera, se mantiene actualizada la información sobre los libros prestados, disponibles y devueltos, permitiendo realizar un seguimiento de los préstamos efectuados y sus respectivas fechas de devolución.

Las operaciones definidas dentro de este proceso de negocio son:

- Seleccionar libro.
- Registrar datos del cliente.
- Registrar préstamo.
- Cobrar el préstamo.
- Emitir factura del préstamo (Operación automática).
- Actualizar disponibilidad del libro (Operación automática).
- Seleccionar préstamo vigente.
- Cobrar recargo del préstamo (si aplica)
- Registrar devolución del libro.
- Emitir comprobante de devolución (Operación automática).

Dichas operaciones son descompuestas en los siguientes pasos lógicos secuenciales determinados por un analista funcional luego de un análisis detallado del proceso de negocio y de los actores involucrados:

Préstamo

1. El Cliente le indica al Vendedor el libro que desea solicitar en préstamo (título, autor/es, editorial, ISBN).
2. El Vendedor selecciona del catálogo el libro solicitado (título, autor/es, editorial, ISBN, precio).
3. El Vendedor le solicita al Cliente su DNI para asociarlo al préstamo junto con el libro solicitado.
4. Si el Cliente no está registrado en la base de datos, el Vendedor lo registra con sus datos personales (DNI, Nombre, Apellido, Correo Electrónico, Teléfono, Dirección).
5. El Vendedor registra el préstamo, incluyendo la fecha de inicio, la fecha prevista de devolución y el Cliente correspondiente.
6. El Cliente le entrega al Vendedor su tarjeta de Débito o Crédito (Banco, Número de la Tarjeta, Fecha de Vencimiento, Número de Seguridad) para pagar su préstamo.

7. El Vendedor utiliza la tarjeta recibida a través de la plataforma de pago para cobrar el préstamo.
8. Si se acepta la operación, el préstamo se confirma y se emite automáticamente su factura (DNI, Teléfono Cliente, Fecha Inicio, Fecha Devolución, Hora, Número de la Factura, Libros adquiridos, Total).
9. Si se rechaza la operación, el préstamo es revocado.
10. El Vendedor le devuelve al Cliente su Tarjeta Bancaria y una copia de la Factura del préstamo (DNI, Teléfono Cliente, Fecha Inicio, Fecha Devolución, Hora, Número de la Factura, Libros adquiridos, Total).
11. Se actualiza automáticamente la disponibilidad del libro, registrándolo como no disponible mientras se encuentre prestado.

Devolución

12. El Cliente entrega al Vendedor el libro que tenía en préstamo (título, autor/es, editorial, ISBN).
13. El Vendedor selecciona el préstamo correspondiente mediante los datos del Cliente (DNI).
14. Si la fecha de devolución excede la fecha indicada en el préstamo, el Vendedor debe cobrarle al Cliente debe un recargo (Fecha actual, Fecha Devolución).
15. Se registra la devolución del ejemplar.
16. Se actualiza automáticamente la disponibilidad del libro, registrándolo nuevamente como disponible (ISBN).
17. Se emite automáticamente el comprobante de devolución (Datos del Cliente, Número de Comprobante, Fecha, Hora, Datos del Libro, Número de la Factura del Préstamo, Estado).
18. El Vendedor entrega al Cliente una copia del comprobante de devolución.

Las especificaciones y descripciones de los colores utilizados para la descripción de procesos se indican por medio de la misma tabla presentada en el Proceso de Negocio previo.

G03. Definiciones, Acrónimos y Abreviaciones

Definiciones	**Descripción**
Cliente	Persona que adquiere o alquila libros dentro del negocio.
Libro	Ejemplar disponible para la venta o préstamo dentro del negocio.

Stock	Cantidad disponible de ejemplares de un libro en el inventario.
Venta	Operación comercial realizada entre el negocio y el cliente mediante la cual se adquieren uno o más libros.
Inventario	Registro y control de los libros disponibles para la venta o préstamo.
Factura	Documento emitido para registrar y validar una venta realizada.
Punto Crítico de Stock	Cantidad mínima de unidades disponibles de un producto que indica que es necesario realizar una reposición para evitar que se agote.
Préstamo	Operación mediante la cual un cliente obtiene un libro por un período determinado, con una fecha establecida para su devolución.
Devolución	Operación mediante la cual un cliente devuelve un libro previamente prestado.
Carrito de compra	Conjunto de libros seleccionados por el cliente durante el proceso de venta.
Catálogo	Conjunto de libros disponibles, con información correspondiente a cada ejemplar, como título, autor, editorial, categoría y precio.
Ejemplar	Unidad física correspondiente a un determinado libro disponible para la venta o préstamo.
Medio de pago	Modalidad utilizada por el cliente para abonar una compra, como efectivo, transferencia, código QR o tarjeta.
Comprobante	Documento o registro que acredita la realización de una operación de venta o pago.
Reposición	Operación mediante la cual se incorporan nuevos ejemplares al inventario para recuperar o aumentar la cantidad disponible.

Acrónimos	**Descripción**
DNI	Documento Nacional de Identidad
ISBN	International Standard Book Number

UDS	Unidades

Abreviaciones	**Descripción**
Prod.	Producto
Prov.	Proveedor
Cant.	Cantidad
Prec.	Precio
Cat.	Categoría
Desc.	Descripción
Máx.	Máximo
Min.	Mínimo

G04. Tabla de Roles

Nombre	**Rol**	**Descripción**	**Acceso a PNs**
Lucía Quiroga	Vendedor	Encargada de atender a los clientes, consultar el catálogo, registrar ventas, cargar libros al carrito, gestionar los cobros y entregar las facturas correspondientes.	PN1: Pasos 2, 3, 5, 7, 8, 9, 11, 12, 13, 14, 15 PN2: Pasos 2, 3, 4, 5, 7, 10, 12, 13, 14, 16, 18
Tomás Benavidez	Encargado de inventario y ventas	Responsable de administrar el inventario y controlar el stock de libros. Puede agregar nuevos libros, actualizar la información de los ejemplares, supervisar las	PN1: Pasos 14, 15 PN2: Pasos 11, 16

		ventas y gestionar la reposición cuando el stock alcanza el punto crítico.	
Camila Peralta	Gerente	Responsable de supervisar el funcionamiento general del negocio. Controla las operaciones de venta y el estado del inventario, consulta la información generada y toma decisiones relacionadas con la gestión comercial y la reposición de ejemplares.	PN1: Pasos 14, 15 PN2: Pasos 16, 17
Agustín Romero	Administrador del Sistema	Responsable de la seguridad informática y el mantenimiento operativo de la plataforma. Gestiona los usuarios del sistema, administra el esquema de perfiles, ejecuta políticas de respaldo y recuperación de base de datos y gestiona la resolución de incidentes de integridad de datos. No interviene en las operaciones comerciales del negocio.	N/A (Rol transversal de seguridad y soporte; no opera sobre los procesos de negocio).
Sofía Martínez	Analista de Sistemas	Responsable de analizar los requisitos del negocio, relevar las necesidades de los usuarios y definir las funcionalidades necesarias para el correcto desarrollo e implementación de la solución.	PN1: Todos PN2: Todos
Martín Salazar	Técnico de Soporte	Responsable del mantenimiento técnico de la solución y de la resolución de errores o inconvenientes relacionados con su funcionamiento.	**PN1:** Todos **PN2:** Todos

Victoria Méndez	Desarrolladora del Sistema	Responsable de implementar las funcionalidades definidas, realizar modificaciones, corregir errores y actualizar la solución de acuerdo con los requisitos establecidos.	**PN1:** Todos **PN2:** Todos
Gabriel García	Analista Tester	Responsable de verificar el correcto funcionamiento de las funcionalidades desarrolladas mediante la ejecución de pruebas y la detección de posibles errores.	**PN1:** Todos **PN2:** Todos

G05. Otros Requisitos

1. Interfaz aplicable



El sistema contará con una interfaz gráfica simple, intuitiva y amigable para el usuario, priorizando la facilidad de navegación y accesibilidad a las distintas funcionalidades. La

pantalla principal incluirá accesos directos a los módulos de ventas, préstamos, productos, stock y reposición.

La interfaz utilizará una paleta de colores relacionada con la temática del negocio, compuesta principalmente por tonos azul petróleo, crema, blanco y dorado suave, manteniendo uniformidad visual dentro de todas las pantallas del sistema.

Los botones, menús y formularios conservarán posiciones consistentes para facilitar el uso del sistema y mejorar la experiencia del usuario. Asimismo, se incluirá el logo representativo del emprendimiento dentro de la pantalla principal.

El sistema seguirá estándares de diseño de interfaces para aplicaciones de escritorio desarrolladas en entorno Windows.

2. Requisitos de desempeño

El sistema deberá responder a las acciones principales del usuario en un tiempo menor a 3 segundos bajo condiciones normales de uso, para evitar que el sistema se ralentice y permitir que el usuario pueda realizar sus actividades de manera rápida. Así, las consultas de productos, ventas y stock serán procesadas de manera efectiva y eficiente para evitar demoras durante la operación de la tienda.

La actualización del stock se realizará automáticamente luego de cada venta registrada, manteniendo la información del inventario actualizada inmediatamente.

El sistema deberá optimizar el uso de memoria y recursos del equipo para garantizar un funcionamiento estable durante jornadas prolongadas de trabajo.

3. Requisitos del sistema y del entorno

Para el correcto funcionamiento del sistema se requerirá:

Procesador Intel i3 / Ryzen 3 o superior.
Memoria RAM mínima de 4 GB.
Espacio libre en disco mínimo de 1 GB.
Sistema gestor de base de datos SQL Server 2022.
Resolución mínima de pantalla de 1366x768.

El sistema será desarrollado utilizando el lenguaje C# y tecnología .NET para aplicaciones de escritorio. Por ende, el sistema funcionará en computadoras con sistema operativo Windows 10 o superior.
El entorno de desarrollo utilizado será Visual Studio junto con el sistema gestor de base de datos Microsoft SQL Server 2019 o superior (compatible con SQL Server 2019 / 2022)

4. Ayuda al usuario

El sistema contará con un manual de usuario en formato digital PDF, redactado de manera clara y comprensible para cualquier usuario. El mismo incluirá explicaciones de acceso al sistema y de uso general de las funcionalidades del sistema, tales como el registro de ventas, la consulta de productos, el control de stock, la reposición de ejemplares, entre otras.

Además, el sistema incluirá ayudas contextuales dentro de las distintas pantallas y formularios, relativas a cada funcionalidad específica. Se incorporarán mensajes descriptivos, validaciones y tooltips sobre botones y campos importantes para facilitar la utilización del sistema, incluyendo una sección de ayuda desde el menú principal del sistema.

5. Guía de instalación y configuración. Archivo LEEME

El sistema contará con una guía detallada en formato PDF que servirá como hoja de ruta para establecer el entorno de ejecución. En la misma, se explicará paso a paso el procedimiento para instalar los archivos ejecutables y las dependencias necesarias, que permitirán al usuario ejecutar la aplicación de escritorio y configurar el motor de base de datos Microsoft SQL Server 2022, asegurando la compatibilidad con el sistema. Asimismo, se detallará el proceso de instalación y configuración de la base de datos necesaria para el funcionamiento de la aplicación.

Finalmente, se describirá el procedimiento de configuración inicial y el listado de requerimientos mínimos de hardware y software necesarios para la correcta ejecución del sistema.

G06. Diagramas de Clases

G07. Diagrama Entidad Relación

N01. Especificación funcional por proceso de negocio

PN1: Gestión de ventas en tienda física

Roles intervinientes

- Cliente (Persona. No es actor directo porque solo provee información, por lo que no usa la GUI)
- Vendedor (Persona. Es un actor primario porque usa la GUI).
- Entidad Bancaria (Sistema Remoto. Se conecta con el Vendedor para procesar un pago).

Diagrama de Roles

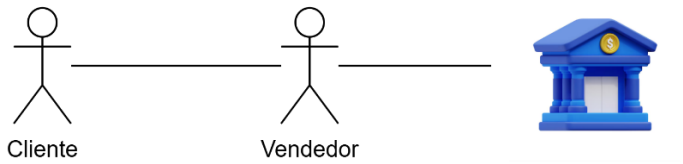


Diagrama de Secuencia

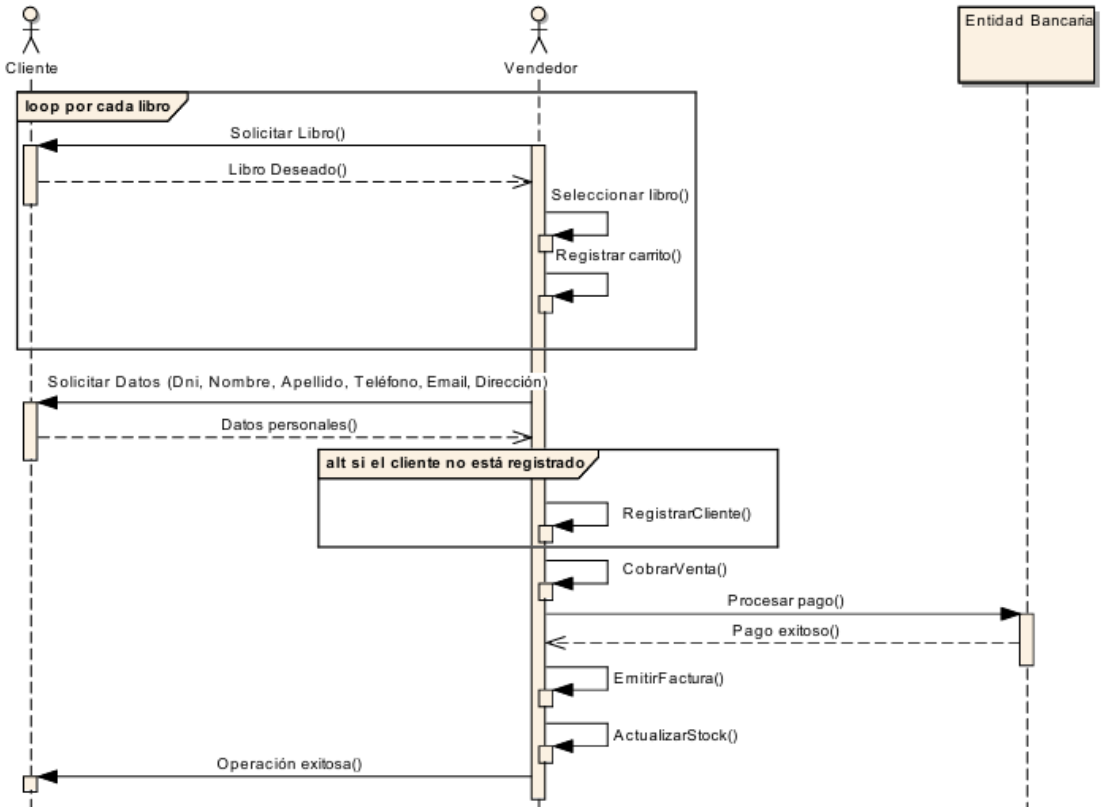


Diagrama Entrada – Comportamiento – Salida

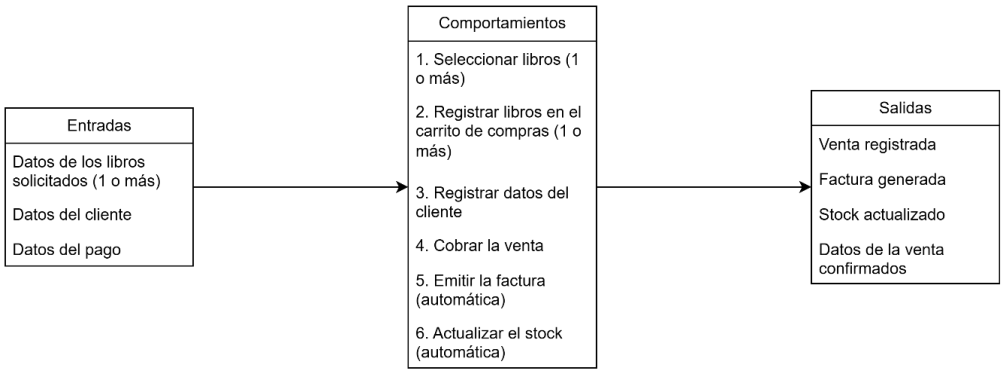
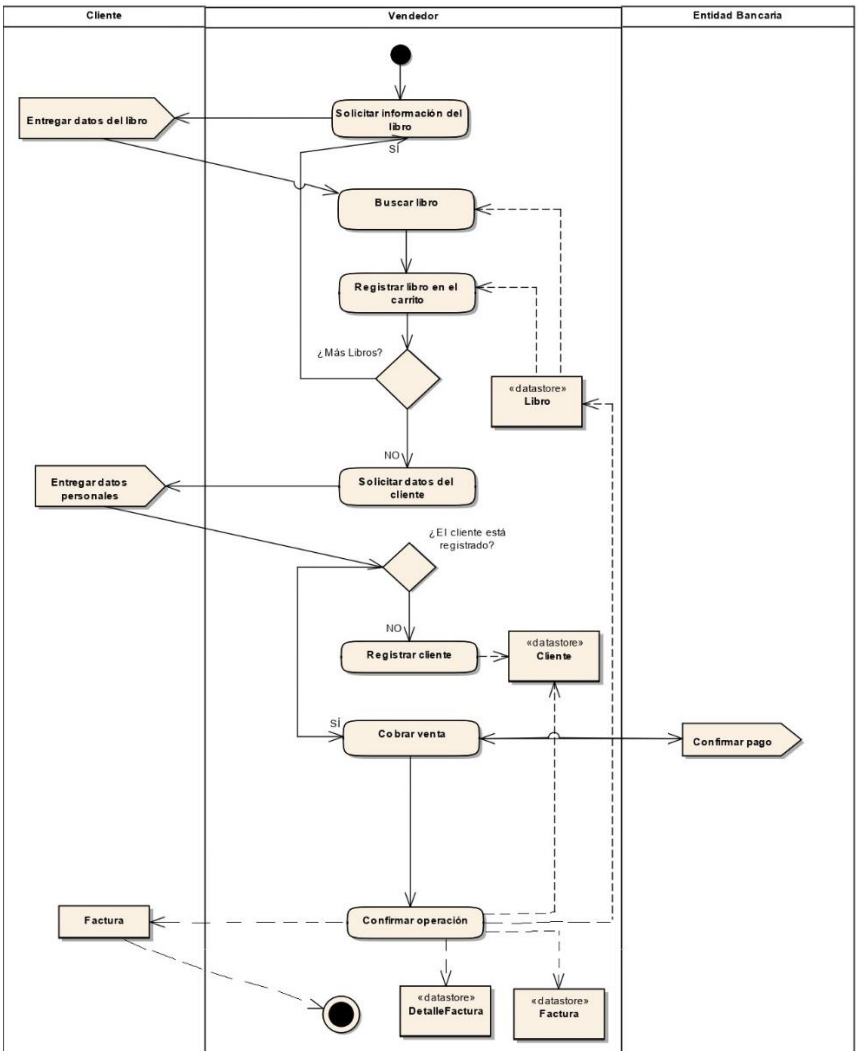
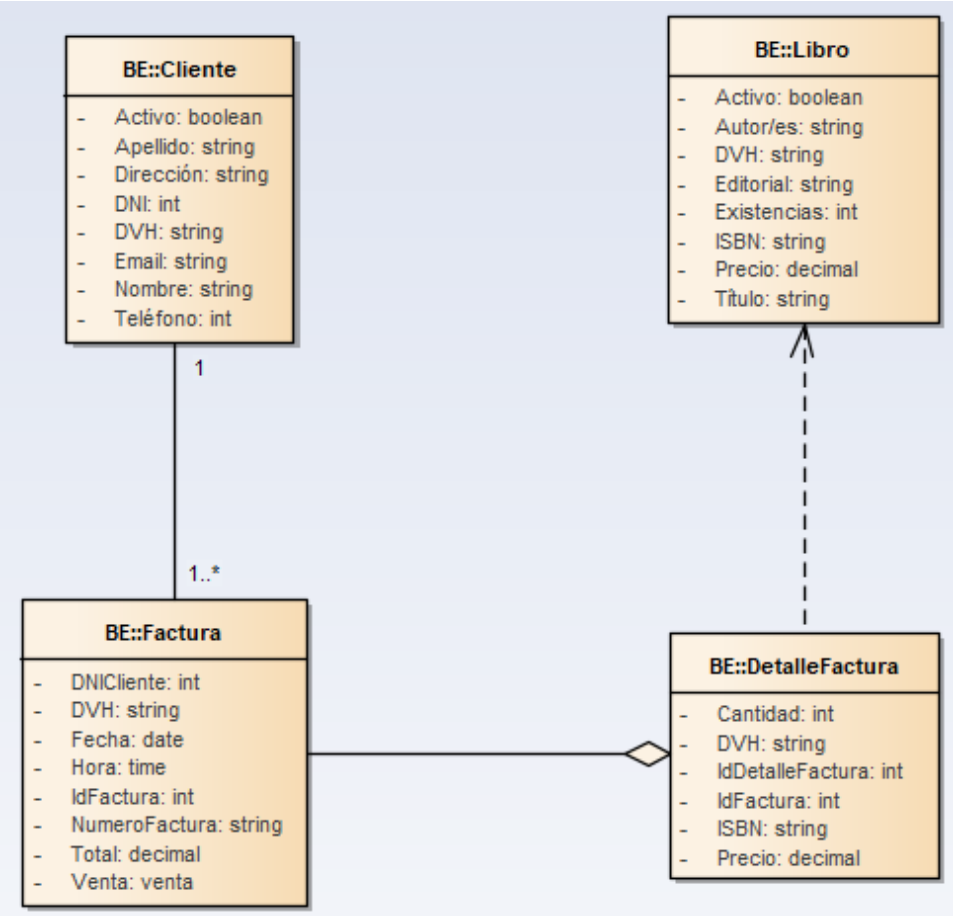


Diagrama de Proceso



Modelo Conceptual



PN2: Gestión de préstamos y devolución de libros en tienda física

Roles intervinientes

- Cliente (Persona. No es actor directo porque solo provee información, por lo que no usa la GUI)
- Vendedor (Persona. Es un actor primario porque usa la GUI).
- Entidad Bancaria (Sistema Remoto. Se conecta con el Vendedor para procesar un pago).

Diagrama de Roles

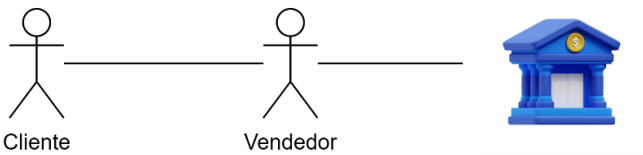


Diagrama de Secuencia

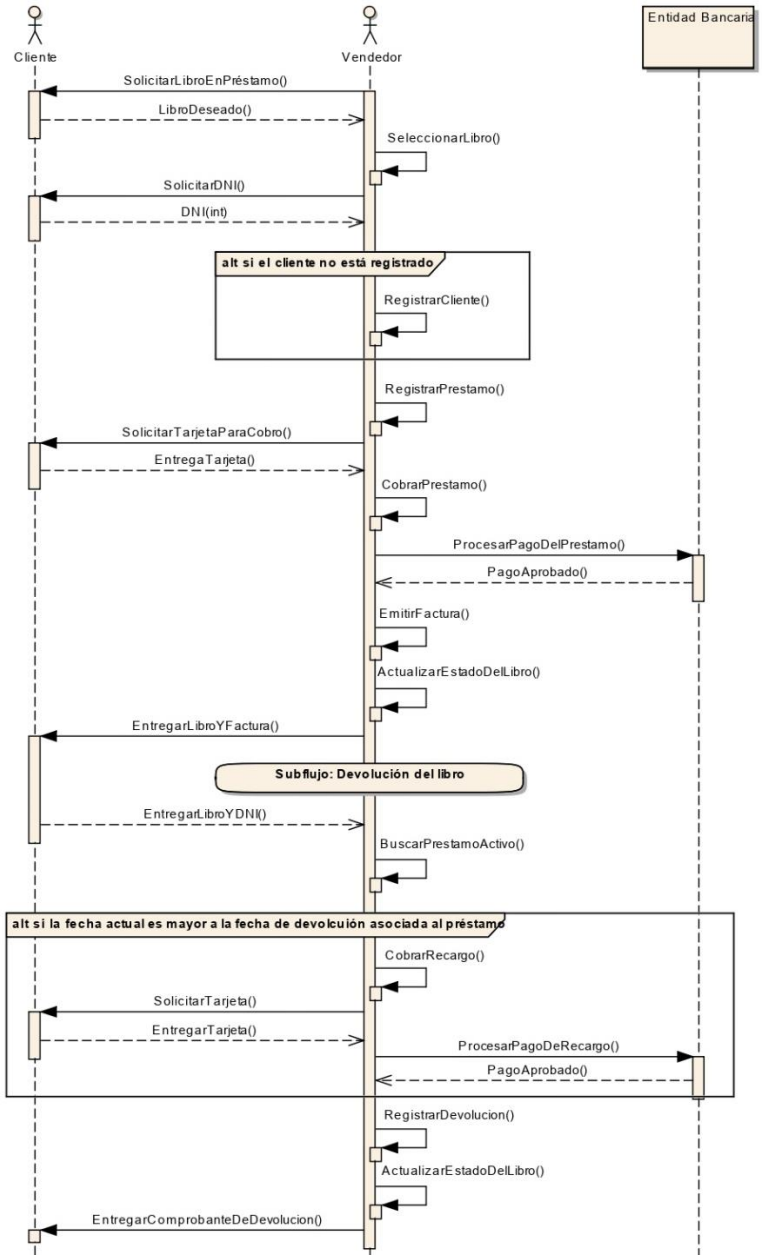


Diagrama Entrada – Comportamiento – Salida

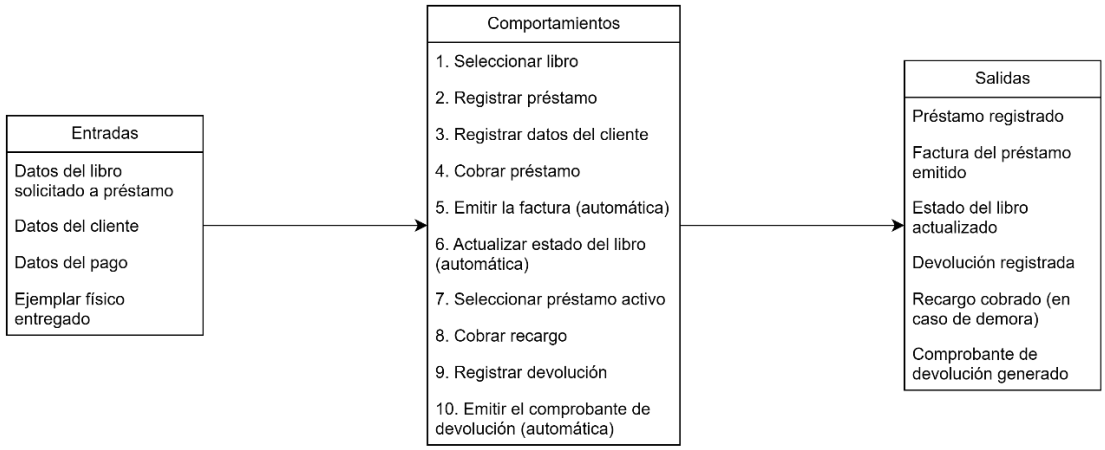
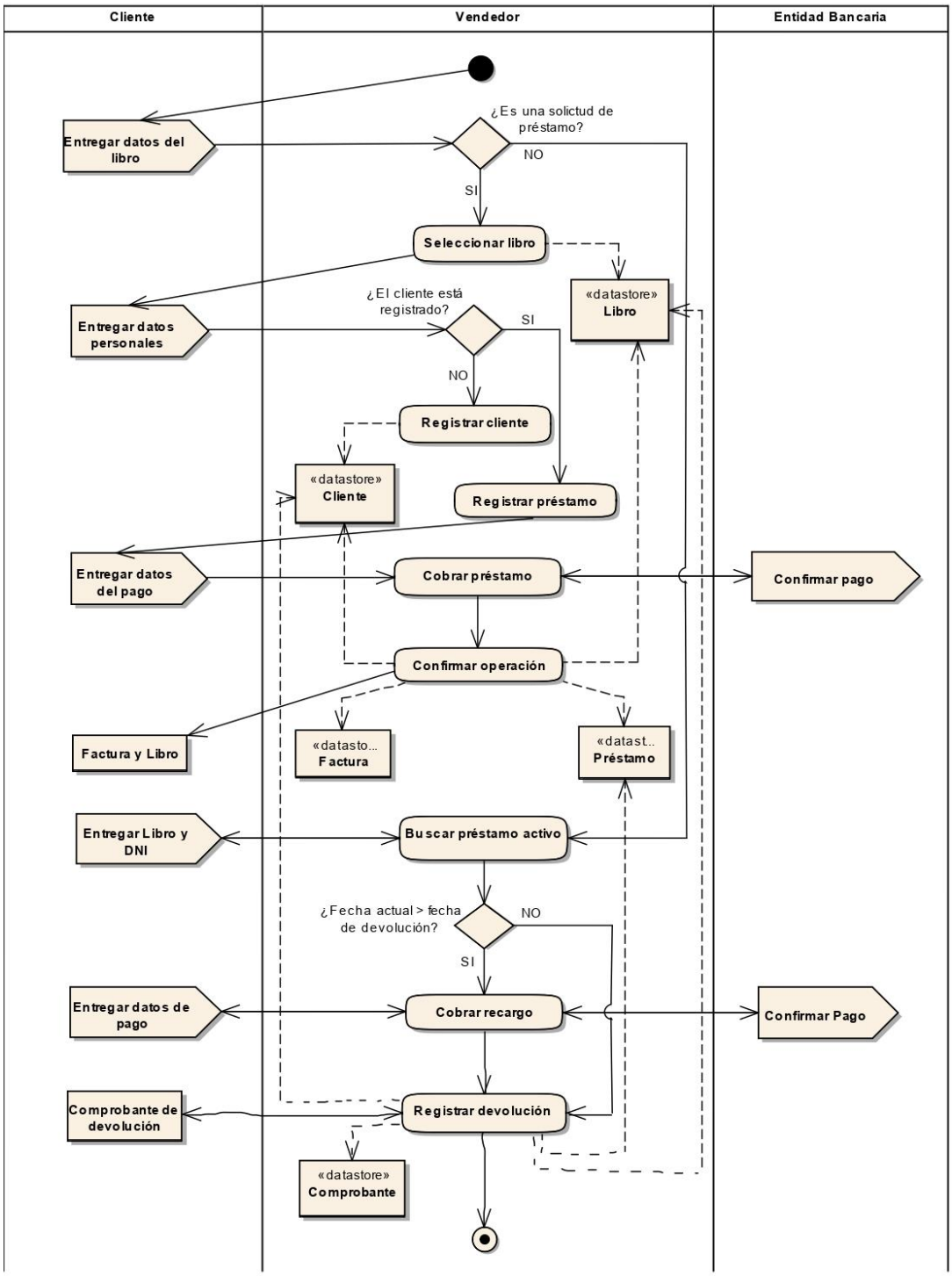
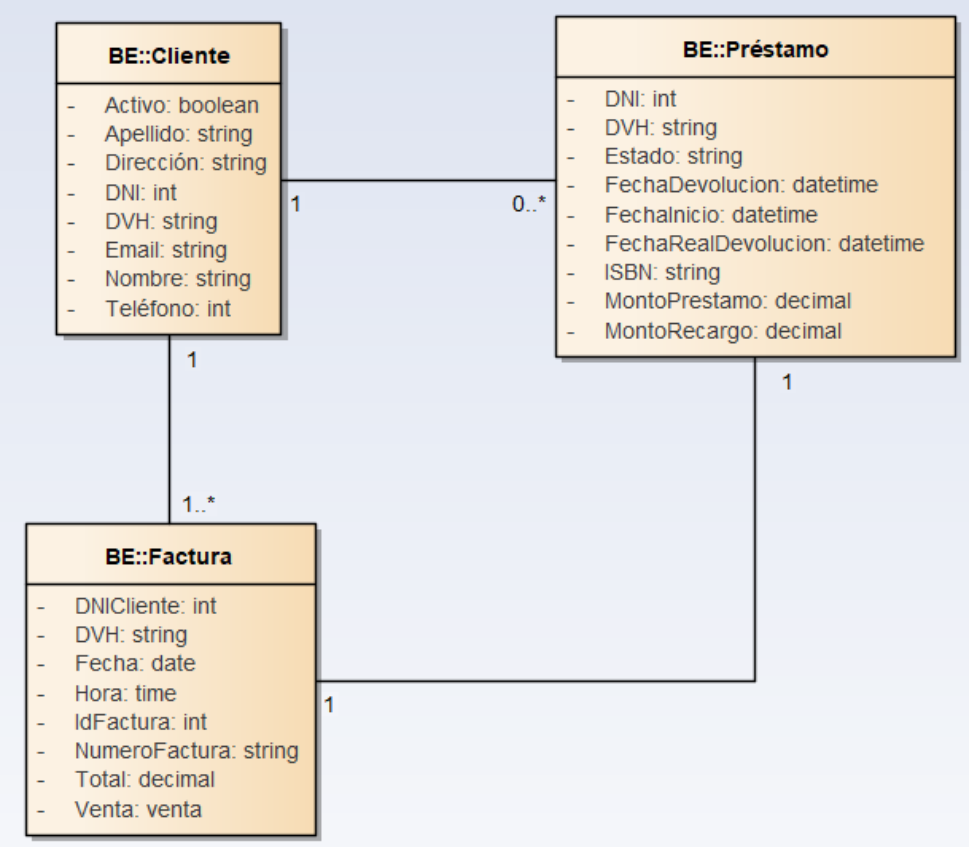


Diagrama de Proceso


Cliente
Vendedor
Entidad Bancaria
Entregar datos del libro
¿Es una solictud de préstamo?
NO
SI
Seleccionar libro
«datastore» Libro
Entregar datos personales
¿El cliente está registrado?
SI
NO
Registrar cliente
«datastore» Cliente
Registrar préstamo
Entregar datos del pago
Cobrar préstamo
Confirmar pago
Confirmar operación
Factura y Libro
«datasto... Factura
«datast.. Préstamo
Entregar Libro y DNI
Buscar préstamo activo
¿Fecha actual > fecha de devolución?
NO
SI
Entregar datos de pago
Cobrar recargo
Confirmar Pago
Comprobante de devolución
Registrar devolución
«datastore» Comprobante

Modelo Conceptual

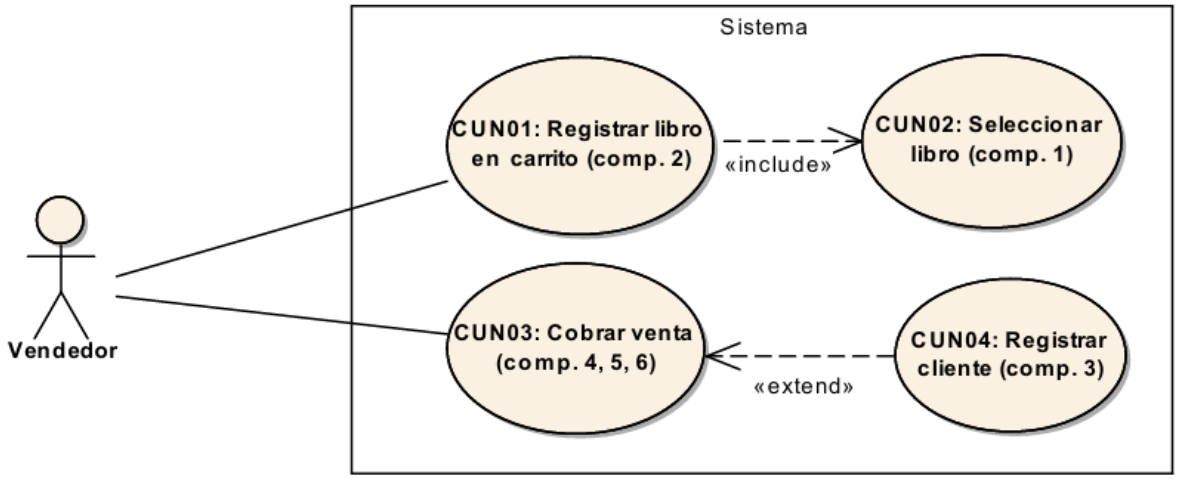


N02. Especificaciones de Casos de Uso por Proceso de Negocio

En este apartado, se especificarán en profundidad los procesos de negocio identificados en el punto G02, documentando los casos de uso correspondientes a cada proceso.

PN1: Gestión de ventas en tienda física

N02.PN1.1 Diagrama de casos de uso general



N02.PN1.2 CUN01 "Registrar libro en el carrito"

Especificación de Caso de Uso

ID y Nombre: CUN01 "Registrar libro en el carrito".
Descripción: Permite al vendedor agregar al carrito el libro seleccionado por el cliente para continuar con el proceso de venta.
Actores: Vendedor.
Precondiciones: • El vendedor debe haber iniciado sesión. • Debe existir un carrito asociado a la venta en curso. • El libro solicitado debe encontrarse registrado en el catálogo. • El libro debe contar con existencias disponibles.
Puntos de extensión: CUN02 – "Seleccionar Libro"
Flujo principal: 1. Se incluye el CUN02 – Seleccionar Libro 2. El vendedor indica la cantidad de unidades que desea agregar al carrito del libro seleccionado. 3. El vendedor confirma la operación. 4. Se verifica que existan unidades disponibles. 5. Se registra el libro en el carrito, actualizando la información del carrito. 6. Se confirma al vendedor que el libro fue agregado correctamente. 7. Se muestra el contenido del carrito con la información actualizada.
Flujo alternativo:

4.1.1 Se detecta que el libro no posee las unidades solicitadas.
4.1.2 Se informa al vendedor que el libro no puede ser agregado al carrito.
5.1.1 Se detecta que el libro ya se encuentra registrado en el carrito.
5.1.2 Se incrementa la cantidad correspondiente en el carrito.

Postcondiciones:

- El libro queda asociado al carrito de compras actual.
- El carrito queda actualizado con el libro seleccionado.
- La información del carrito queda disponible para continuar con la venta.

Diagrama de secuencia

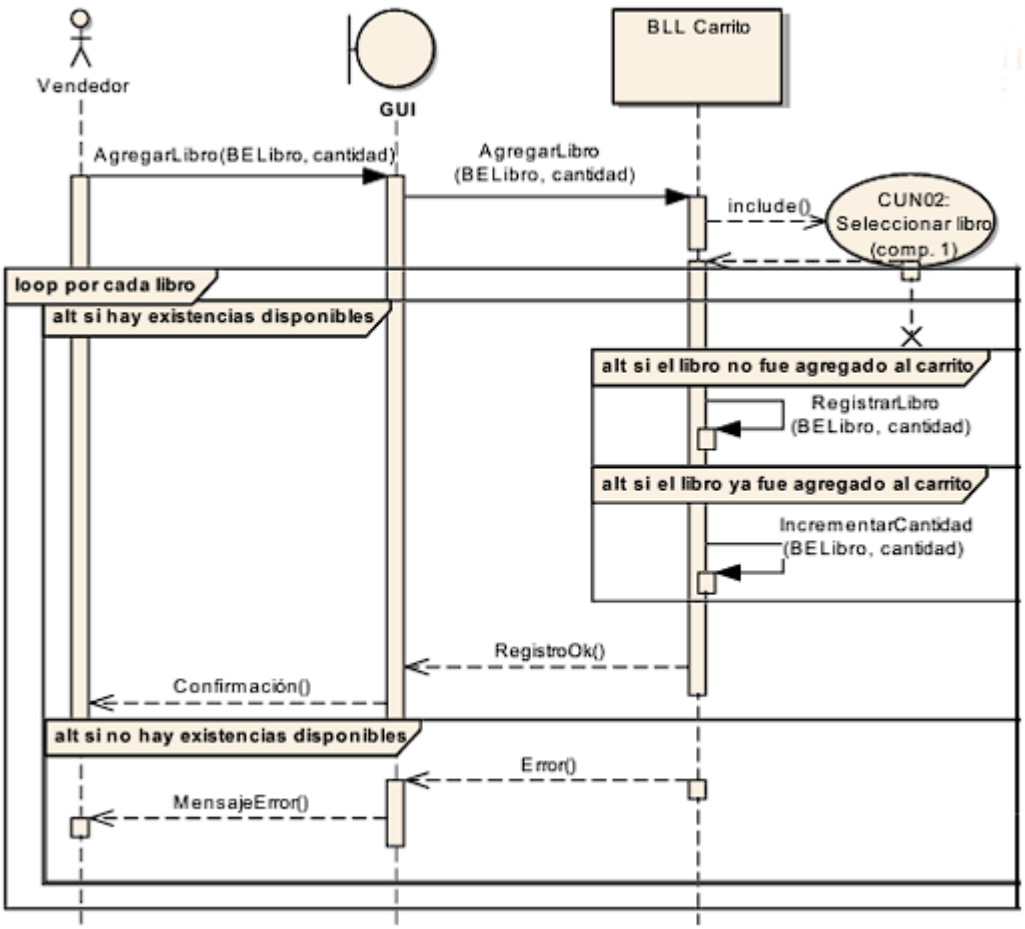


Diagrama de clases

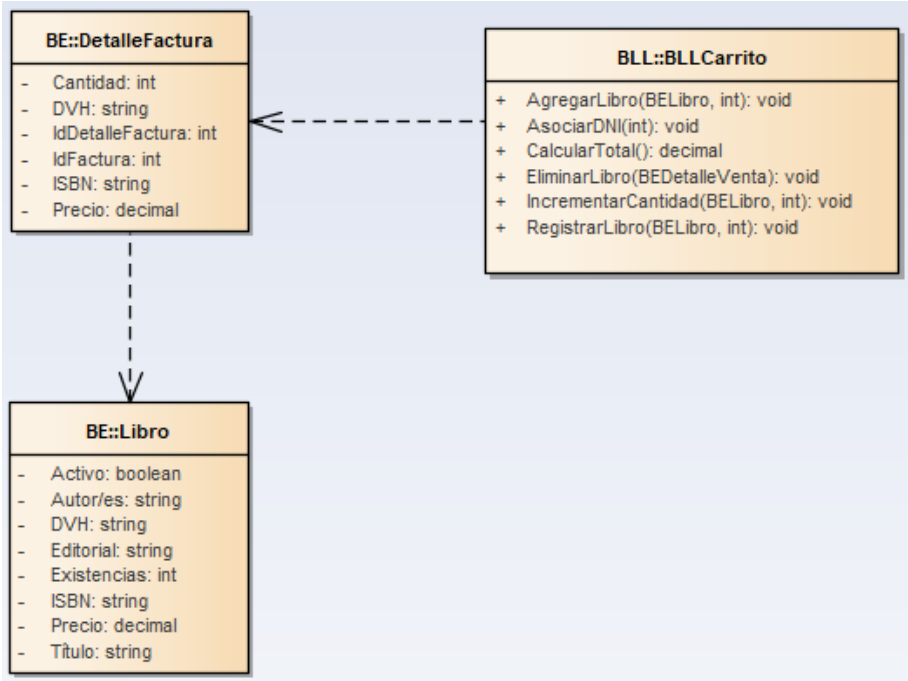
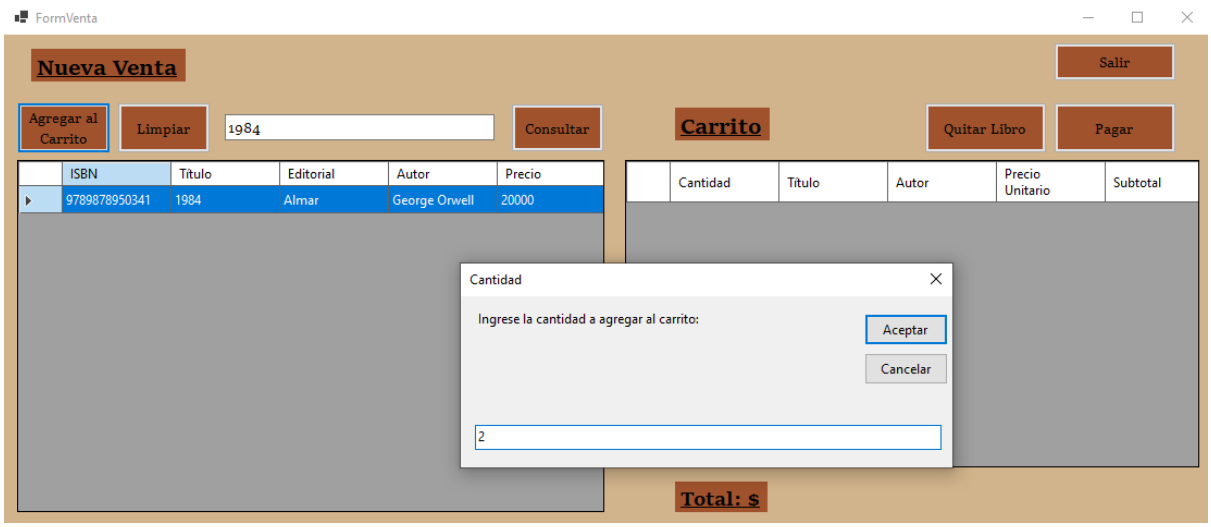
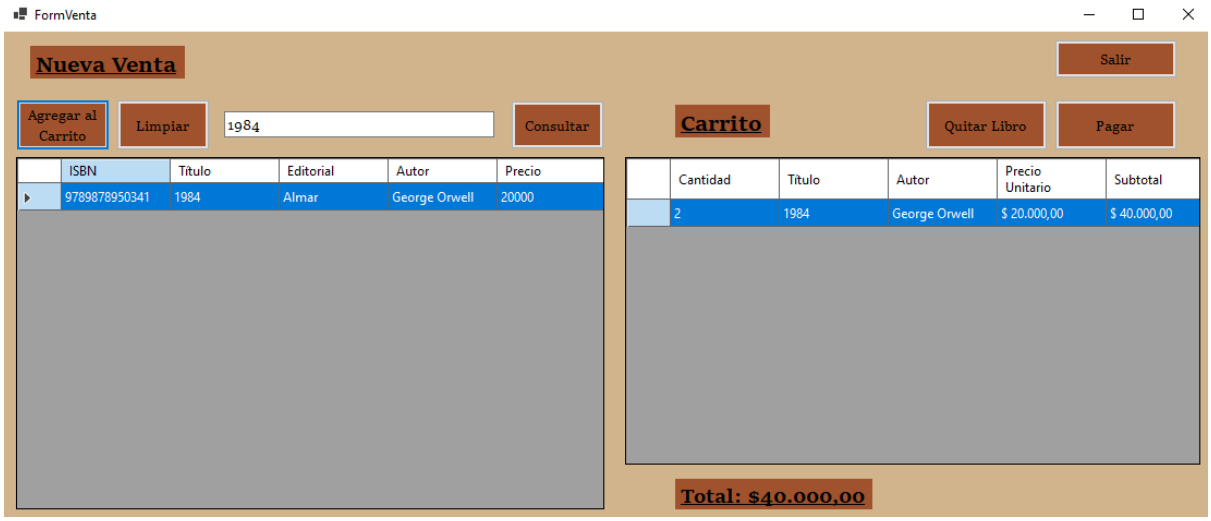


Diagrama DER

No existe un modelo de base de datos para este caso de uso

GUI





N02.PN1.3 CUN02 “Seleccionar libro”

Especificación de Caso de Uso

ID y Nombre: CUN02 “Seleccionar libro”.
Descripción: Permite al vendedor buscar y seleccionar del catálogo el libro solicitado por el cliente para continuar con el proceso de registro en el carrito.
Actores: Vendedor.
Precondiciones: • El vendedor debe haber iniciado sesión. • El catálogo de libros debe encontrarse disponible. • El cliente debe haber indicado el libro que desea comprar.
Puntos de extensión: N/A
Flujo principal: 1. Se consulta en la base de datos el catálogo de libros. 2. Se muestra el catálogo completo de libros 3. El cliente indica al vendedor el libro que desea comprar. 4. El vendedor ingresa los datos del libro solicitado para realizar la búsqueda. 5. Se busca en la base de datos los resultados en base a la información ingresada. 6. Se muestran los libros que coinciden con los datos ingresados. 7. El vendedor selecciona el libro solicitado. 8. Se muestra la información correspondiente al libro seleccionado. 9. Se incluye la opción de registrar el libro en el carrito de compras.

Flujo alternativo:
6.1.1 Se detecta que no existen libros que coincidan con los datos ingresados.
6.1.2 Se informa al vendedor que no se encontraron resultados.
6.1.3 El vendedor puede modificar los datos de búsqueda y realizar una nueva consulta.

Postcondiciones:
- El libro solicitado queda seleccionado.
- La información del libro seleccionado queda a disposición del vendedor.

Diagrama de secuencia

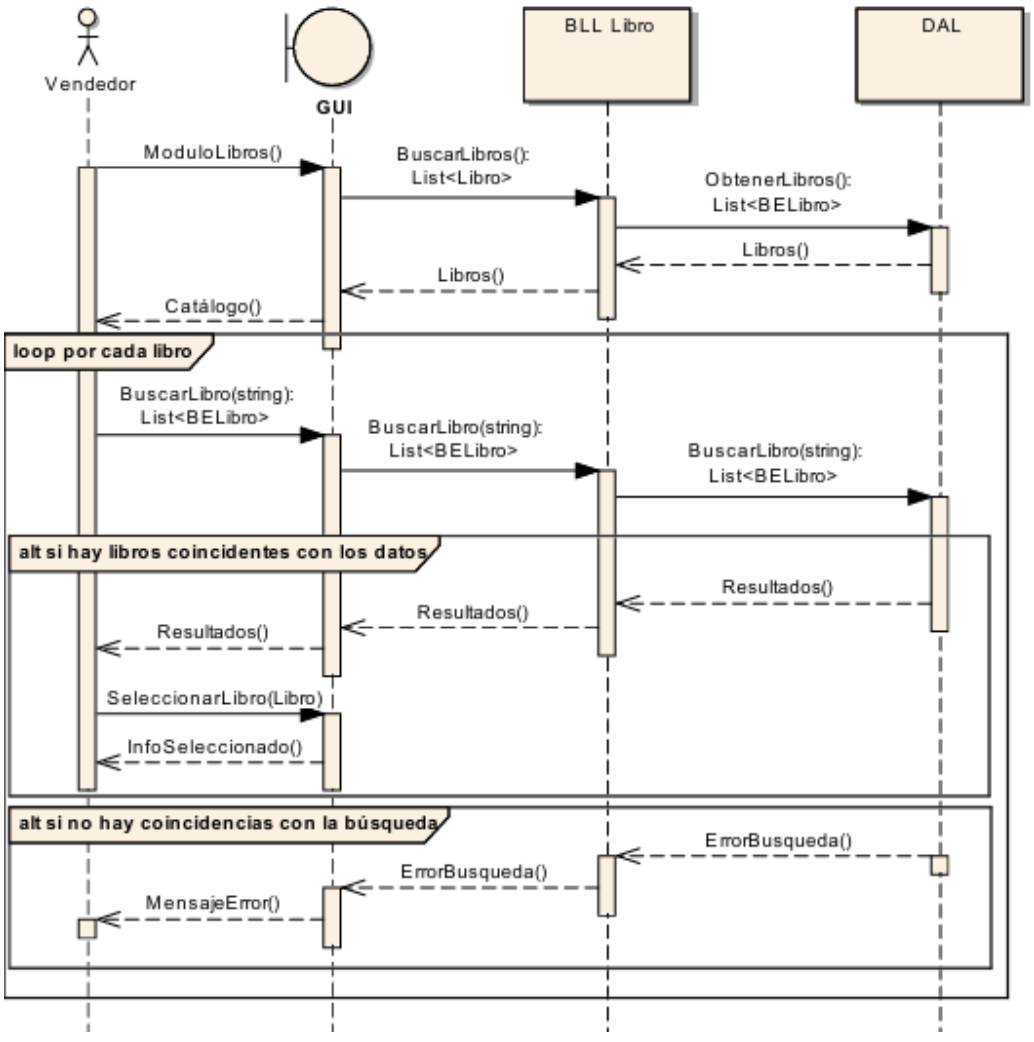


Diagrama de clases

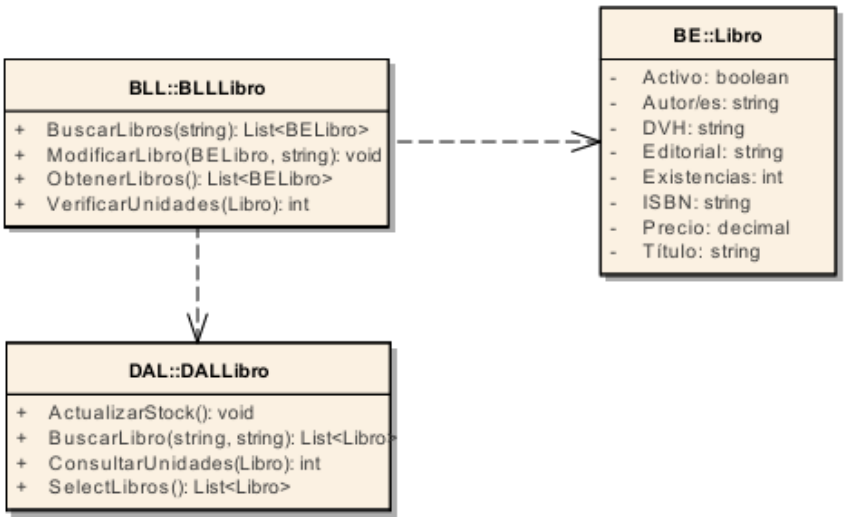
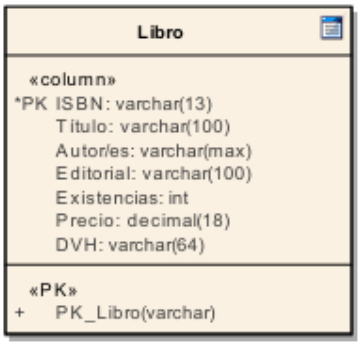
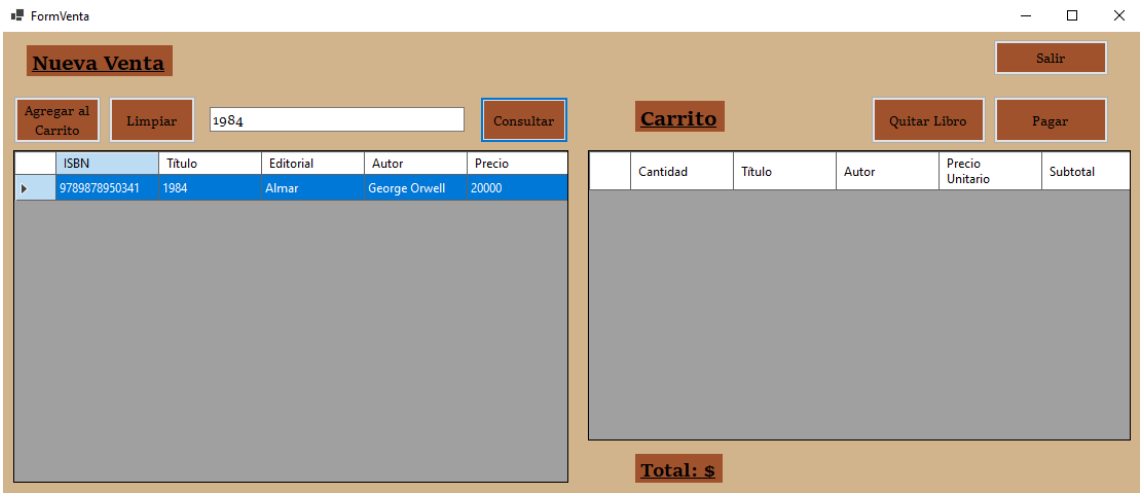


Diagrama DER



GUI



N02.PN1.4 CUN03 “Cobrar venta”

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Especificación de Caso de Uso

ID y Nombre: CUN03 "Cobrar venta".

Descripción: Permite al vendedor registrar y procesar el pago correspondiente a una venta, utilizando el medio de pago seleccionado por el cliente, para confirmar o rechazar la operación.

Actores: Vendedor.

Precondiciones:

- El vendedor debe haber iniciado sesión.
- Debe existir una venta en curso.
- La venta debe contener al menos un libro registrado en el carrito.
- El importe total de la venta debe estar calculado.

Puntos de extensión: CUN04 – "Registrar cliente"

Flujo principal:

1. El vendedor selecciona la opción de "Finalizar compra" dentro del carrito.
2. El vendedor asocia el DNI del cliente al carrito de compras.
3. El vendedor indica al cliente el total de la venta.
4. El cliente indica con qué tipo de tarjeta desea pagar (débito / crédito).
5. El vendedor registra el medio de pago seleccionado.
6. El vendedor procesa el pago correspondiente con la tarjeta del cliente.
7. La Entidad Bancaria verifica el resultado de la operación.
8. Se genera la factura correspondiente a la venta.
9. Se actualiza el stock de los libros vendidos.
10. Se registra la venta en el historial de operaciones.
11. El vendedor confirma la venta.
12. El vendedor le entrega al cliente una copia de la factura correspondiente.

Flujo alternativo:

2.1.1 El cliente no está registrado en la base de datos.
2.1.2 Se extiende al caso de uso CUN04 – "Registrar cliente".
7.1.1 El vendedor detecta que el pago fue rechazado por la Entidad Bancaria.
7.1.2 La venta queda cancelada.

Postcondiciones:

- El pago queda registrado.
- La venta queda confirmada.
- Se genera la factura correspondiente.
- El stock de los libros vendidos queda actualizado.

- La venta queda registrada en el historial de operaciones.

Diagrama de secuencia

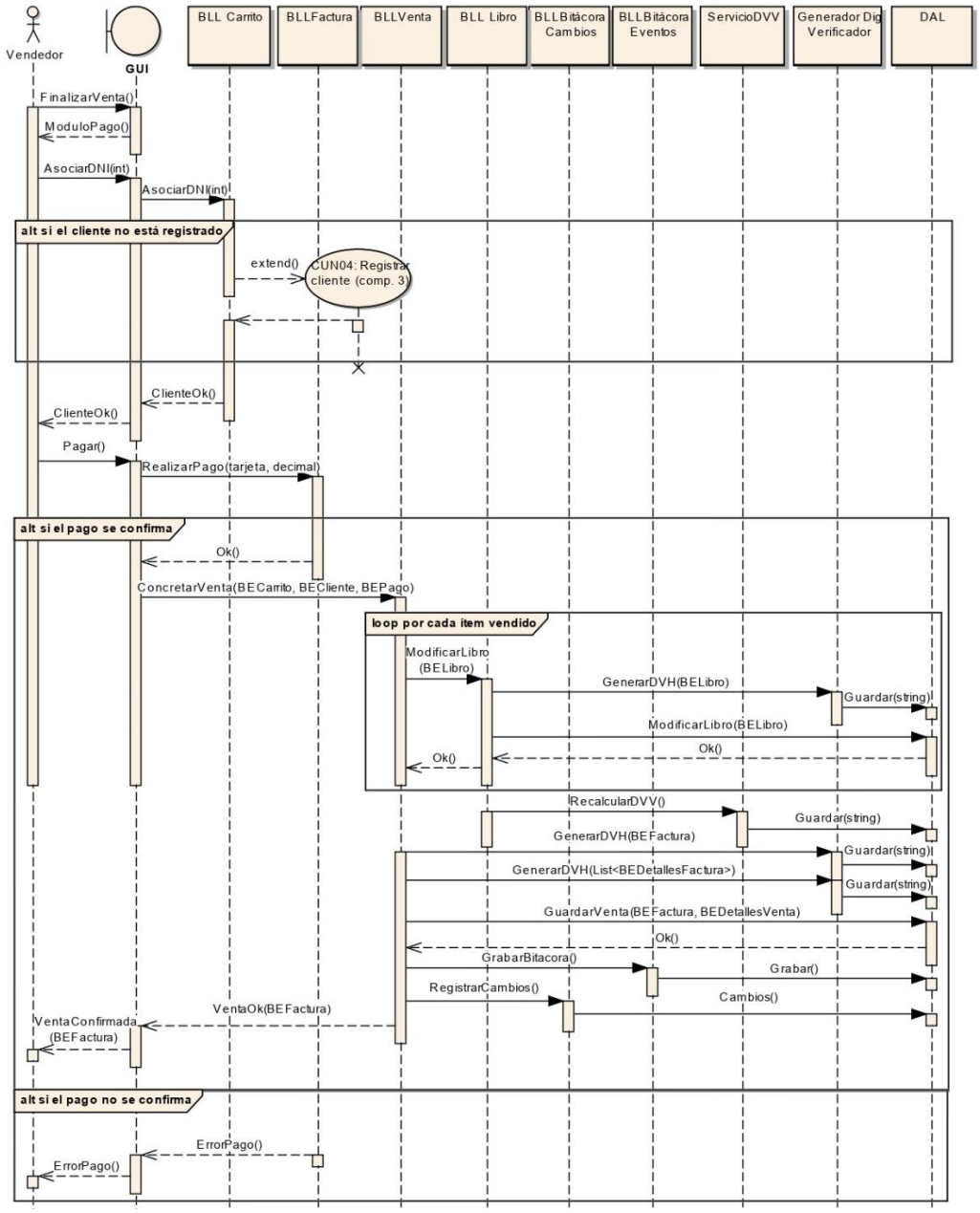


Diagrama de clases

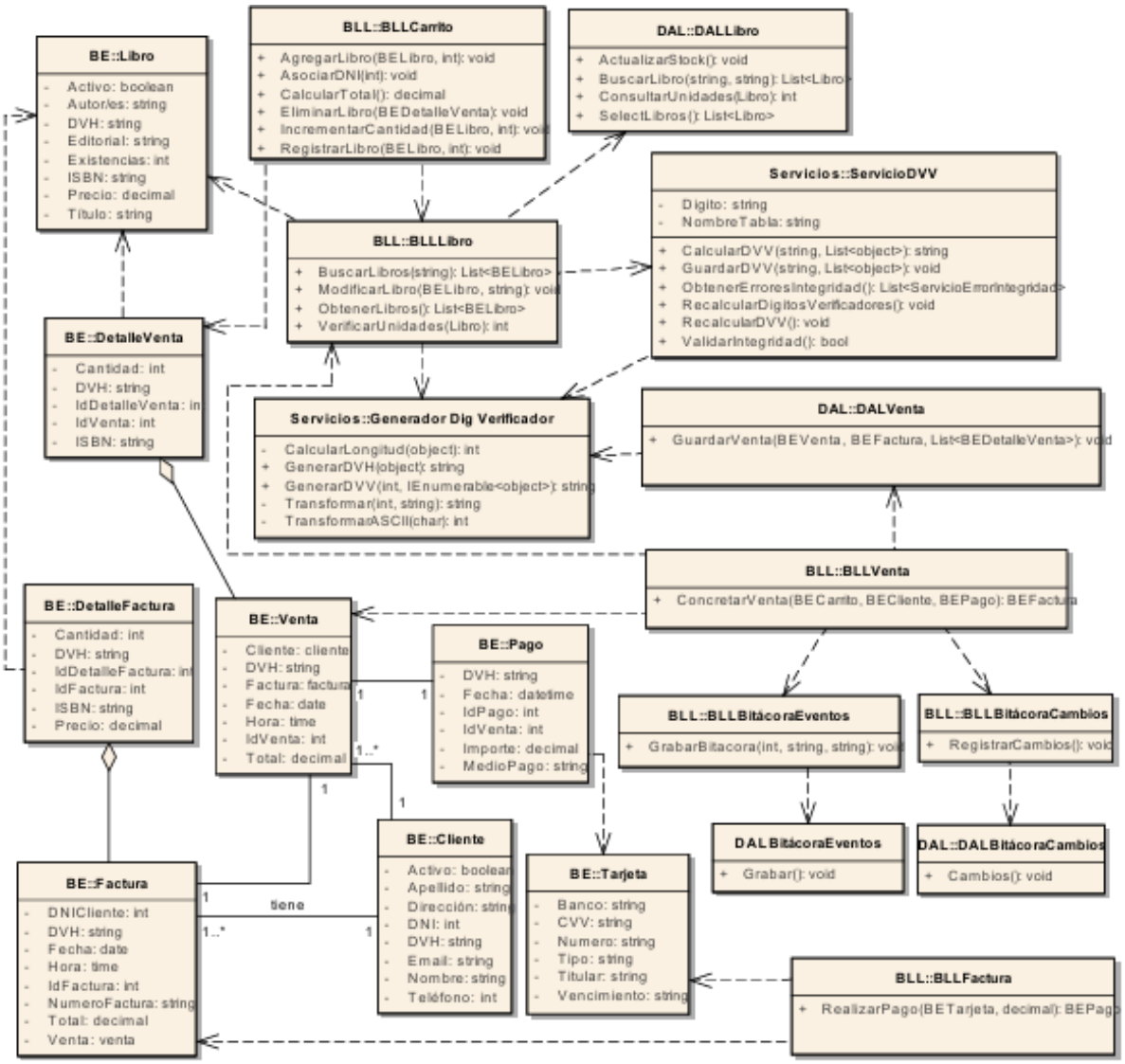
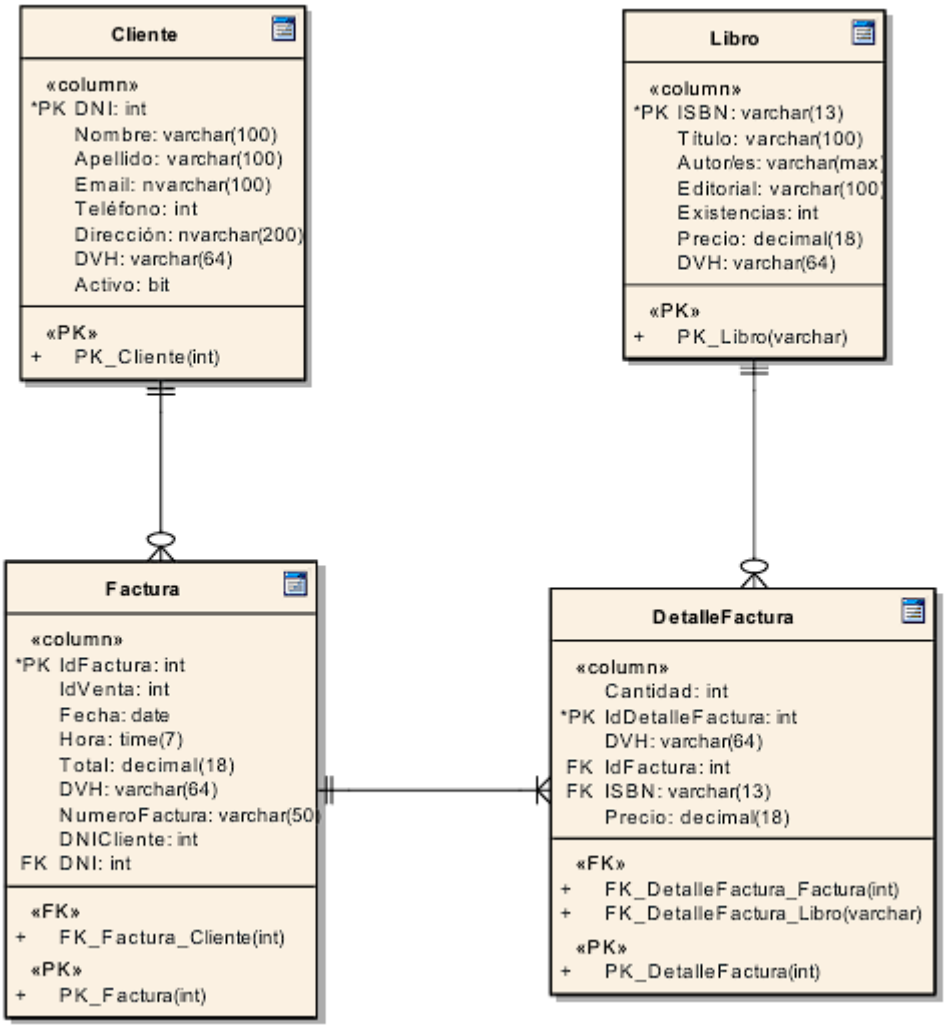


Diagrama DER



GUI



N02.PN1.5 CUN04 “Registrar cliente”

Especificación de Caso de Uso

ID y Nombre: CUN04 “Registrar usuario”.
Descripción: Permite al vendedor registrar los datos personales de un cliente que no se encuentra registrado en el sistema, para asociarlo posteriormente a la venta en curso.
Actores: Vendedor.
Precondiciones: • El vendedor debe haber iniciado sesión. • Debe existir una venta en curso. • El cliente no debe encontrarse registrado en la base de datos.
Puntos de extensión: N/A
Flujo principal:

1. El vendedor solicita al cliente sus datos personales.
2. El cliente proporciona su DNI, nombre, apellido, correo electrónico, teléfono y dirección.
3. El vendedor ingresa los datos del cliente.
4. Se verifica que el DNI no se encuentre registrado.
5. Se registra al nuevo cliente en la base de datos.
6. El vendedor es informado que el cliente fue registrado correctamente.
7. El cliente queda disponible para ser asociado a la venta en curso.

Flujo alternativo:

4.1.2 El DNI ingresado ya se encuentra registrado.

4.1.2 El vendedor puede utilizar los datos del cliente existente y continuar con el CUN03 – "Cobrar venta".

Postcondiciones:

- El cliente queda registrado en la base de datos.
- Los datos del cliente quedan disponibles para la venta en curso.

Diagrama de secuencia

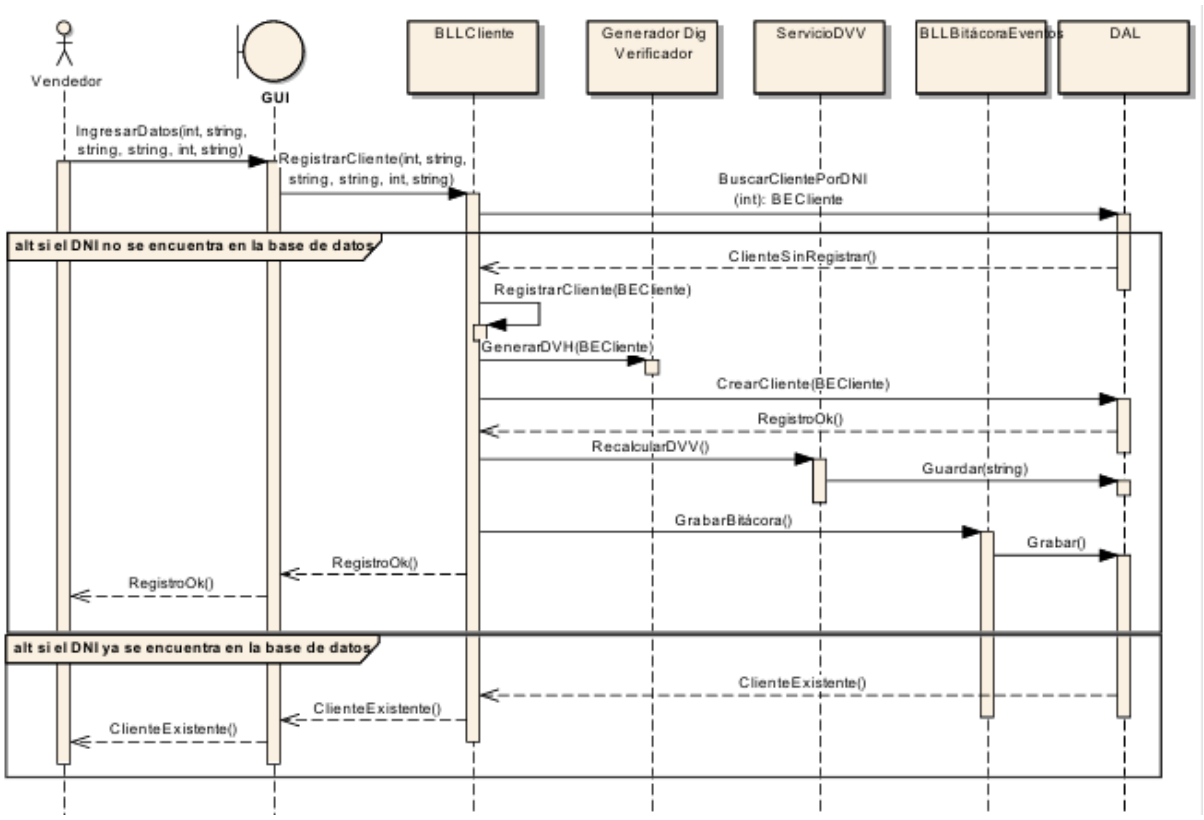
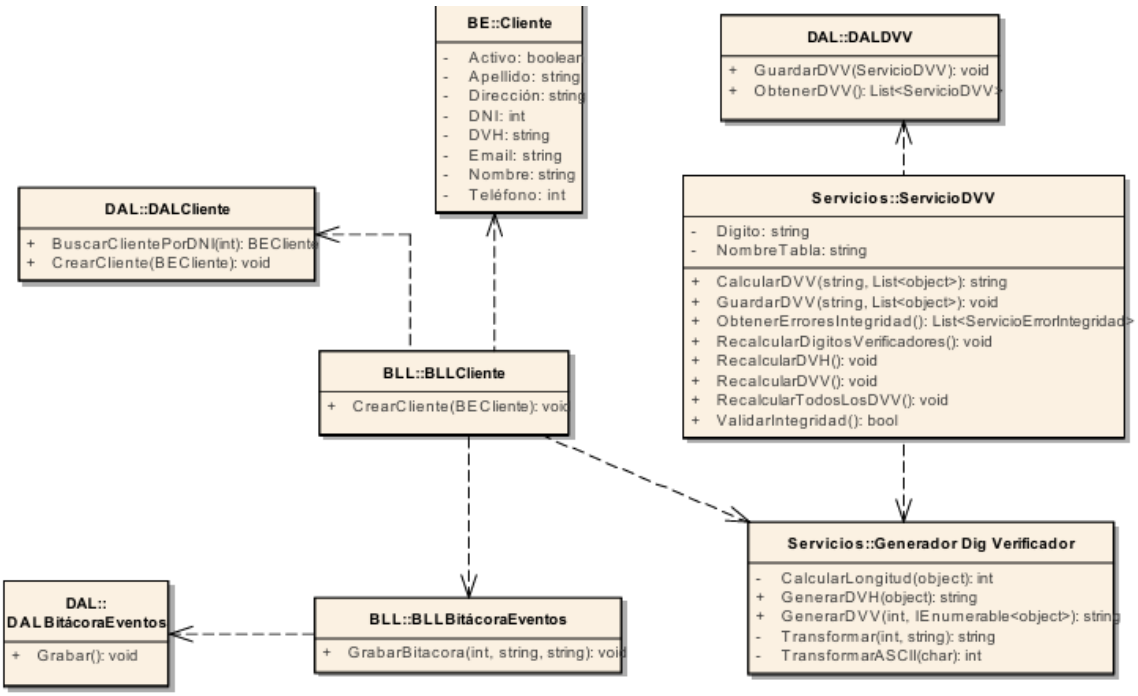
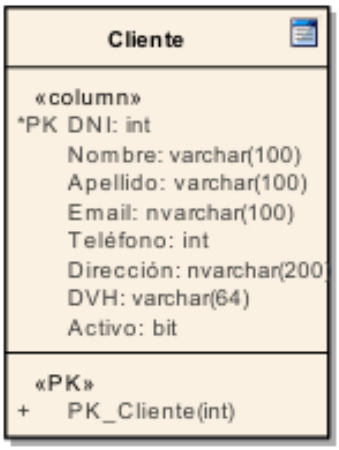


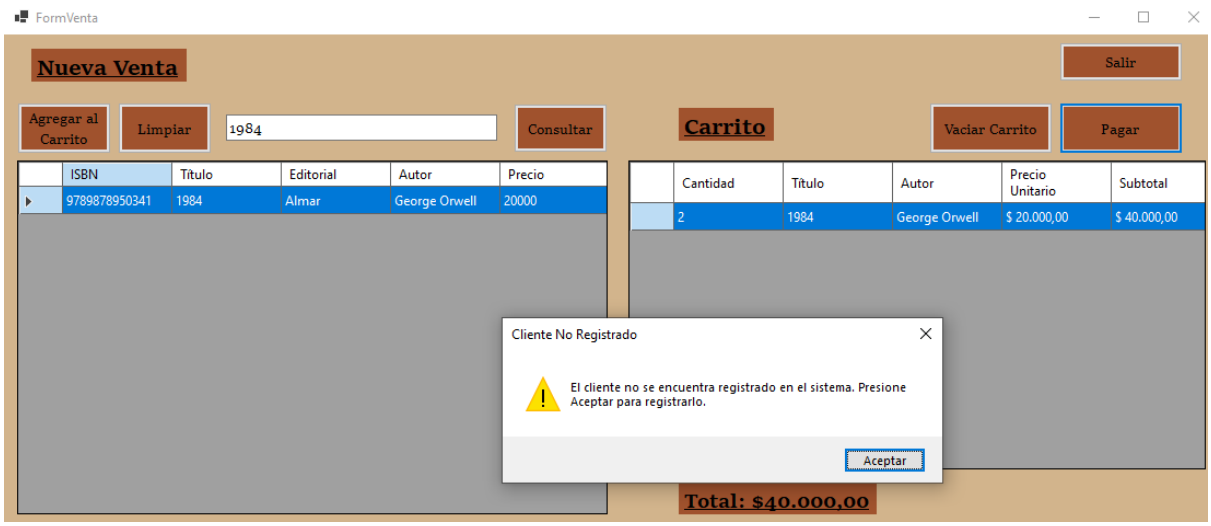
Diagrama de clases

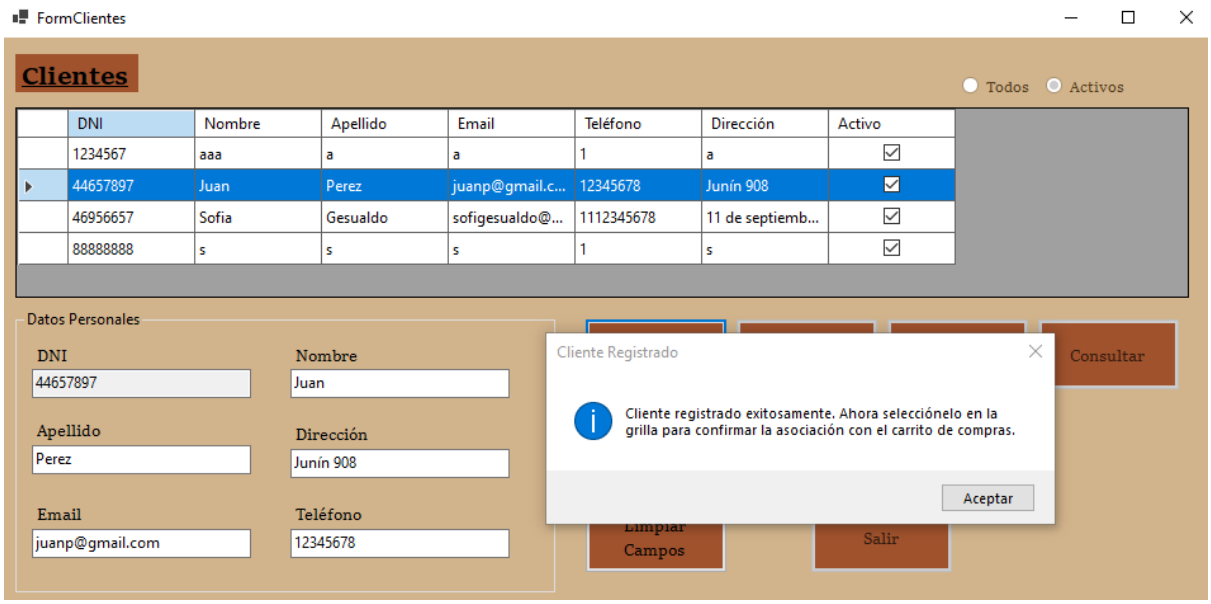


DER



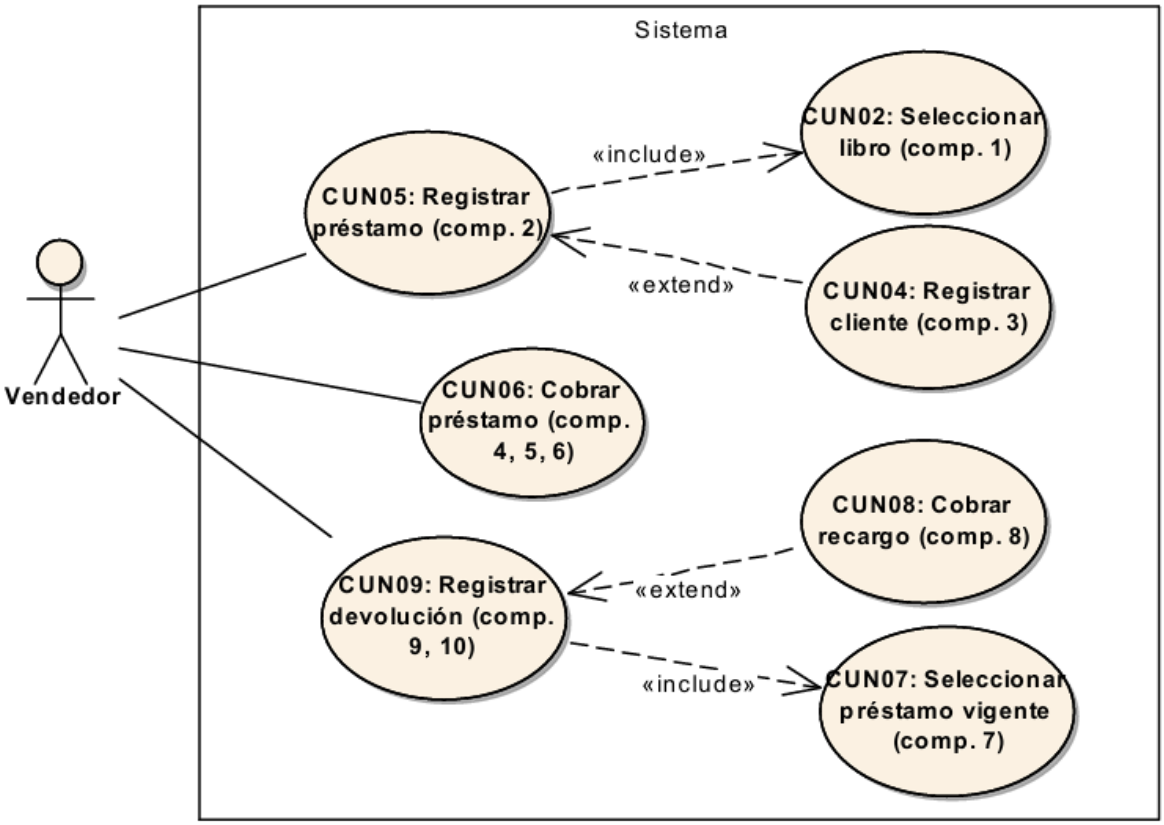
GUI





PN2: Gestión de préstamos y devolución de libros en tienda física

N02.PN2.1 Diagrama de casos de uso general



Nota: los casos de uso de negocio 2 y 4 ya fueron especificados en el proceso de negocio previo

N02.PN2.2 CUN05 "Registrar préstamo"

Especificación de Caso de Uso

ID y Nombre: CUN05 "Registrar préstamo".
Descripción: Permite al vendedor registrar el préstamo de un libro a un cliente, verificando la disponibilidad del ejemplar y validando que el cliente no posea ningún otro préstamo activo vigente.
Actores: Vendedor.
Precondiciones: • El vendedor debe haber iniciado sesión. • El catálogo de libros debe encontrarse disponible. • El ejemplar debe encontrarse disponible para préstamo.

Puntos de extensión: CUN02 – “Seleccionar Libro”, CUN04 – “Registrar Cliente”

Flujo principal:

1. Se incluye el CUN02 – Seleccionar libro.
2. Se verifica que el libro seleccionado tenga existencias y se encuentre disponible para préstamo.
3. El vendedor solicita el DNI del cliente para asociarlo al préstamo.
4. Se verifica que el DNI ingresado se encuentre registrado en la base de datos.
5. Se verifica que el cliente no tenga ningún préstamo activo pendiente de devolución.
6. El vendedor selecciona o ingresa la fecha prevista de devolución (ej. plazo estándar de 7 o 14 días).
7. Se calcula y muestra el importe a abonar por el concepto de préstamo.
8. El vendedor confirma la solicitud de préstamo.

Flujo alternativo:

2.1.1 Se notifica al vendedor que el ejemplar no cuenta con stock disponible.
2.1.2 Se permite volver a realizar una búsqueda en el catálogo.
4.1.1 El cliente no se encuentra registrado en la base de datos:
4.1.2 Se extiende al caso de uso CUN04 – Registrar cliente.
4.1.3 Tras completar el alta del cliente, se retoma el flujo en el paso 5.
5.1.1 Se detecta que el cliente ya tiene un préstamo registrado con estado "Activo".
5.1.2 Se bloquea la operación y se muestra un mensaje de error.
5.1.3 El caso de uso finaliza sin registrar el nuevo préstamo.

Postcondiciones:

- Se registra el préstamo asociado al cliente con el libro, las fechas pactadas y el estado "Pendiente de Cobro”.
- El cliente queda inhabilitado para solicitar nuevos préstamos hasta devolver el libro.

Diagrama de secuencia

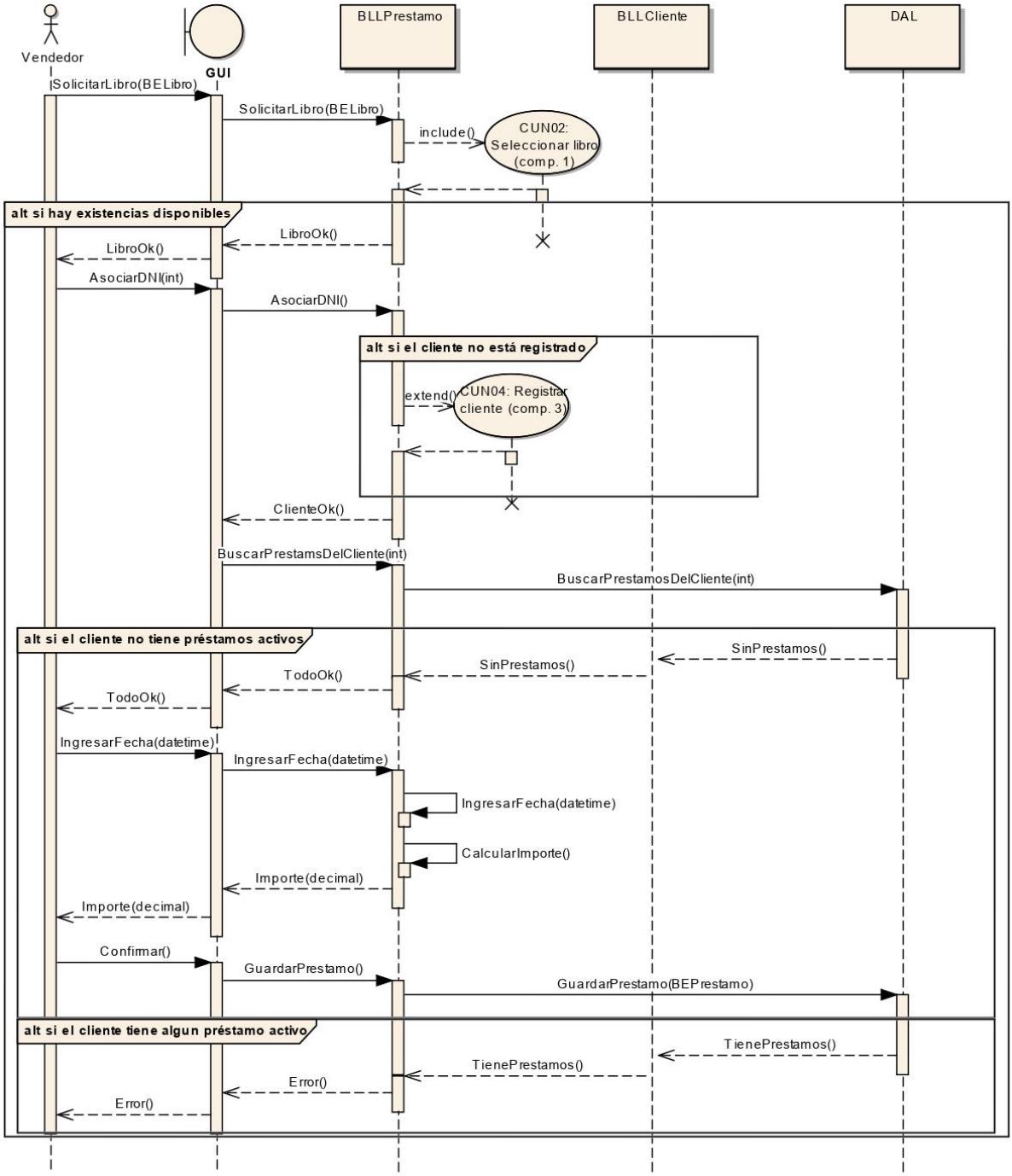
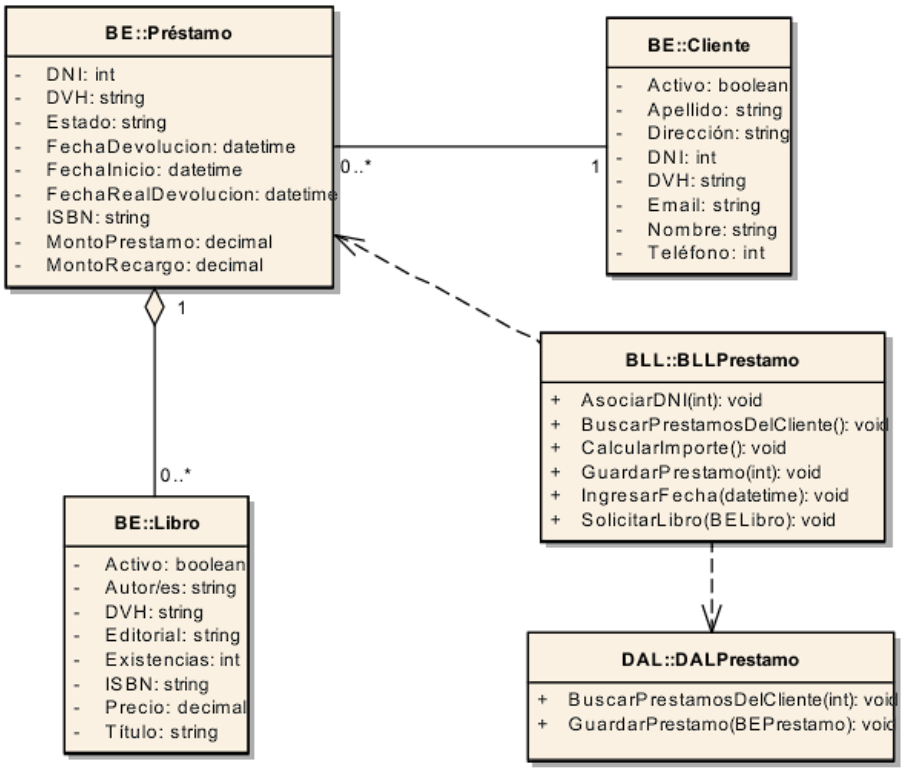
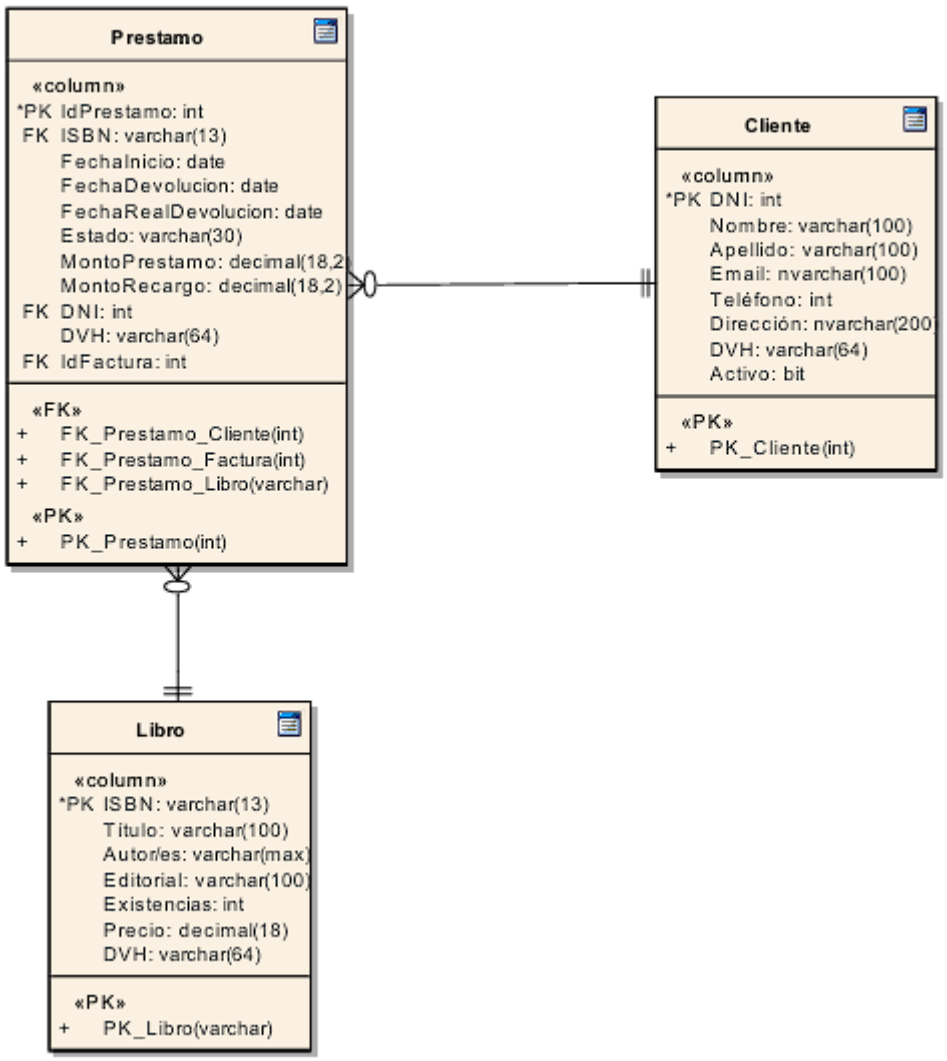

Vendedor
GUI
BLLPrestamo
BLLCliente
DAL
SolicitarLibro(BELibro)
SolicitarLibro(BELibro)
include()
CUN02: Seleccionar libro (comp. 1)
alt si hay existencias disponibles
LibroOk()
LibroOk()
AsociarDNI(int)
AsociarDNI()
alt si el cliente no está registrado
extend()
CUN04: Registrar cliente (comp. 3)
ClienteOk()
BuscarPrestamsDelCliente(int)
BuscarPrestamosDelCliente(int)
alt si el cliente no tiene préstamos activos
SinPrestamos()
SinPrestamos()
TodoOk()
TodoOk()
IngresarFecha(datetime)
IngresarFecha(datetime)
IngresarFecha(datetime)
CalcularImporte()
Importe(decimal)
Importe(decimal)
Confirmar()
GuardarPrestamo()
GuardarPrestamo(BEPrestamo)
alt si el cliente tiene algun préstamo activo
TienePrestamos()
TienePrestamos()
Error()
Error()

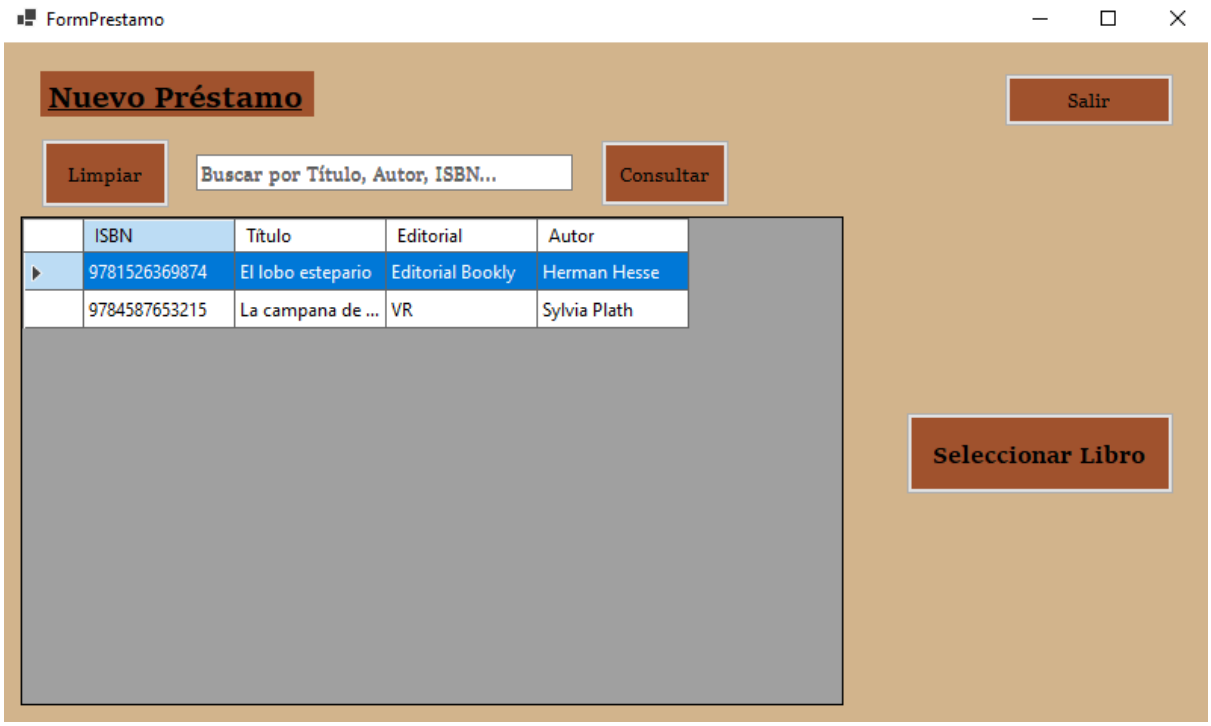
Diagrama de clases



DER


Prestamo
«column»
*PK IdPrestamo: int
FK ISBN: varchar(13)
FechaInicio: date
FechaDevolucion: date
FechaRealDevolucion: date
Estado: varchar(30)
MontoPrestamo: decimal(18,2)
MontoRecargo: decimal(18,2)
FK DNI: int
DVH: varchar(64)
FK IdFactura: int
«FK»
+ FK_Prestamo_Cliente(int)
+ FK_Prestamo_Factura(int)
+ FK_Prestamo_Libro(varchar)
«PK»
+ PK_Prestamo(int)
Cliente
«column»
*PK DNI: int
Nombre: varchar(100)
Apellido: varchar(100)
Email: nvarchar(100)
Teléfono: int
Dirección: nvarchar(200)
DVH: varchar(64)
Activo: bit
«PK»
+ PK_Cliente(int)
Libro
«column»
*PK ISBN: varchar(13)
Titulo: varchar(100)
Autor/es: varchar(max)
Editorial: varchar(100)
Existencias: int
Precio: decimal(18)
DVH: varchar(64)
«PK»
+ PK_Libro(varchar)

GUI



N02.PN2.3 CUN06 “Cobrar préstamo”

Especificación de Caso de Uso

ID y Nombre: CUN06 “Cobrar préstamo”
Descripción: Permite al vendedor procesar el cobro correspondiente al préstamo del libro a través de la plataforma de pago (tarjeta de débito/crédito), emitir la factura del préstamo, actualizar la disponibilidad del ejemplar a no disponible y asentar la operación en la bitácora del sistema.
Actores: Vendedor.
Precondiciones: • El vendedor debe haber iniciado sesión. • Debe existir una solicitud de préstamo en curso.
Puntos de extensión: N/A
Flujo principal: 1. El vendedor selecciona la opción de “Cobrar préstamo” luego de haber registrado un nuevo préstamo. 2. El vendedor visualiza en la interfaz de cobro el importe total del préstamo y los datos del cliente asociado.

3. El cliente indica con qué tipo de tarjeta desea pagar (débito / crédito).
4. El vendedor registra el medio de pago seleccionado.
5. El vendedor procesa el pago correspondiente con la tarjeta del cliente.
6. El vendedor solicita el procesamiento del pago.
7. La Entidad Bancaria verifica el resultado de la operación.
8. El sistema confirma el préstamo cambiando su estado a “Activo”.
9. Se genera la factura correspondiente al préstamo.
10. El sistema actualiza automáticamente la disponibilidad del libro en el inventario, marcándolo como “Prestado”).
11. Se registra el préstamo en el historial de operaciones.
12. El vendedor le entrega al cliente el ejemplar solicitado y una copia de la factura correspondiente.

Flujo alternativo:

8.1.1 El vendedor detecta que el pago fue rechazado por la Entidad Bancaria.
8.1.2 El préstamo queda cancelado.

Postcondiciones:

- El cobro queda acreditado.
- El préstamo queda confirmado y activo en la base de datos.
- Se emite la factura correspondiente al alquiler.
- El ejemplar queda registrado como no disponible en el inventario.
- Queda asentada la acción en la bitácora de eventos.

Diagrama de secuencia

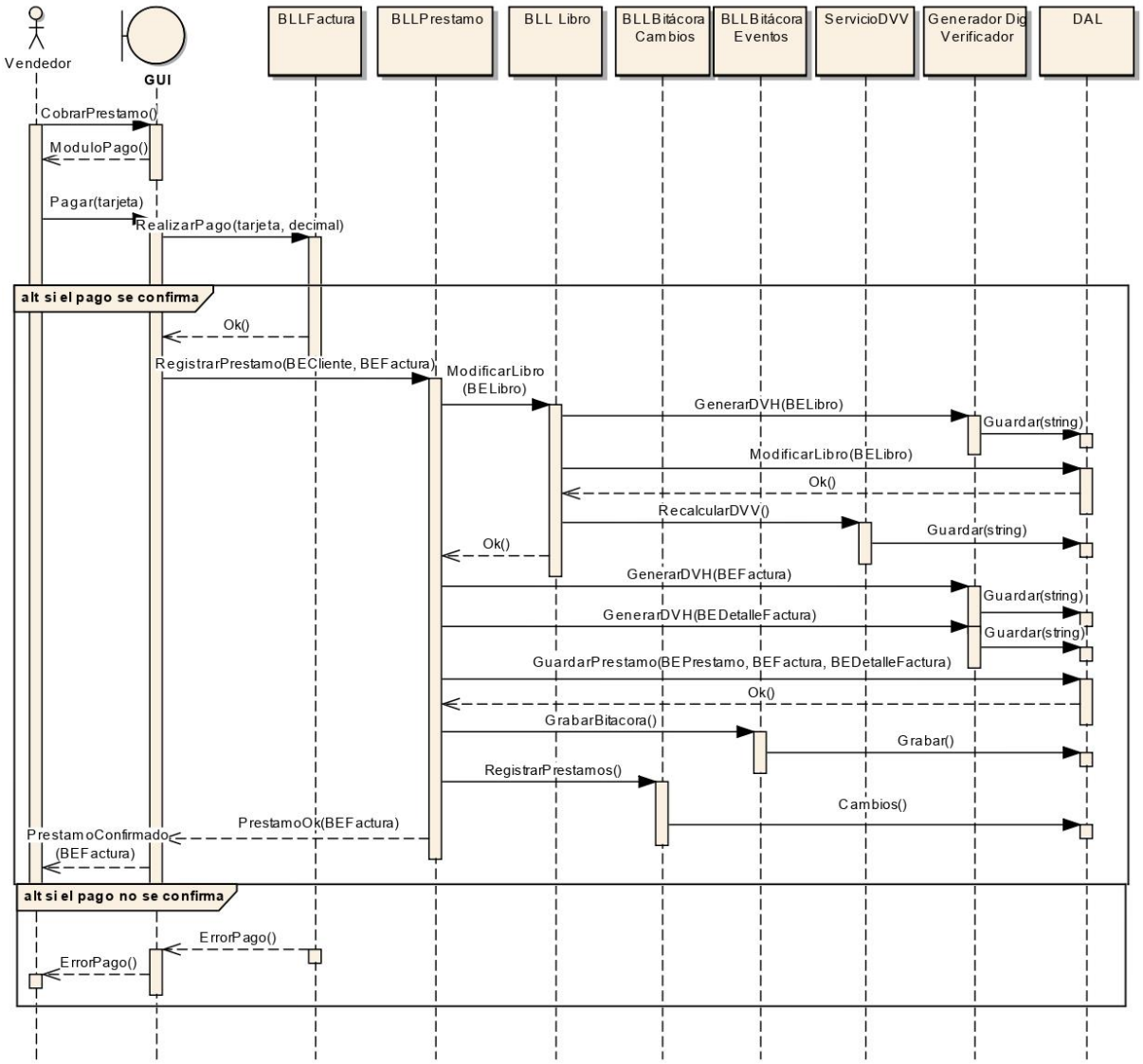
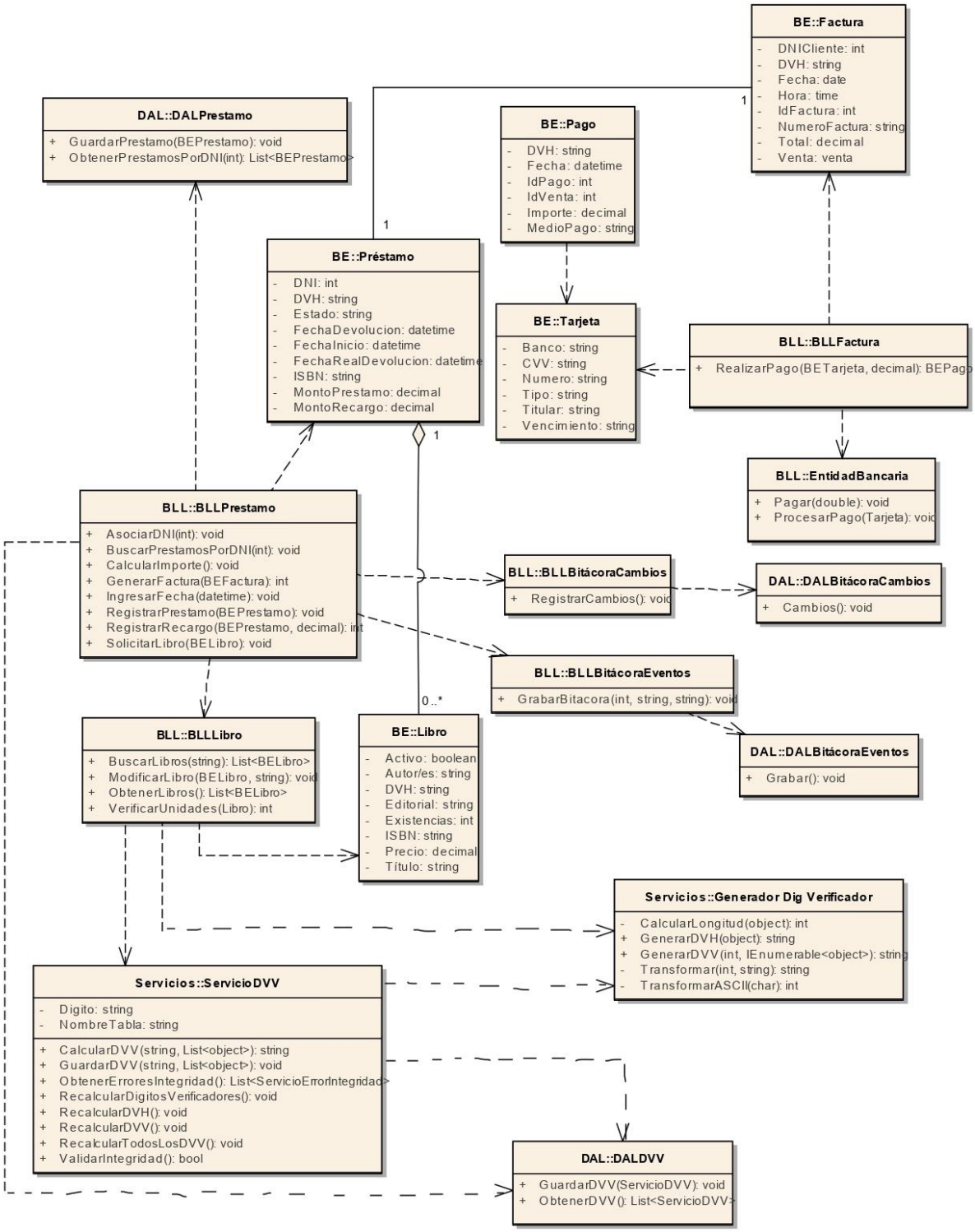
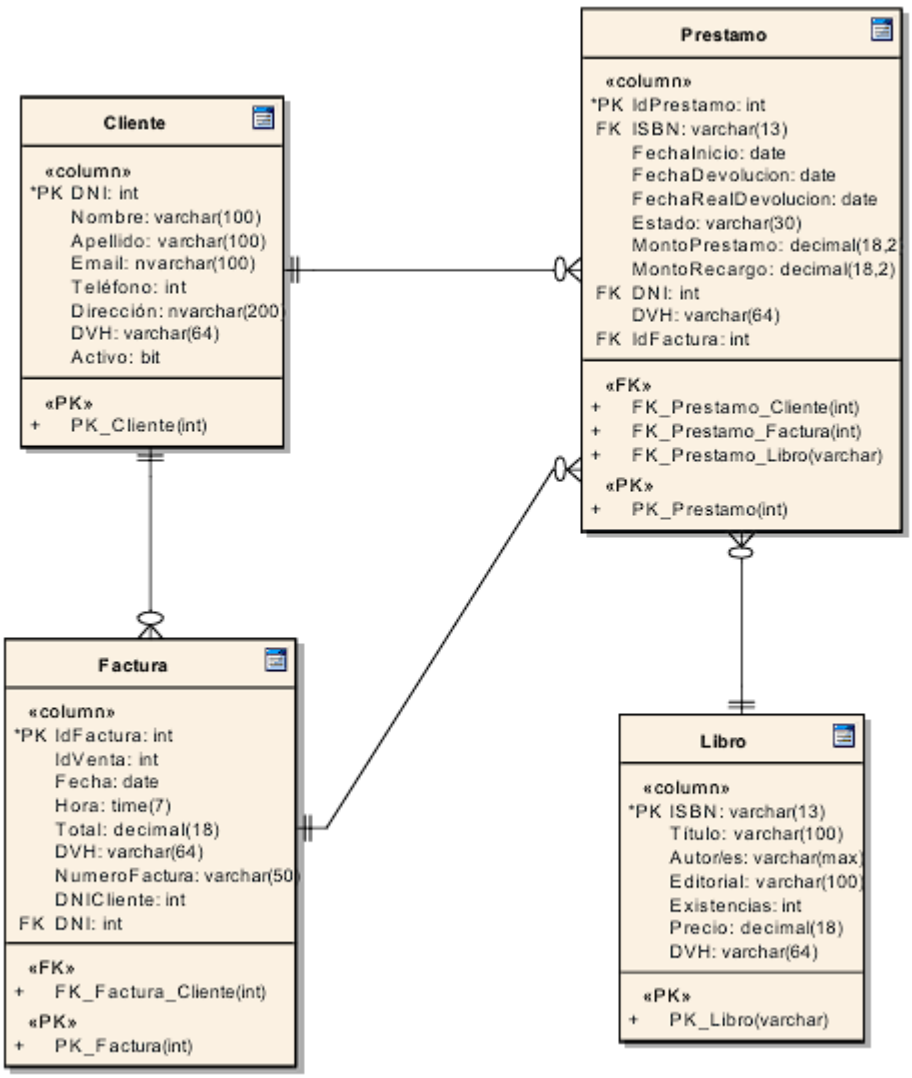


Diagrama de clases

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026



DER


Prestamo
«column»
*PK IdPrestamo: int
FK ISBN: varchar(13)
FechaInicio: date
FechaDevolucion: date
FechaRealDevolucion: date
Estado: varchar(30)
MontoPrestamo: decimal(18,2
MontoRecargo: decimal(18,2)
FK DNI: int
DVH: varchar(64)
FK IdFactura: int
«FK»
+ FK_Prestamo_Cliente(int)
+ FK_Prestamo_Factura(int)
+ FK_Prestamo_Libro(varchar)
«PK»
+ PK_Prestamo(int)
Cliente
«column»
*PK DNI: int
Nombre: varchar(100)
Apellido: varchar(100)
Email: nvarchar(100)
Teléfono: int
Dirección: nvarchar(200
DVH: varchar(64)
Activo: bit
«PK»
+ PK_Cliente(int)
Factura
«column»
*PK IdFactura: int
IdVenta: int
Fecha: date
Hora: time(7)
Total: decimal(18)
DVH: varchar(64)
NumeroFactura: varchar(50)
DNICliente: int
FK DNI: int
«FK»
+ FK_Factura_Cliente(int)
«PK»
+ PK_Factura(int)
Libro
«column»
*PK ISBN: varchar(13)
Titulo: varchar(100)
Autor/es: varchar(max)
Editorial: varchar(100)
Existencias: int
Precio: decimal(18)
DVH: varchar(64)
«PK»
+ PK_Libro(varchar)

GUI



N02.PN2.4 CUN07 “Seleccionar préstamo vigente”

Especificación de Caso de Uso

ID y Nombre: CUN07 “Seleccionar préstamo vigente”.
Descripción: Permite al vendedor buscar y seleccionar en el sistema el préstamo activo de un cliente ingresando su número de DNI, recuperando los datos del libro adeudado, la fecha de inicio y la fecha pactada de devolución para continuar con el proceso de devolución.
Actores: Vendedor.
Precondiciones: • El vendedor debe haber iniciado sesión. • El cliente debe encontrarse registrado en la base de datos.
Puntos de extensión: N/A
Flujo principal: 1. El cliente informa su número de DNI para devolver el libro. 2. El vendedor ingresa el número de DNI del cliente para buscar un préstamo asociado al mismo. 3. Se consulta en la base de datos si el cliente existe y/o si posee un registro en la tabla Prestamo con

estado "Activo".

4. Se muestra en pantalla la información del préstamo vigente y del libro asociado.
5. El vendedor selecciona el préstamo visualizado.
6. La información del préstamo activo queda disponible en la interfaz para proceder con el registro de la devolución.

Flujo alternativo:

3.1.1 El DNI ingresado no corresponde a ningún cliente registrado en el sistema.
3.1.2 Se muestra un mensaje de error.
3.2.1 El cliente está registrado, pero no posee préstamos activos pendientes.
3.2.1 Se muestra un mensaje de error.

Postcondiciones:

- El préstamo activo queda seleccionado y cargado en la interfaz.
- La información del préstamo queda a disposición del vendedor para continuar con la verificación de fechas y la registración de la devolución en CUN09.

Diagrama de secuencia

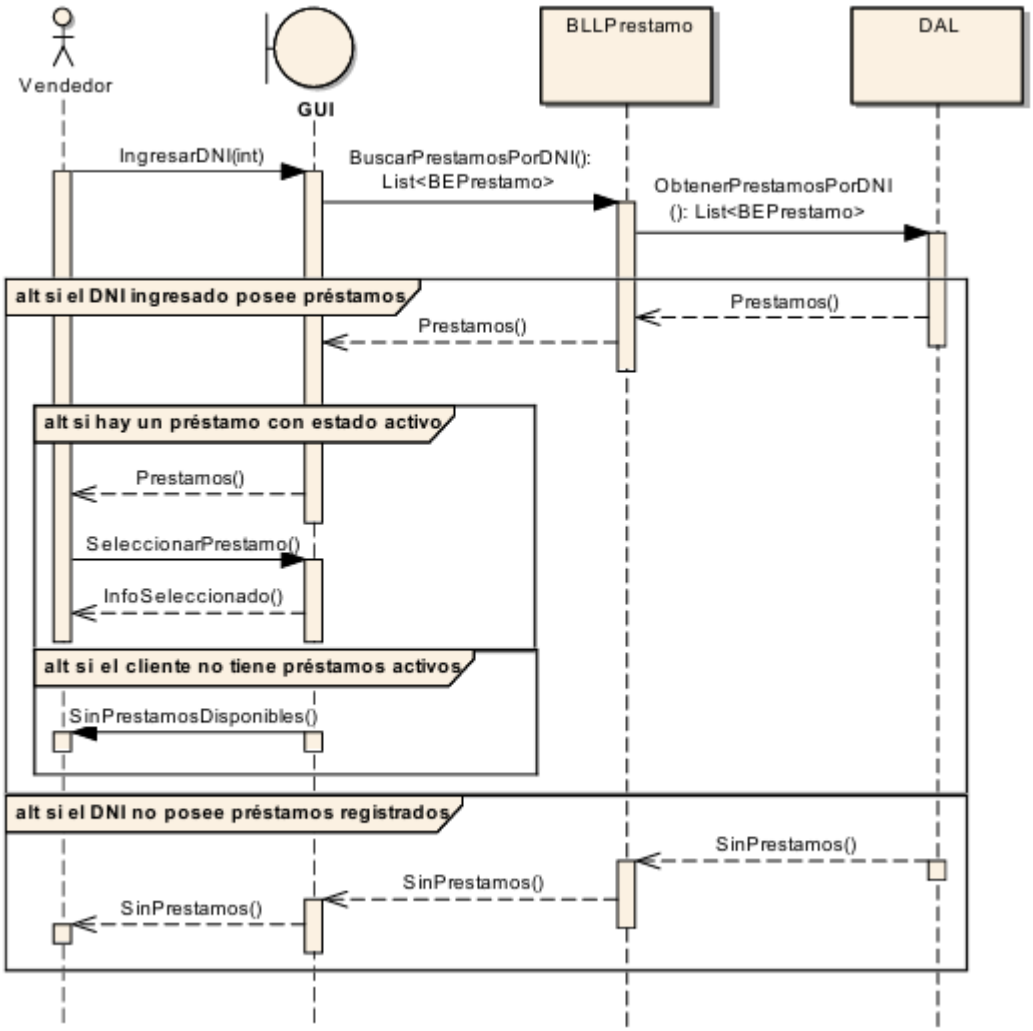
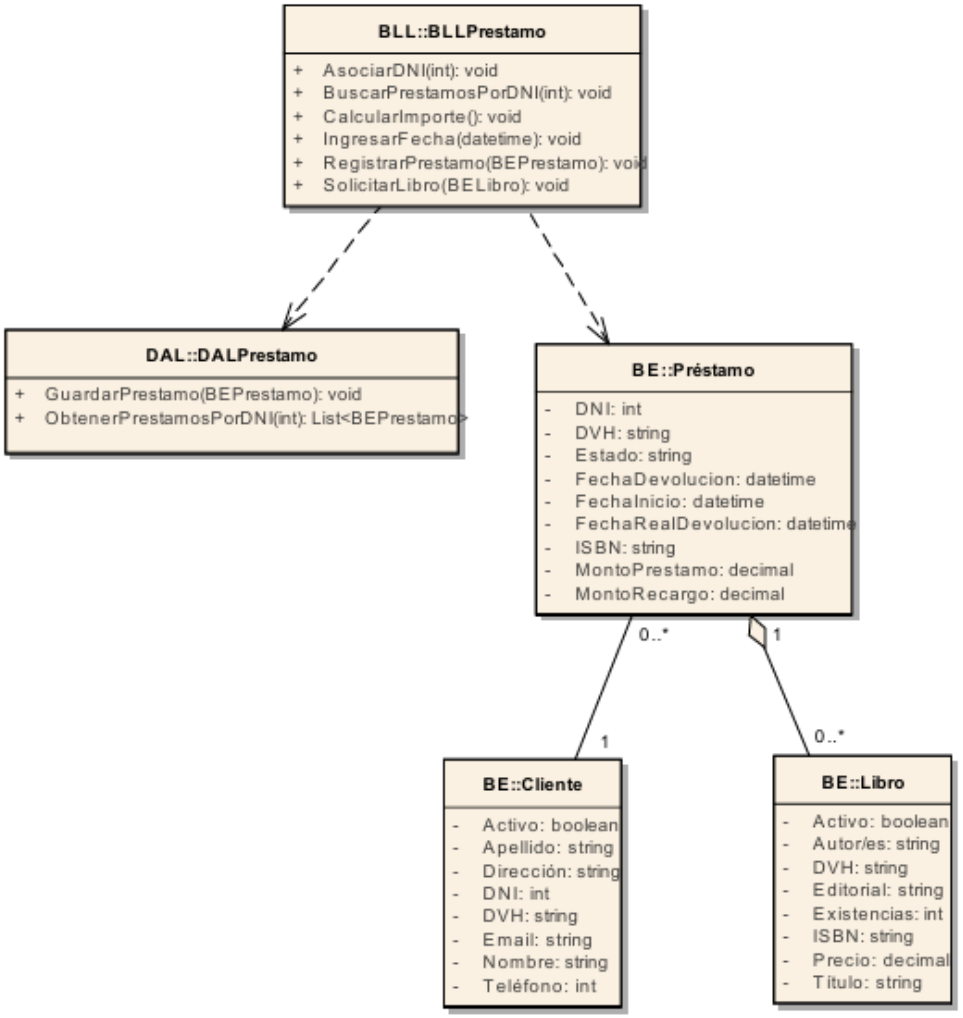
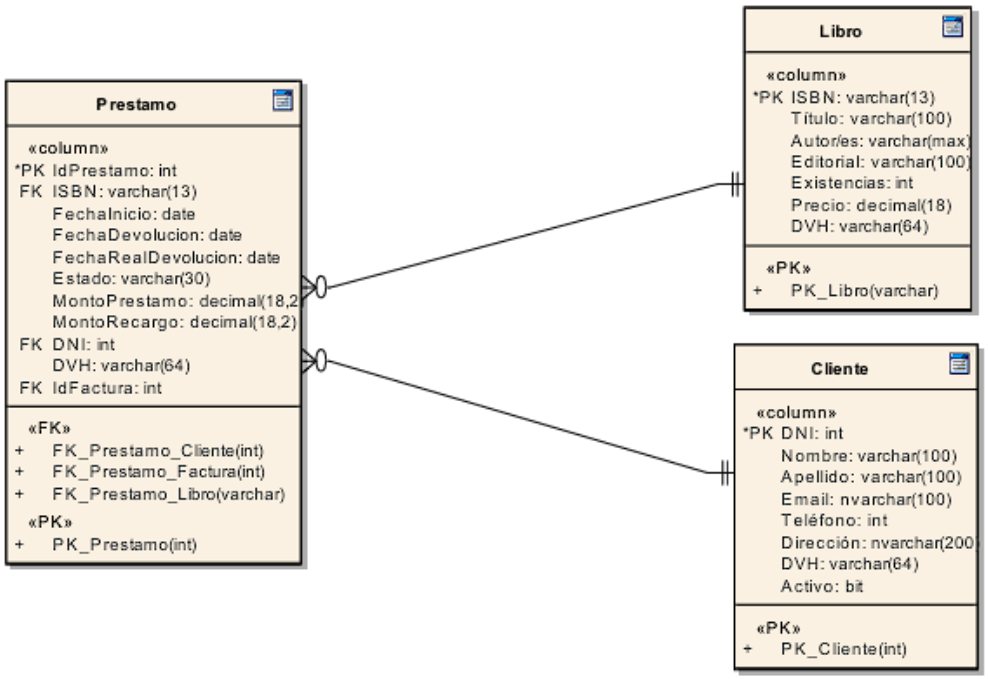


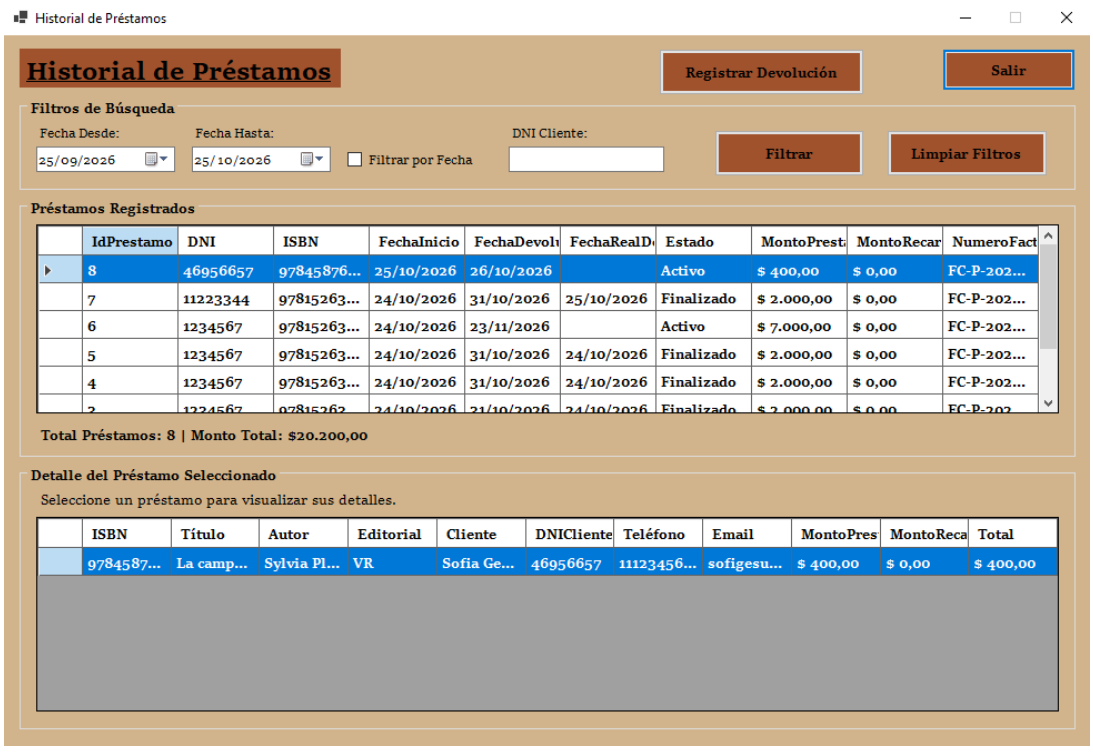
Diagrama de Clases


BLL::BLLPrestamo
+ AsociarDNI(int): void
+ BuscarPrestamosPorDNI(int): void
+ CalcularImporte(): void
+ IngresarFecha(datetime): void
+ RegistrarPrestamo(BEPrestamo): void
+ SolicitarLibro(BELibro): void
DAL::DALPrestamo
+ GuardarPrestamo(BEPrestamo): void
+ ObtenerPrestamosPorDNI(int): List<BEPrestamo>
BE::Préstamo
- DNI: int
- DVH: string
- Estado: string
- FechaDevolucion: datetime
- FechaInicio: datetime
- FechaRealDevolucion: datetime
- ISBN: string
- MontoPrestamo: decimal
- MontoRecargo: decimal
0..*
1
1
0..*
BE::Cliente
- Activo: boolean
- Apellido: string
- Dirección: string
- DNI: int
- DVH: string
- Email: string
- Nombre: string
- Teléfono: int
BE::Libro
- Activo: boolean
- Autor/es: string
- DVH: string
- Editorial: string
- Existencias: int
- ISBN: string
- Precio: decimal
- Título: string

DER


Prestamo
«column»
*PK IdPrestamo: int
FK ISBN: varchar(13)
FechaInicio: date
FechaDevolucion: date
FechaRealDevolucion: date
Estado: varchar(30)
MontoPrestamo: decimal(18,2
MontoRecargo: decimal(18,2)
FK DNI: int
DVH: varchar(64)
FK IdFactura: int
«FK»
+ FK_Prestamo_Cliente(int)
+ FK_Prestamo_Factura(int)
+ FK_Prestamo_Libro(varchar)
«PK»
+ PK_Prestamo(int)
Libro
«column»
*PK ISBN: varchar(13)
Título: varchar(100)
Autor/es: varchar(max)
Editorial: varchar(100)
Existencias: int
Precio: decimal(18)
DVH: varchar(64)
«PK»
+ PK_Libro(varchar)
Cliente
«column»
*PK DNI: int
Nombre: varchar(100)
Apellido: varchar(100)
Email: nvarchar(100)
Teléfono: int
Dirección: nvarchar(200
DVH: varchar(64)
Activo: bit
«PK»
+ PK_Cliente(int)

GUI


Historial de Préstamos
Historial de Préstamos
Registrar Devolución
Salir
Filtros de Búsqueda
Fecha Desde:
25/09/2026
Fecha Hasta:
25/10/2026
Filtrar por Fecha
DNI Cliente:
Filtrar
Limpiar Filtros
Préstamos Registrados
IdPrestamo DNI ISBN FechaInicio FechaDevolu FechaRealD Estado MontoPrest MontoRecar NumeroFact
8 46956657 97845876... 25/10/2026 26/10/2026 Activo $ 400,00 $ 0,00 FC-P-202...
7 11223344 97815263... 24/10/2026 31/10/2026 25/10/2026 Finalizado $ 2.000,00 $ 0,00 FC-P-202...
6 1234567 97815263... 24/10/2026 23/11/2026 Activo $ 7.000,00 $ 0,00 FC-P-202...
5 1234567 97815263... 24/10/2026 31/10/2026 24/10/2026 Finalizado $ 2.000,00 $ 0,00 FC-P-202...
4 1234567 97815263... 24/10/2026 31/10/2026 24/10/2026 Finalizado $ 2.000,00 $ 0,00 FC-P-202...
Total Préstamos: 8 | Monto Total: $20.200,00
Detalle del Préstamo Seleccionado
Seleccione un préstamo para visualizar sus detalles.
ISBN Título Autor Editorial Cliente DNICliente Teléfono Email MontoPres MontoReca Total
9784587... La camp... Sylvia Pl... VR Sofia Ge... 46956657 11123456... sofigesu... $ 400,00 $ 0,00 $ 400,00

N02.PN2.5 CUN08 “Cobrar recargo”

Especificación de Caso de Uso

ID y Nombre: CUN08 “Cobrar recargo”.

Descripción: Permite al vendedor procesar el cobro de la penalización por mora cuando un ejemplar es devuelto luego de la fecha límite pactada, validando la transacción mediante la entidad bancaria, emitiendo una factura y registrando la operación en la bitácora.

Actores: Vendedor

Precondiciones:

- El vendedor debe haber iniciado sesión.
- Debe existir una devolución en curso donde se detectó que la fecha actual es mayor a su fecha pactada de devolución.

Puntos de extensión: N/A

Flujo principal:

1. Se ingresa a la interfaz de cobro exhibiendo los días de retraso transcurridos y el monto total a pagar en concepto de recargo por mora.
2. El vendedor comunica al cliente el importe adeudado por la penalización.
3. El cliente indica el tipo de tarjeta (Débito o Crédito) y entrega el medio de pago.
4. El vendedor registra el medio de pago seleccionado.
5. El vendedor procesa el pago correspondiente con la tarjeta del cliente.
6. La Entidad Bancaria verifica el resultado de la operación.
7. Se emite automáticamente la Factura correspondiente al recargo por mora.
8. Se asocia el importe cobrado al préstamo en curso.
9. Se registra el evento de cobro de penalización en la Bitácora de auditoría.
10. Se retorna el control a CUN09 para finalizar la devolución del libro.

Flujo alternativo:

6.1.1 La Entidad Bancaria rechaza la operación
6.1.2 Se muestra un mensaje de advertencia informando el rechazo bancario.
6.1.3 El préstamo no es cerrado como devuelto.

Postcondiciones:

- El pago del recargo queda acreditado.
- Se emite la factura correspondiente al pago.
- El préstamo queda habilitado para concretar su cierre en CUN09.
- La operación queda auditada en la Bitácora.

Diagrama de secuencia

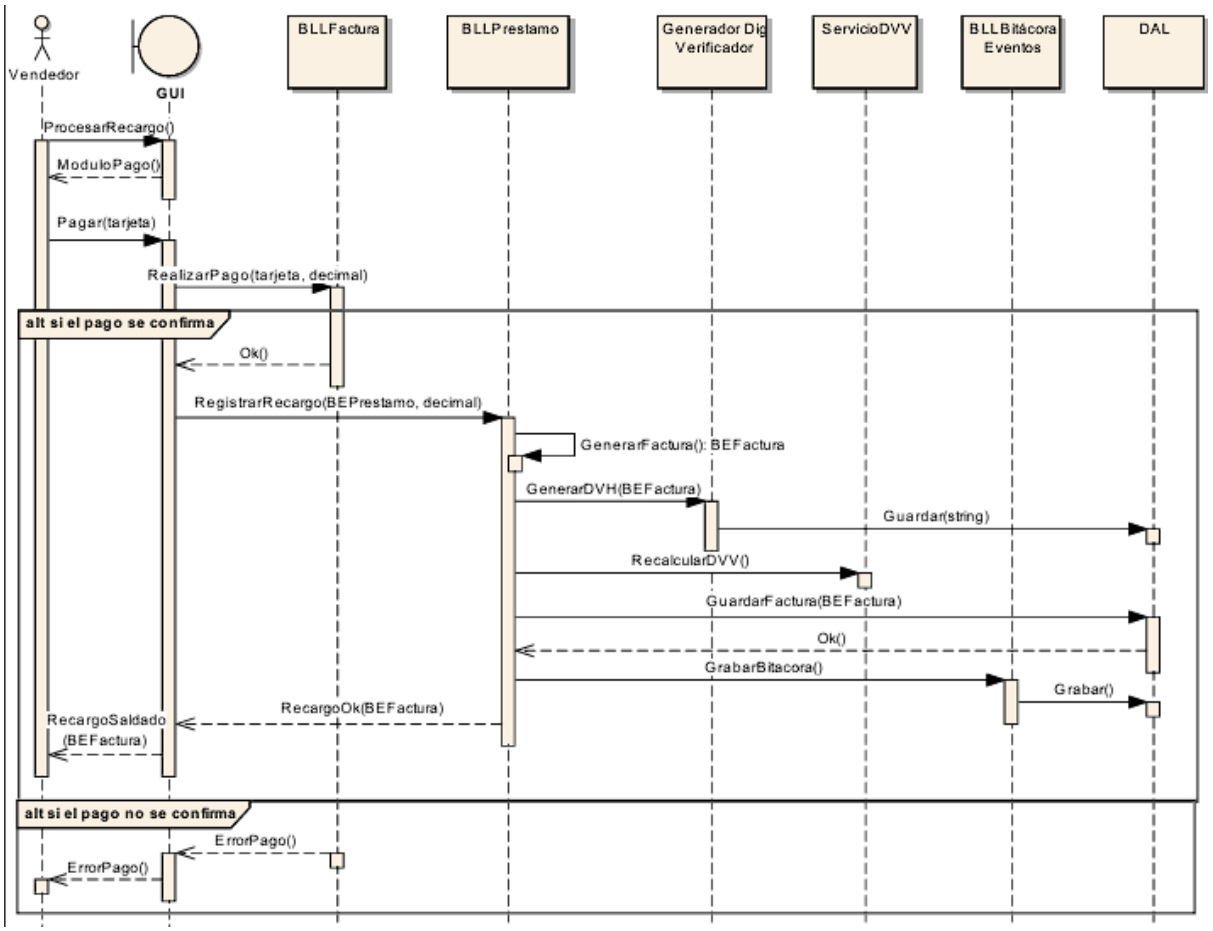
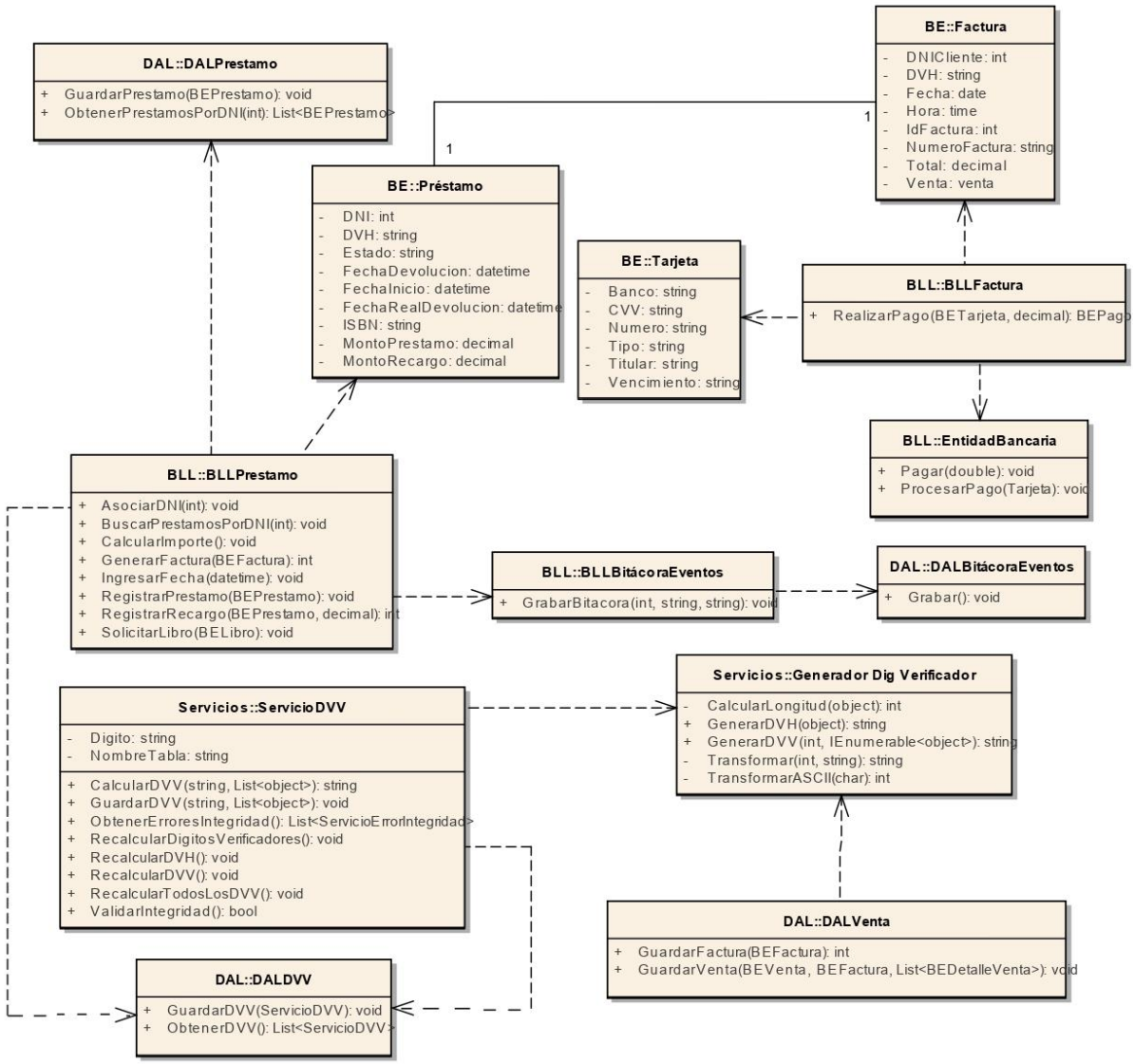
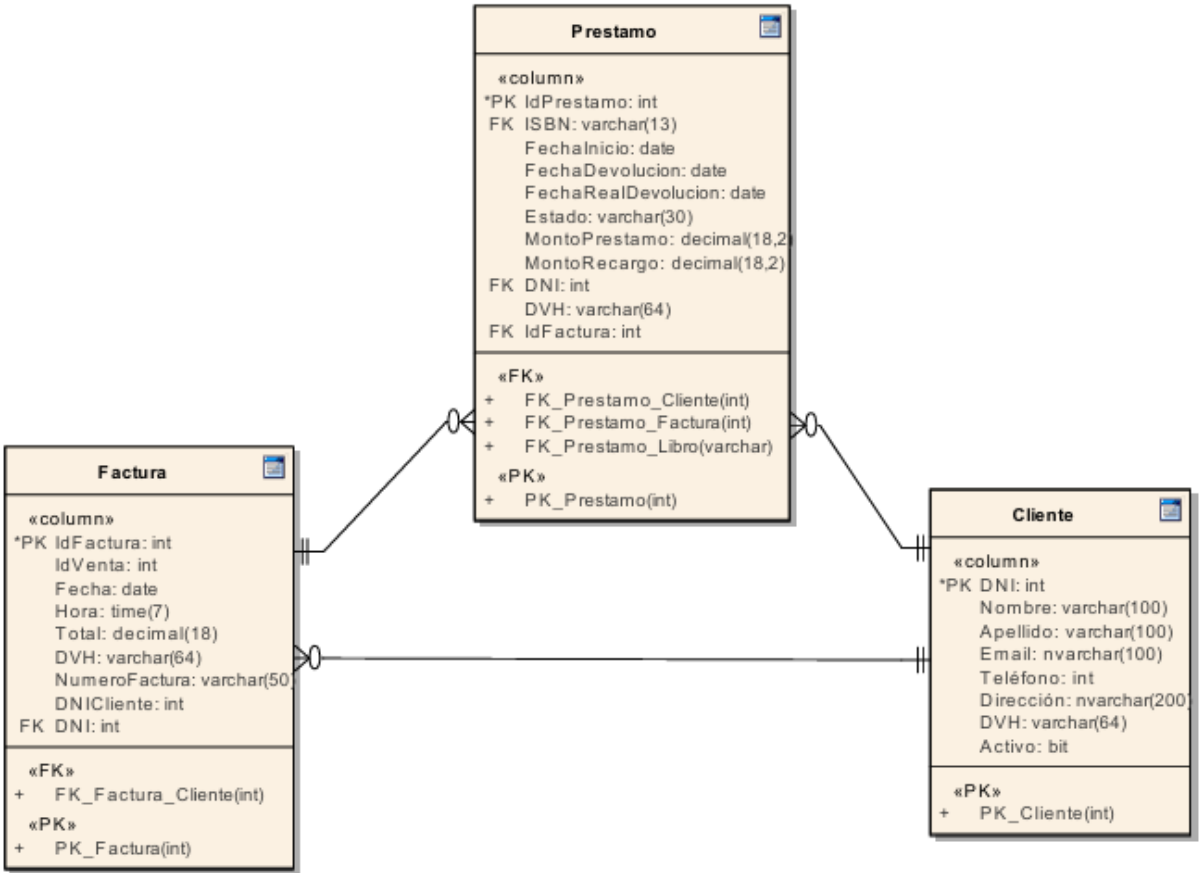


Diagrama de Clases


DAL::DALPrestamo
+ GuardarPrestamo(BEPrestamo): void
+ ObtenerPrestamosPorDNI(int): List<BEPrestamo>
BE::Factura
- DNICliente: int
- DVH: string
- Fecha: date
- Hora: time
- IdFactura: int
- NumeroFactura: string
- Total: decimal
- Venta: venta
1
1
BE::Préstamo
- DNI: int
- DVH: string
- Estado: string
- FechaDevolucion: datetime
- FechaInicio: datetime
- FechaRealDevolucion: datetime
- ISBN: string
- MontoPrestamo: decimal
- MontoRecargo: decimal
BE::Tarjeta
- Banco: string
- CVV: string
- Numero: string
- Tipo: string
- Titular: string
- Vencimiento: string
BLL::BLLFactura
+ RealizarPago(BETarjeta, decimal): BEPago
BLL::EntidadBancaria
+ Pagar(double): void
+ ProcesarPago(Tarjeta): void
BLL::BLLPrestamo
+ AsociarDNI(int): void
+ BuscarPrestamosPorDNI(int): void
+ CalcularImporte(): void
+ GenerarFactura(BEFactura): int
+ IngresarFecha(datetime): void
+ RegistrarPrestamo(BEPrestamo): void
+ RegistrarRecargo(BEPrestamo, decimal): int
+ SolicitarLibro(BELibro): void
BLL::BLLBitácoraEventos
+ GrabarBitacora(int, string, string): void
DAL::DALBitácoraEventos
+ Grabar(): void
Servicios::ServicioDVV
- Digito: string
- NombreTabla: string
+ CalcularDVV(string, List<object>): string
+ GuardarDVV(string, List<object>): void
+ ObtenerErroresIntegridad(): List<ServicioErrorIntegridad>
+ RecalcularDigitosVerificadores(): void
+ RecalcularDVH(): void
+ RecalcularDVV(): void
+ RecalcularTodosLosDVV(): void
+ ValidarIntegridad(): bool
Servicios::Generador Dig Verificador
- CalcularLongitud(object): int
+ GenerarDVH(object): string
+ GenerarDVV(int, IEnumerable<object>): string
- Transformar(int, string): string
- TransformarASCII(char): int
DAL::DALVenta
+ GuardarFactura(BEFactura): int
+ GuardarVenta(BEVenta, BEFactura, List<BEDetalleVenta>): void
DAL::DALDVV
+ GuardarDVV(ServicioDVV): void
+ ObtenerDVV(): List<ServicioDVV>

DER



GUI

N02.PN2.6 CUN09 “Registrar devolución”

Especificación de Caso de Uso

ID y Nombre: CUN09 “Registrar devolución”.
Descripción: Permite al vendedor registrar la devolución física de un libro prestado, verificar si la entrega se realiza dentro del plazo pactado o si tuvo demora, reintegrar el libro al stock disponible y emitir el comprobante de devolución correspondiente.
Actores: Vendedor.
Precondiciones: • El vendedor debe haber iniciado sesión.

- El cliente debe tener un préstamo registrado en estado "Activo".
- El ejemplar entregado debe corresponder al ISBN del préstamo activo.

Puntos de extensión: CUN07 – "Seleccionar préstamo vigente", CUN08 – "Cobrar recargo"

Flujo principal:

1. El cliente entrega el ejemplar físico en el mostrador e indica su DNI.
2. Se incluye el caso de uso CUN07 – Seleccionar préstamo vigente para recuperar los datos del préstamo activo y del libro correspondiente.
3. Se compara la fecha actual con la fecha pactada de devolución.
4. El vendedor verifica el buen estado del libro y confirma la devolución.
5. Se actualiza el registro del préstamo, asignando la fecha real de devolución como el día de la fecha y cambiando su estado a "Finalizado".
6. Se reestablece la disponibilidad del libro para que vuelva a estar óptimo para su préstamo.
7. Se registra el evento en la bitácora de auditoría.
8. Se genera automáticamente el comprobante de devolución.
9. El vendedor entrega una copia del comprobante de devolución al cliente.

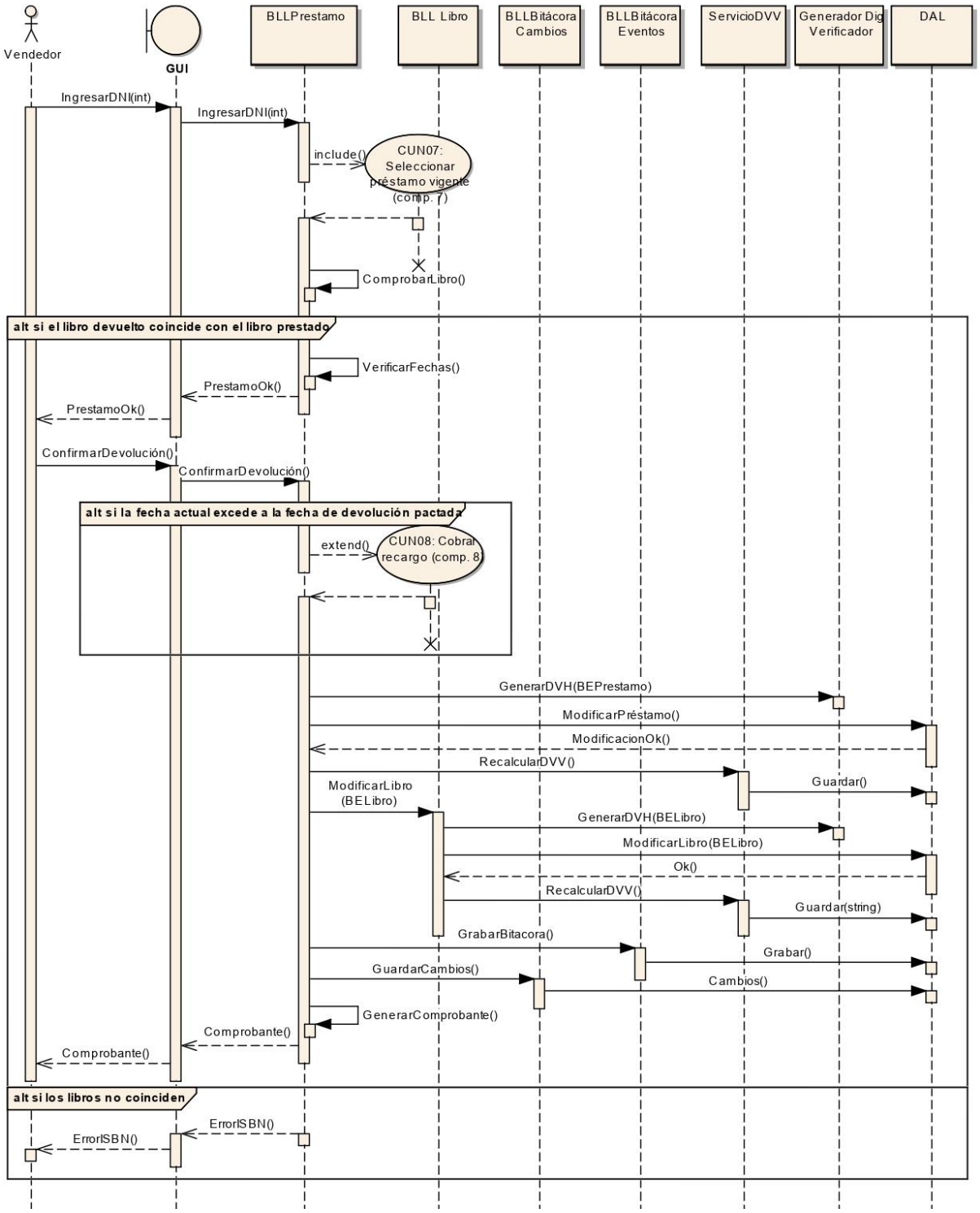
Flujo alternativo:

2.1.1 El ejemplar entregado no coincide con el ISBN del préstamo activo.
2.1.2 Se advierte al vendedor acerca de la disconformidad del libro.
3.1.1 Se detecta que la fecha actual es mayor a la fecha de devolución pactada.
3.1.2 Se extiende al caso de uso CUN08 – Cobrar recargo.
3.1.3 Una vez aprobado y emitido el cobro del recargo, el sistema actualiza el estado del préstamo a "Finalizado con Mora", registra el monto cobrado y se retoma el flujo en el paso 6.

Postcondiciones:

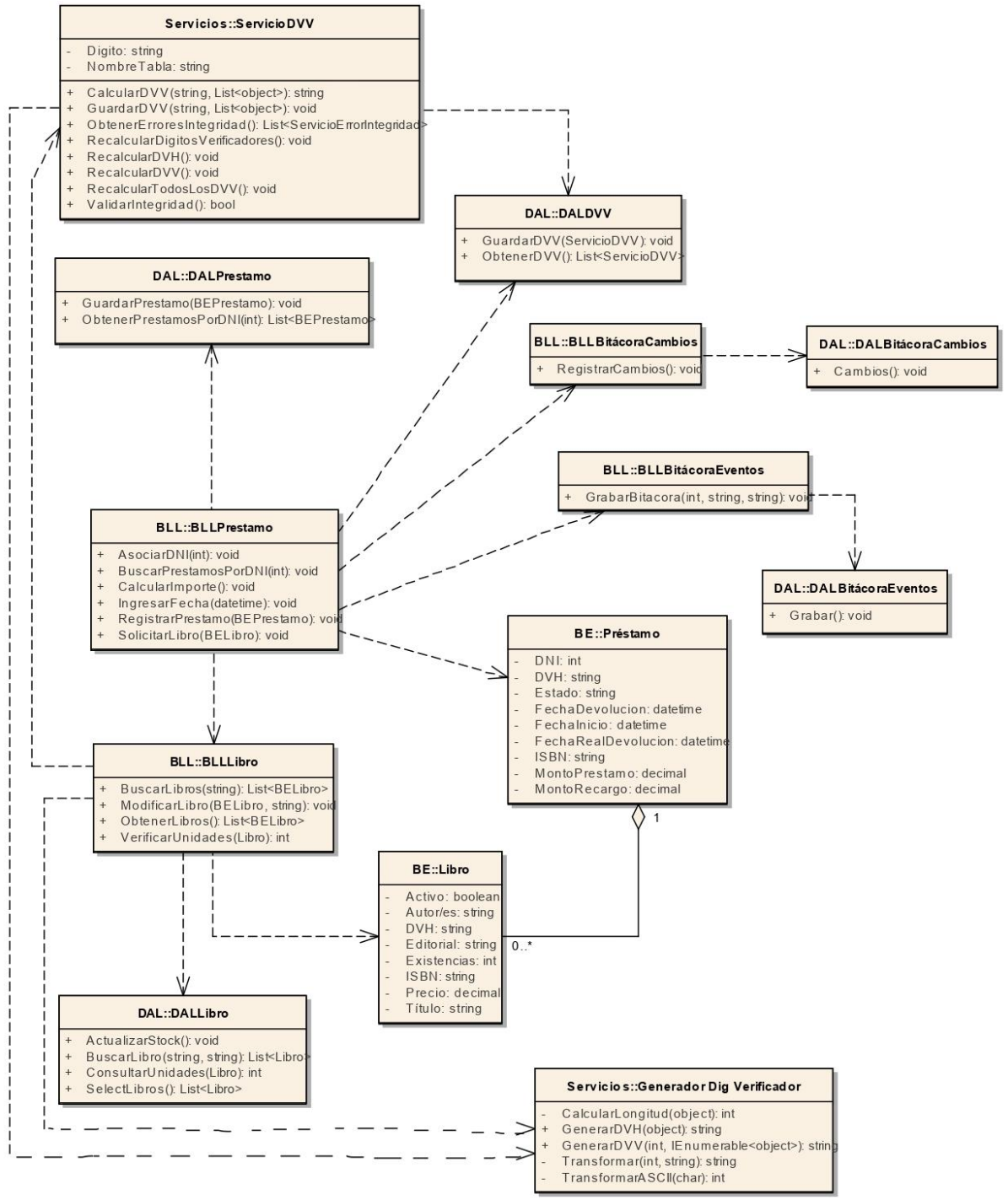
- El préstamo queda finalizado con estado "Finalizado" o "Finalizado con Mora").
- El cliente queda nuevamente habilitado para solicitar futuros préstamos.
- El ejemplar vuelve a estar disponible para la venta o préstamo.
- Se emite el Comprobante de Devolución.

Diagrama de secuencia


Vendedor
GUI
BLLPrestamo
BLL Libro
BLLBitácora Cambios
BLLBitácora Eventos
ServicioDVV
Generador Dig Verificador
DAL
IngresarDNI(int)
IngresarDNI(int)
include()
CUN07: Seleccionar préstamo vigente (comp. 7)
ComprobarLibro()
alt si el libro devuelto coincide con el libro prestado
VerificarFechas()
PrestamoOk()
PrestamoOk()
ConfirmarDevolución()
ConfirmarDevolución()
alt si la fecha actual excede a la fecha de devolución pactada
extend()
CUN08: Cobrar recargo (comp. 8)
GenerarDVH(BEPrestamo)
ModificarPréstamo()
ModificacionOk()
RecalcularDVV()
Guardar()
ModificarLibro (BELibro)
GenerarDVH(BELibro)
ModificarLibro(BELibro)
Ok()
RecalcularDVV()
Guardar(string)
GrabarBitacora()
Grabar()
GuardarCambios()
Cambios()
GenerarComprobante()
Comprobante()
Comprobante()
alt si los libros no coinciden
ErrorISBN()
ErrorISBN()

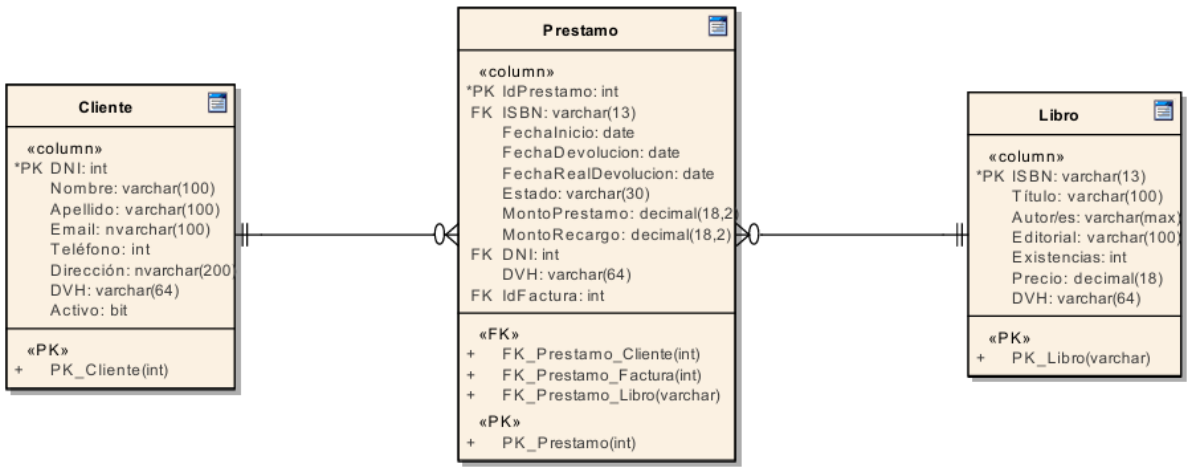
UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Diagrama de Clases

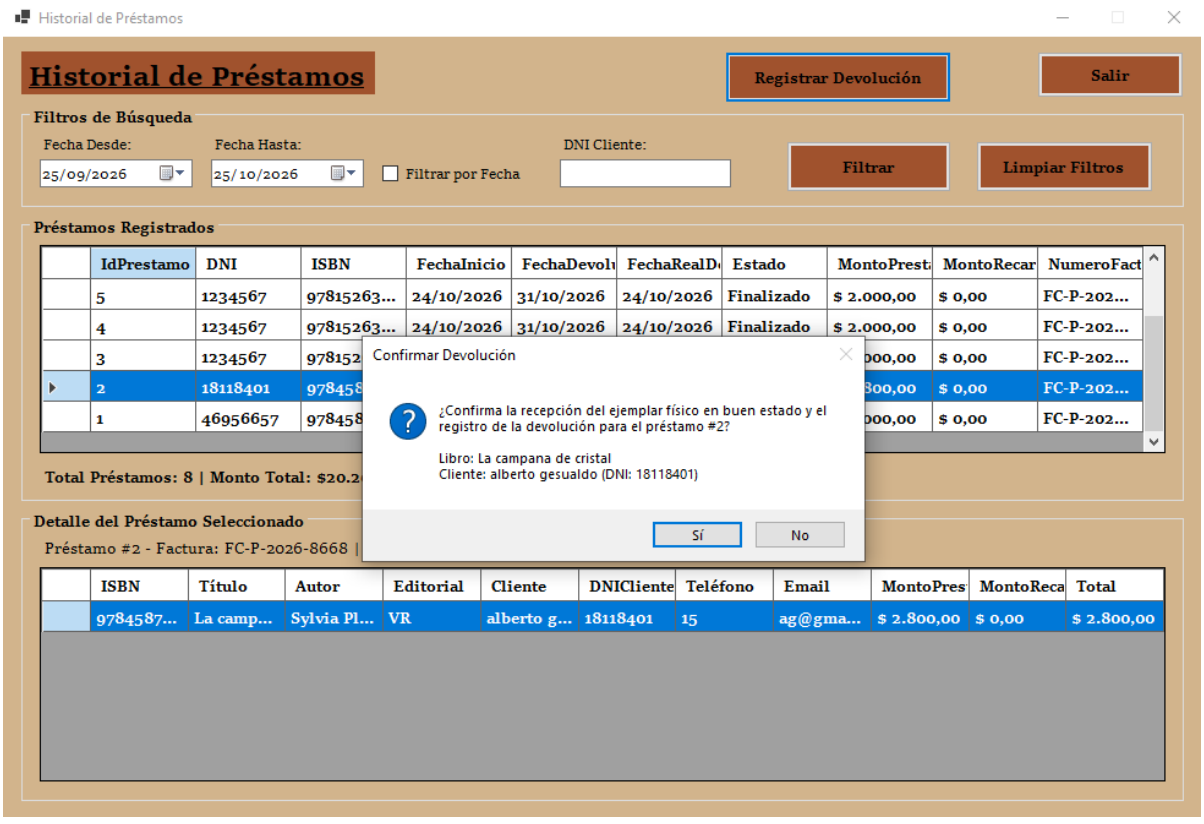


UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

DER

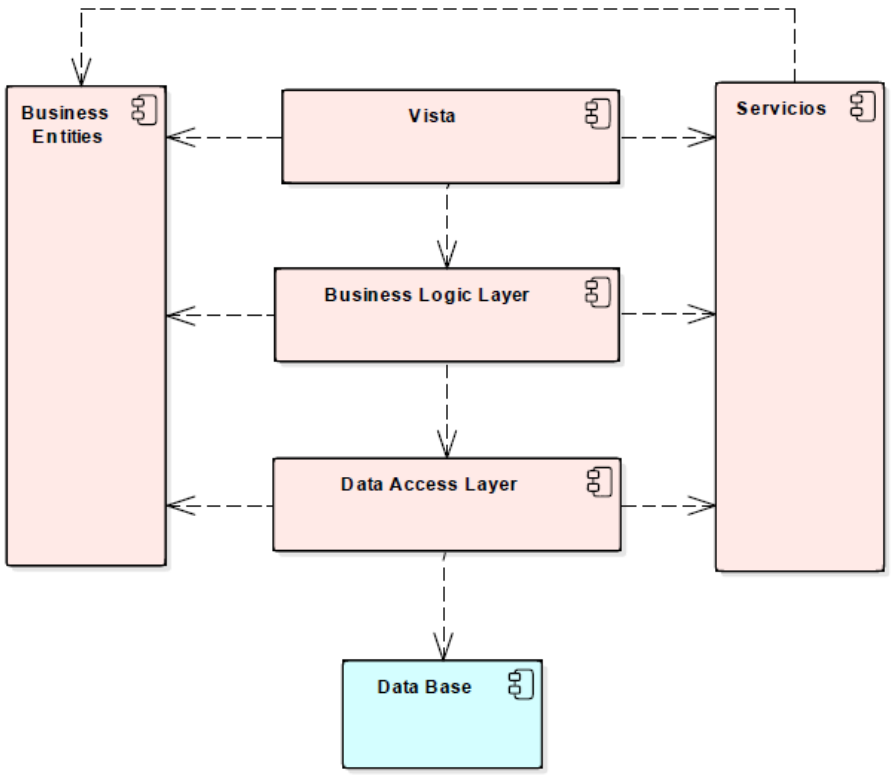


GUI



T01. Arquitectura

T01.1. Diseño de la Arquitectura



Capa de Vista:
Corresponde a la interfaz gráfica del sistema, mediante la cual los usuarios interactúan con la aplicación. Su objetivo principal es mostrar la información de forma comprensible y registrar las acciones realizadas por el usuario para enviarlas a las demás capas.

Capa de Dominio (Business Entities):
Esta capa agrupa los objetos y estructuras de datos que representan las entidades principales del negocio (en nuestro caso, ventas, productos, empleados, entre otros). Se encarga de definir sus atributos, comportamientos y las relaciones existentes entre ellas.

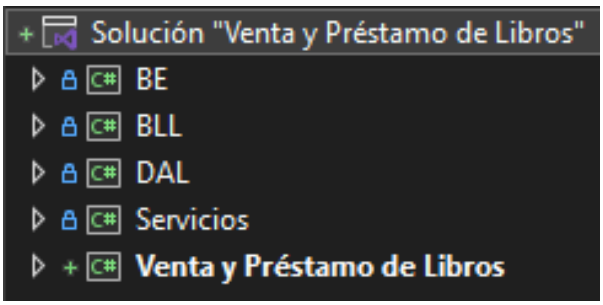
Capa de Lógica de Negocio (Business Logic Layer):
Es la base funcional del sistema, ya que define las reglas y procesos del negocio correspondientes a las necesidades del cliente. Recibe los datos provenientes de la capa de

vista, realiza las validaciones necesarias y ejecuta las decisiones correspondientes según dichas reglas, interactuando con la capa de datos para obtener información almacenada en la base de datos..

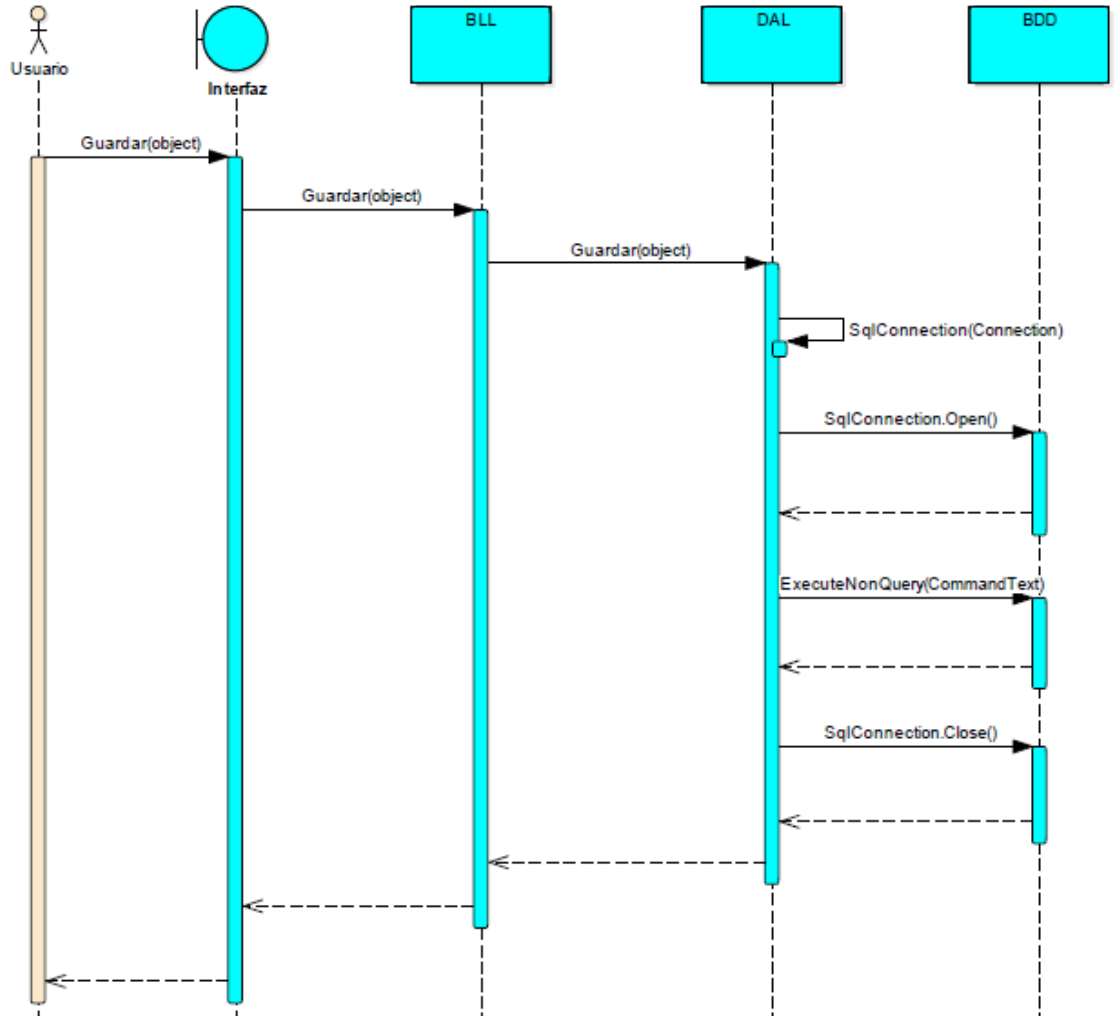
Capa de Acceso a Datos (Data Access Layer):
Tiene la responsabilidad de comunicarse directamente con la base de datos (construida en SQL Server). Gestiona las operaciones de consulta, inserción, modificación y eliminación de datos, permitiendo que el resto del sistema no dependa directamente de las consultas SQL y pueda acceder a los datos mediante consultas de lenguaje natural.

Capa de Servicios:
Esta capa agrupa las entidades y patrones/estructuras utilizadas exclusivamente para el funcionamiento y soporte del sistema, sin interferir con las reglas del negocio. Brinda acceso a funcionalidades del sistema mediante interfaces que pueden ser utilizadas por distintos módulos o por aplicaciones externas. La capa incluye aspectos técnicos como seguridad, envío de notificaciones e integración con otros sistemas, entre otros.

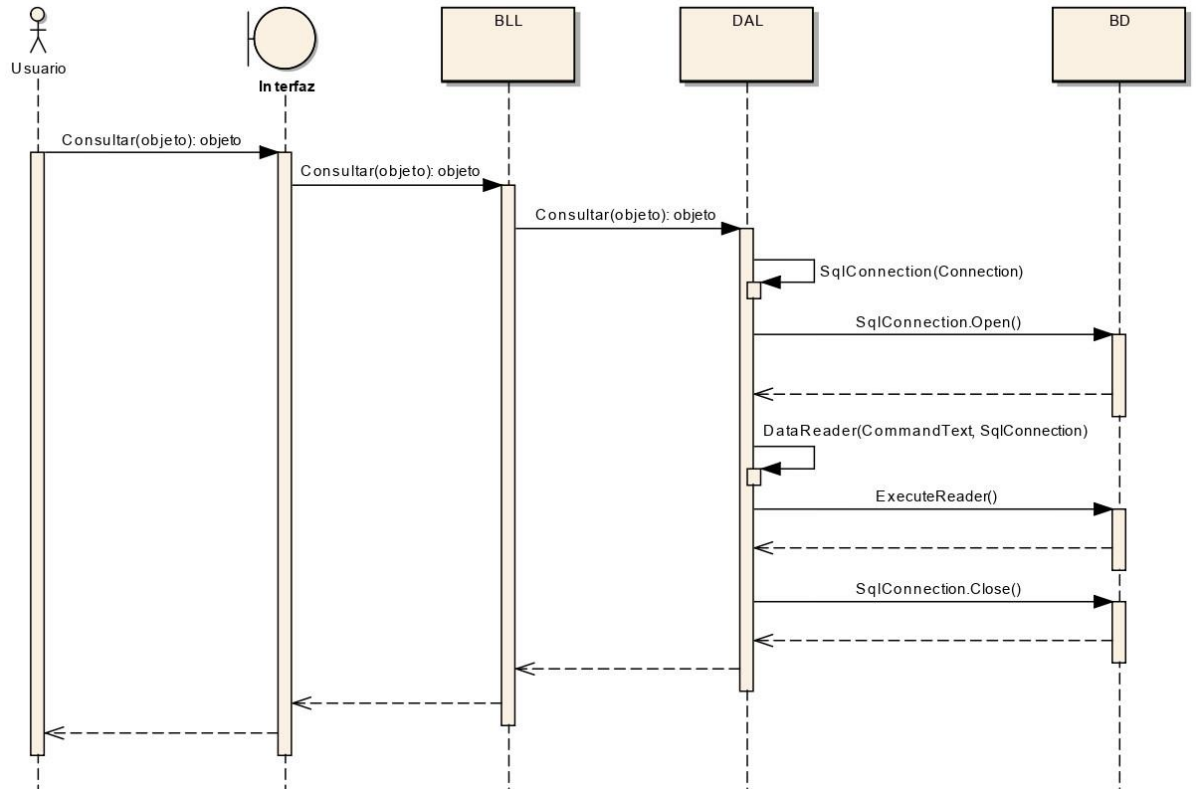
T01.2. Configuración de la Arquitectura en Visual Studio.



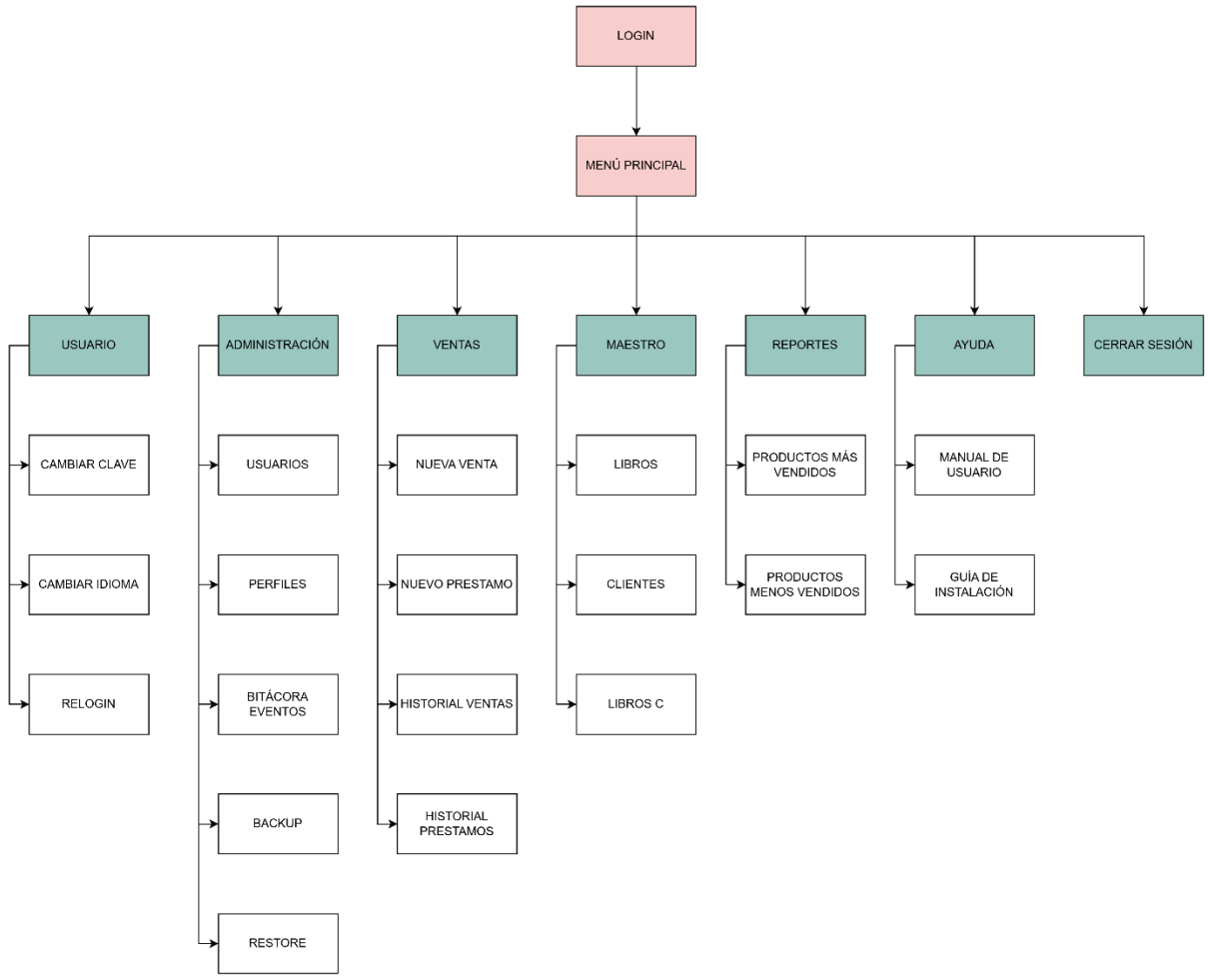
T01.3. Diagrama de Secuencia de Persistencia de Datos.


Usuario
Interfaz
BLL
DAL
BDD
Guardar(object)
Guardar(object)
Guardar(object)
SqlConnection(Connection)
SqlConnection.Open()
ExecuteNonQuery(CommandText)
SqlConnection.Close()

T01.4. Diagrama de Secuencia de Consulta de Datos.



T01.5. Esquema Jerárquico.



T01.6. GUI (Menú Principal)



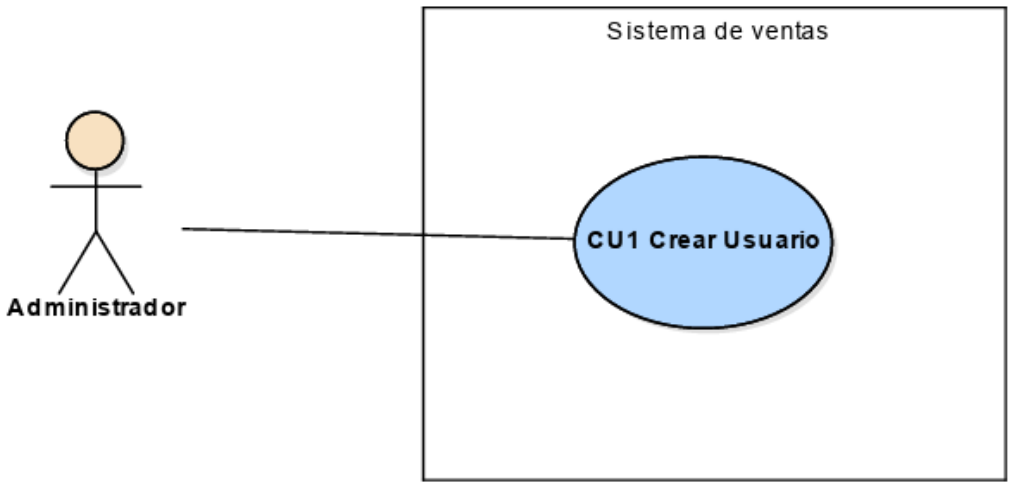
T02. Usuarios

T02.1. Diseño Crear Usuario

1.1. Descomposición Funcional

1. El administrador ingresa al módulo de usuarios.
2. El sistema muestra el formulario de creación de usuario.
3. El administrador ingresa los datos del nuevo usuario y presiona el botón confirmar.
4. El sistema valida los datos ingresados.
5. El sistema verifica que no exista un usuario registrado con el mismo DNI o email.
6. El sistema encripta la contraseña del usuario.
7. El sistema guarda el nuevo usuario en la base de datos.
8. El sistema recalcula los dígitos verificadores horizontales y verticales.
9. El sistema registra la operación en la bitácora.
10. El sistema informa que el usuario fue creado correctamente.

1.2. Diagrama de Caso de Uso



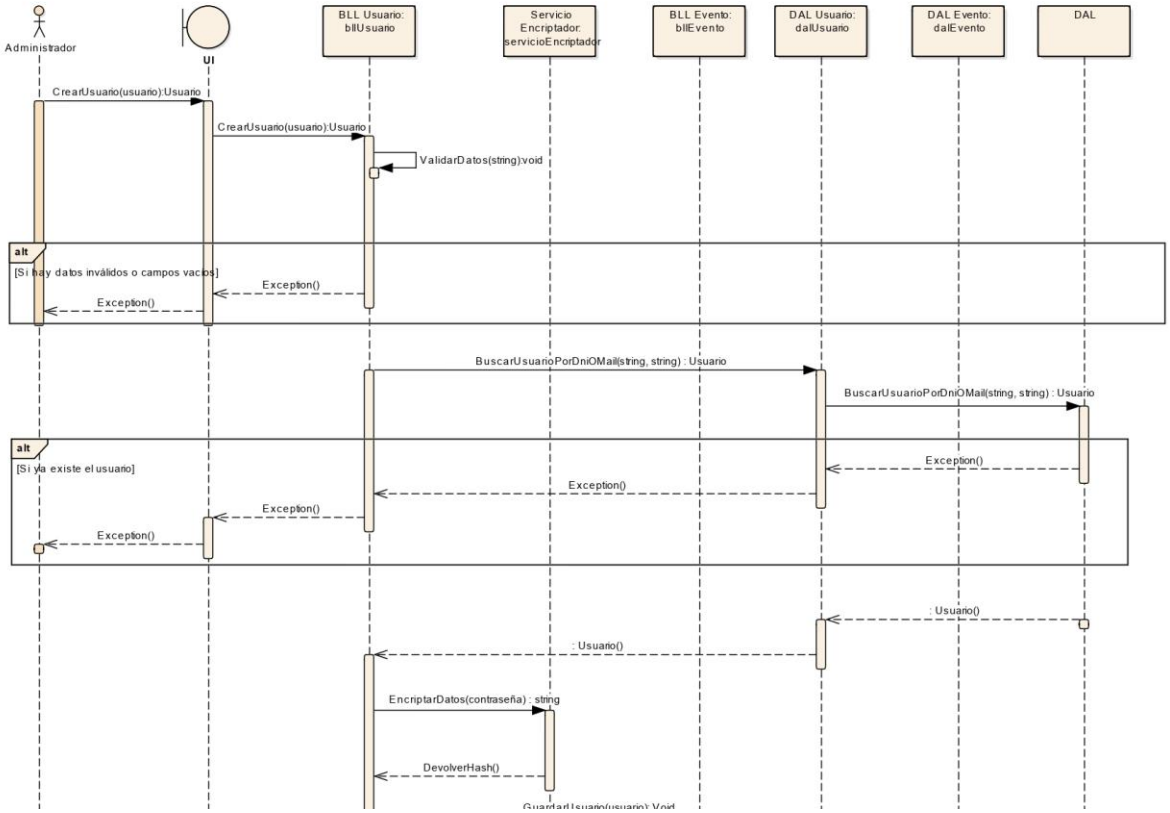
1.3. Especificación de Caso de Uso

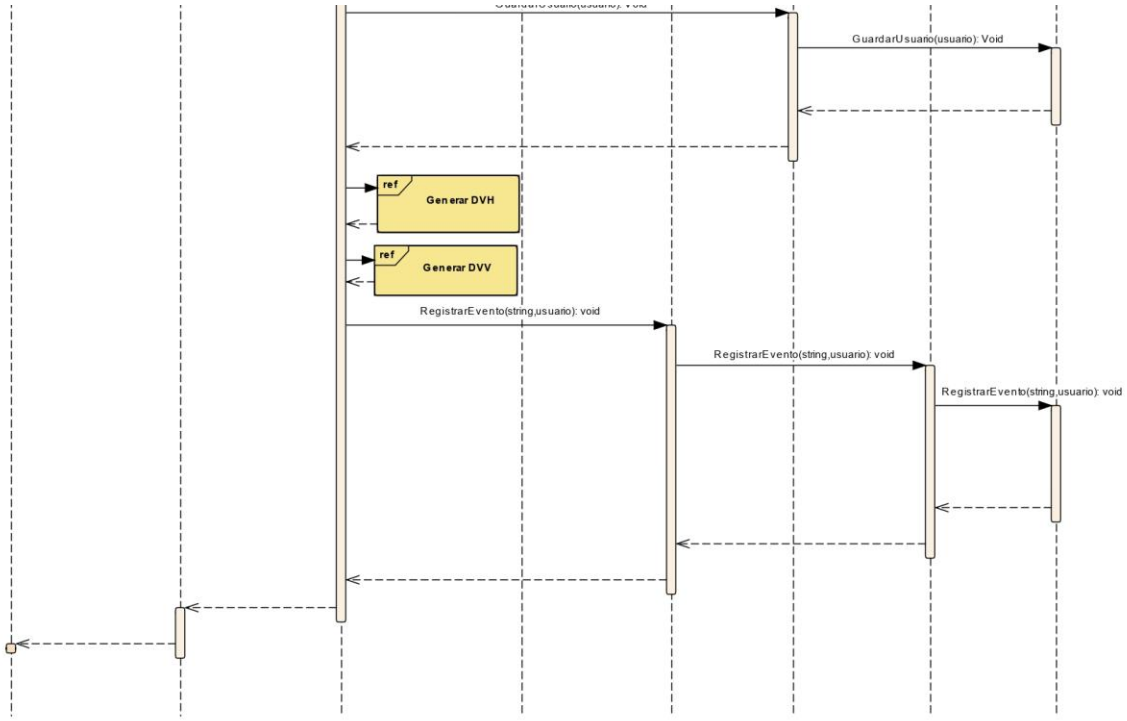
CU1 - CREAR USUARIO

Descripción:	El sistema permite al administrador registrar nuevos usuarios en la aplicación, asignándoles un rol y almacenando sus datos de forma segura.
Actores:	Administrador
Precondiciones:	• El administrador debe haber iniciado sesión en el sistema. • El administrador debe poseer permisos para gestionar usuarios. • El sistema debe encontrarse operativo y con conexión activa a la base de datos SQL Server.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador ingresa al módulo de usuarios.
2	El sistema muestra el formulario de creación de usuario.
3	El administrador ingresa DNI, nombre, apellido, email, contraseña y rol del nuevo usuario y presiona el botón confirmar.
4	El sistema valida que los datos ingresados no se encuentren vacíos.
5	El sistema valida el formato de los datos ingresados.
6	El sistema verifica que no exista un usuario registrado con el mismo dni o mail.
7	El sistema encripta la contraseña mediante funciones criptográficas de seguridad.
8	El sistema guarda el nuevo usuario en la base de datos.
9	El sistema hace el cálculo de los dígitos verificadores horizontales y verticales.
10	El sistema registra la operación en la bitácora del sistema.
11	El sistema informa que el usuario fue creado correctamente.
Flujo alternativo:	
4.1.1	El administrador dejó alguno de los campos vacíos.
4.1.2	El sistema informa que debe completar todos los campos para continuar.
5.1.1	Alguno de los datos ingresados posee un formato inválido.
5.1.2	El sistema informa que los datos ingresados no son válidos.
6.1.1	Ya existe un usuario registrado con el mismo DNI o email.
6.1.2	El sistema informa que el usuario ya se encuentra registrado.

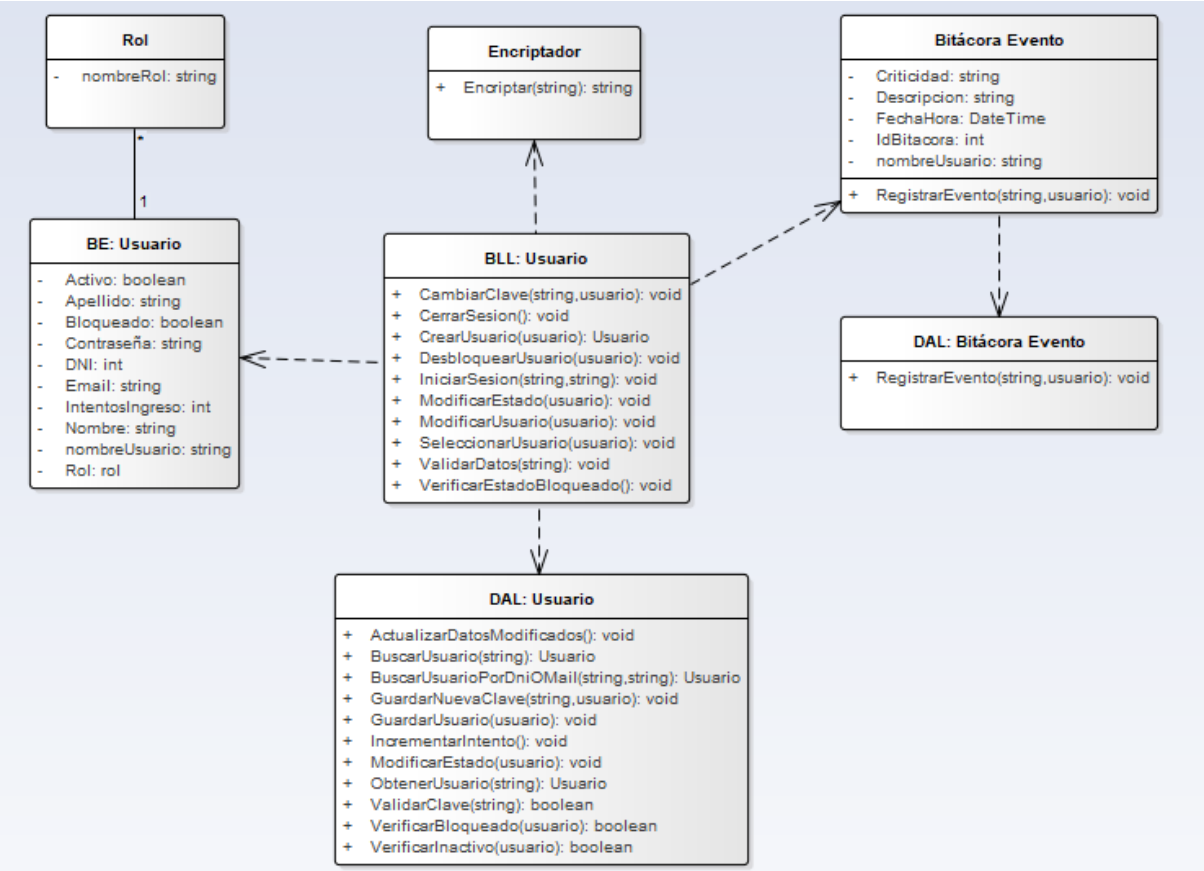
Poscondición:	El nuevo usuario queda registrado correctamente en el sistema con su contraseña encriptada, rol asignado y registro correspondiente en la bitácora.

1.4. Diagrama de Secuencia

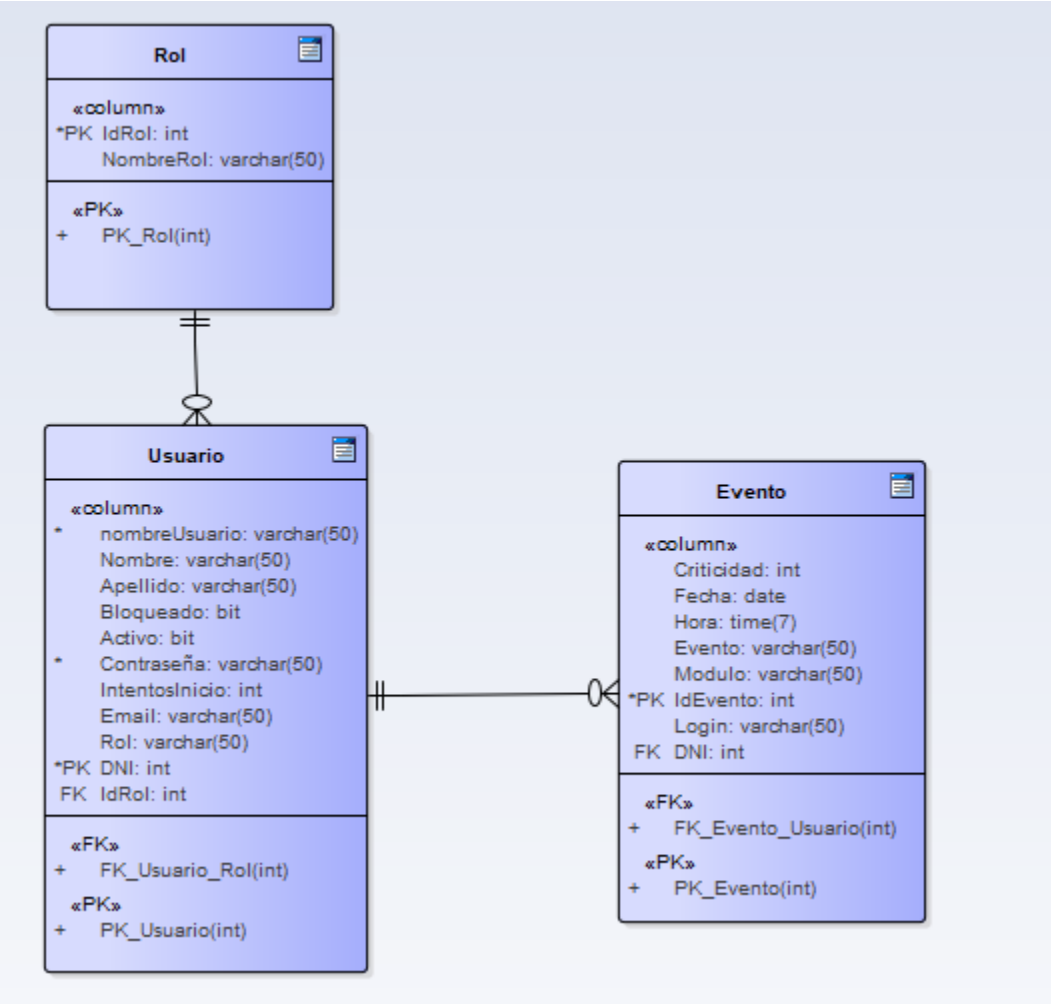

Administrador
UI
BLL Usuario: bllUsuario
Servicio Encriptador: servicioEncriptador
BLL Evento: bllEvento
DAL Usuario: dalUsuario
DAL Evento: dalEvento
DAL
CrearUsuario(usuario):Usuario
CrearUsuario(usuario):Usuario
ValidarDatos(string):void
alt
[Si hay datos inválidos o campos vacios]
Exception()
Exception()
BuscarUsuarioPorDniOMail(string, string) : Usuario
BuscarUsuarioPorDniOMail(string, string) : Usuario
alt
[Si ya existe el usuario]
Exception()
Exception()
Exception()
Exception()
: Usuario()
: Usuario()
EncriptarDatos(contraseña) : string
DevolverHash()



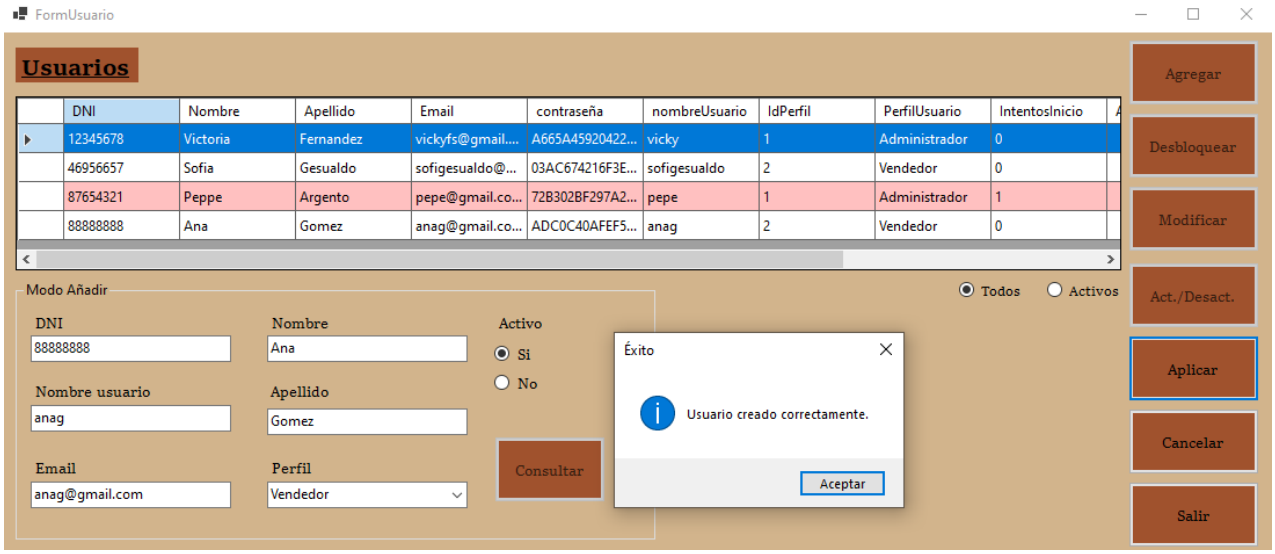
1.5. Diagrama de Clase



1.6. Diagrama Entidad Relación


Rol
«column»
*PK IdRol: int
NombreRol: varchar(50)
«PK»
+ PK_Rol(int)
Usuario
«column»
* nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
* Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
Rol: varchar(50)
*PK DNI: int
FK IdRol: int
«FK»
+ FK_Usuario_Rol(int)
«PK»
+ PK_Usuario(int)
Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
FK DNI: int
«FK»
+ FK_Evento_Usuario(int)
«PK»
+ PK_Evento(int)

1.7. GUI



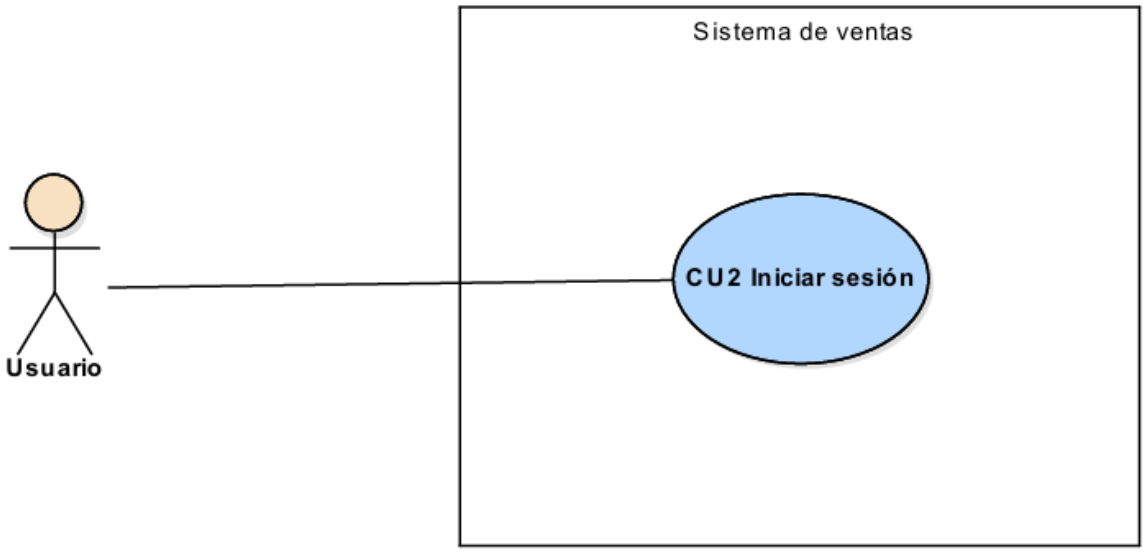
T02.2. Diseño Login

2.1. Descomposición Funcional CU2 “Iniciar Sesión”

1. El usuario ingresa en el módulo de inicio de sesión.
2. El sistema muestra la interfaz gráfica solicitando el nombre de usuario y la contraseña.
3. El usuario ingresa las credenciales de acceso solicitadas y presiona el botón confirmar.
4. Se valida la integridad de la base de datos y si hay errores, se extiende al CU20: Administrar Error de Integridad.
5. El sistema verifica que no exista una sesión activa en el sistema. En caso de que exista, el sistema informa que no se puede iniciar sesión y descarta la solicitud.
6. El sistema valida que los datos ingresados no estén vacíos. Si el usuario dejó algún campo en blanco, se informa que debe completar todos los datos para continuar.
7. El sistema encripta la contraseña ingresada mediante el encriptador de la capa de servicios.
8. El sistema verifica que el usuario ingresado exista en la base de datos, caso contrario, informa que las credenciales son incorrectas.
9. El sistema verifica que el usuario no se encuentre bloqueado, sino informa que la cuenta está inhabilitada temporalmente por seguridad.
10. El sistema verifica que el usuario se encuentre en estado activo (no posea baja lógica). Si figura inactivo, informa que el usuario se encuentra dado de baja.

11. El sistema verifica que la contraseña ingresada sea válida; si no coincide, suma un intento fallido y, en caso de alcanzar los 3 intentos fallidos, cambia su estado a bloqueado e informa la situación.
12. El sistema carga los permisos asignados al rol correspondiente del usuario.
13. El sistema registra el ingreso exitoso en la bitácora de eventos y muestra la pantalla del menú principal habilitando las opciones correspondientes a sus accesos.

2.2. Diagrama de Caso de Uso



2.3. Especificación de Caso de Uso

CU2 - INICIAR SESIÓN

Descripción:	El sistema permite al usuario autenticarse en la aplicación para acceder a las funcionalidades del negocio.
Actores:	Usuario

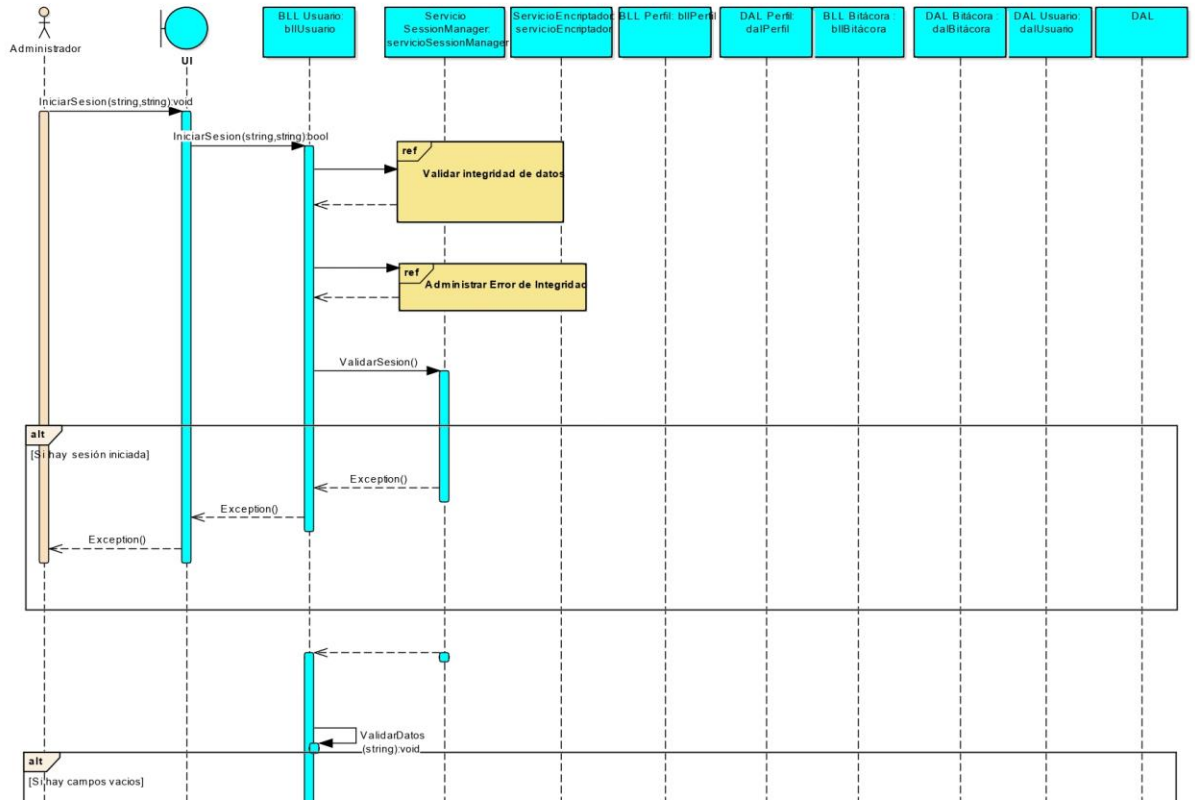
Precondiciones:	1. El usuario debe contar con una cuenta registrada en la base de datos. 2. El sistema debe encontrarse operativo y con conexión activa a la base de datos en SQL Server.
Puntos de extensión:	CU20: Administrar Error de Integridad
Flujo Principal	
1	El usuario ingresa en el módulo de inicio de sesión.
2	El sistema solicita el nombre de usuario y la contraseña.
3	El usuario ingresa el nombre de usuario y la contraseña, y presiona el botón confirmar.
4	Si el sistema detecta que hay errores de integridad en la base de datos al momento de inicio de la aplicación, se extiende al CU20: Administrar Error de Integridad.
5	El sistema verifica que no exista una sesión activa iniciada.
6	El sistema valida que los datos ingresados no estén vacíos.

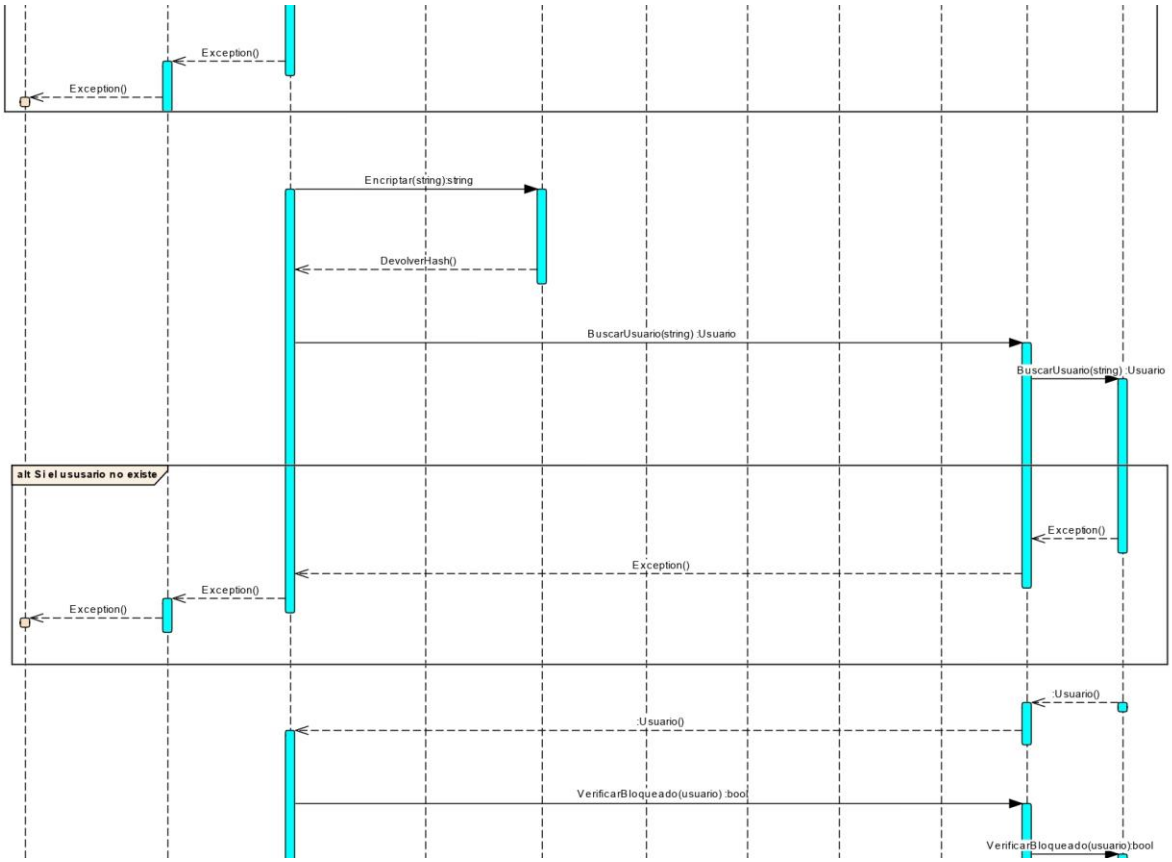
7	El sistema encripta la contraseña ingresada mediante funciones criptográficas de seguridad.
8	El sistema verifica que el usuario ingresado existe en la base de datos.
9	El sistema verifica que el usuario no se encuentra en estado bloqueado por seguridad.
10	El sistema verifica que el usuario se encuentra en estado activo (no posee baja lógica).
11	El sistema verifica que la contraseña ingresada es válida y coincide con la almacenada.
12	El sistema carga los permisos asignados al usuario.
13	El sistema registra el ingreso exitoso y muestra la pantalla del menú principal habilitando las opciones correspondientes a sus permisos.
Flujo Alternativo	
5.1.1	Ya existe una sesión iniciada activa en la terminal.

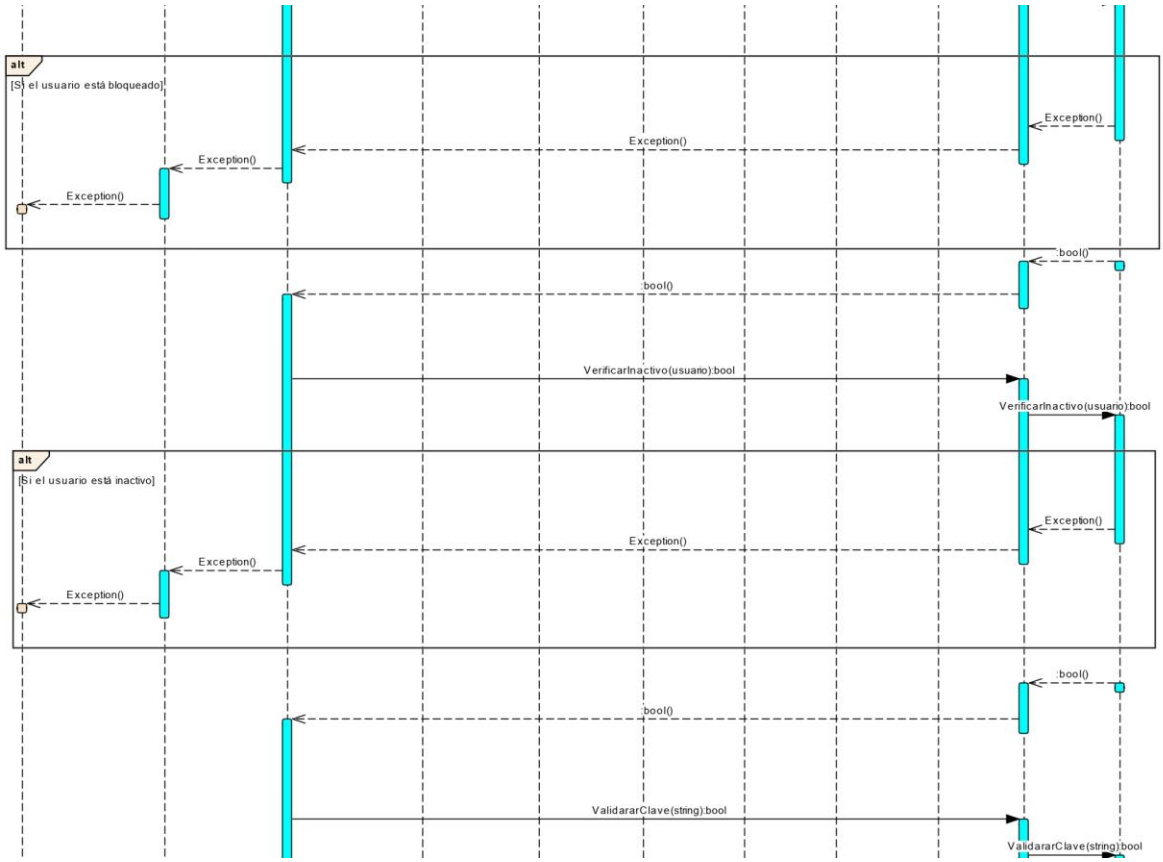
5.1.2	El sistema informa que no se permiten sesiones concurrentes y descarta la solicitud.
6.1.1	El usuario dejó alguno de los campos vacíos.
6.1.2	El sistema informa que debe completar todos los datos para continuar.
8.1.1	El sistema no encuentra ningún usuario con los datos ingresados.
8.1.2	El sistema informa que el usuario no existe.
9.1.1	El usuario figura en estado bloqueado.
9.1.2	El sistema informa que la cuenta está bloqueada temporalmente por seguridad.
10.1.1	El usuario figura en estado inactivo.
10.1.2	El sistema informa que el usuario se encuentra dado de baja en la tienda.

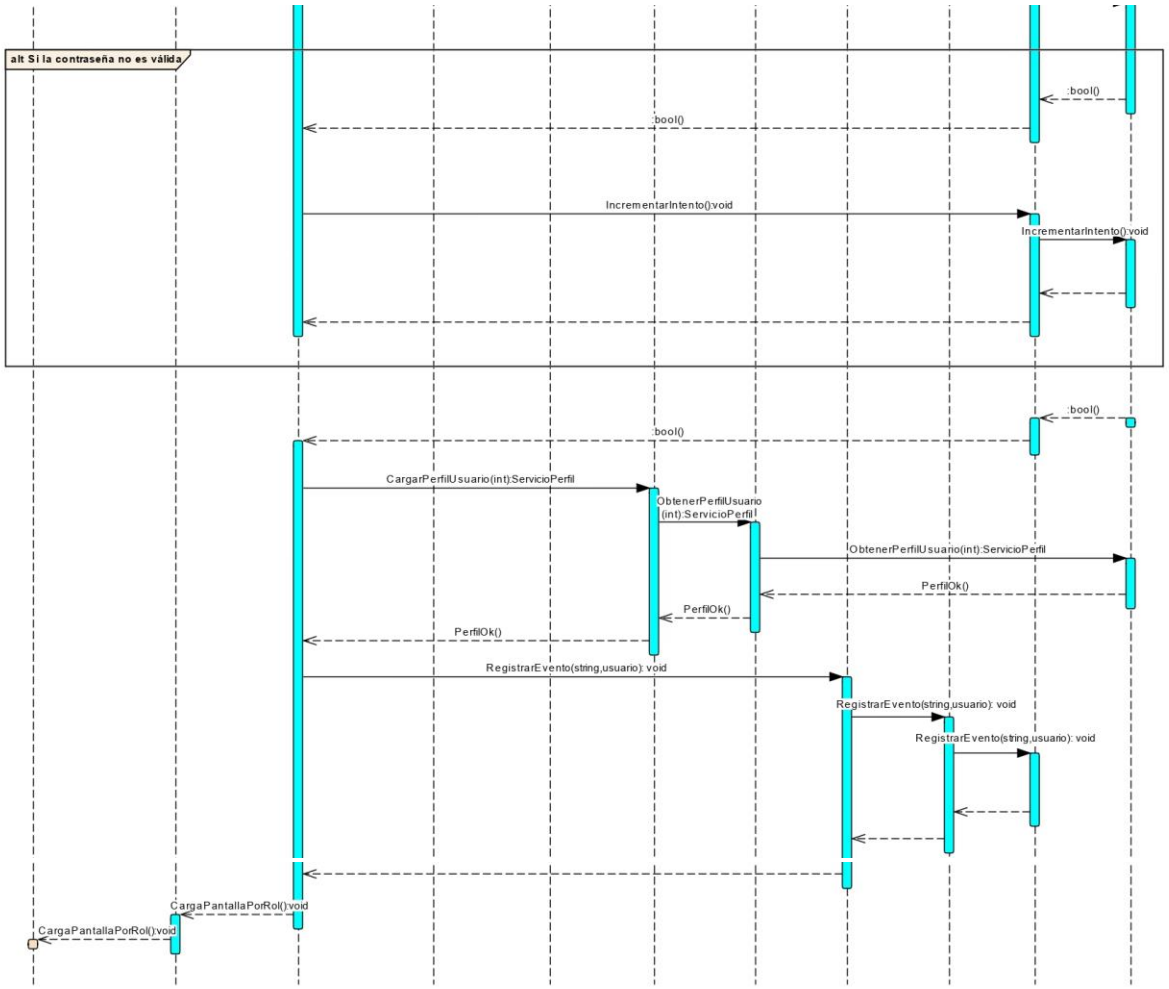
11.1.1	La contraseña ingresada no es válida.
11.1.2	El sistema suma un intento fallido al registro del usuario.
11.1.3	Si el usuario alcanza o supera los 3 intentos fallidos, el sistema cambia su estado a bloqueado e informa la situación.
Poscondición:	El usuario queda correctamente autenticado en el sistema y posicionado en la pantalla del menú principal con sus respectivos permisos de acceso cargados.

2.4. Diagrama de Secuencia

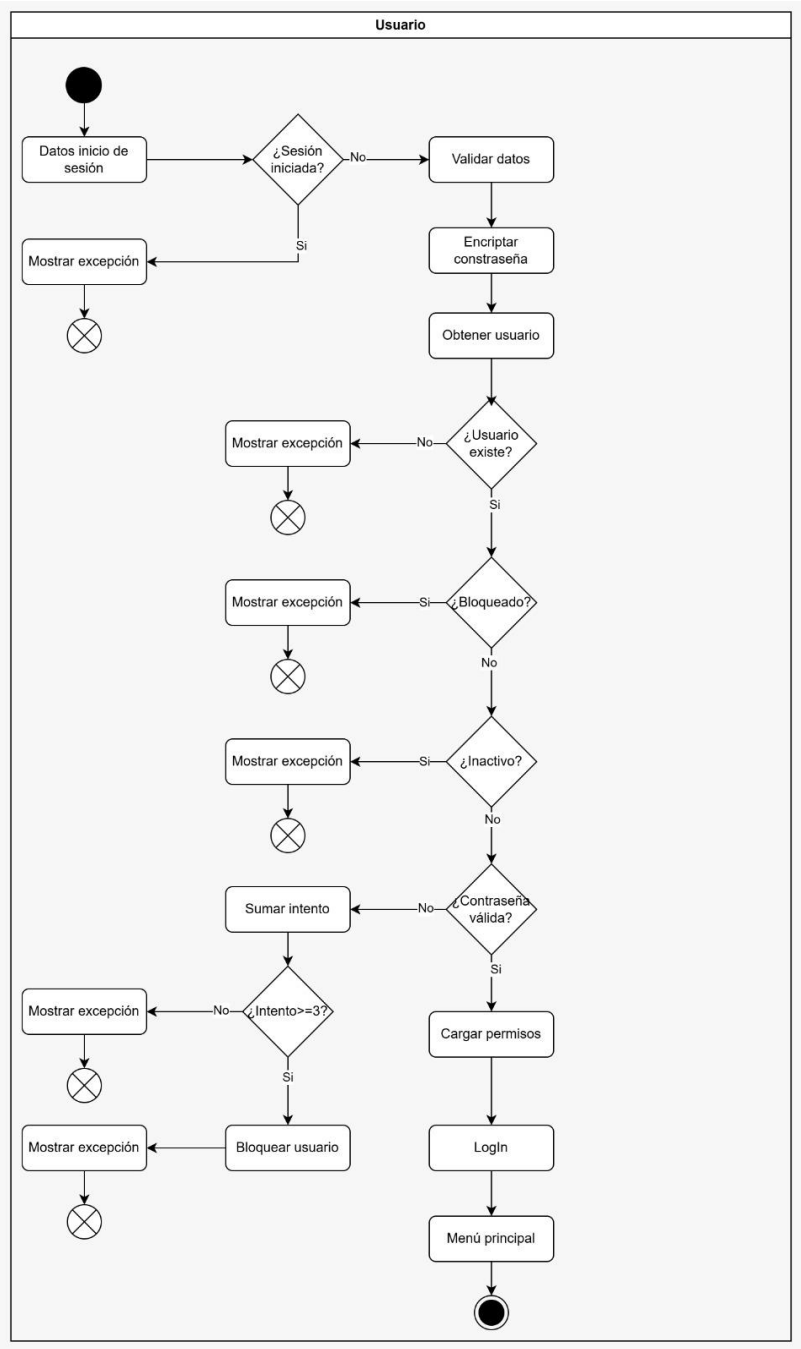



Exception()
Exception()
Encriptar(string):string
DevolverHash()
BuscarUsuario(string):Usuario
BuscarUsuario(string):Usuario
alt Si el ususario no existe
Exception()
Exception()
Exception()
Exception()
:Usuario()
:Usuario()
VerificarBloqueado(usuario):bool
VerificarBloqueado(usuario):bool

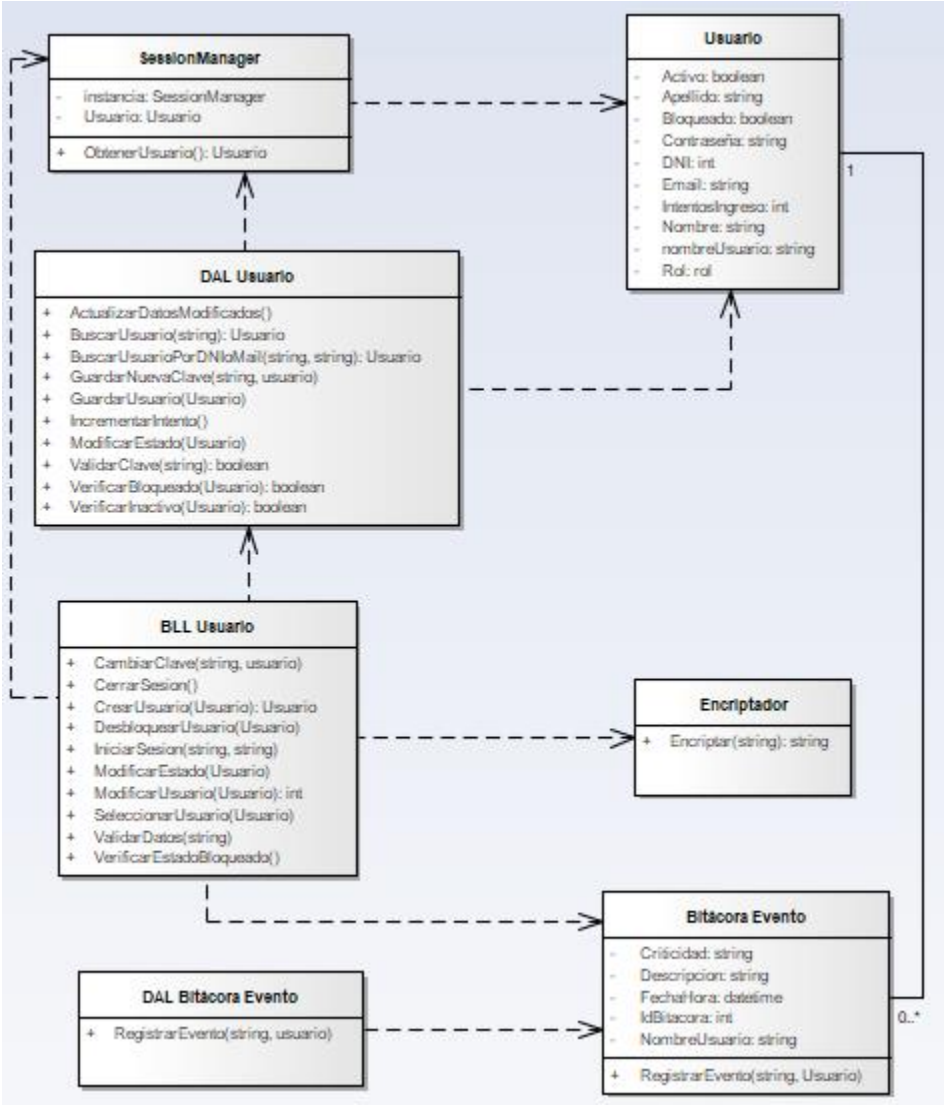

alt
[Si el usuario está bloqueado]
Exception()
Exception()
Exception()
Exception()
:bool()
:bool()
VerificarInactivo(usuario):bool
VerificarInactivo(usuario):bool
alt
[Si el usuario está inactivo]
Exception()
Exception()
Exception()
Exception()
:bool()
:bool()
ValidararClave(string):bool
ValidararClave(string):bool



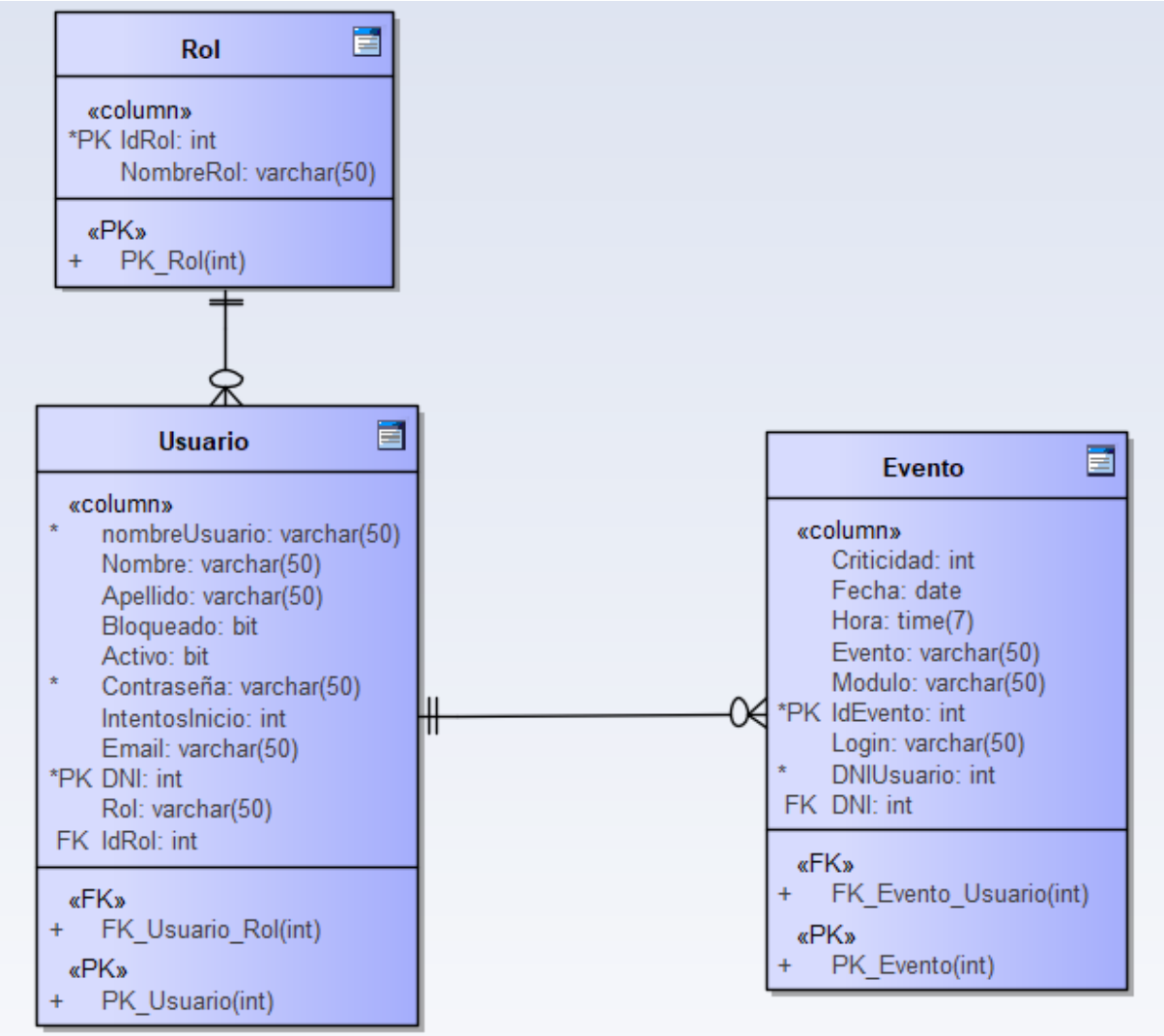
2.5. Diagrama de Actividad



2.6. Diagrama de Clase



2.7. Diagrama Entidad Relación


Rol
«column»
*PK IdRol: int
NombreRol: varchar(50)
«PK»
+ PK_Rol(int)
Usuario
«column»
* nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
* Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
*PK DNI: int
Rol: varchar(50)
FK IdRol: int
«FK»
+ FK_Usuario_Rol(int)
«PK»
+ PK_Usuario(int)
Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
* DNIUsuario: int
FK DNI: int
«FK»
+ FK_Evento_Usuario(int)
«PK»
+ PK_Evento(int)

2.8. GUI

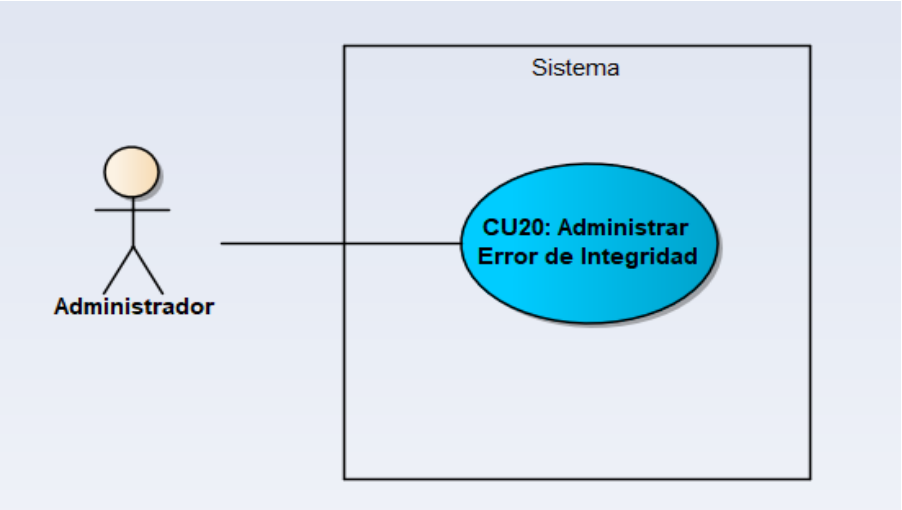


CU20 - ADMINISTRAR ERROR DE INTEGRIDAD

Descomposición Funcional

1. El Sistema bloquea el acceso y detiene el procesamiento del login (Capa GUI).
2. El Sistema guarda registro del fallo crítico en el historial de auditoría en bitácora (Capa DAL).
3. El Sistema despliega la pantalla crítica de error recibiendo la señal de fallo de consistencia de datos desde el proceso de autenticación, produciendo la interfaz restrictiva de seguridad (Capa GUI).
4. El Administrador gestiona los errores encontrar y, al finalizar, solicita la opción de recalcular dígitos verificadores (Capa GUI).
5. El Sistema ejecuta el recálculo de dígitos verificadores recibiendo la colección de filas de la tabla afectada, y produciendo los nuevos hashes válidos horizontales y verticales (Capa BLL y DAL).
6. El Sistema actualiza los registros con los nuevos valores corregidos, produciendo la actualización física de las tablas en la base de datos (Capa DAL).
7. El Sistema notifica la restauración del estado seguro y libera la aplicación recibiendo la confirmación de la persistencia correcta, y produciendo el mensaje de éxito junto con el redireccionamiento a la pantalla de inicio (Capa GUI).

Diagrama de Caso de Uso

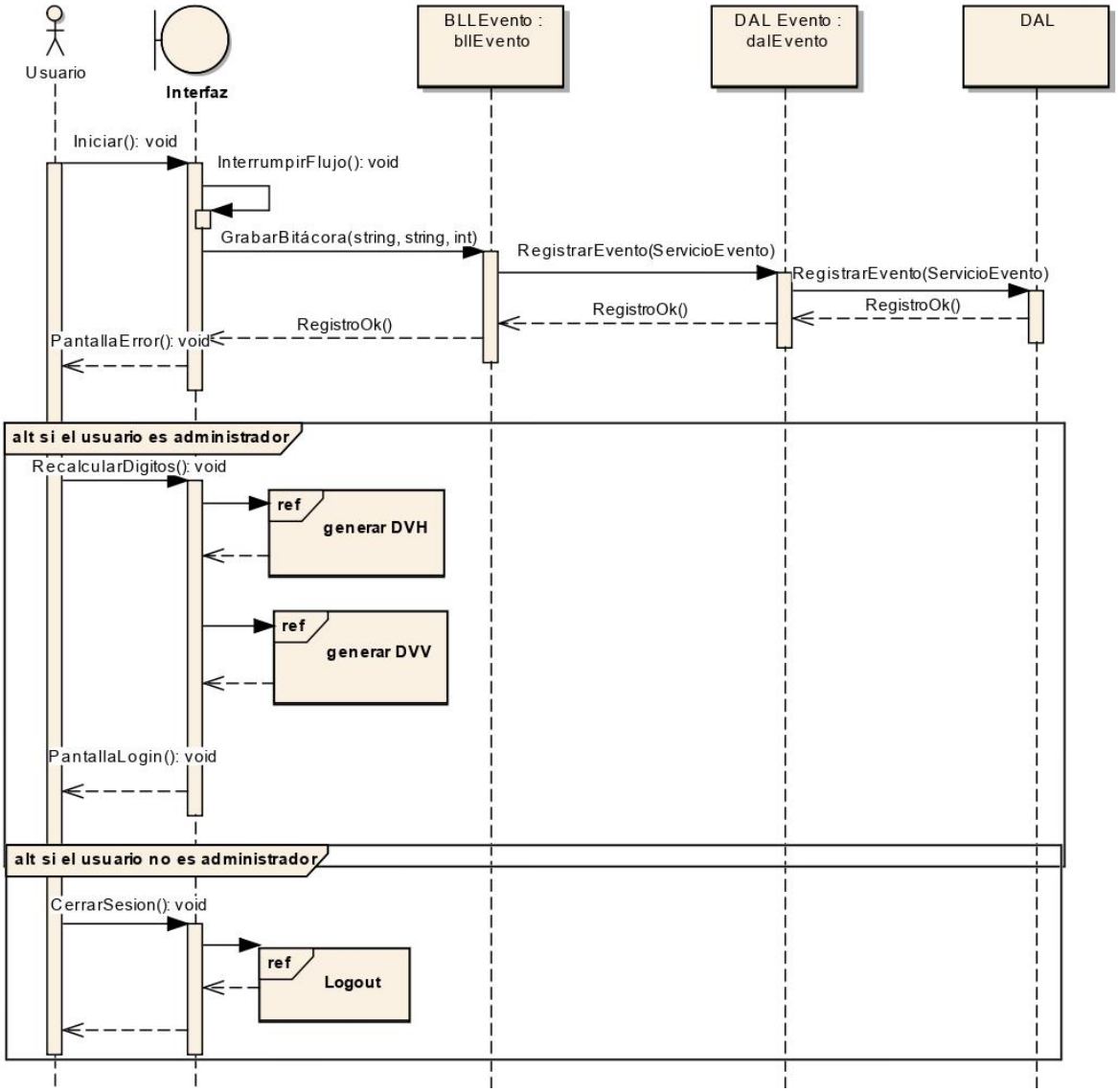


Especificación de Caso de Uso

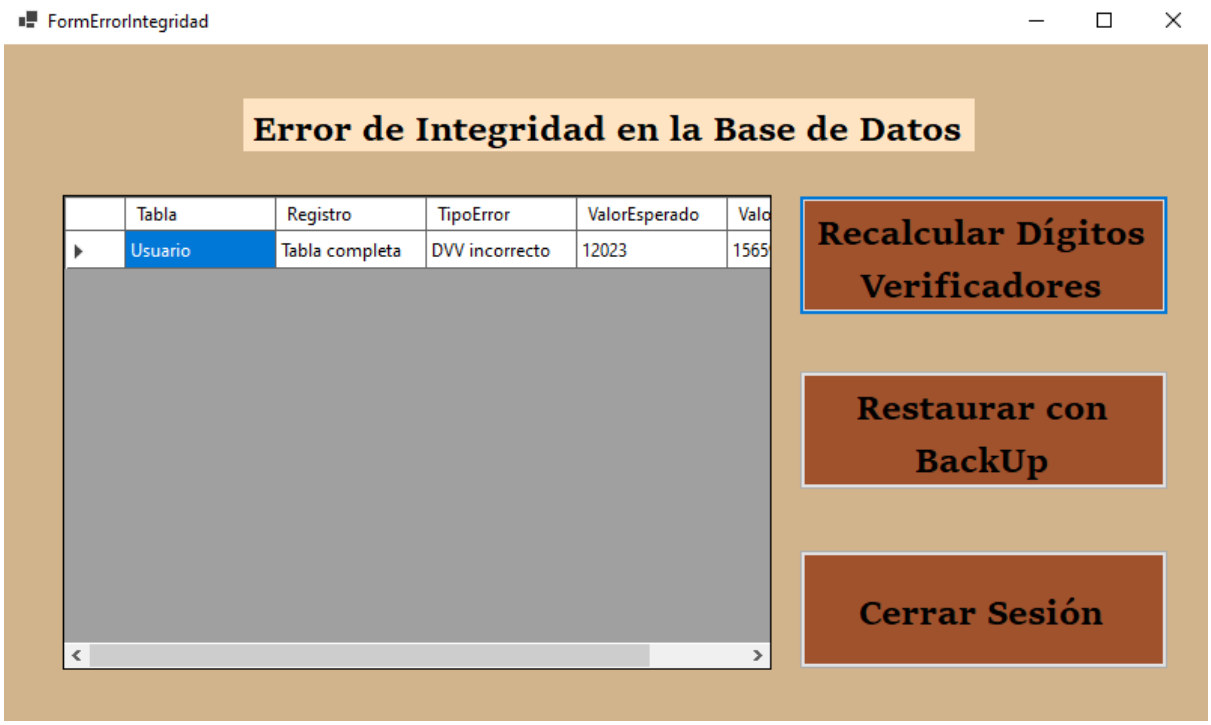
CU20 - ADMINISTRAR ERROR DE INTEGRIDAD

Descripción:	El sistema bloquea el acceso general de la aplicación ante una falla de consistencia de datos y fuerza al administrador a solucionar la inconsistencia.
Actores:	Administrador
Precondiciones:	1. El sistema detectó un fallo en la verificación de DVV/DVH durante el proceso de Login o inicio del sistema. 2. El usuario ingresado debe tener permisos de administrador 3. El sistema debe tener conexión activa a la base de datos en SQL Server.
Puntos de extensión:	N/A
Flujo principal:	
1	El sistema interrumpe el flujo de Login normal y bloquea las funciones operativas de la aplicación.
2	El sistema registra de forma automática el fallo crítico de integridad en la bitácora del sistema de forma persistente.
3	El sistema muestra una interfaz restrictiva informando detalladamente qué tabla o registro perdió la integridad.
4	El administrador gestiona en la base de datos las tablas y los registros afectados para subsanar el error y luego selecciona la opción de "Recalcular Dígitos Verificadores".
5	El sistema recalcula los valores de DVH y DVV sobre la base de datos para verificar los datos legítimos.
6	El sistema carga los datos con los nuevos dígitos correctos.
7	El sistema informa que la integridad fue restablecida con éxito y redirige a la pantalla de Login.
Flujo alternativo:	
5.1.1	El sistema detecta que el recálculo no logra solucionar la inconsistencia o que los datos de origen están físicamente dañados.
5.1.2	El sistema le indica al usuario que los errores no fueron resueltos y se deben administrar nuevamente.
Poscondición:	La base de datos del sistema recupera su consistencia e integridad, se desbloquean las funciones generales de la aplicación y la acción de reparación queda registrada de forma exitosa en la bitácora.

Diagrama de Secuencia



GUI

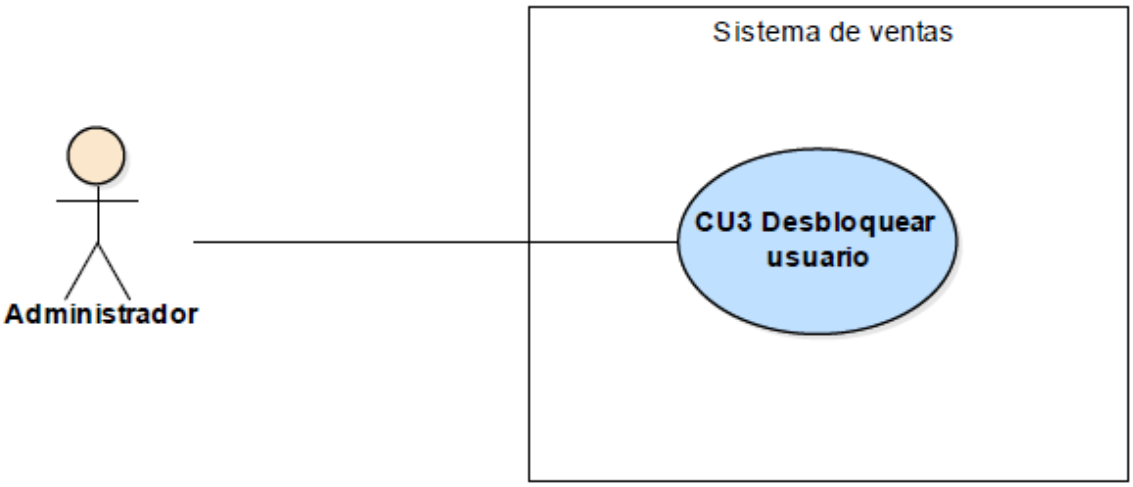


T02.3. Diseño Desbloquear Usuario

3.1. Descomposición Funcional

1. El administrador ingresa al menú principal y accede al módulo de usuarios.
2. El sistema muestra la pantalla de gestión de usuarios y visualiza los usuarios bloqueados en una grilla.
3. El administrador selecciona un usuario bloqueado y presiona el botón desbloquear.
4. El sistema modifica el estado del usuario a desbloqueado.
5. El sistema recalcula los dígitos verificadores.
6. El sistema registra la acción en la bitácora.
7. El sistema actualiza la grilla y muestra el usuario desbloqueado correctamente.

3.2. Diagrama de Caso de Uso

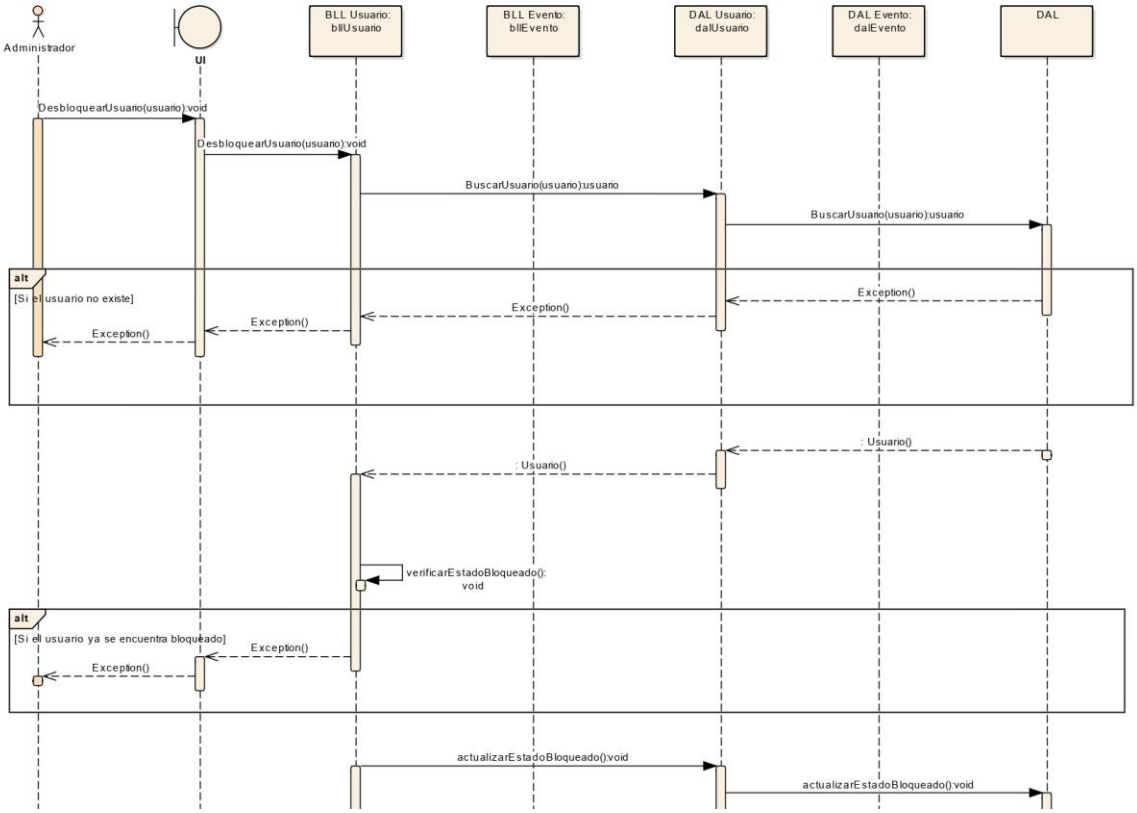


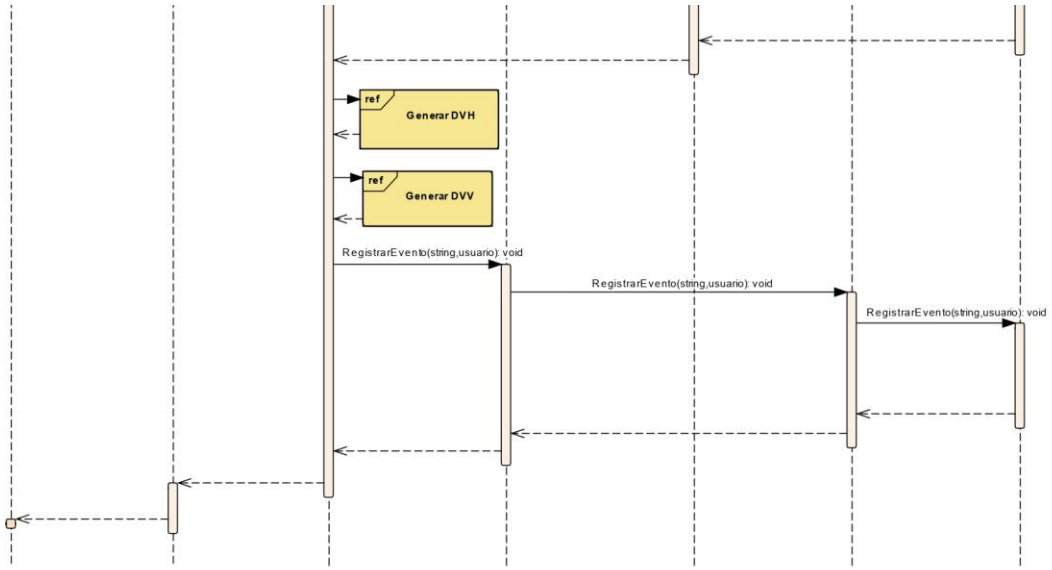
3.3. Especificación de Caso de Uso

CU3 - DESBLOQUEAR USUARIO

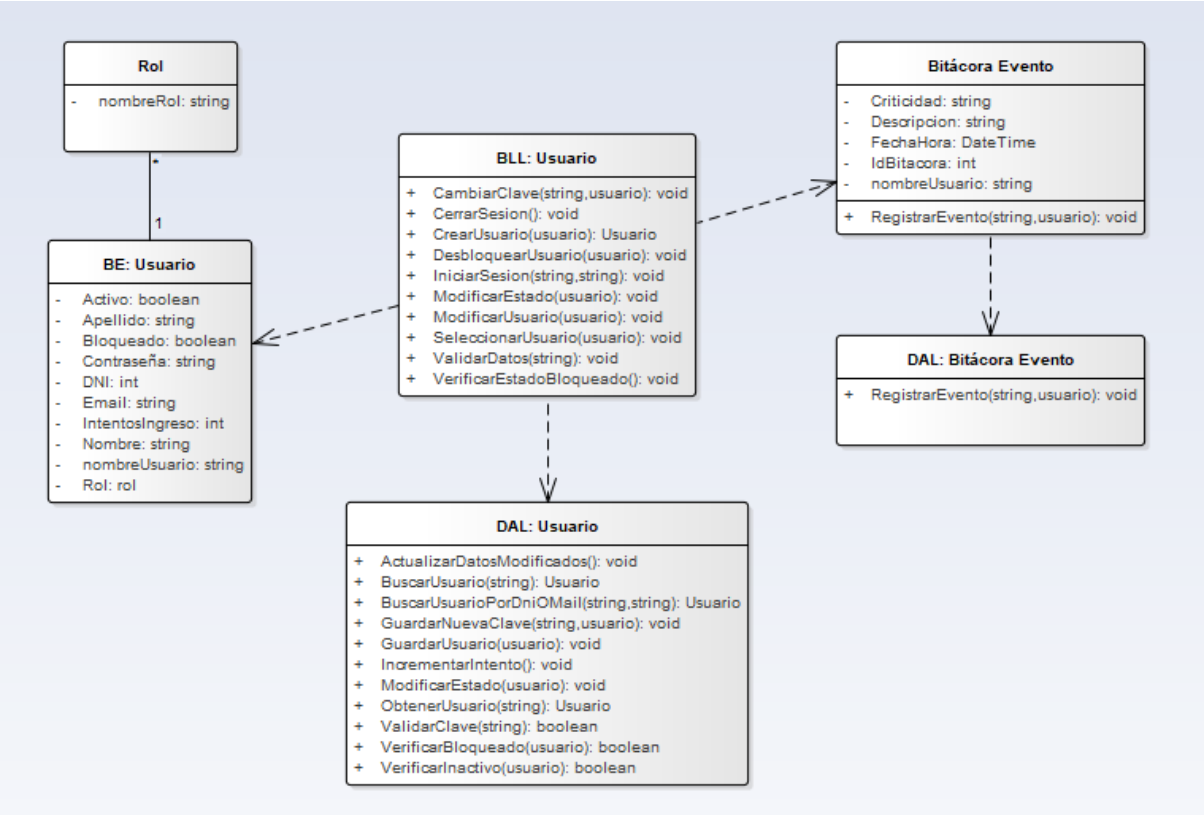
Descripción:	El sistema permite al administrador desbloquear usuarios bloqueados para restablecer su acceso al sistema.
Actores:	Administrador
Precondiciones:	4. El administrador debe haber iniciado sesión en el sistema. 5. El administrador debe poseer permisos para gestionar usuarios. 6. El sistema debe encontrarse operativo y con conexión activa a la base de datos SQL Server.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador ingresa al módulo de usuarios.
2	El sistema muestra la lista de usuarios bloqueados.
3	El administrador selecciona un usuario bloqueado y presiona el botón desbloquear.
4	El sistema verifica que el usuario seleccionado exista.
5	El sistema verifica que el usuario se encuentre bloqueado.
6	El sistema actualiza el estado bloqueado del usuario.
7	El sistema recalcula los dígitos verificadores horizontales y verticales de la base de datos.
8	El sistema registra la acción en la bitácora.
9	El sistema informa que el usuario fue desbloqueado correctamente.
Flujo alternativo:	
4.1.1	El usuario seleccionado no existe.
4.1.2	El sistema informa que el usuario no fue encontrado.
5.1.1	El usuario seleccionado no se encuentra bloqueado.
5.1.2	El sistema informa que el usuario ya se encuentra desbloqueado.
Poscondición:	El usuario queda desbloqueado en el sistema y la acción realizada queda registrada en la bitácora.

3.4. Diagrama de Secuencia

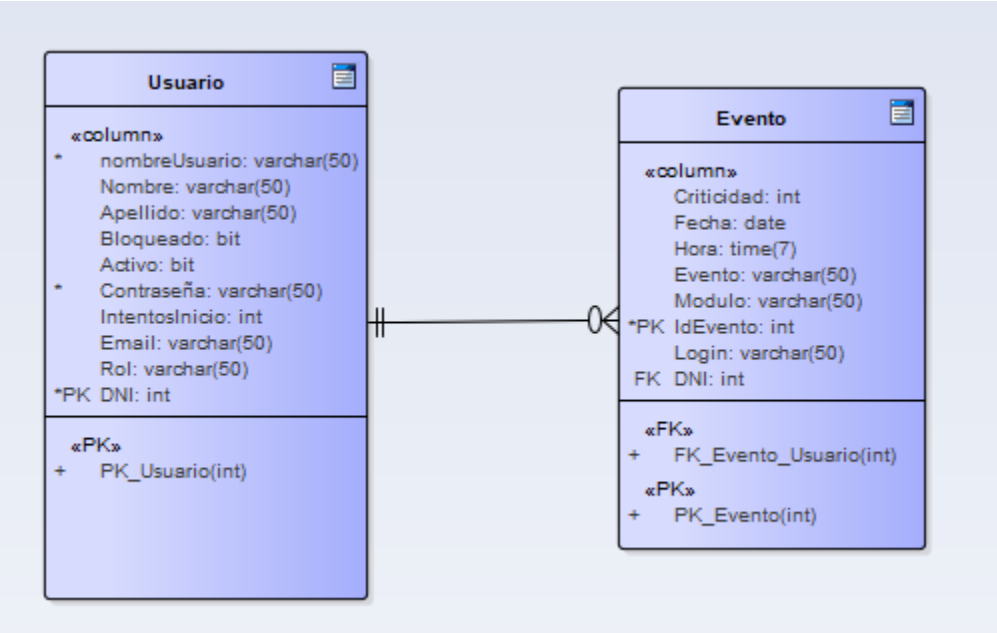

Administrador
UI
BLL Usuario: bllUsuario
BLL Evento: bllEvento
DAL Usuario: dalUsuario
DAL Evento: dalEvento
DAL
DesbloquearUsuario(usuario):void
DesbloquearUsuario(usuario):void
BuscarUsuario(usuario):usuario
BuscarUsuario(usuario):usuario
alt
[Si el usuario no existe]
Exception()
Exception()
Exception()
Exception()
: Usuario()
: Usuario()
verificarEstadoBloqueado(): void
alt
[Si el usuario ya se encuentra bloqueado]
Exception()
Exception()
actualizarEstadoBloqueado():void
actualizarEstadoBloqueado():void



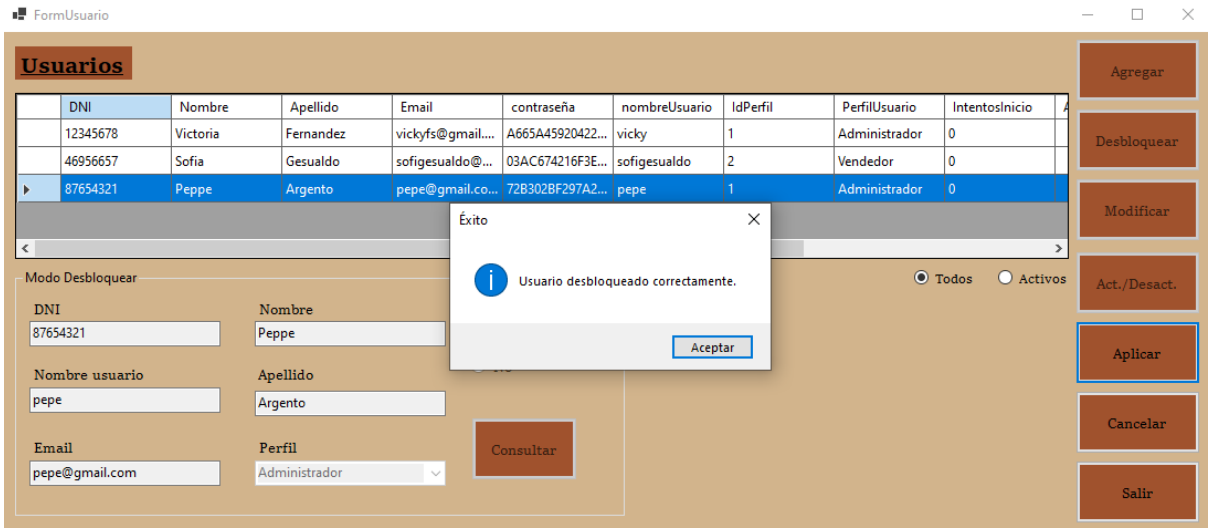
3.5. Diagrama de Clase



3.6. Diagrama Entidad Relación



3.7. GUI

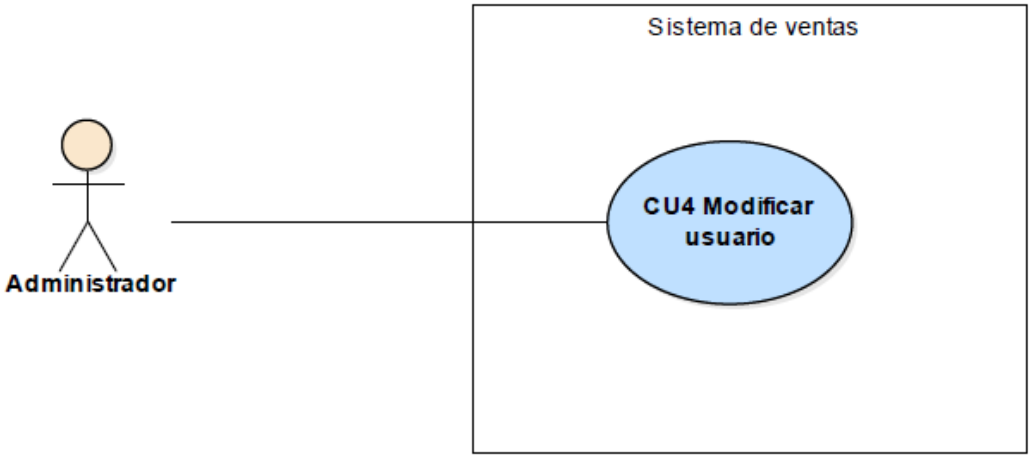


T02.4. Diseño Modificar Usuario

4.1. Descomposición Funcional

1. El administrador ingresa al módulo de usuarios.
2. El sistema muestra la lista de usuarios registrados.
3. El administrador selecciona un usuario y presiona modificar.
4. El sistema muestra los datos actuales del usuario.
5. El administrador modifica los datos y presiona guardar.
6. El sistema actualiza los datos del usuario.
7. El sistema registra la acción en la bitácora.
8. El sistema informa el resultado de la operación.

4.2. Diagrama de Caso de Uso



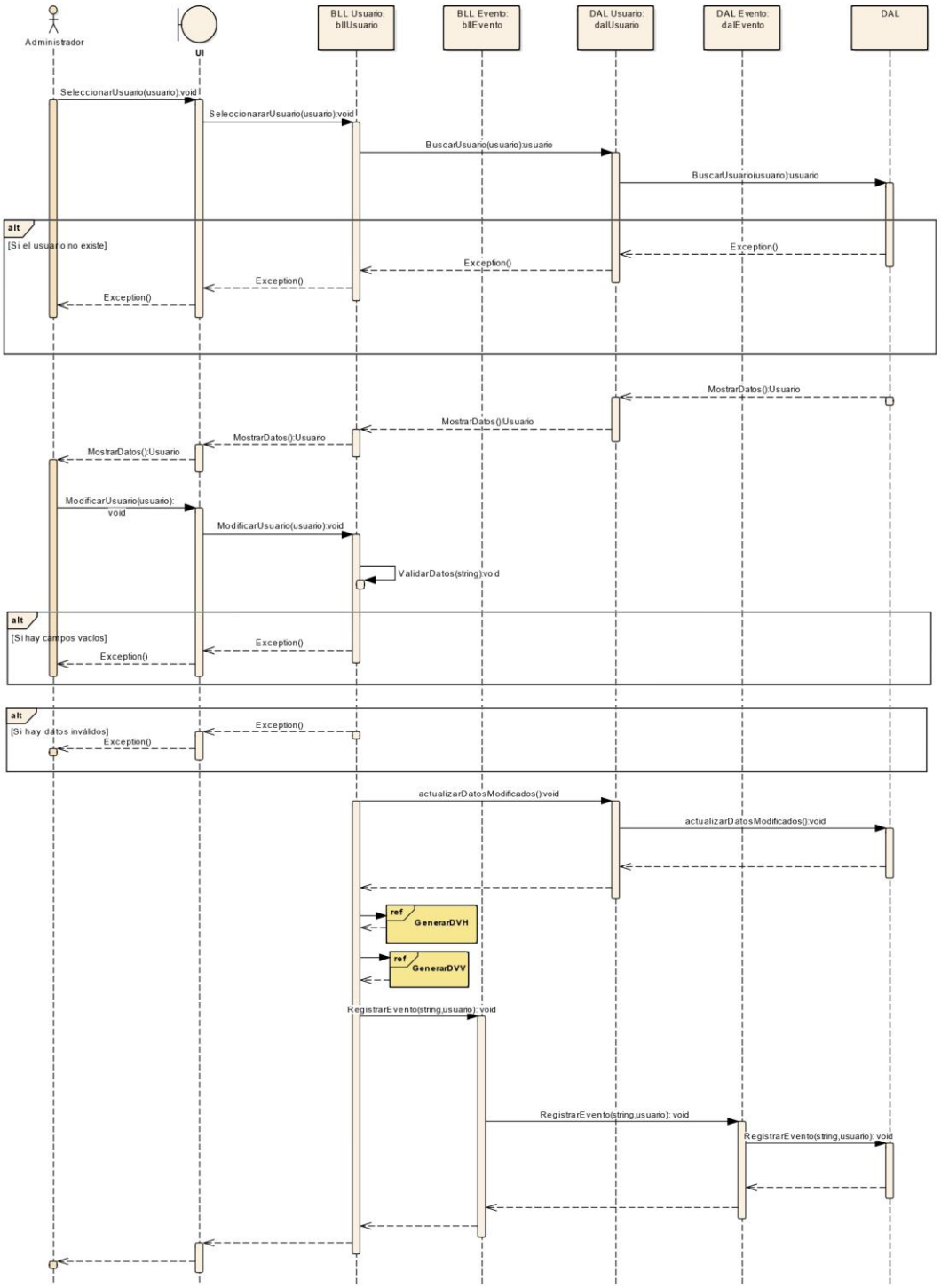
4.3. Especificación de Caso de Uso

CU4 - MODIFICAR USUARIO

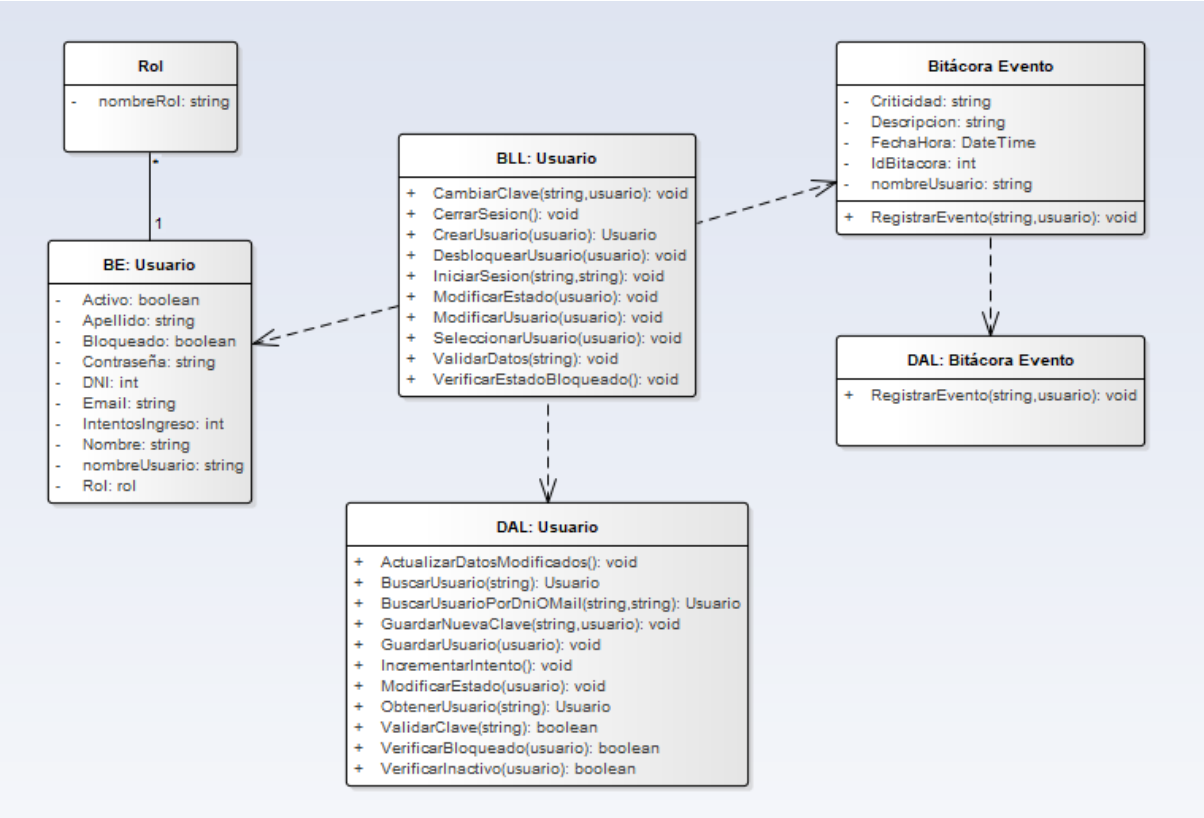
Descripción:	El sistema permite al administrador modificar los datos de un usuario registrado en la aplicación.
Actores:	Administrador
Precondiciones:	1. El administrador debe haber iniciado sesión en el sistema. 2. El administrador debe poseer permisos para gestionar usuarios. 3. El sistema debe encontrarse operativo y con conexión activa a la base de datos SQL Server.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador ingresa al módulo de usuarios.
2	El sistema muestra la lista de usuarios registrados.
3	El administrador selecciona un usuario y presiona el botón modificar.
4	El sistema verifica que el usuario seleccionado exista.
5	El sistema muestra los datos actuales del usuario seleccionado.
6	El administrador modifica los datos del usuario y presiona el botón guardar.
7	El sistema valida que los datos ingresados no se encuentren vacíos.
8	El sistema valida el formato de los datos ingresados.
9	El sistema actualiza los datos modificados del usuario en la base de datos.
10	El sistema calcula nuevamente los dígitos verificadores horizontales y verticales en la base de datos.
11	El sistema registra la acción en la bitácora.
12	El sistema informa que el usuario fue modificado correctamente.
Flujo alternativo:	
4.1.1	El usuario seleccionado no existe.
4.1.2	El sistema informa que el usuario no fue encontrado.
7.1.1	El usuario seleccionado no se encuentra bloqueado.
7.1.2	El sistema informa que el usuario ya se encuentra desbloqueado.
8.1.1	Alguno de los datos ingresados posee un formato inválido.
8.1.2	El sistema informa que los datos ingresados no son válidos.

Poscondición:	El usuario queda desbloqueado en el sistema y la acción realizada queda registrada en la bitácora.

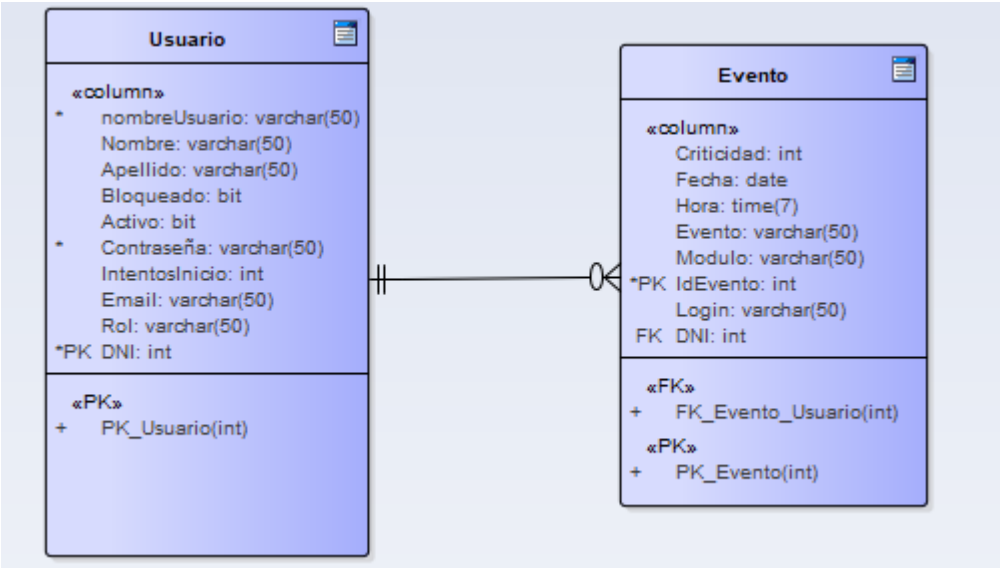
4.4. Diagrama de Secuencia


Administrador
UI
BLL Usuario: bllUsuario
BLL Evento: bllEvento
DAL Usuario: dalUsuario
DAL Evento: dalEvento
DAL
SeleccionarUsuario(usuario):void
SeleccionararUsuario(usuario):void
BuscarUsuario(usuario):usuario
BuscarUsuario(usuario):usuario
alt
[Si el usuario no existe]
Exception()
Exception()
Exception()
Exception()
MostrarDatos():Usuario
MostrarDatos():Usuario
MostrarDatos():Usuario
MostrarDatos():Usuario
ModificarUsuario(usuario): void
ModificarUsuario(usuario):void
ValidarDatos(string):void
alt
[Si hay campos vacios]
Exception()
Exception()
alt
[Si hay datos inválidos]
Exception()
Exception()
actualizarDatosModificados():void
actualizarDatosModificados():void
ref
GenerarDVH
ref
GenerarDVV
RegistrarEvento(string,usuario): void
RegistrarEvento(string,usuario): void
RegistrarEvento(string,usuario): void

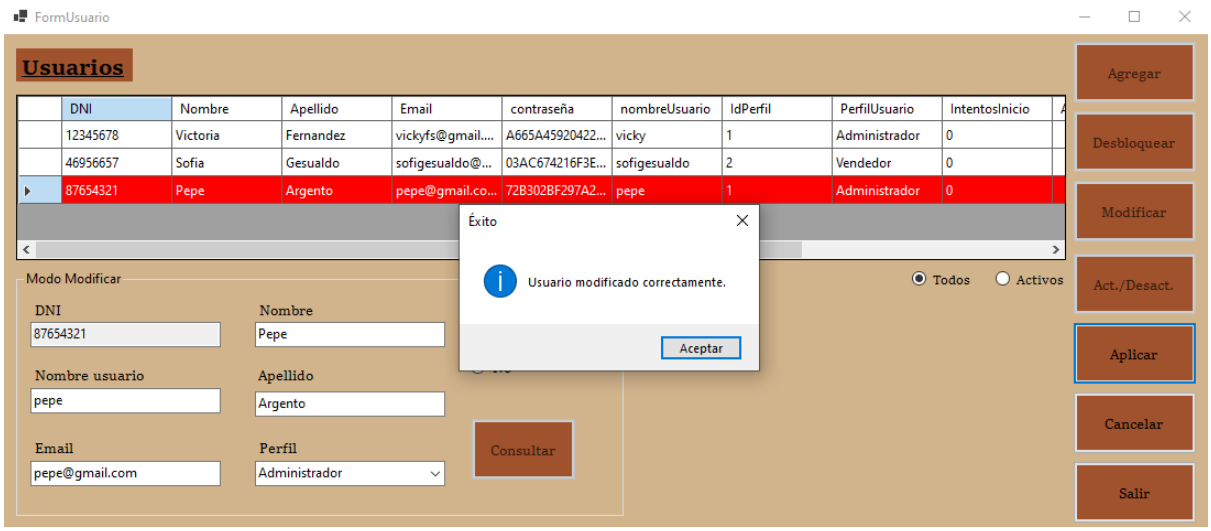
4.5. Diagrama de Clase



4.6. Diagrama Entidad Relación



4.7. GUI



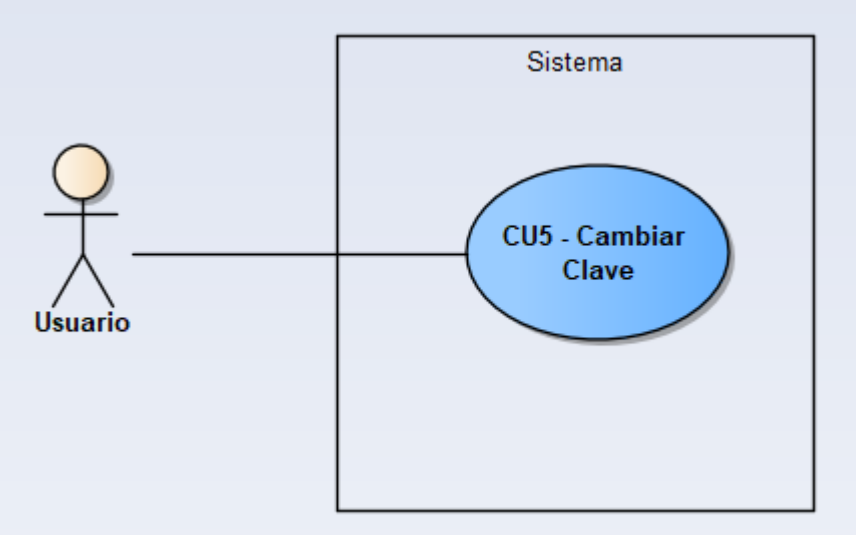
T02.5. Diseño Cambiar Clave

5.1. Descomposición Funcional CU5 “Cambiar Clave”

1. El usuario ingresa al sistema con sus credenciales.
2. El sistema muestra la pantalla de menú principal.
3. El usuario ingresa en el apartado de “Usuario” y luego selecciona el botón de “Cambiar Clave”.
4. El sistema muestra el módulo de cambio de contraseña y solicita los campos de Clave Actual, Clave Nueva y Confirmación de Clave.
5. El usuario ingresa los datos requeridos y confirma la acción.
6. El sistema valida que el usuario no haya ingresado valores vacíos y que el campo Clave Nueva coincida con el campo Confirmación de Clave.
7. El sistema obtiene a través del Session Manager los datos del usuario activo en la sesión.
8. El sistema encripta la Clave Actual ingresada en el cambio de clave para validar que coincida con los datos del usuario registrados en la base de datos. En caso de que no coincida, el sistema muestra un mensaje de error por pantalla.
9. El sistema encripta la nueva clave suministrada para actualizar el registro del hash de la contraseña del usuario en la base de datos.

10. El sistema recalcula los dígitos verificadores en la base de datos.
11. El sistema registra el cambio de clave en la bitácora de eventos y muestra un mensaje de confirmación por pantalla al usuario.

5.2. Diagrama de Caso de Uso



5.3. Especificación de Caso de Uso

CU5 - CAMBIAR CLAVE

Descripción:	El sistema permite al usuario autenticado modificar su contraseña actual por una nueva clave de seguridad.
Actores:	Usuario
Precondiciones:	1. El usuario debe contar con una sesión activa registrada en la base de datos.

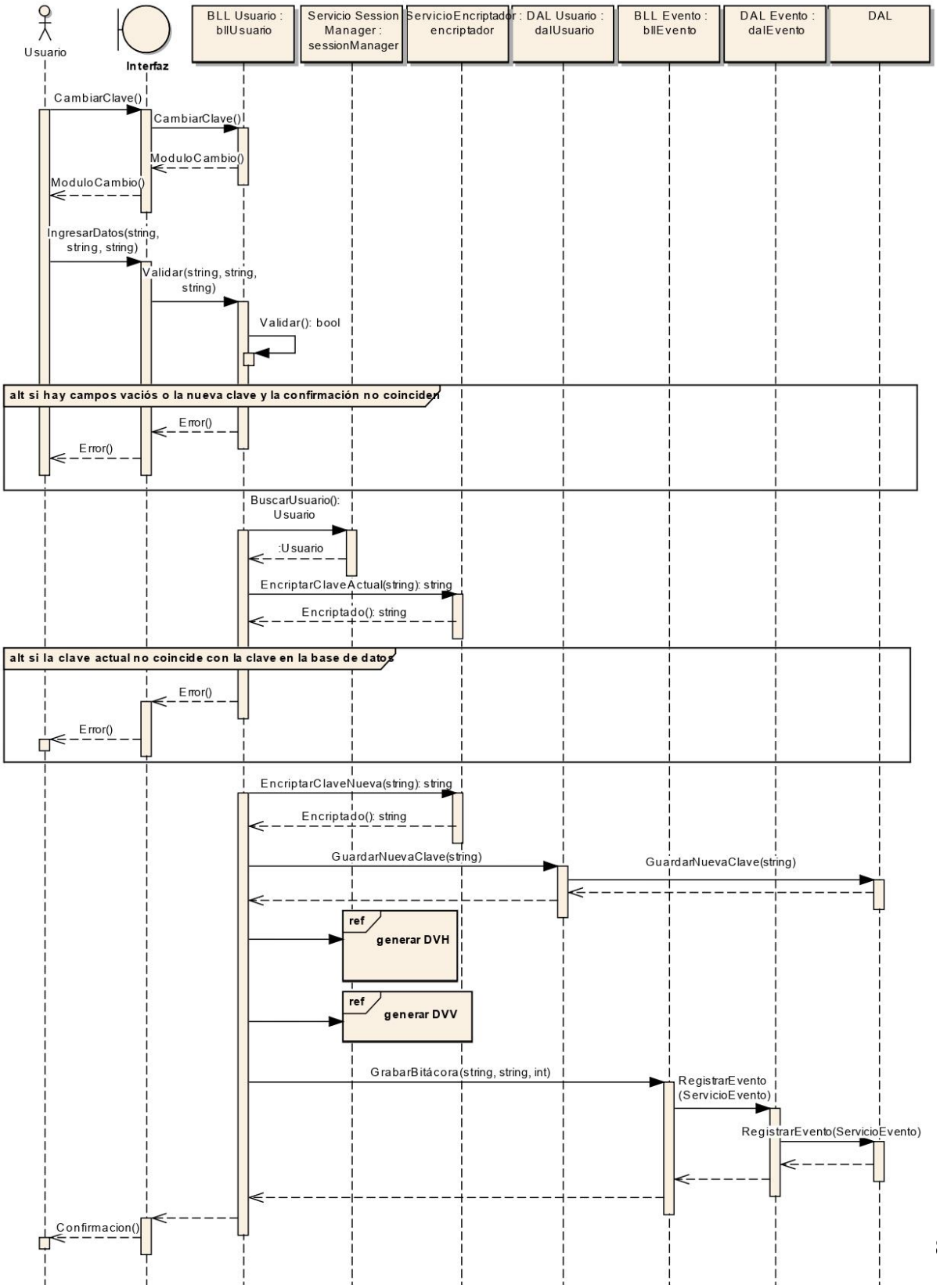
	2. El sistema debe encontrarse operativo y con conexión activa a la base de datos en SQL Server.
Puntos de extensión:	N/A
Flujo Principal	
1	El usuario ingresa en el apartado de “Usuario” y luego selecciona el botón de “Cambiar Clave”.
2	El sistema muestra el módulo de cambio de contraseña y solicita los campos de Clave Actual, Clave Nueva y Confirmación de Clave.
3	El usuario ingresa los datos requeridos y confirma la acción.
4	El sistema valida que el usuario no haya ingresado valores vacíos y que el campo Clave Nueva coincida con el campo Confirmación de Clave.
5	El sistema obtiene a través del session manager los datos del usuario activo en la sesión.

6	El sistema encripta la Clave Actual ingresada en el cambio de clave para validar que coincida con los datos del usuario registrados en la base de datos.
7	El sistema encripta la nueva clave suministrada para actualizar el registro del hash de la contraseña del usuario en la base de datos.
8	El sistema recalcula los dígitos verificadores horizontales y verticales luego de actualizar la base de datos.
9	El sistema registra el cambio de clave en la bitácora de eventos y muestra un mensaje de confirmación por pantalla al usuario.
Flujo Alternativo	
4.1.1	El usuario ingresó uno o más valores vacíos en el formulario.
4.1.2	El sistema informa mediante una excepción que todos los campos son obligatorios para continuar.
4.2.1	El campo Clave Nueva y el campo Confirmación de Clave no coinciden sintácticamente.

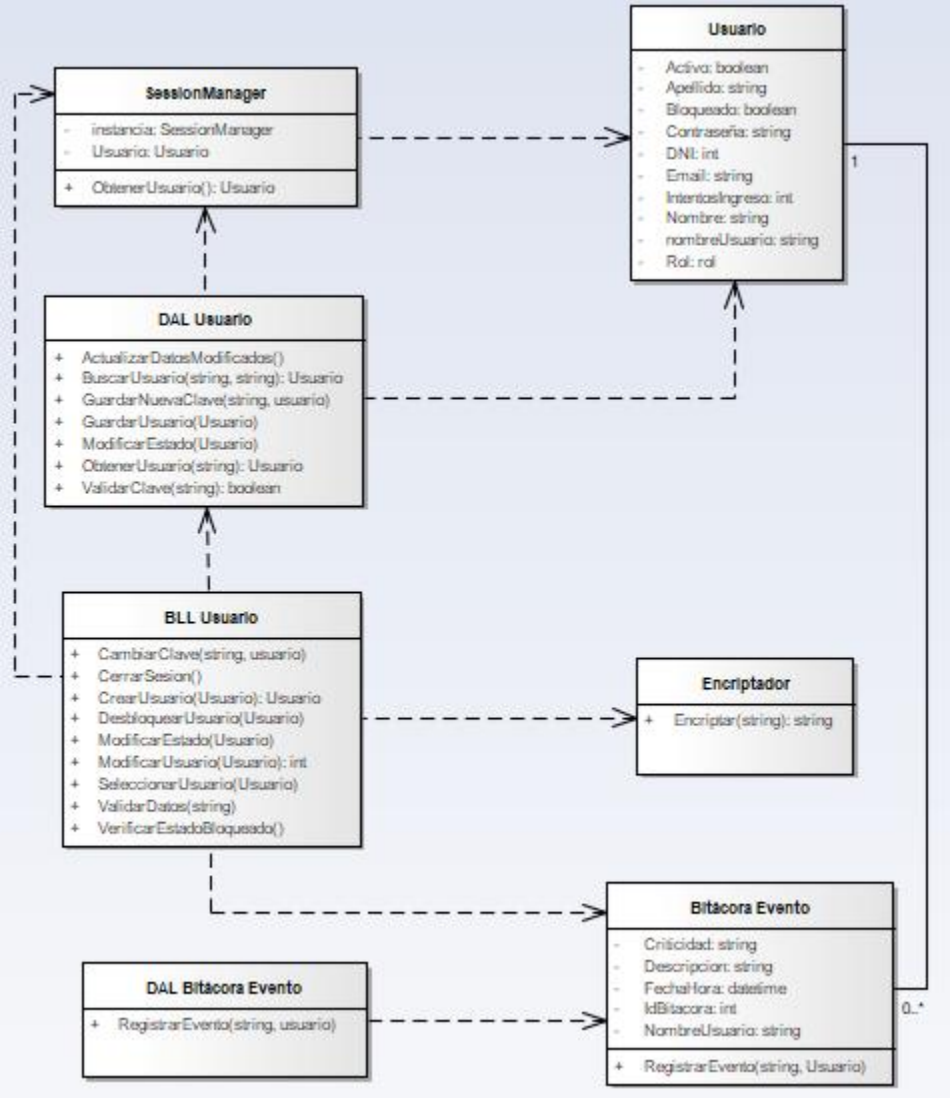
4.2.2	El sistema muestra un mensaje de advertencia indicando la discrepancia y solicita corregir los campos.
6.1.1	La Clave Actual encriptada no coincide con el hash registrado en la base de datos para el usuario de la sesión.
6.1.2	El sistema interrumpe el proceso, muestra un mensaje de error por pantalla informando que la credencial es inválida y cancela la actualización por seguridad.
Poscondición:	La contraseña del usuario es modificada de manera persistente en la base de datos bajo un nuevo hash, manteniendo la sesión actual activa.

5.4. Diagrama de Secuencia

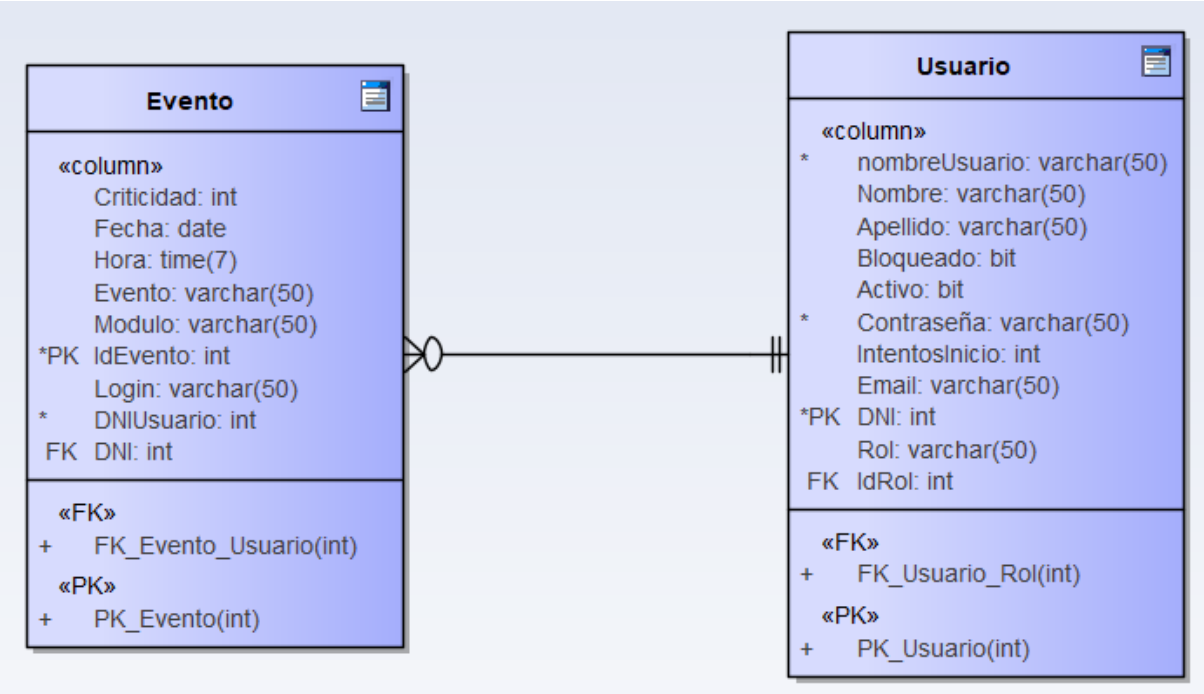
UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026



5.5. Diagrama de Clase



5.6. Diagrama Entidad Relación


Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
* DNIUsuario: int
FK DNI: int
«FK»
+ FK_Evento_Usuario(int)
«PK»
+ PK_Evento(int)
Usuario
«column»
* nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
* Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
*PK DNI: int
Rol: varchar(50)
FK IdRol: int
«FK»
+ FK_Usuario_Rol(int)
«PK»
+ PK_Usuario(int)

5.7. GUI

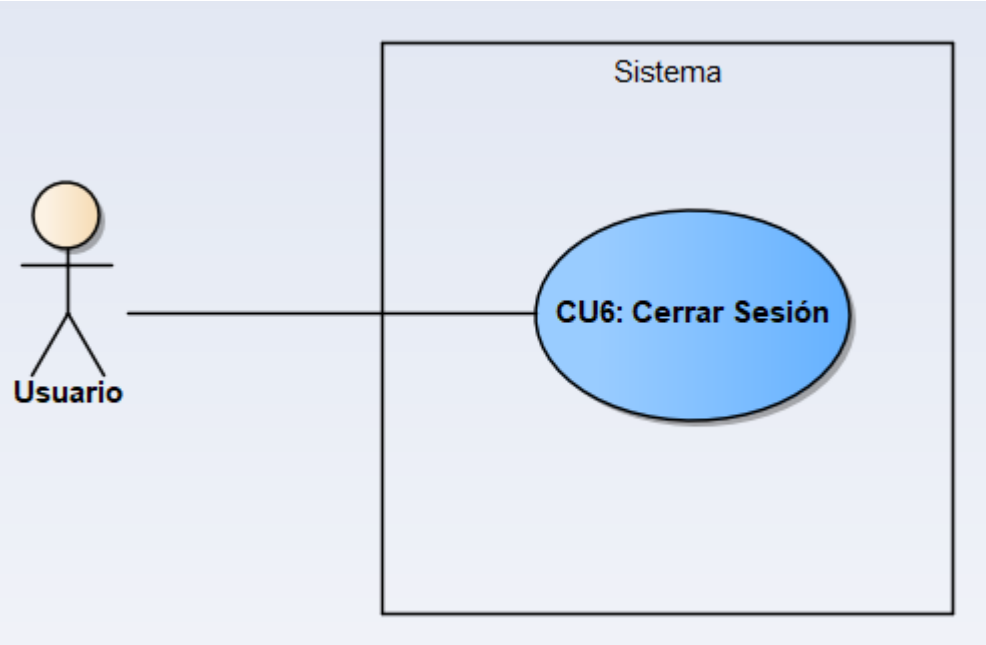

FormCambioClave
BOOKLY
LIBROS QUE TE ACOMPAÑAN
Cambio de Clave
Clave Actual:
Nueva Clave:
Confirmación:
Salir
Confirmar

T02.6. Diseño Logout

6.1. Descomposición Funcional CU6 “Cerrar Sesión”

1. El usuario está ingresado en el sistema y se encuentra en la pantalla del menú principal.
2. El usuario selecciona el botón de “Cerrar Sesión” desde la barra de herramientas.
3. El sistema muestra un cuadro de diálogo flotante solicitando la confirmación de la acción por parte del usuario.
4. El usuario confirma la acción.
5. El sistema obtiene con el Session Manager los datos del usuario activo y su configuración de idioma actual.
6. El sistema invoca al método de actualización en la capa de lógica para persistir en la base de datos la preferencia de idioma del usuario en la tabla Usuario.
7. El sistema recalcula los dígitos verificadores en la base de datos.
8. El sistema invoca al método de cierre de sesión en la capa de lógica.
9. El sistema ejecuta el método interno de limpieza en el Session Manager, asignando un valor nulo al usuario activo.
10. El sistema registra la desconexión del usuario en la bitácora de eventos.
11. El sistema cierra todos los módulos abiertos y muestra nuevamente la pantalla de inicio de sesión.

6.2. Diagrama de Caso de Uso



6.3. Especificación de Caso de Uso

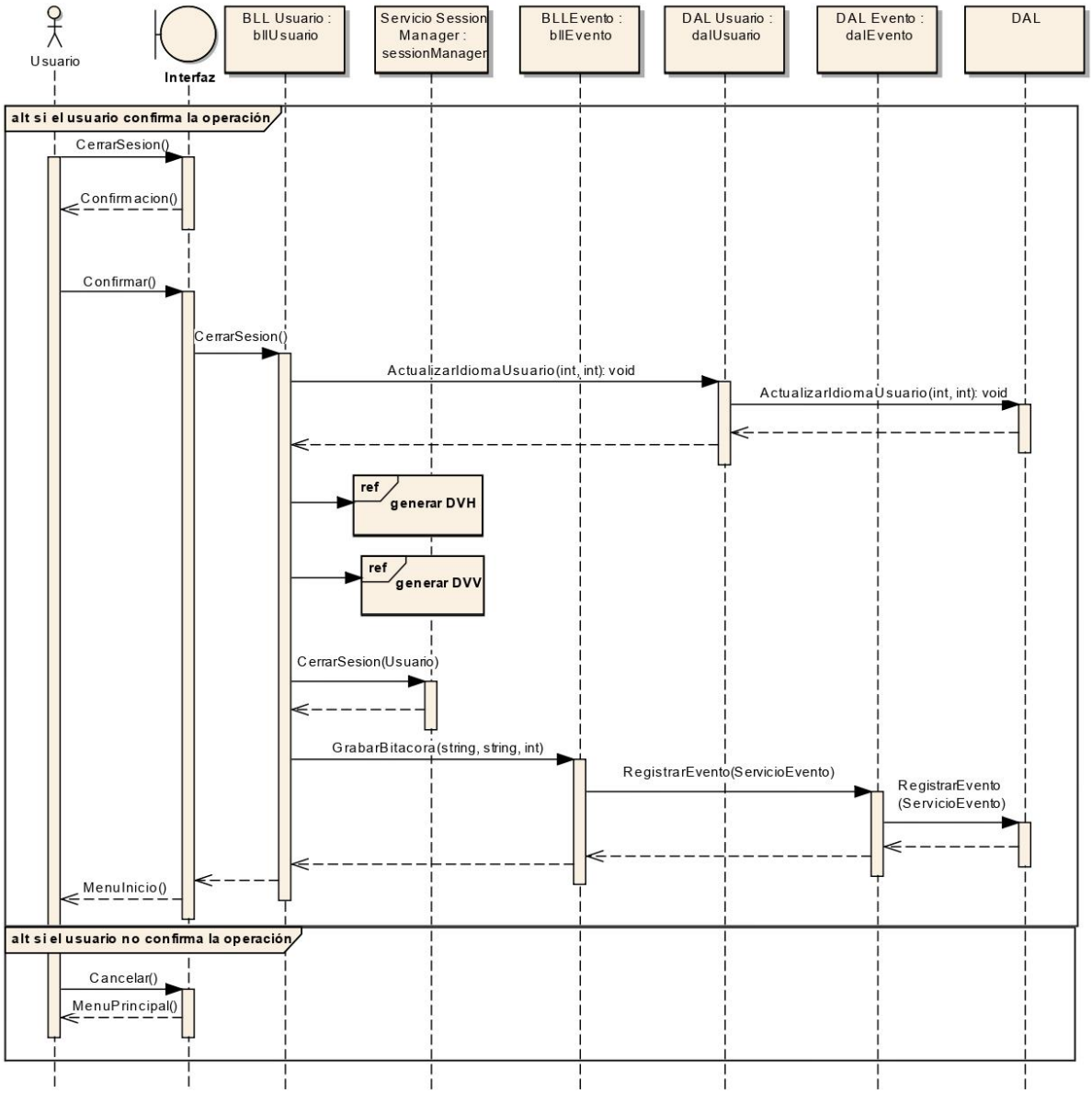
CU6 - CERRAR SESIÓN

Descripción:	El sistema permite al usuario salir de su sesión activa, registrando en la base de datos la preferencia de idioma elegida por el usuario, restringiendo nuevamente los accesos a los módulos de la tienda y retornando a la pantalla de Login.
Actores:	Usuario
Precondiciones:	1. El usuario debe contar con una sesión activa registrada en el Session Manager.

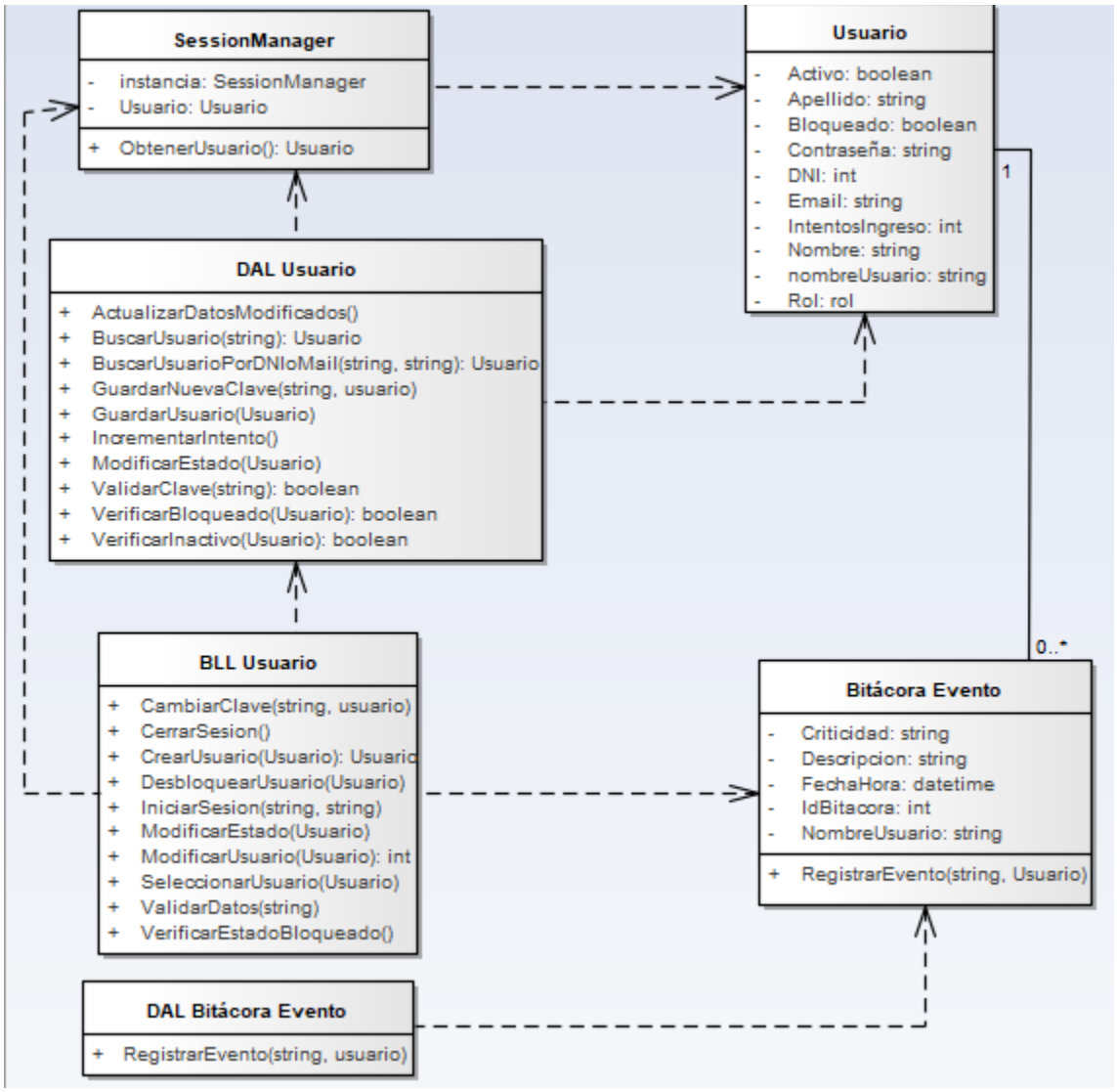
	2. El sistema debe encontrarse operativo y ejecutando la interfaz principal.
Puntos de extensión:	N/A
Flujo Principal	
1	El usuario selecciona el botón de “Cerrar Sesión” en la barra del menú principal.
2	El sistema muestra una ventana emergente de confirmación de salida.
3	El usuario confirma la acción presionando el botón "Sí".
4	El sistema obtiene a través del SessionManager los datos del usuario activo y el código de cultura actualmente seleccionado para el idioma.
5	El sistema envía el identificador de idioma a la capa BLL, la cual actualiza de forma persistente el registro en la tabla Usuario.

6	El sistema recalcula automáticamente los Dígitos Verificadores horizontales y verticales al modificar la base de datos.
7	El sistema destruye la instancia del usuario, asignando el objeto como nulo dentro del SessionManager.
8	El sistema registra el evento de cierre de sesión dentro de la bitácora de eventos.
9	El sistema cierra la pantalla del menú principal y despliega la pantalla de inicio de sesión.
Flujo Alternativo	
3.1.1	El usuario cancela la acción presionando el botón "No" en la ventana emergente.
3.1.2	El sistema descarta la solicitud de cierre de sesión y retorna al control del menú principal.
Poscondición:	La preferencia de idioma se guarda de forma persistente en la base de datos, la sesión del usuario en memoria se destruye y se impide el uso de las funciones hasta un nuevo inicio de sesión.

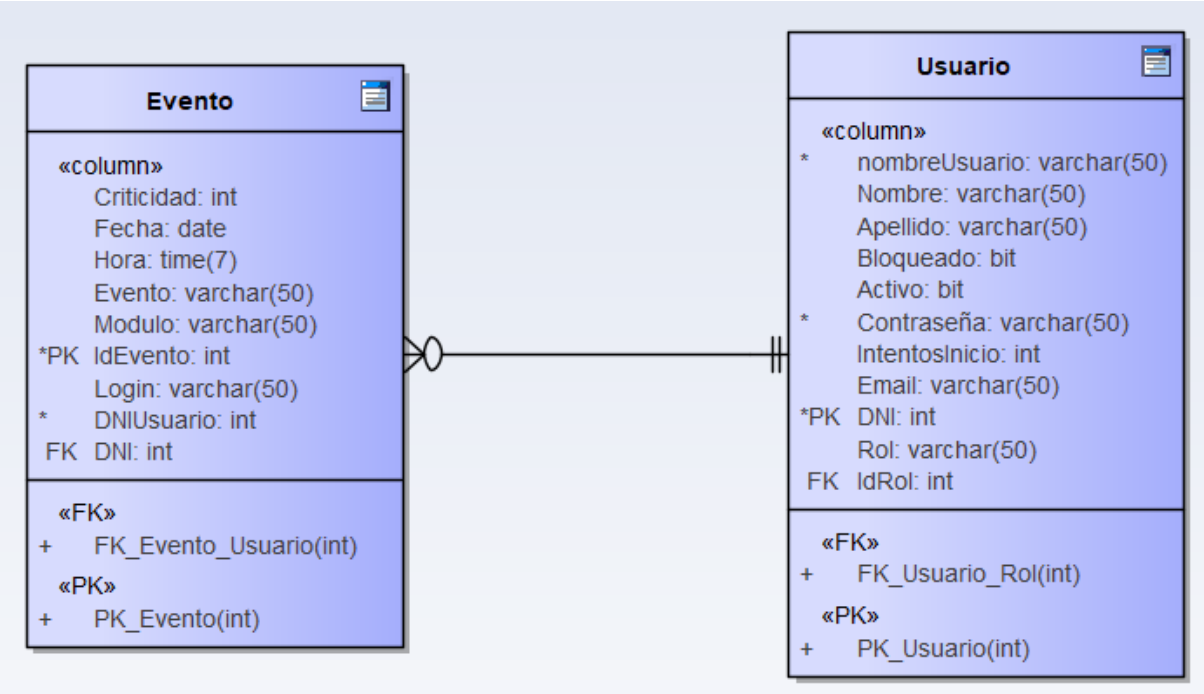
6.4. Diagrama de Secuencia



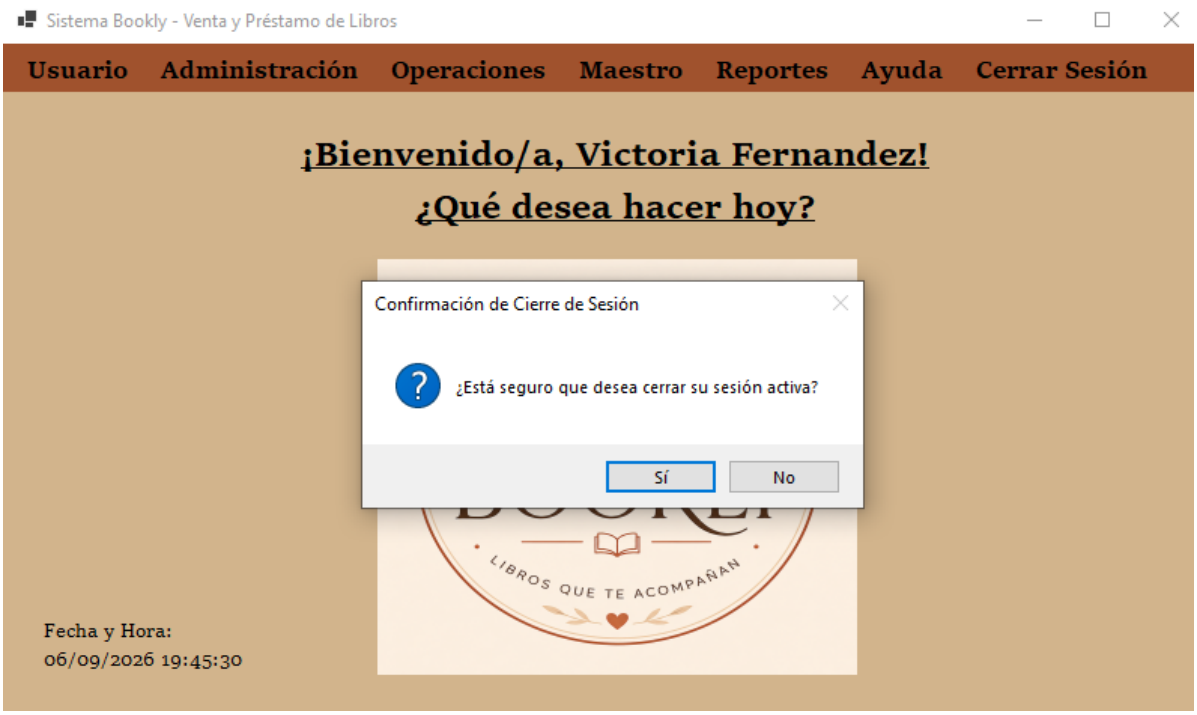
6.5. Diagrama de Clase



6.6. Diagrama Entidad Relación


Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
* DNIUsuario: int
FK DNI: int
«FK»
+ FK_Evento_Usuario(int)
«PK»
+ PK_Evento(int)
Usuario
«column»
* nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
* Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
*PK DNI: int
Rol: varchar(50)
FK IdRol: int
«FK»
+ FK_Usuario_Rol(int)
«PK»
+ PK_Usuario(int)

6.7. GUI

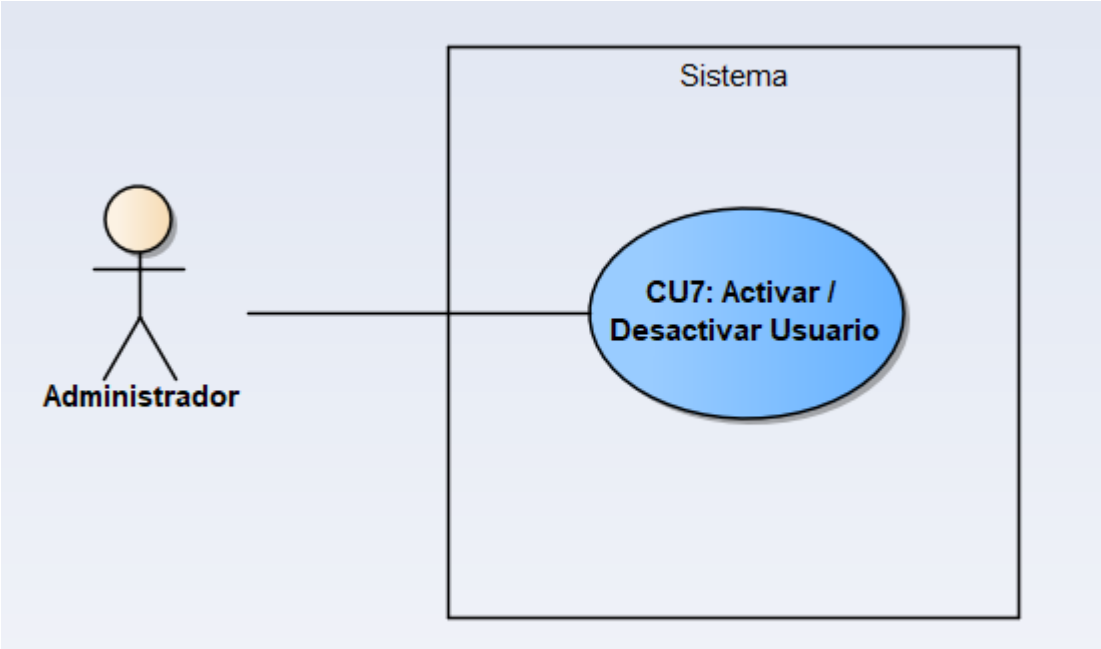

Sistema Bookly - Venta y Préstamo de Libros
Usuario
Administración
Operaciones
Maestro
Reportes
Ayuda
Cerrar Sesión
¡Bienvenido/a, Victoria Fernandez!
¿Qué desea hacer hoy?
Confirmación de Cierre de Sesión
¿Está seguro que desea cerrar su sesión activa?
Sí
No
LIBROS QUE TE ACOMPAÑAN
Fecha y Hora:
06/09/2026 19:45:30

T02.7. Diseño Activar/Desactivar Usuario

7.1. Descomposición Funcional CU7 “Activar/Desactivar Usuario”.

1. El usuario administrador ingresa al sistema con sus credenciales.
2. El sistema muestra la pantalla de menú principal.
3. El administrador selecciona el botón de “Usuarios” en el apartado de “Administración”.
4. El sistema muestra el módulo de gestión de usuarios con el listado de las cuentas registradas y su estado actual (Activo/Inactivo).
5. El administrador selecciona un usuario de la grilla y presiona el botón "Activar / Desactivar".
6. El sistema valida que el usuario seleccionado no sea la misma cuenta que se encuentra actualmente logueada en la sesión para evitar el autobloqueo.
7. El sistema actualiza la base de datos invirtiendo el estado lógico del usuario seleccionado.
8. El sistema recalcula los dígitos verificadores en la base de datos.
9. El sistema registra la acción en la bitácora de eventos guardando el detalle de la cuenta afectada.
10. El sistema actualiza la grilla de la interfaz gráfica mostrando el nuevo estado y despliega un mensaje de confirmación por pantalla.

7.2. Diagrama de Caso de Uso



7.3. Especificación de Caso de Uso

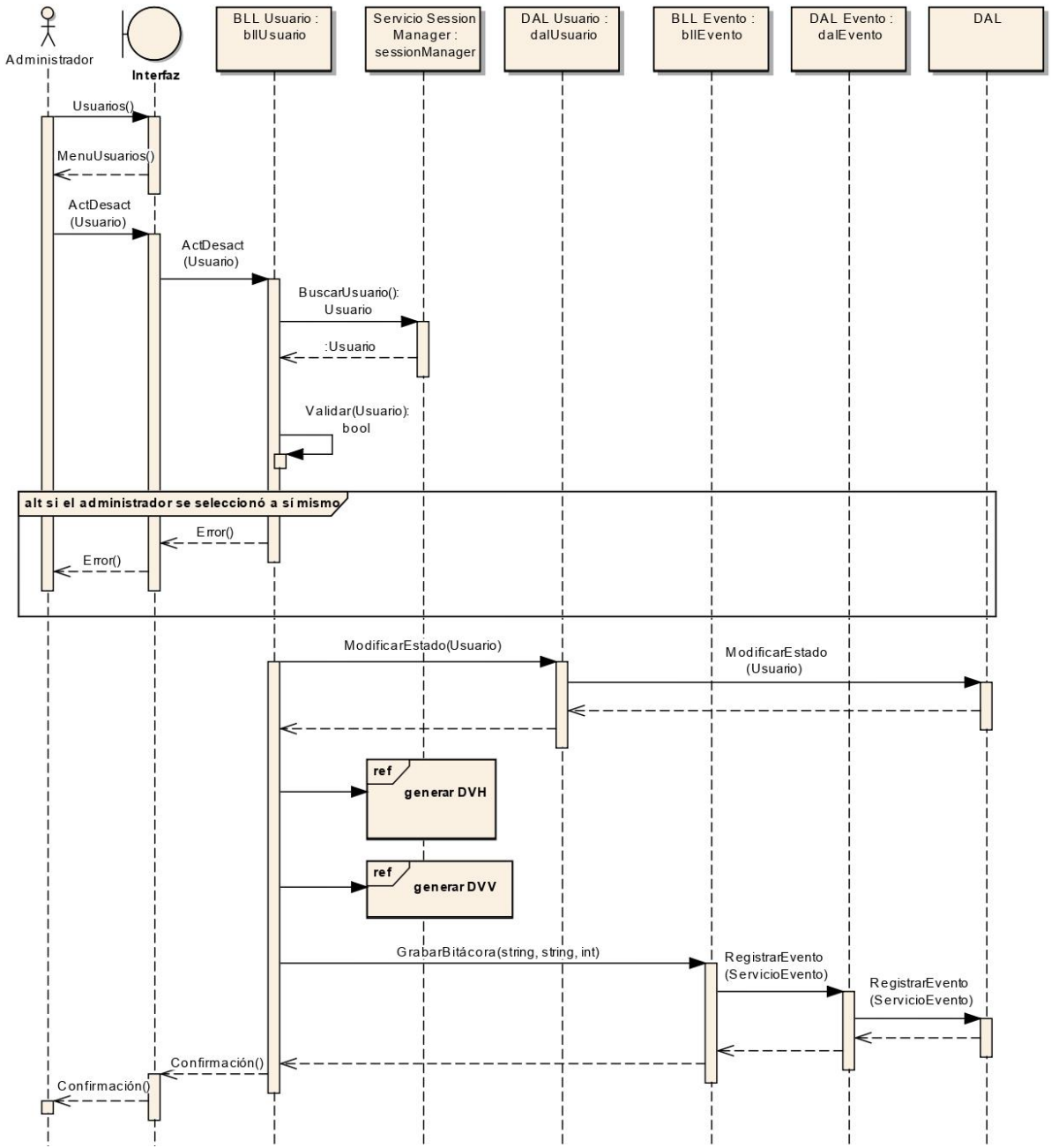
CU7 - ACTIVAR / DESACTIVAR USUARIO

Descripción:	El sistema permite al administrador modificar el estado de habilitación de una cuenta de usuario, aplicando una baja lógica (desactivar) o una reincorporación (activar).
Actores:	Administrador
Precondiciones:	1. El administrador debe contar con una sesión activa en el sistema. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server.

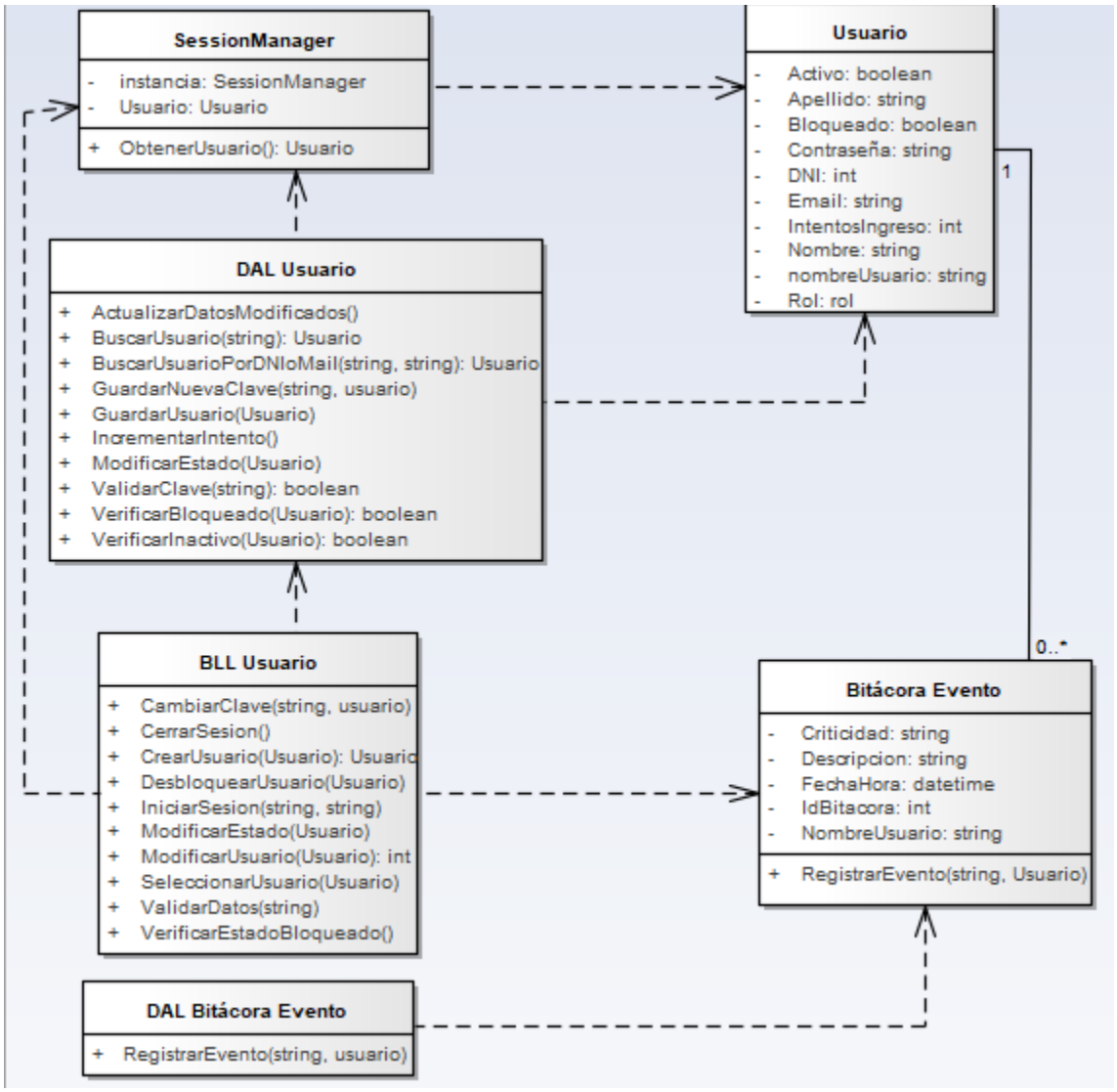
Puntos de extensión:	N/A
Flujo Principal	
1	El administrador selecciona la opción “Usuarios” dentro del apartado de “Administración” en el menú principal.
2	El sistema despliega la interfaz con la grilla que contiene todas las cuentas registradas de la tienda.
3	El administrador selecciona una cuenta de la grilla y presiona el botón "Activar / Desactivar".
4	El sistema valida que el usuario seleccionado no coincida con el usuario activo en el Session Manager.
5	El sistema modifica el registro del usuario seleccionado en SQL Server invirtiendo el valor del campo activo.
6	El sistema recalcula los dígitos verificadores horizontales y verticales luego de actualizar la base de datos.
7	El sistema registra el cambio de estado en la bitácora de eventos

8	El sistema actualiza la grilla visual de la pantalla con los nuevos datos y muestra un mensaje de confirmación.
Flujo Alternativo	
4.1.1	El administrador intentó desactivar su propia cuenta en uso.
4.1.2	El sistema interrumpe el proceso mediante una excepción y cancela la operación.
Poscondición:	El estado del usuario es actualizado en la base de datos, automáticamente impidiendo su inicio de sesión si la cuenta fue desactivada.

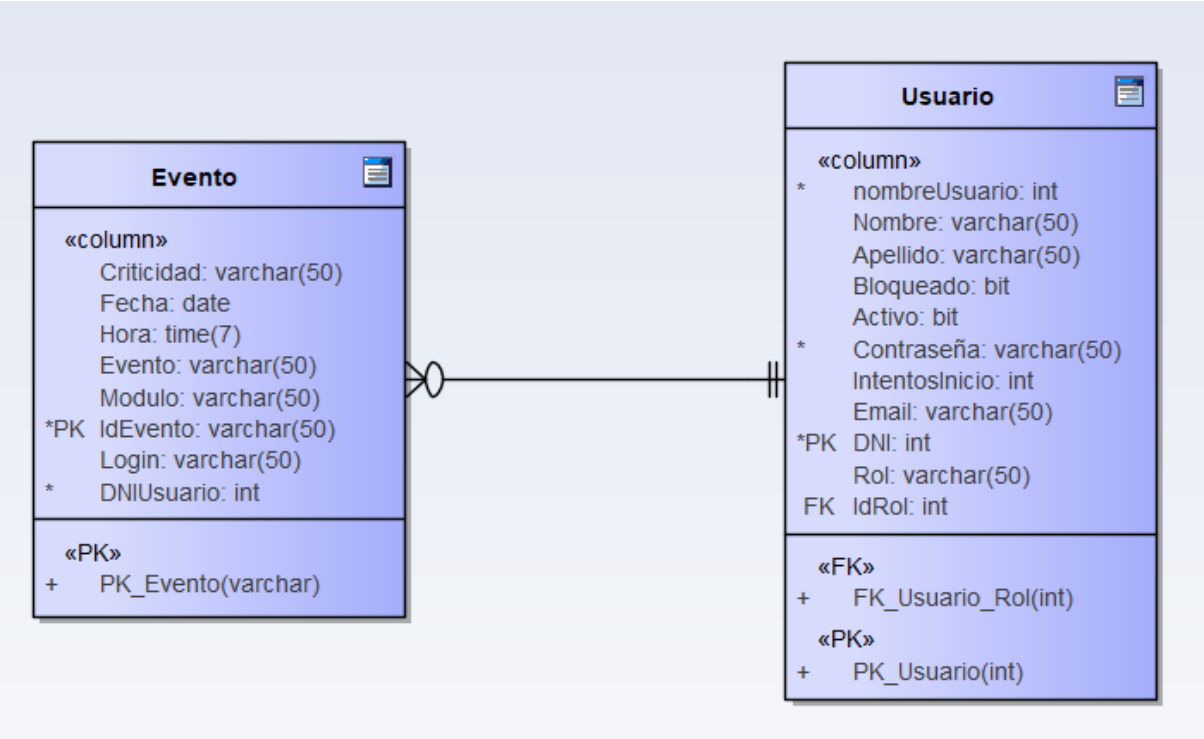
7.4. Diagrama de Secuencia



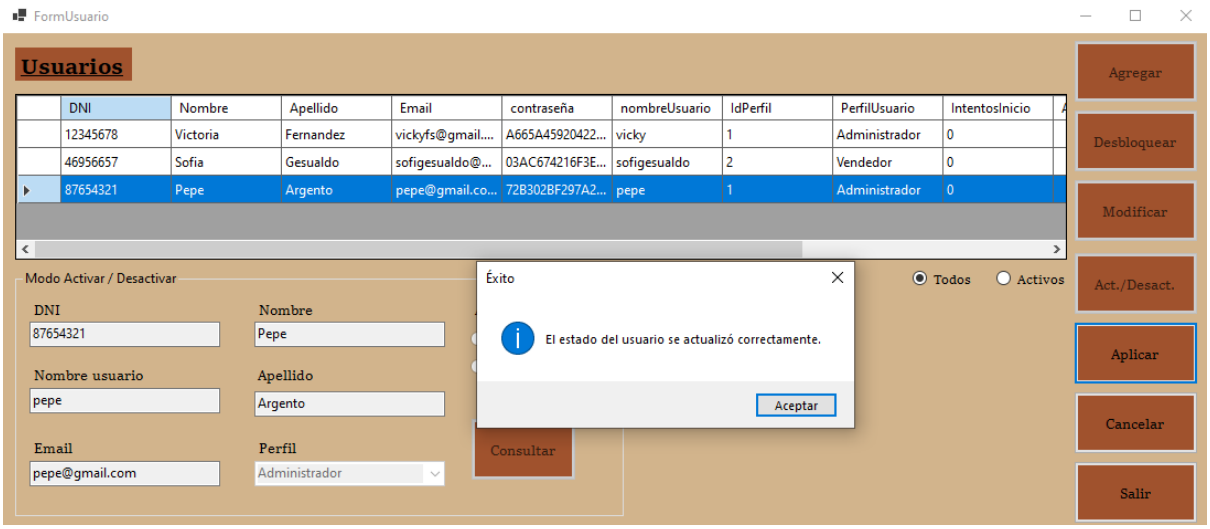
7.5. Diagrama de Clase



7.6. Diagrama Entidad Relación



7.7. GUI



T03. Encriptación

1. Encriptación Irreversible (Hashing)

La encriptación irreversible, conocida técnicamente como Hashing, es un proceso criptográfico que toma un flujo de datos de entrada de cualquier tamaño y lo transforma en una cadena de caracteres de longitud fija (denominada hash). Su propiedad fundamental es la unidireccionalidad: el proceso está diseñado matemáticamente para ser una función de un solo sentido, lo que significa que es imposible revertir el hash obtenido para recuperar los datos originales.

Aplicación: Seguridad de Contraseñas

En el ámbito de la seguridad, el hashing es el estándar obligatorio para el manejo de contraseñas. El sistema nunca debe almacenar las contraseñas en texto plano. En su lugar, cuando un usuario se registra, el sistema calcula el hash de su contraseña y lo almacena en la base de datos. Durante el inicio de sesión, el sistema calcula el hash de la contraseña ingresada y lo compara con el almacenado; si coinciden, se otorga el acceso. De este modo, si la base de datos se ve comprometida, las contraseñas reales de los usuarios permanecen protegidas.

Características Principales

- *Longitud de Salida Fija*: el hash resultante siempre tiene la misma cantidad de caracteres independientemente de la longitud del texto de entrada.
- *Determinismo Absoluto*: Una misma entrada siempre producirá el mismo hash. Esto sirve para verificar si una contraseña es correcta comparando los hashes resultantes.
- *Efecto Avalancha*: Un cambio minúsculo en la entrada produce un hash de salida completamente diferente, evitando que se puedan deducir patrones.
- *Resistencia a Colisiones*: El algoritmo está diseñado para que sea matemáticamente improbable que dos datos de entrada diferentes generen el mismo hash de salida.

Algoritmo Elegido: SHA-256 (Secure Hash Algorithm de 256 bits), es la técnica más utilizada actualmente en la tecnología blockchain y gestión de contraseñas para garantizar la integridad de los datos. Pertenece a la familia SHA (Secure Hash Algorithm), que es el estándar actual desarrollado por la Agencia de Seguridad Nacional (NSA) de Estados Unidos.

Ejemplo Práctico de Negocio

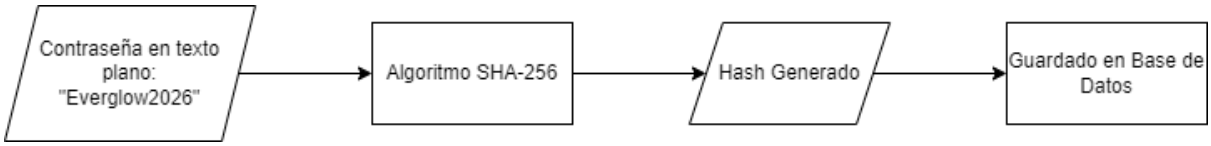
1. Cuando la vendedora Lucía Quiroga ingresa por primera vez al sistema, se le asigna la contraseña temporal en texto plano "**Everglow2026**".
2. El sistema toma la cadena "**Everglow2026**" y lo procesa a través del algoritmo SHA-256.
3. Se genera y guarda en la base de datos el siguiente Hash único de longitud fija:

8D969EEF6ECAD3C29A3A629280E686CF0C3F5D5A86AFF3CA12020C923ADC6C92

4. Demostración del Efecto Avalancha: Si Lucía se equivoca por una minúscula al loguearse y escribe "**everglow2026**", el hash resultante cambia drásticamente a:

4F82BBA827A119D384D1E8F2768FEAA4A1C89874FE10A7426A9E9F97123BCDE1

5. Como los hashes no coinciden, el sistema rechaza el ingreso sin saber qué letra falló.



2. Encriptación Reversible

La encriptación reversible es un proceso criptográfico bidireccional. Esto quiere decir que altera datos legibles mediante un algoritmo matemático para transformarlos en un formato indescifrable (cifrado) y que, a diferencia del hashing, este método permite realizar el proceso inverso (descifrado) para recuperar por completo la información original.

Aplicación: Campos Sensibles del Negocio
Se utiliza para proteger datos confidenciales que el negocio sí necesita leer y procesar en su flujo operativo (por ejemplo: números de documentos de identidad, saldos, tarjetas de crédito, etc.). Los datos se almacenan cifrados en la base de datos para protegerlos de accesos no autorizados, pero los módulos autorizados del software pueden descifrarlos en memoria en el momento exacto en que se requiere operar con ellos.

Características Principales

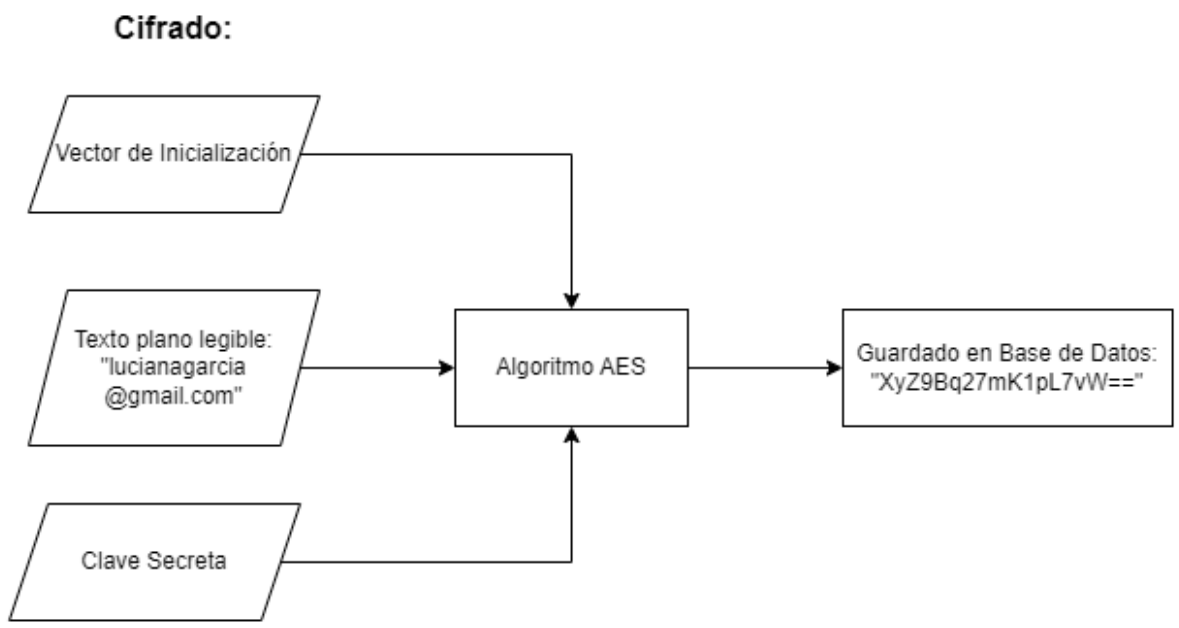
- *Confidencialidad de Datos*: Utiliza la misma clave secreta para cifrar y descifrar, lo que requiere un manejo extremadamente seguro y restringido de dicha clave.
- *Fortaleza del Algoritmo*: El estándar de la industria es AES (Advanced Encryption Standard). Su seguridad radica en la longitud de sus claves, lo que vuelve el cifrado inmune a ataques con la tecnología actual.
- *Vector de Inicialización*: Para evitar que un mismo dato sensible genere siempre el mismo texto cifrado y se puedan deducir patrones, se utiliza un componente aleatorio llamado IV. Esto garantiza que si dos clientes tienen el mismo número de

documento, sus datos cifrados en la base de datos se vean completamente diferentes.

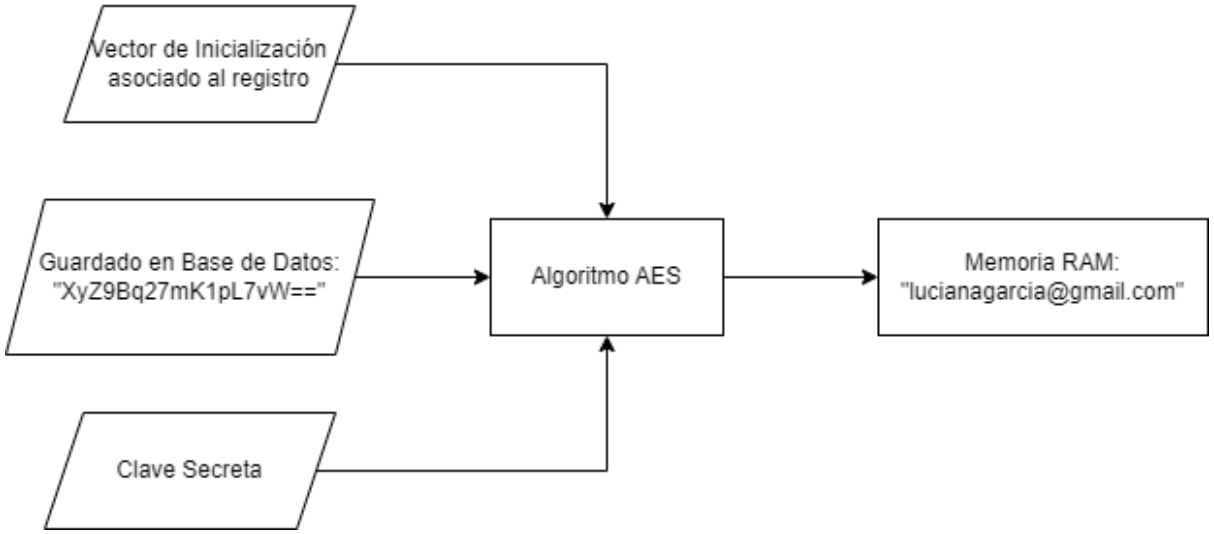
- *Seguridad basada en el Secreto de la Clave*: Sigue el principio de Kerckhoffs: la seguridad del sistema reside exclusivamente en mantener la clave en secreto, y no en ocultar el algoritmo matemático utilizado.

Ejemplo Práctico de Negocio

1. Queremos proteger el correo de una clienta de la tienda: "**lucianagarcia@gmail.com**".
2. Utilizamos la Clave Secreta del sistema, guardada de forma segura en los archivos de configuración del servidor.
3. Agregamos el Vector de Inicialización de esta transacción.
4. Proceso de Cifrado (Guardado en BD): El sistema toma el texto plano, le aplica el algoritmo AES-256 combinado con la Key y el IV, y guarda en la tabla de clientes la siguiente cadena ilegible: **XyZ9Bq27mK1pL7vW==**
5. Proceso de Descifrado (Lectura en Memoria): Cuando el módulo de marketing de Andrea Mitre quiere enviar un catálogo de labiales nuevos, el software toma **XyZ9Bq27mK1pL7vW==** de la base de datos, le aplica la misma Key y el IV, y recupera en la memoria RAM el string original: "**lucianagarcia@gmail.com**".



Descifrado:



T04. Perfiles (Documentación)

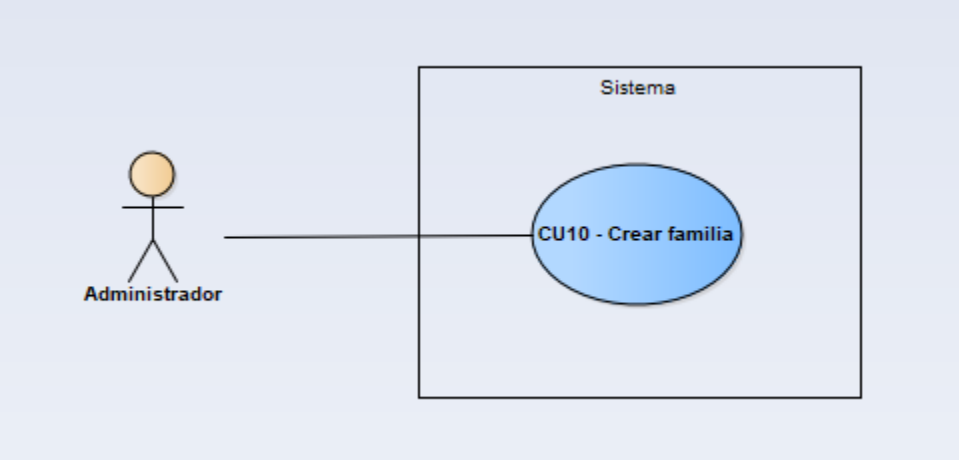
1. Crear Familia

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad. Recibe la selección del apartado "Perfiles" y produce la petición de verificación y lectura hacia el componente de negocio (Capa GUI).
2. El Sistema recupera la totalidad de los componentes configurados (permisos, perfiles y familias) para popular la interfaz. Recibe la orden de lectura y produce las Colecciones de objetos desde la persistencia (Capa DAL).
3. El Sistema procesa el intento de creación disparado por el usuario tras ingresar el nombre y seleccionar los componentes. Recibe los datos del formulario desde la interfaz (Capa GUI).
4. El Sistema valida que el nombre ingresado para la nueva familia no exista previamente en los registros. Recibe el string con el nombre propuesto, ejecuta la subfunción interna de validación de unicidad en el negocio y produce un veredicto booleano de disponibilidad (Capa BLL).

5. El Sistema registra la nueva Familia junto con sus respectivas asociaciones jerárquicas complejas. Recibe la entidad Familia y produce la inserción de los registros correspondientes en la persistencia (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación para el control de seguridad. Recibe la información del evento de creación y del usuario logueado, y produce un nuevo asiento persistido en la Bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso y actualiza la vista estructural. Recibe la confirmación de la persistencia y produce el despliegue del mensaje de éxito junto con el refresco automático del árbol visual (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU10 - CREAR FAMILIA

Descripción:	El sistema permite al Administrador dar de alta una nueva Familia en el sistema, asignándole un nombre único y asociándole de forma obligatoria un conjunto inicial de componentes (permisos, perfiles u otras familias).
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. Debe existir al menos un componente (permiso, perfil o familia) registrado previamente en el sistema para poder ser asignado como hijo.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El Administrador selecciona de la lista los componentes que desea asignar, presiona el botón "Crear familia", y escribe el nombre de la nueva familia.
4	El sistema verifica que el nombre ingresado no exista previamente en el sistema.
5	El sistema registra la nueva Familia con sus asociaciones jerárquicas correspondientes.
6	El sistema registra el evento de creación de familia en la Bitácora.
7	El sistema informa que la familia fue creada correctamente y actualiza el árbol visual
Flujo alternativo:	
4.1.1	El sistema detecta que el nombre ingresado ya se encuentra registrado.
4.1.2	El sistema muestra un mensaje de alerta informando que el nombre de la familia ingresado ya existe.
Poscondición:	La nueva Familia queda registrada en el sistema con sus componentes heredados, impactada en la Bitácora y visible en el listado jerárquico.

Diagrama de Secuencia

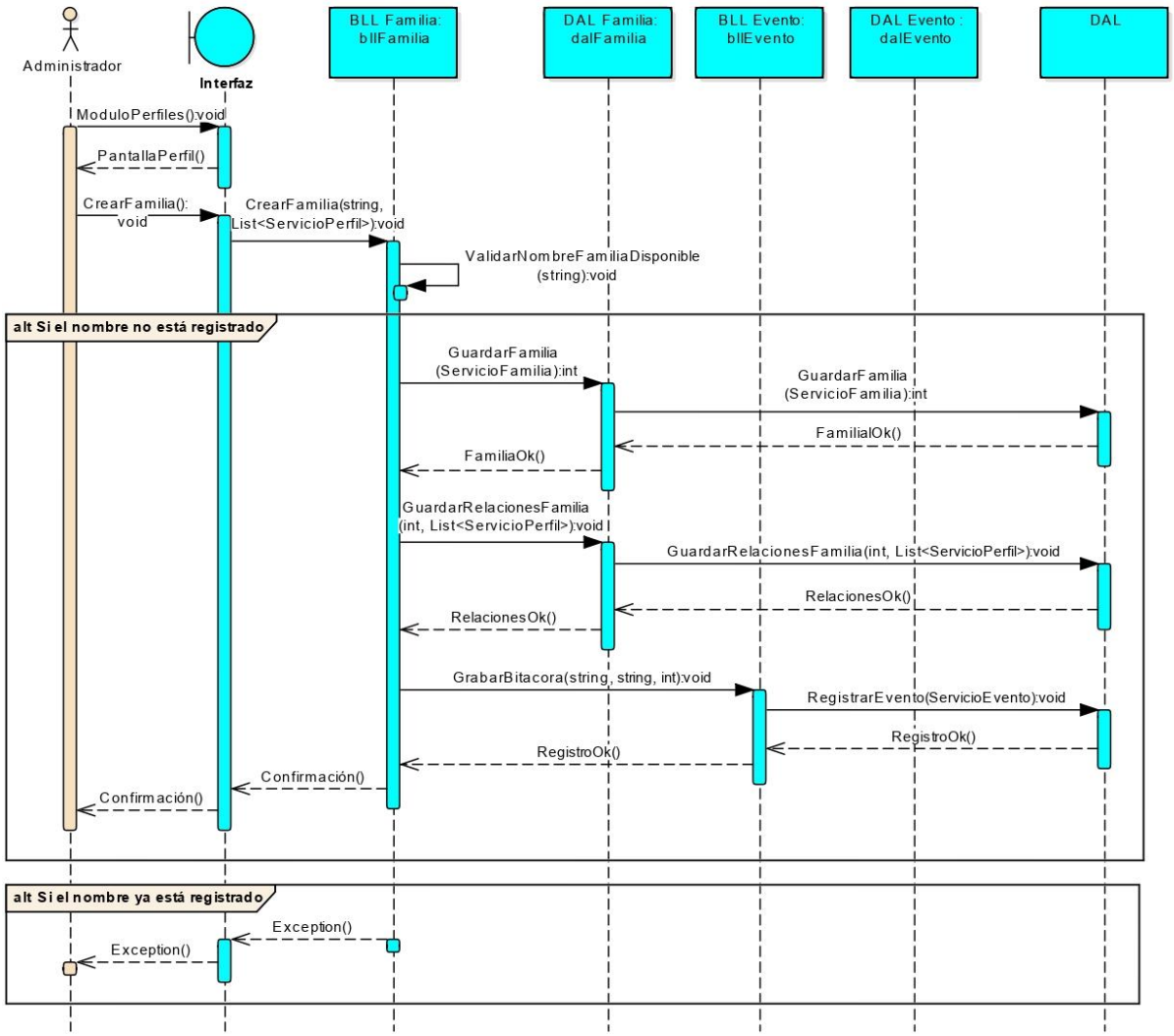


Diagrama de Actividad

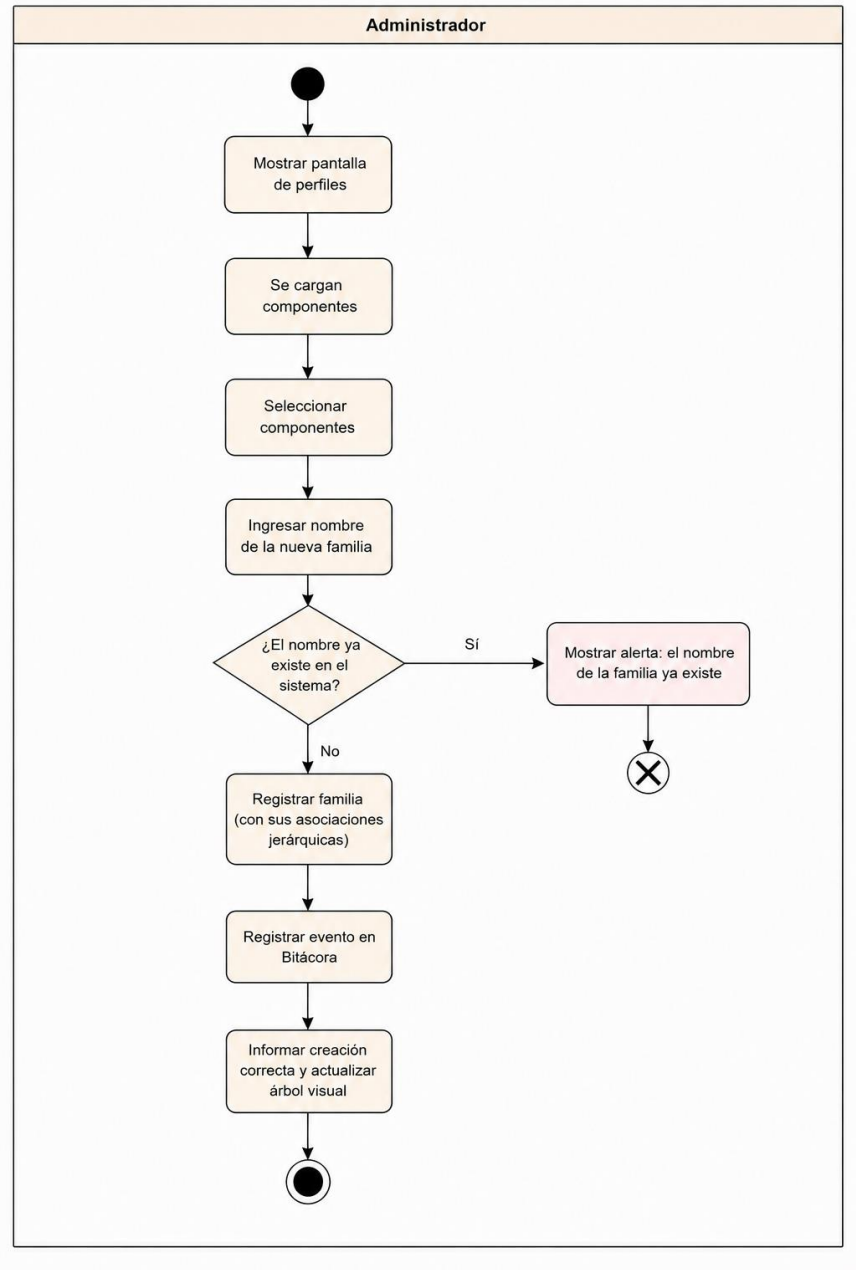
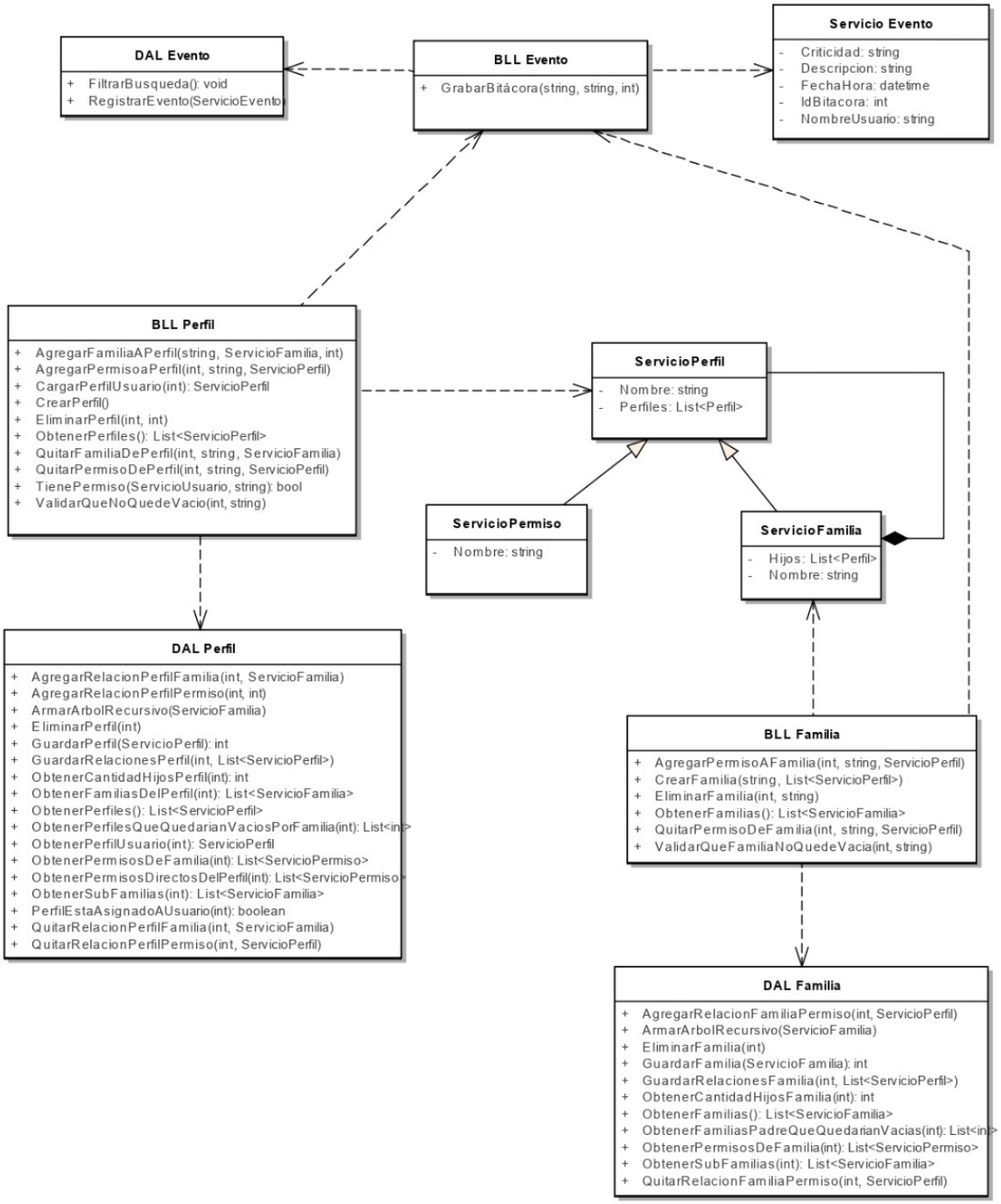


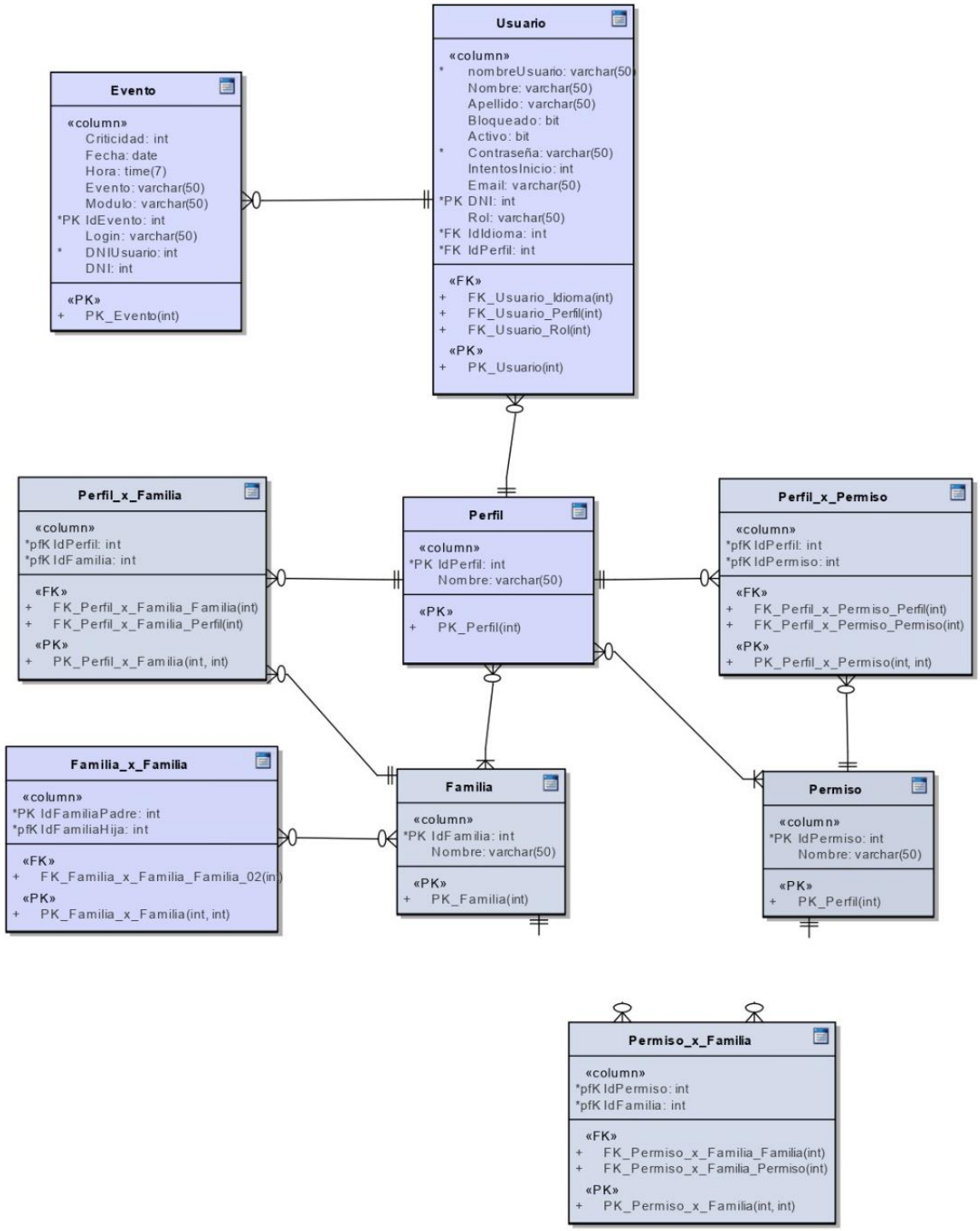
Diagrama de Clases



Nota: El método TienePermiso() utiliza un algoritmo recursivo basado en la estructura jerárquica del patrón Composite para verificar las autorizaciones de un usuario. Cuando se invoca sobre un componente base, el método evalúa su propio identificador; si este coincide con el permiso buscado, retorna verdadero de inmediato (caso base). Si el componente

actual es un objeto compuesto (una Familia), el método itera secuencialmente sobre su colección interna de hijos (listaperfil) y realiza una llamada recursiva a TienePermiso() en cada uno de ellos. Este proceso de ramificación continúa descendiendo por los niveles del árbol hasta encontrar una coincidencia o hasta que todas las hojas (permisos individuales) hayan sido evaluadas, garantizando la herencia automática de accesos a lo largo de toda la estructura de perfiles sin importar su nivel de profundidad.

Diagrama Entidad Relación

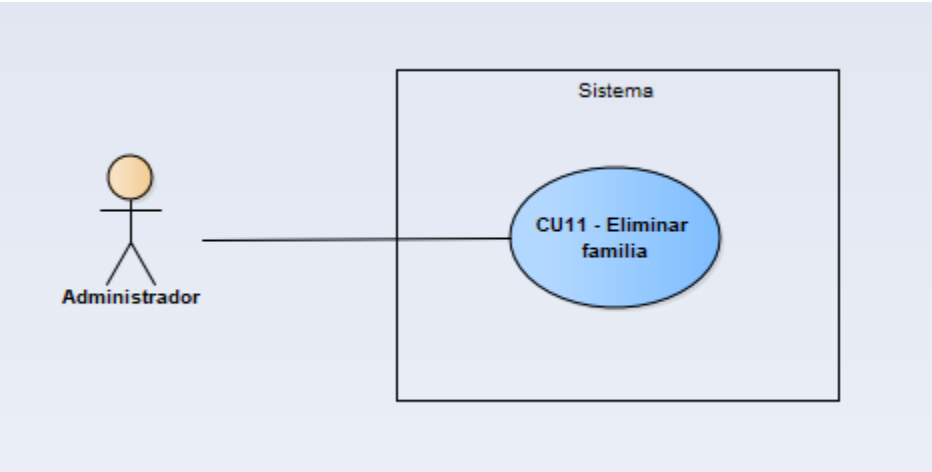

Usuario
«column»
nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
*PK DNI: int
Rol: varchar(50)
*FK IdIdioma: int
*FK IdPerfil: int
«FK»
FK_Usuario_Idioma(int)
FK_Usuario_Perfil(int)
FK_Usuario_Rol(int)
«PK»
PK_Usuario(int)
Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
DNIUsuario: int
DNI: int
«PK»
PK_Evento(int)
Perfil_x_Familia
«column»
*pfK IdPerfil: int
*pfK IdFamilia: int
«FK»
FK_Perfil_x_Familia_Familia(int)
FK_Perfil_x_Familia_Perfil(int)
«PK»
PK_Perfil_x_Familia(int, int)
Perfil
«column»
*PK IdPerfil: int
Nombre: varchar(50)
«PK»
PK_Perfil(int)
Perfil_x_Permiso
«column»
*pfK IdPerfil: int
*pfK IdPermiso: int
«FK»
FK_Perfil_x_Permiso_Perfil(int)
FK_Perfil_x_Permiso_Permiso(int)
«PK»
PK_Perfil_x_Permiso(int, int)
Familia_x_Familia
«column»
*PK IdFamiliaPadre: int
*pfK IdFamiliaHija: int
«FK»
FK_Familia_x_Familia_Familia_02(in
«PK»
PK_Familia_x_Familia(int, int)
Familia
«column»
*PK IdFamilia: int
Nombre: varchar(50)
«PK»
PK_Familia(int)
Permiso
«column»
*PK IdPermiso: int
Nombre: varchar(50)
«PK»
PK_Perfil(int)
Permiso_x_Familia
«column»
*pfK IdPermiso: int
*pfK IdFamilia: int
«FK»
FK_Permiso_x_Familia_Familia(int)
FK_Permiso_x_Familia_Permiso(int)
«PK»
PK_Permiso_x_Familia(int, int)

2. Eliminar Familia

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad. Recibe la selección del apartado "Perfiles" y produce la petición de verificación y lectura hacia el componente de negocio (Capa GUI).
2. El Sistema recupera la totalidad de los componentes configurados (permisos, perfiles y familias) para llenar la interfaz. Recibe la orden de lectura y produce las colecciones de objetos desde la persistencia (Capa DAL).
3. El Sistema procesa la solicitud de baja disparada por el usuario tras seleccionar el elemento a remover. Recibe la identificación de la familia desde la interfaz (Capa GUI).
4. El Sistema remueve de manera definitiva la Familia y elimina en cascada todas sus asociaciones jerárquicas relacionadas. Recibe la orden de borrado de la entidad y produce la actualización correspondiente en la persistencia (Capa DAL).
5. El Sistema genera el registro de auditoría de la operación para el control de seguridad. Recibe la información del evento de eliminación y del usuario logueado, y produce un nuevo asiento persistido en la Bitácora (Capa DAL).
6. El Sistema notifica al usuario el resultado exitoso y actualiza la vista estructural. Recibe la confirmación de la persistencia y produce el despliegue del mensaje de éxito junto con el refresco automático del árbol visual (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU11 - ELIMINAR FAMILIA

Descripción:	El sistema permite al Administrador dar de baja una Familia existente en el sistema, eliminando de forma automática sus relaciones con los componentes hijos y desvinculándola de los perfiles que la contenían.
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. La Familia que se desea eliminar debe existir y estar registrada en el sistema.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El Administrador selecciona de la lista la Familia que desea dar de baja y presiona el botón "Eliminar familia".
4	El sistema remueve la Familia de la base de datos y elimina en cascada todas sus asociaciones jerárquicas.
5	El sistema registra el evento de eliminación de familia en la Bitácora.
6	El sistema informa que la familia fue eliminada correctamente y actualiza el árbol visual.
Flujo alternativo:	N/A
Poscondición:	La Familia seleccionada es removida del sistema junto con sus vínculos, el cambio queda impactado en la Bitácora de auditoría y se refresca el listado jerárquico de la pantalla.

Diagrama de Secuencia

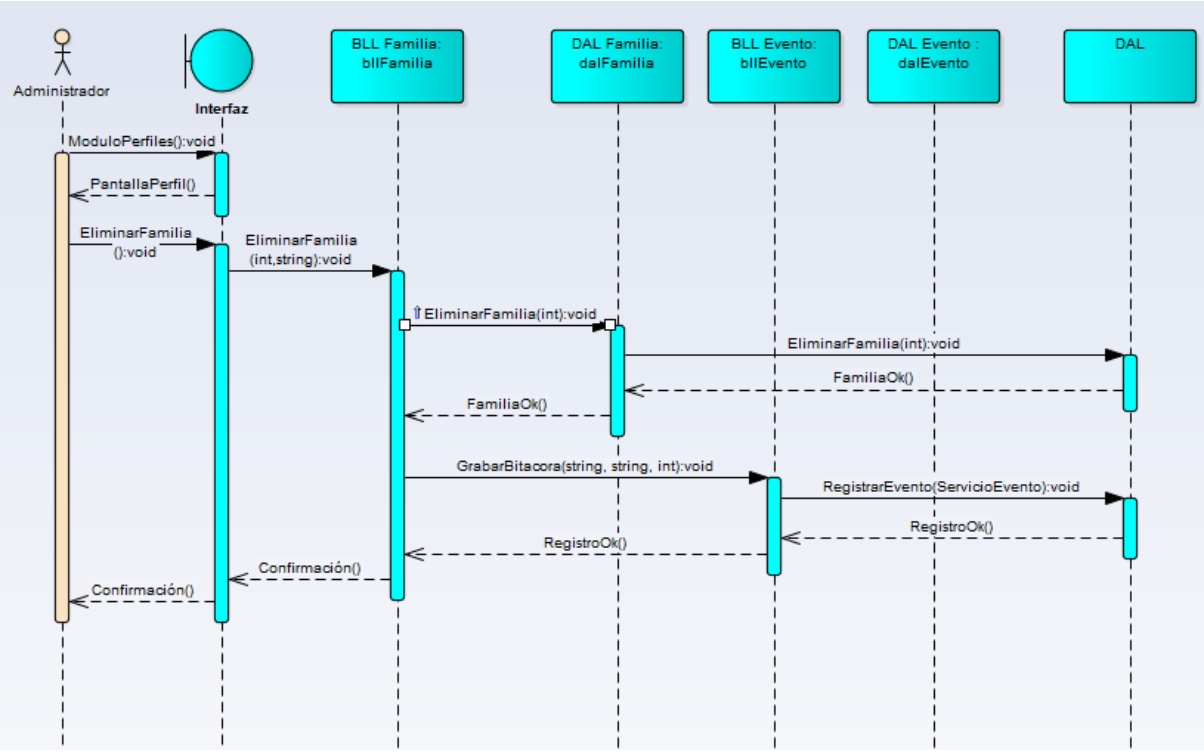


Diagrama de Actividad

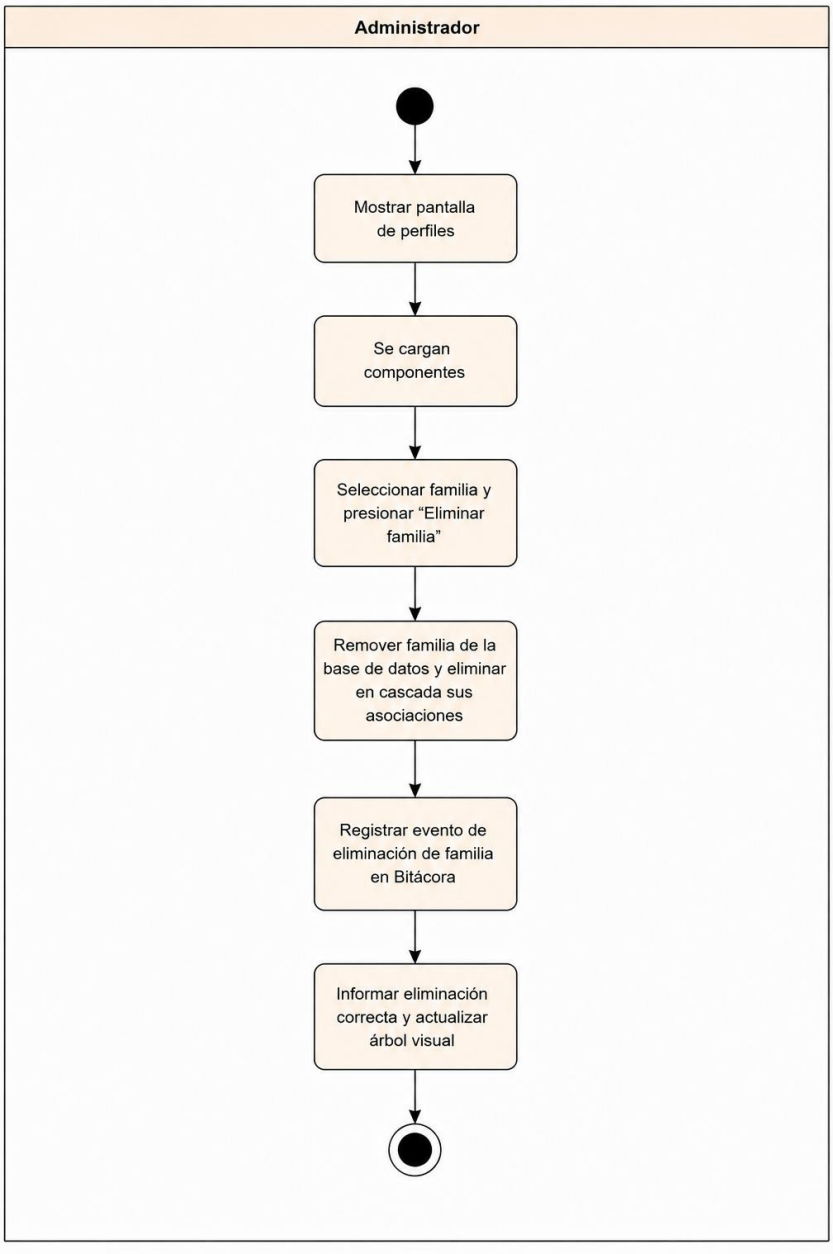
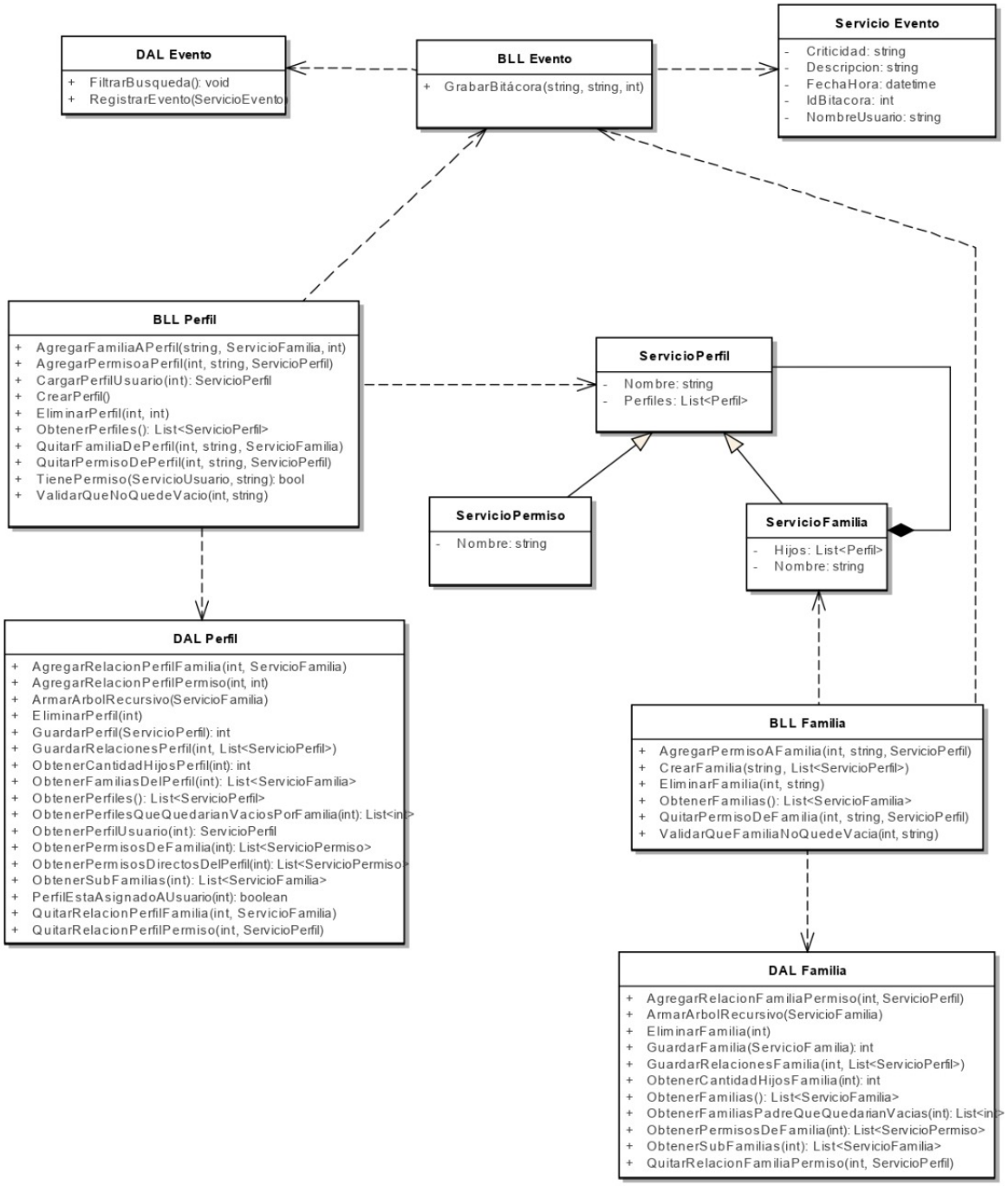


Diagrama de Clases

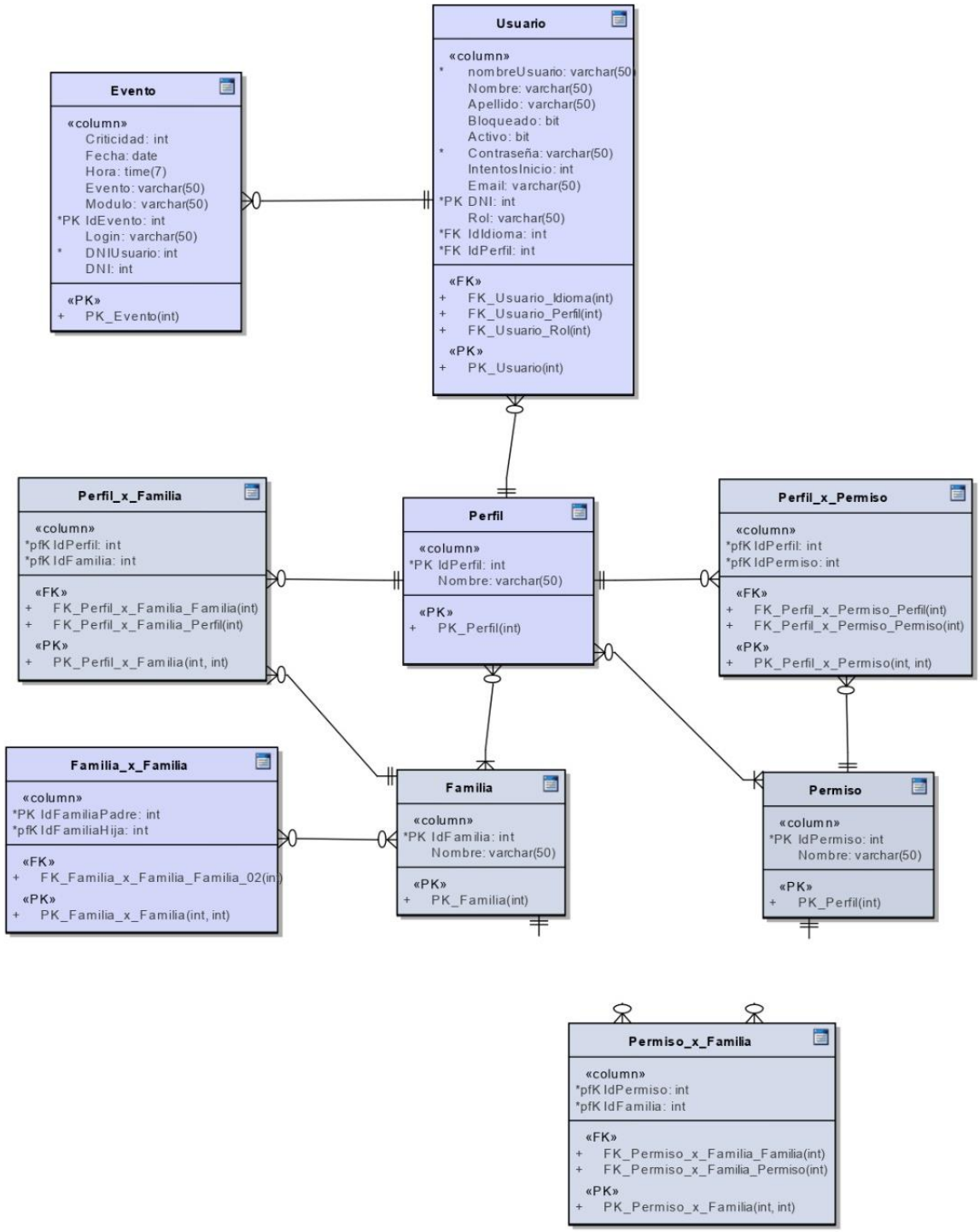
UNIVERSIDAD ABIERTA INTERAMERICANA
Facultad de Tecnología Informática

	Materia: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026



UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Diagrama Entidad Relación

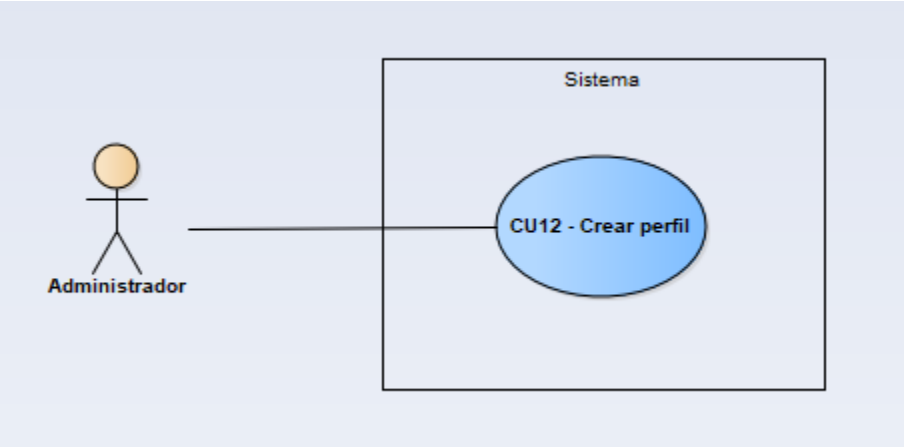


3. Crear Perfil

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad. Recibe la selección del apartado "Perfiles" y produce la petición de verificación y lectura hacia el componente de negocio (Capa GUI).
2. El Sistema recupera la totalidad de los componentes configurados (permisos, perfiles y familias) para popular la interfaz. Recibe la orden de lectura y produce las colecciones de objetos desde la persistencia (Capa DAL).
3. El Sistema procesa el intento de creación disparado por el usuario tras ingresar el nombre y seleccionar los componentes a heredar. Recibe los datos del formulario desde la interfaz (Capa GUI).
4. El Sistema valida que el nombre ingresado para el nuevo perfil no exista previamente en los registros. Recibe el string con el nombre propuesto, ejecuta la subfunción interna de validación de unicidad en el negocio y produce un veredicto booleano de disponibilidad (Capa BLL).
5. El Sistema registra el nuevo Perfil junto con sus respectivas asociaciones y dependencias de herencia. Recibe la entidad Perfil y produce la inserción de los registros correspondientes en la persistencia (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación para el control de seguridad. Recibe la información del evento de creación y del usuario logueado, y produce un nuevo asiento persistido en la Bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso de la operación. Recibe la confirmación de la persistencia y produce el despliegue del mensaje de éxito en la interfaz (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU12 - CREAR PERFIL

Descripción:	El sistema permite al Administrador dar de alta un nuevo Perfil en el sistema, asignándole un nombre único y asociándole de forma obligatoria un conjunto inicial de permisos o familias.
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. Debe existir al menos un permiso o familia registrado previamente en el sistema para poder ser asignado.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El Administrador selecciona de la lista los componentes (permisos/familias) que desea heredarle, presiona el botón "Crear perfil" y escribe el nombre del nuevo perfil.
4	El sistema verifica que el nombre ingresado no exista previamente en el sistema.
5	El sistema registra el nuevo Perfil con sus asociaciones.
6	El sistema registra el evento de creación de perfil en la Bitácora.
7	El sistema informa que el perfil fue creado correctamente.
Flujo alternativo:	
4.1.1	El sistema detecta que el nombre ingresado ya se encuentra registrado.
4.1.2	El sistema muestra un mensaje de alerta informando que el nombre del perfil ingresado ya existe.
Poscondición:	El nuevo Perfil queda registrado en el sistema con sus componentes heredados, impactado en la Bitácora y visible en el listado jerárquico.

Diagrama de Secuencia

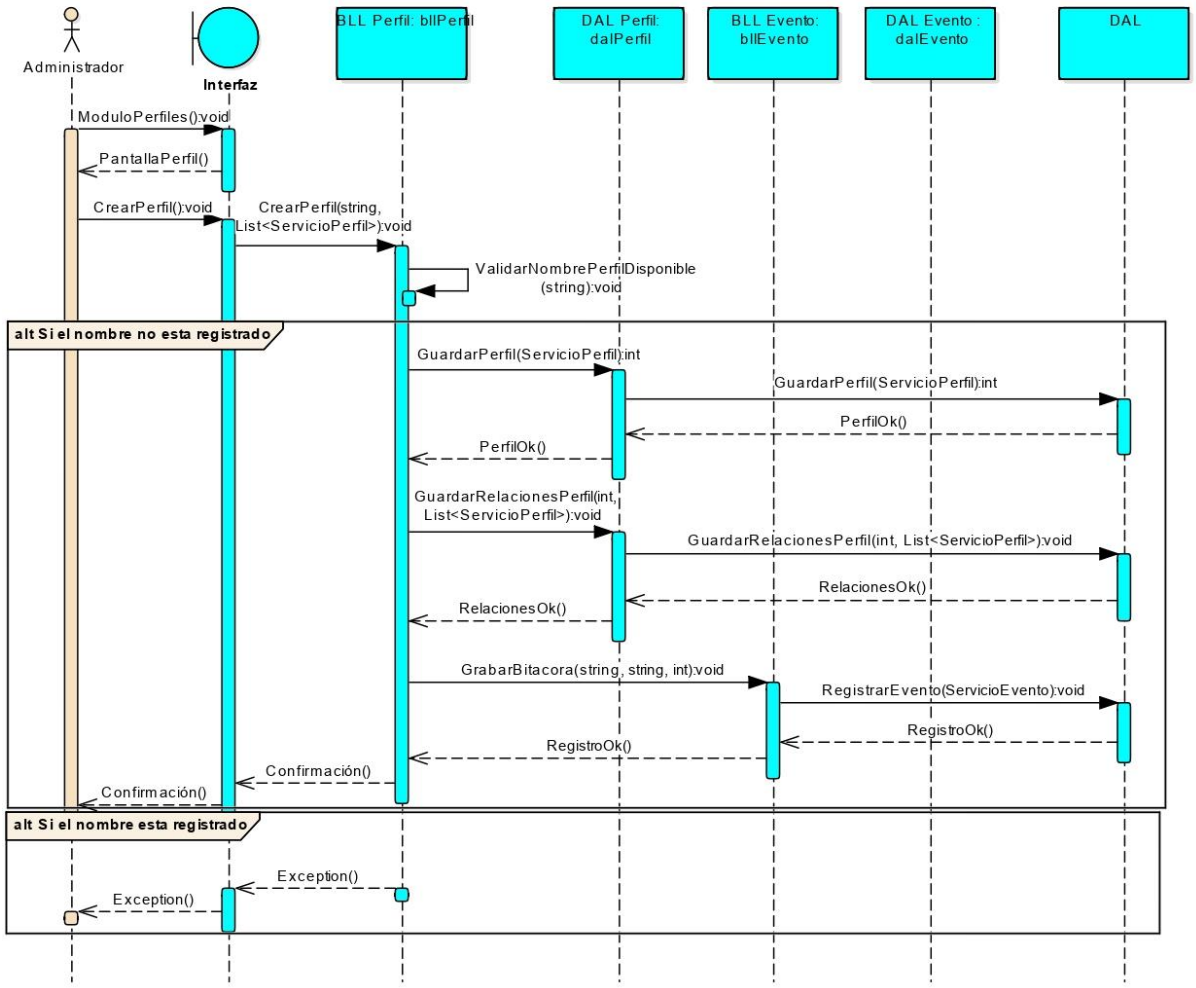


Diagrama de Actividad

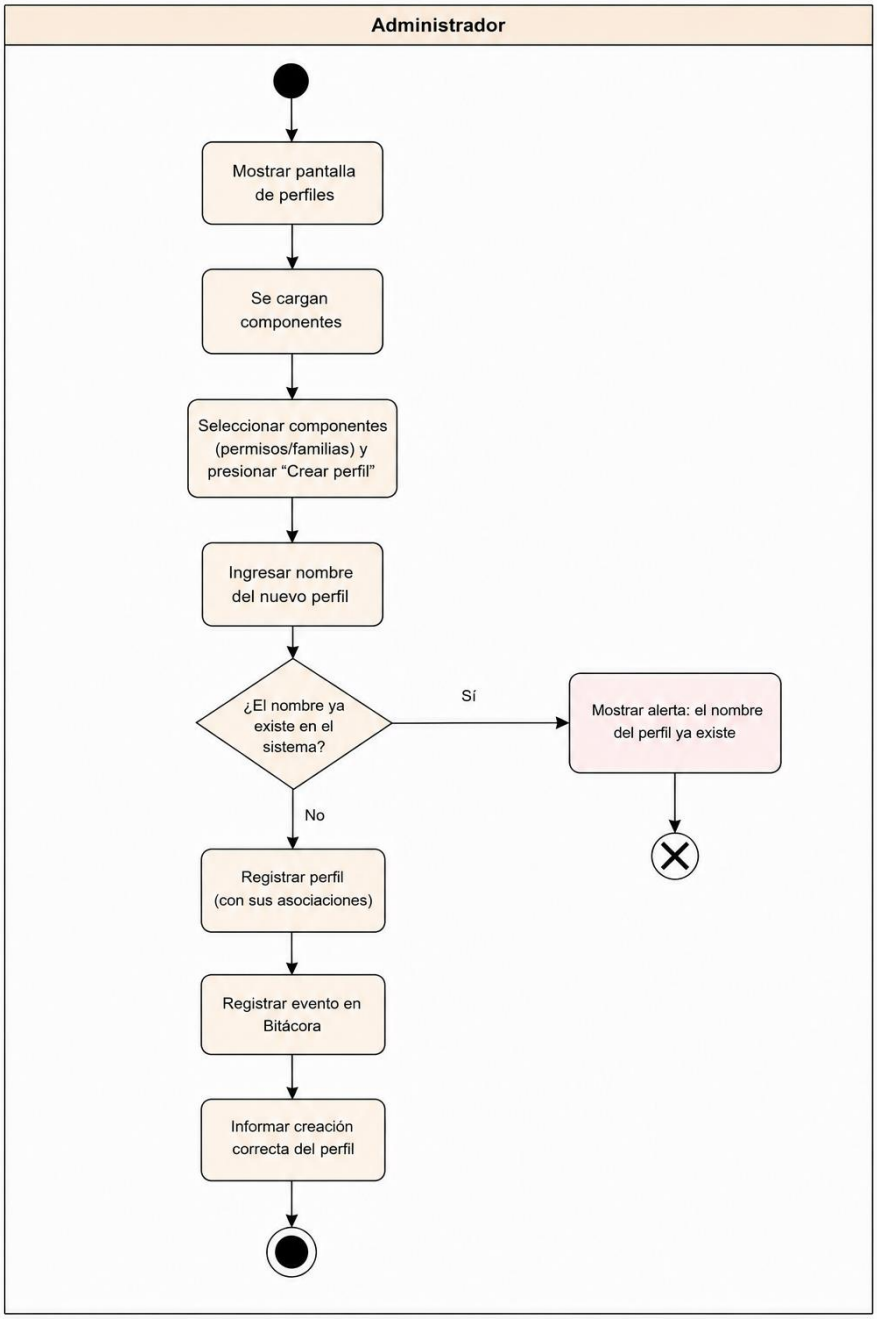


Diagrama de Clases

UNIVERSIDAD ABIERTA INTERAMERICANA
Facultad de Tecnología Informática

Materia: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel	
Alumno: Gesualdo Antonini, Sofía.			
Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

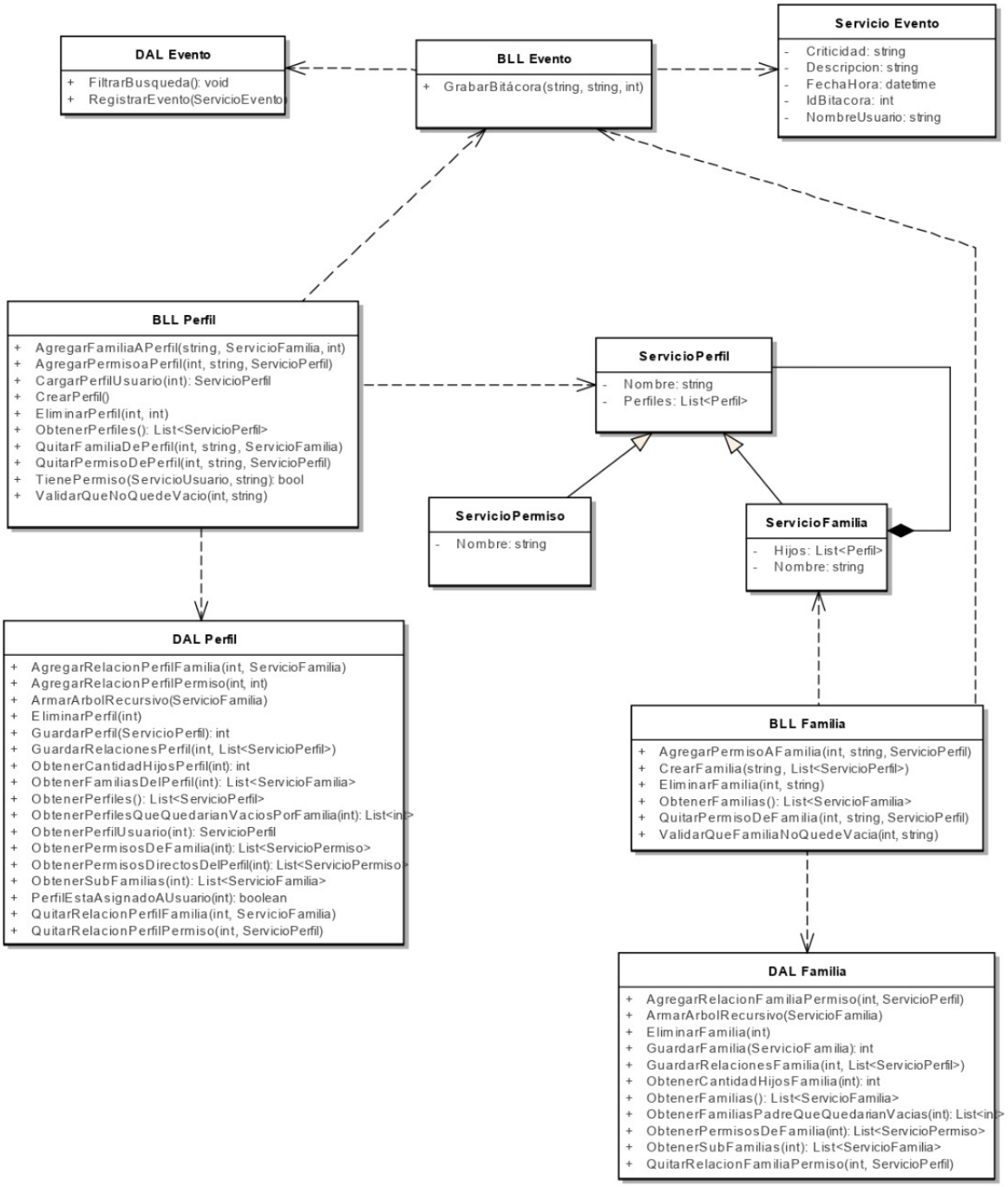
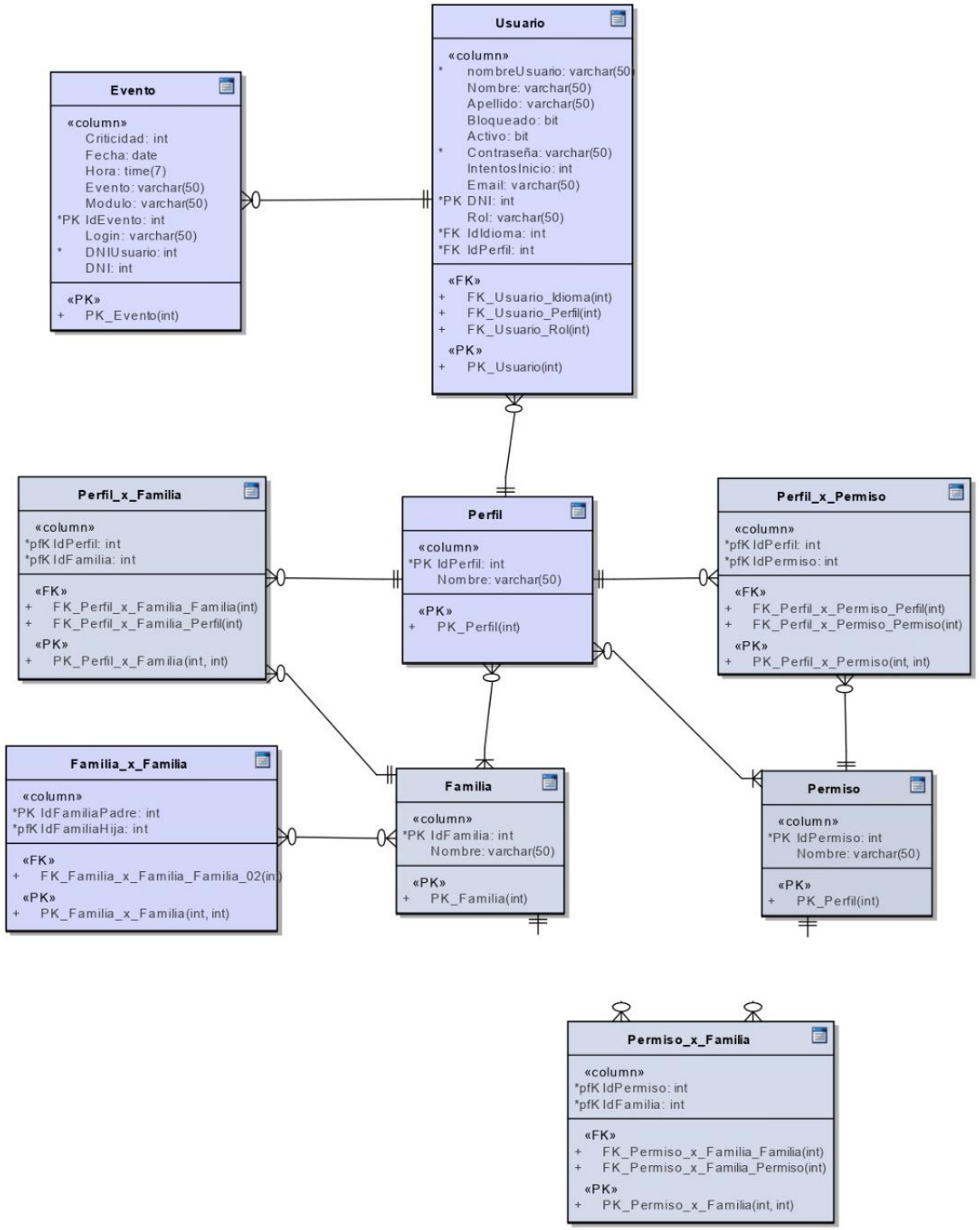


Diagrama Entidad Relación

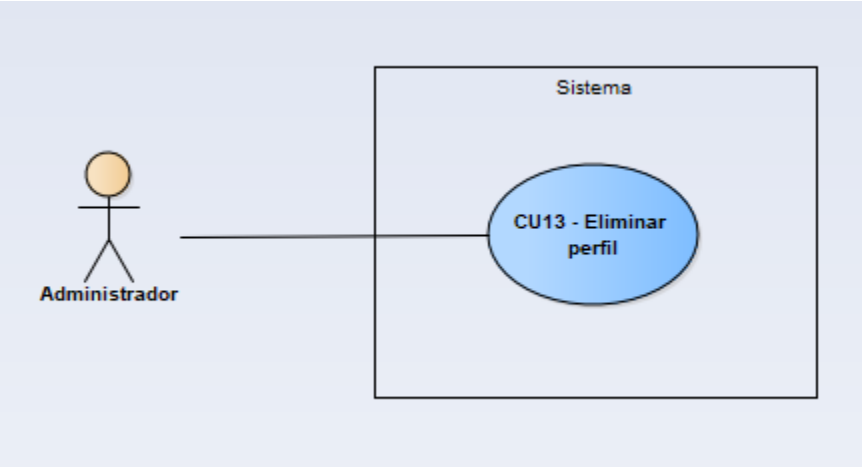


4. Eliminar Perfil

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad. Recibe la selección del apartado "Perfiles" y produce la petición de verificación y lectura hacia el componente de negocio (Capa GUI).
2. El Sistema recupera la totalidad de los componentes configurados (permisos, perfiles y familias) para popular la interfaz. Recibe la orden de lectura y produce las colecciones de objetos desde la persistencia (Capa DAL).
3. El Sistema procesa la solicitud de baja disparada por el usuario tras seleccionar el perfil a remover. Recibe la identificación del perfil desde la interfaz (Capa GUI).
4. El Sistema verifica que el Perfil seleccionado no se encuentre asignado a ningún usuario en el sistema. Recibe el identificador del perfil, ejecuta la subfunción interna de validación de dependencias en el negocio y produce un veredicto booleano de viabilidad para el borrado (Capa BLL y DAL).
5. El Sistema remueve de manera definitiva el Perfil de los registros junto con sus asociaciones relacionales. Recibe la orden de borrado de la entidad validada y produce la actualización correspondiente en la persistencia (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación para el control de seguridad. Recibe la información del evento de eliminación y del usuario logueado, y produce un nuevo asiento persistido en la Bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso y actualiza la vista estructural. Recibe la confirmación de la persistencia y produce el despliegue del mensaje de éxito junto con el refresco automático del árbol visual (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU13 - ELIMINAR PERFIL

Descripción:	El sistema permite al Administrador dar de baja un Perfil existente en el sistema, asegurando previamente que ningún usuario activo lo tenga asignado para mantener la integridad de la persistencia.
Actores:	Administrador
Precondiciones:	4. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 5. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 6. El Perfil que se desea eliminar debe existir y estar registrado en el sistema.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El Administrador selecciona de la lista el Perfil que desea dar de baja y presiona el botón "Eliminar perfil".
4	El sistema verifica que el Perfil seleccionado no se encuentre asignado a ningún usuario en la base de datos.
5	El sistema elimina el Perfil y sus asociaciones relacionales de forma permanente.
6	El sistema registra el evento de eliminación de perfil en la Bitácora.
7	El sistema informa que el perfil fue eliminado correctamente y actualiza el árbol visual.
Flujo alternativo:	
4.1.1	El sistema detecta que existen usuarios vinculados al Perfil que se intenta borrar.
4.1.2	El sistema muestra un mensaje de alerta informando que no se puede eliminar el perfil y que se deben modificar los usuarios asociados antes de proceder.
Poscondición:	El Perfil seleccionado es removido del sistema, el cambio queda impactado en la Bitácora de auditoría y se actualiza el listado jerárquico de la pantalla.

Diagrama de Secuencia

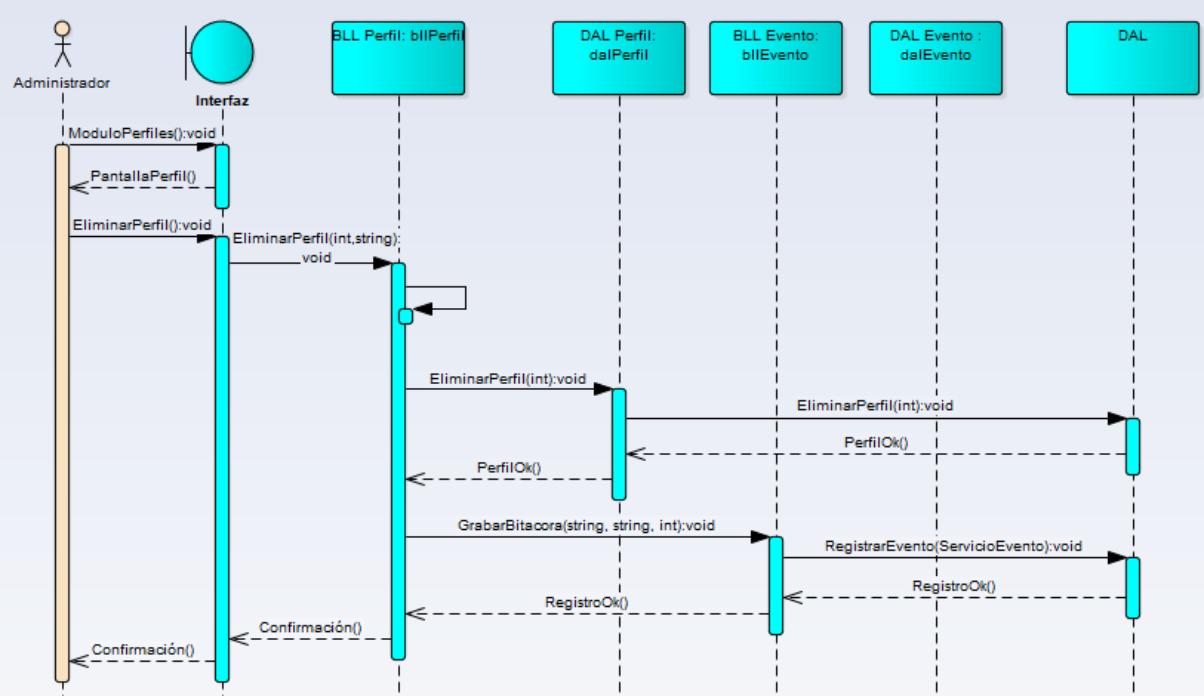


Diagrama de Actividad

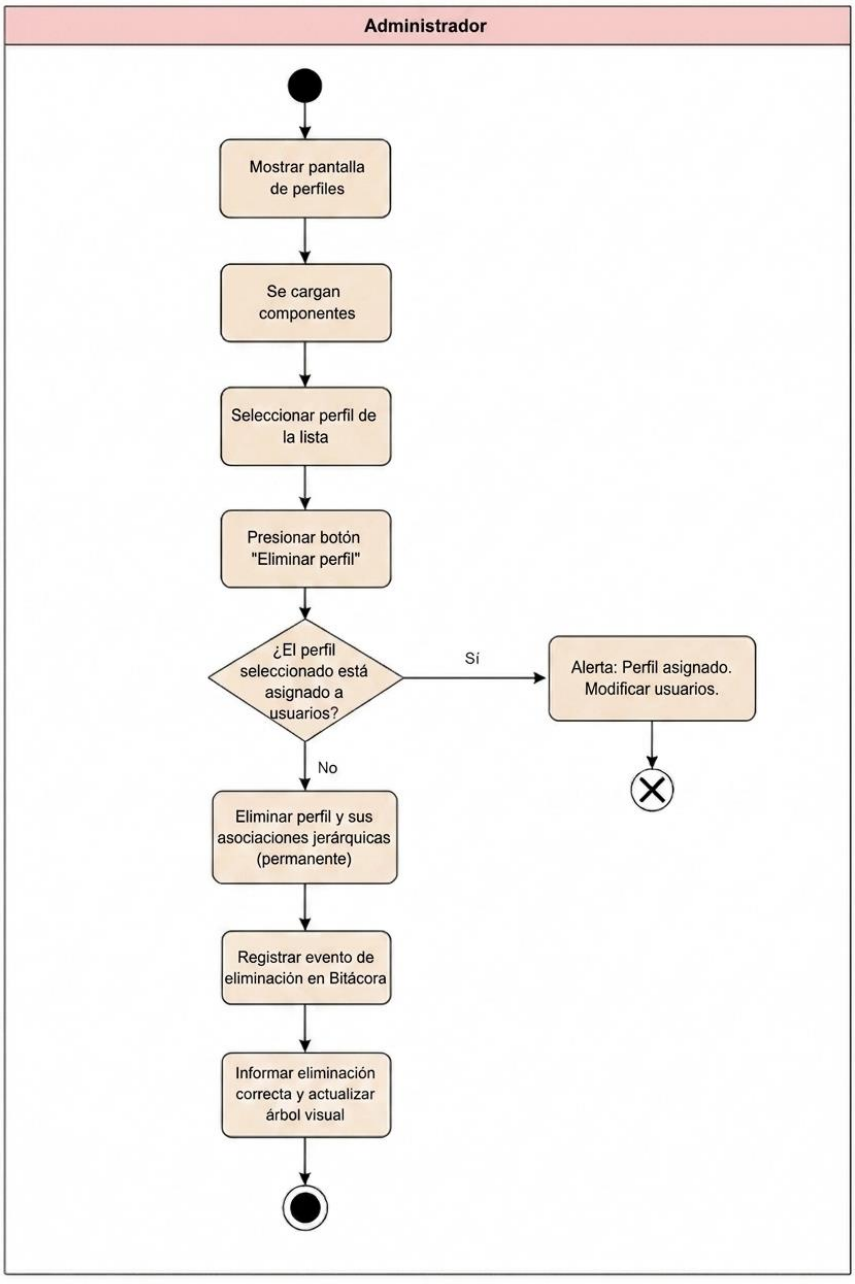


Diagrama de Clases

UNIVERSIDAD ABIERTA INTERAMERICANA
Facultad de Tecnología Informática

	Materia: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

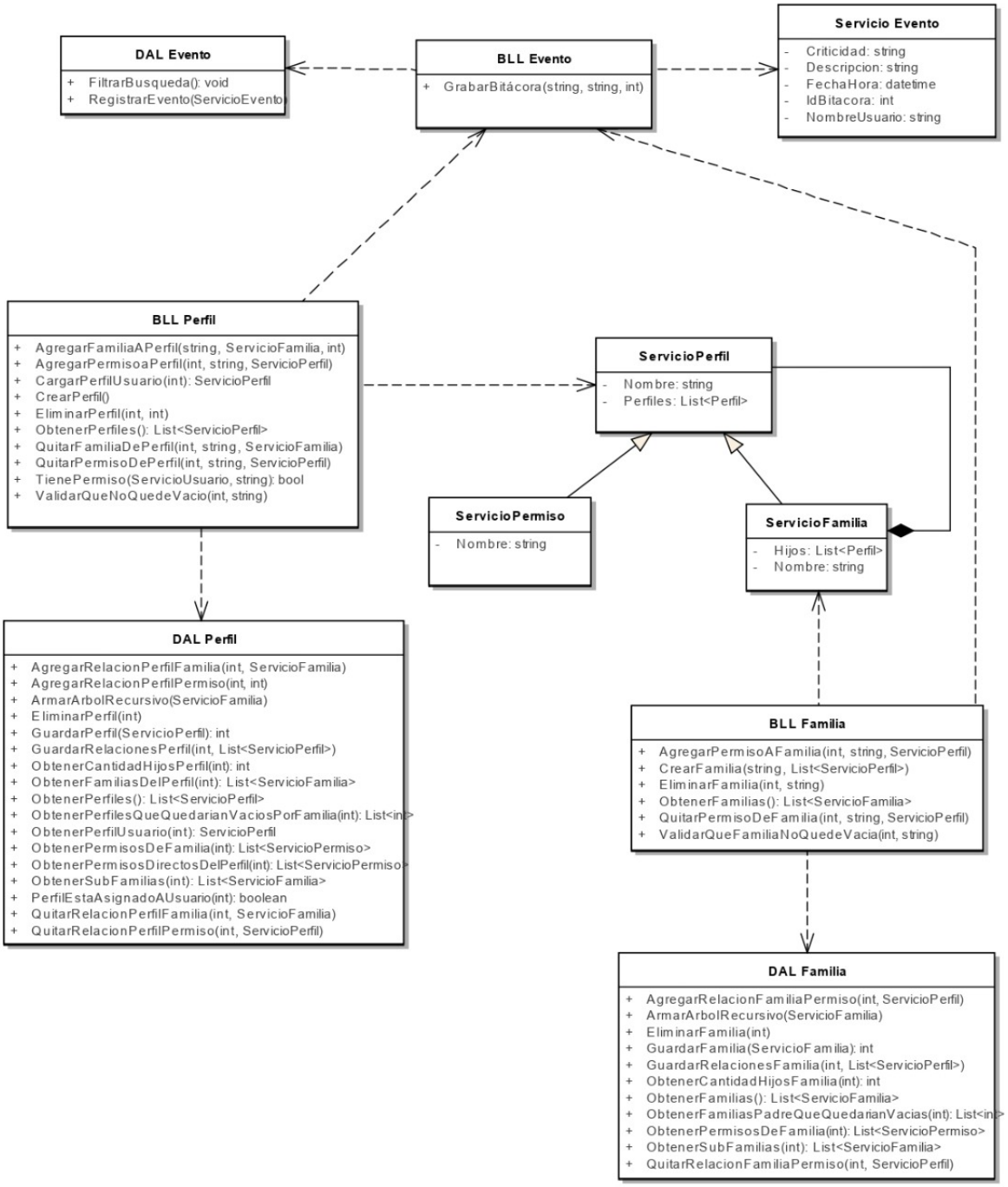
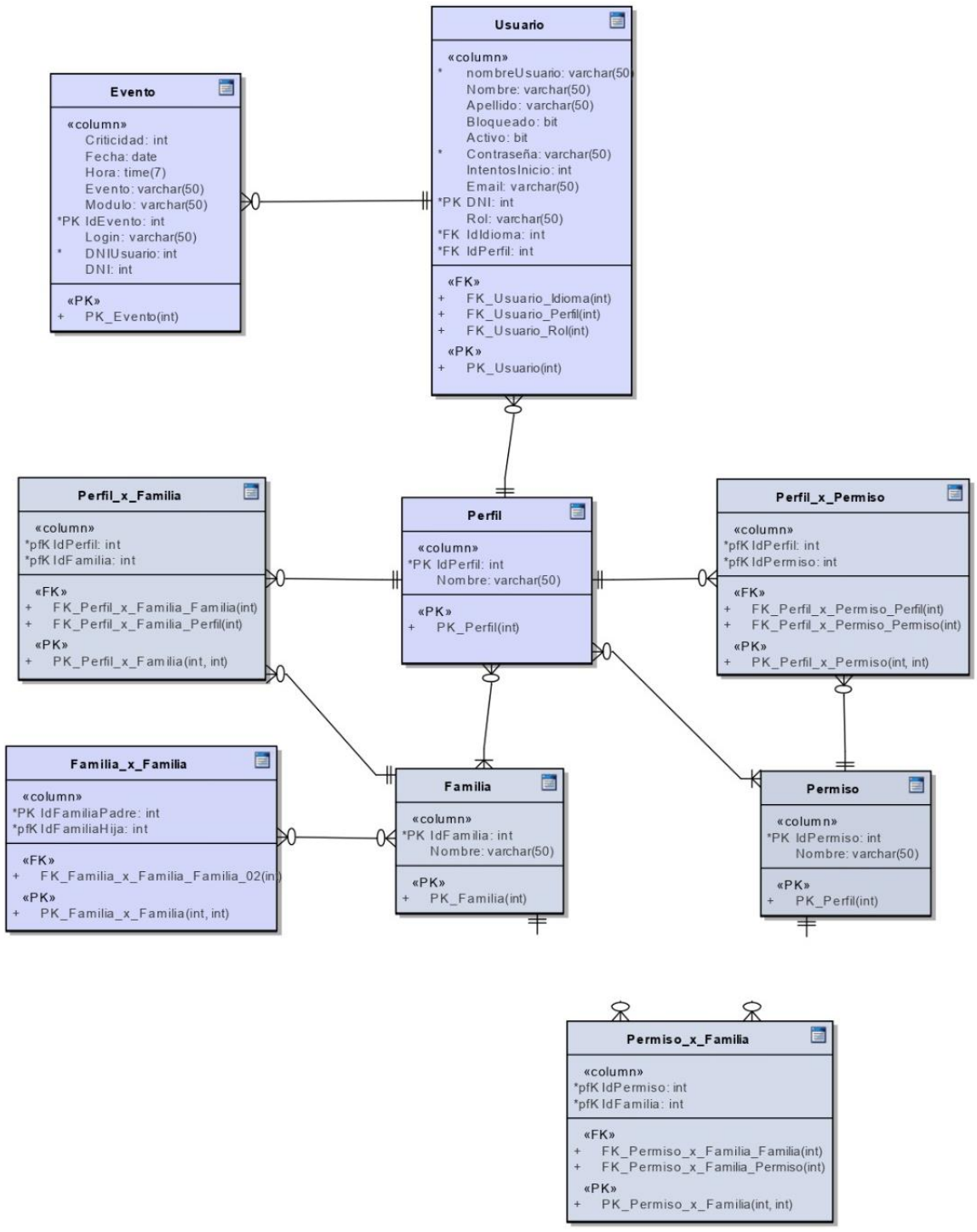


Diagrama Entidad Relación

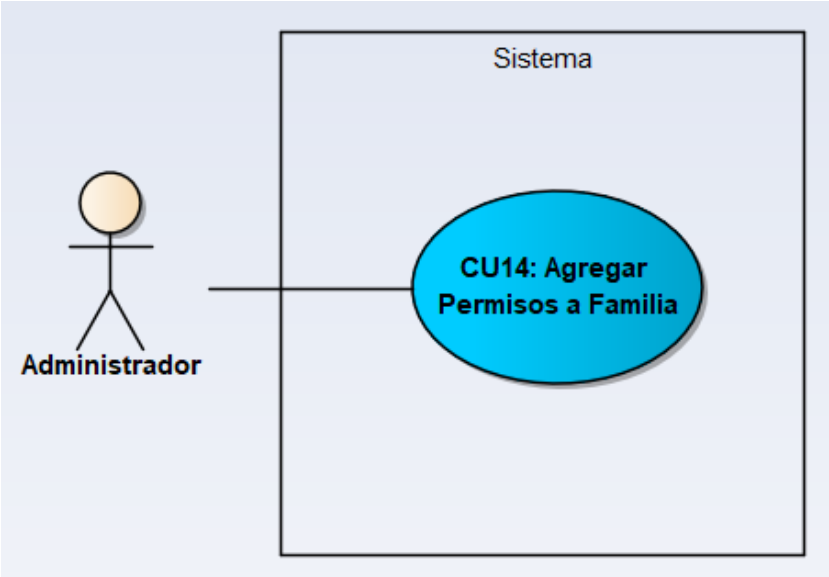


5. Agregar Permisos a Familia

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad recibiendo la selección del apartado "Administración" y la opción "Perfiles" por parte del Administrador, y produciendo la petición de visualización en la Capa GUI.
2. El Sistema recupera y despliega la totalidad de los componentes configurados recibiendo la solicitud de consulta de patentes, familias y perfiles, y produciendo las colecciones de objetos correspondientes desde la Capa DAL.
3. El Sistema procesa el intento de asociación de nuevos elementos recibiendo la selección de la familia destino y los permisos a agregar junto con la pulsación del botón "Agregar Permisos a Familia", y produciendo el envío de los datos hacia el negocio (Capa GUI y BLL).
4. El Sistema valida la viabilidad estructural y la ausencia de relaciones circulares recibiendo los identificadores de los componentes seleccionados, y produciendo un veredicto de integridad (Capa GUI y BLL).
5. El Sistema verifica la inexistencia previa de los vínculos seleccionados recibiendo los datos de la familia y los permisos a asociar, y produciendo una validación de unicidad (Capa BLL y DAL).
6. El Sistema inserta de forma permanente las nuevas relaciones estructurales recibiendo la confirmación de las validaciones de negocio aprobadas, y produciendo la escritura de los nuevos registros en las tablas correspondientes (Capa DAL).
7. El Sistema genera el registro de auditoría de la operación recibiendo la información del evento de asignación y los datos del Administrador logueado (Capa DAL).
8. El Sistema notifica el resultado exitoso y actualiza la vista estructural recibiendo la confirmación de la persistencia de los datos, y produciendo el mensaje de éxito en pantalla junto al refresco del módulo de perfiles (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU14 - AGREGAR PERMISOS A FAMILIA

Descripción:	El sistema permite al administrador agregar nuevos permisos (ya sean patentes o familias) a una familia existente en el sistema.
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. Deben existir patentes o familias creadas previamente en el sistema para poder ser asignadas.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El administrador selecciona una familia y los permisos que desea agregar a dicha familia y presiona el botón "Agregar Permisos a Familia".
4	El sistema consulta la base de datos para verificar que una familia no se agregue a sí misma o a una familia que ya la contiene, y que los permisos seleccionados no estén agregados previamente a la familia elegida
5	El sistema inserta las nuevas relaciones en las tablas correspondientes en la base de datos.
6	El sistema registra la acción en la bitácora.
7	El sistema informa que los permisos fueron agregados correctamente a la familia.
Flujo alternativo:	
4.1.1	El sistema detecta que el administrador desea realizar una relación circular entre permisos.
4.1.2	El sistema bloquea la operación e informa el error.
Poscondición:	La familia queda actualizada en la base de datos con sus nuevos permisos asociados y la acción queda registrada con éxito en la bitácora del sistema.

Diagrama de Secuencia

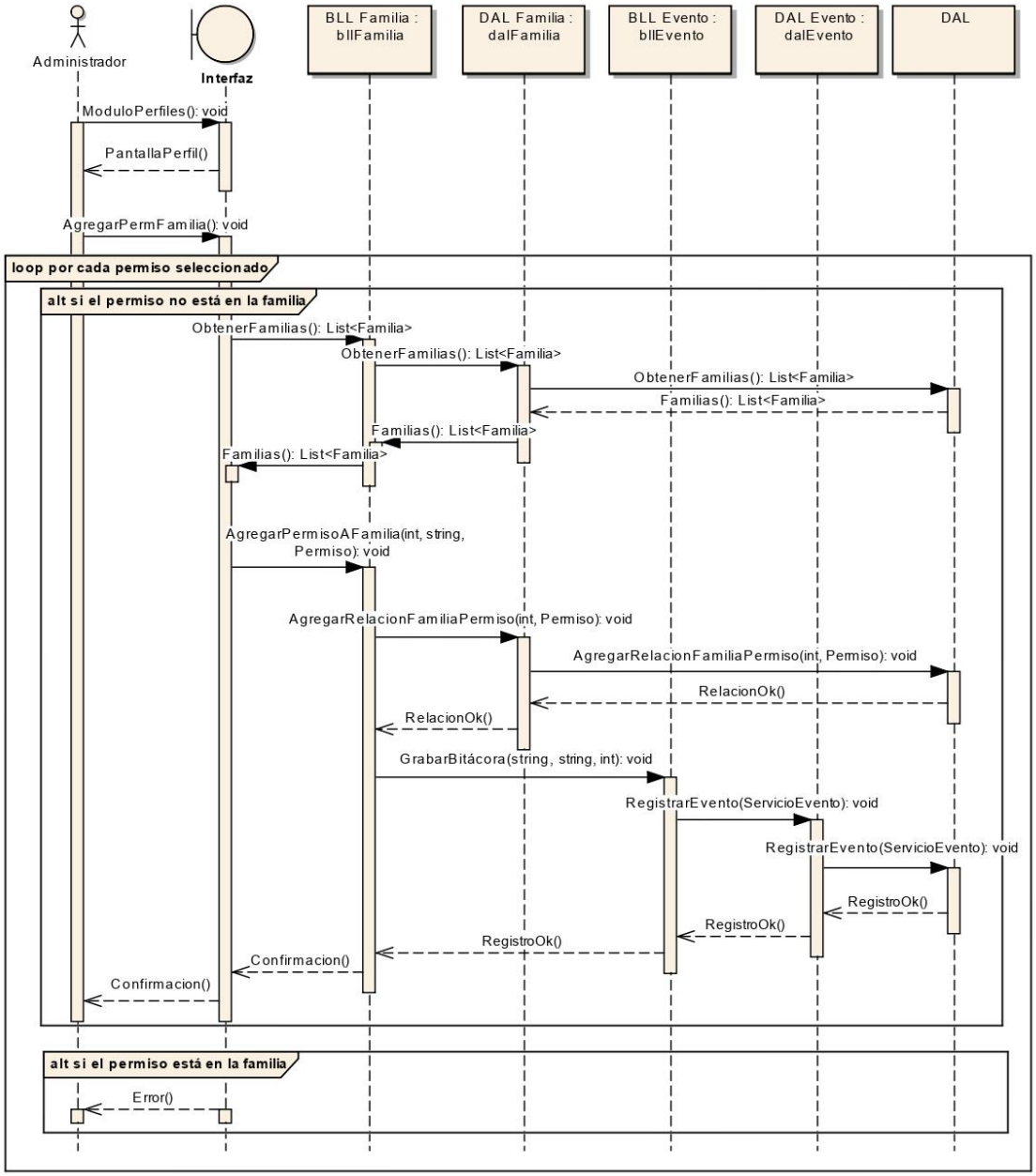


Diagrama de Actividad

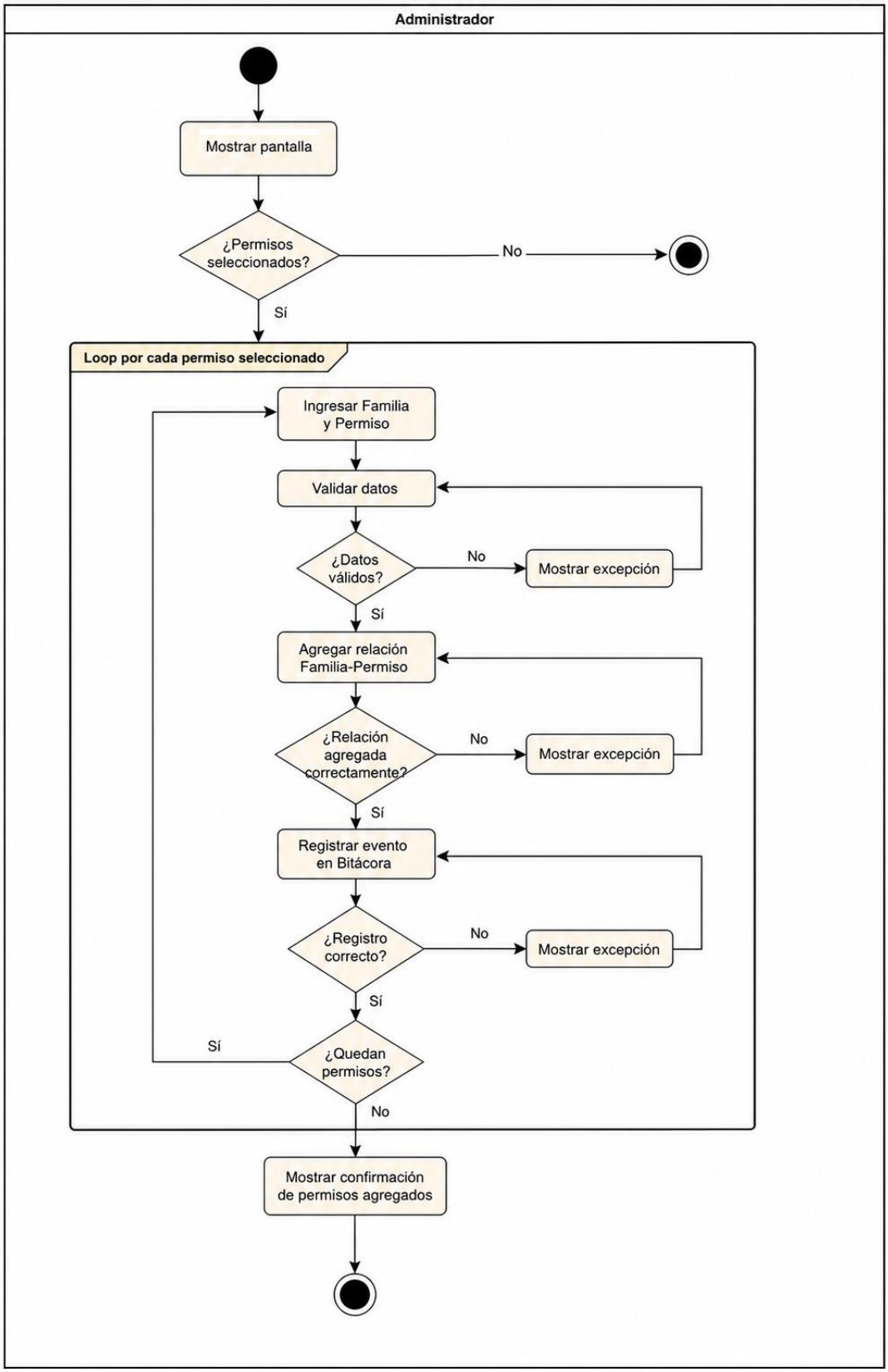

Administrador
Mostrar pantalla
¿Permisos seleccionados?
No
Sí
Loop por cada permiso seleccionado
Ingresar Familia y Permiso
Validar datos
¿Datos válidos?
No
Mostrar excepción
Sí
Agregar relación Familia-Permiso
¿Relación agregada correctamente?
No
Mostrar excepción
Sí
Registrar evento en Bitácora
¿Registro correcto?
No
Mostrar excepción
Sí
¿Quedan permisos?
Sí
No
Mostrar confirmación de permisos agregados

Diagrama de Clases

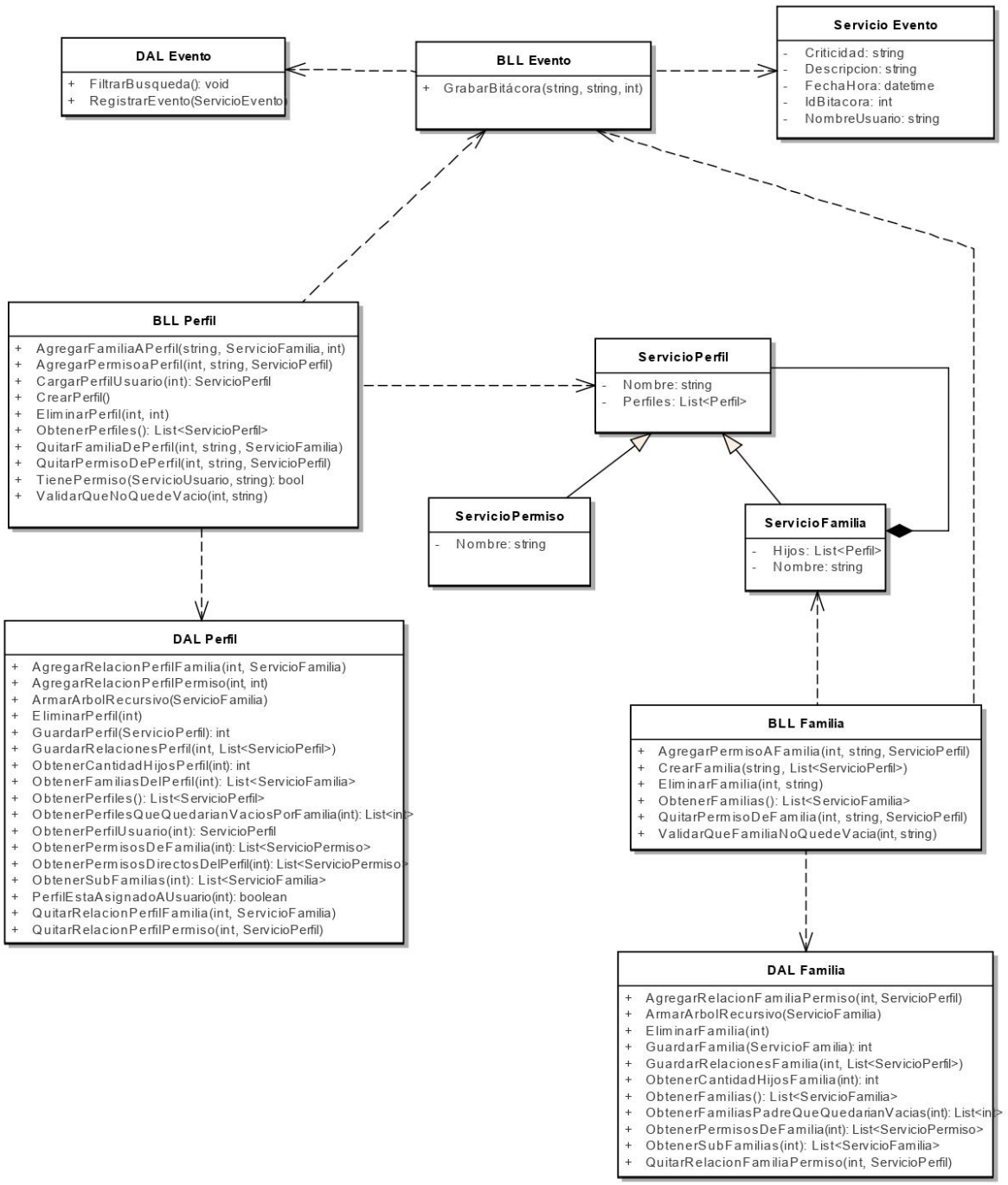
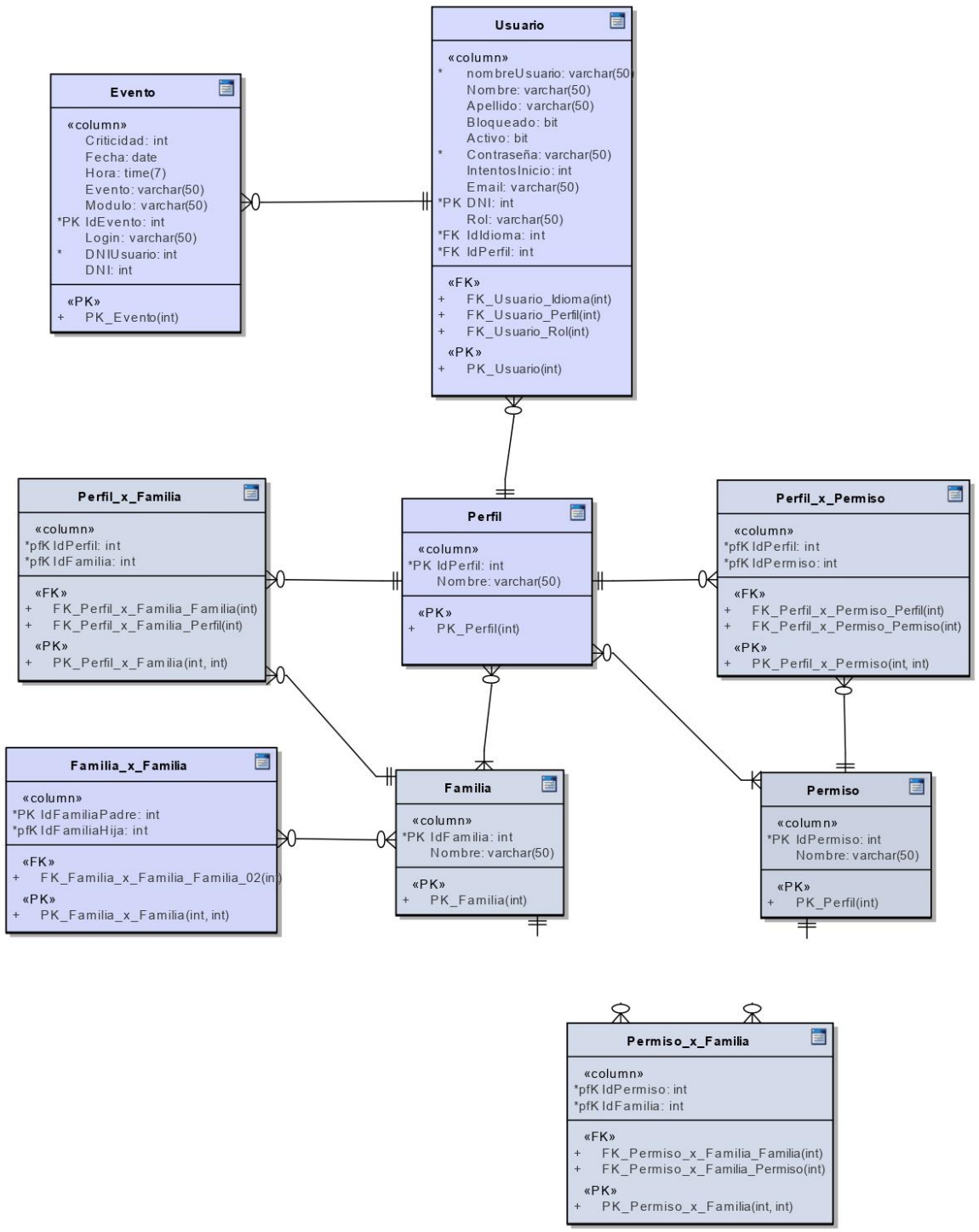


Diagrama Entidad Relación

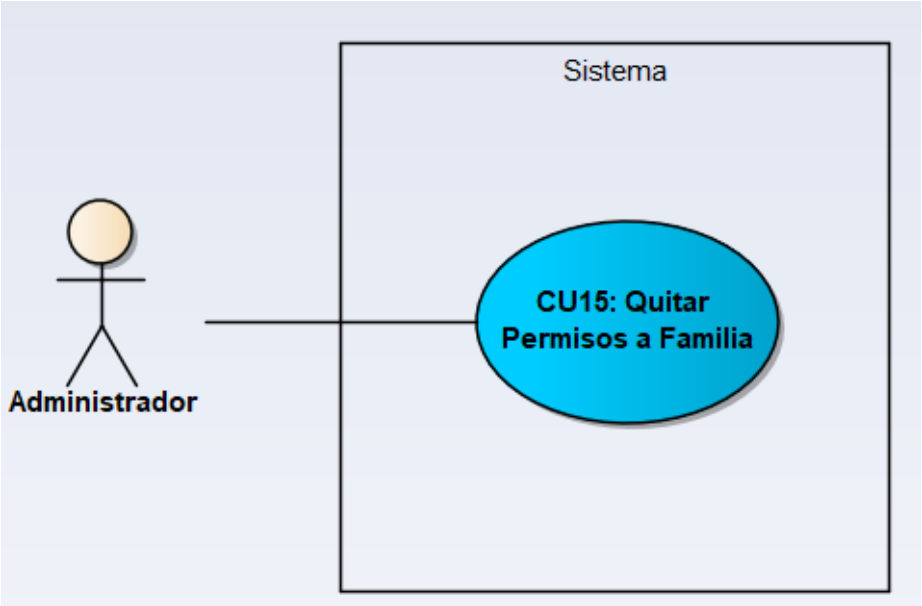


6. Quitar Permisos a Familia

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad recibiendo la selección del apartado "Administración" y la opción "Perfiles" por parte del Administrador, y produciendo la petición de visualización en la Capa GUI.
2. El Sistema recupera y despliega la totalidad de los componentes configurados recibiendo la solicitud de consulta de patentes, familias y perfiles, y produciendo las colecciones de objetos correspondientes desde la Capa DAL.
3. El Sistema procesa el intento de desasociación de elementos recibiendo la selección de la familia y los permisos a remover junto con la pulsación del botón "Quitar Permisos a Familia", y produciendo el envío de los datos hacia el negocio (Capa GUI y BLL).
4. El Sistema valida que la estructura resultante no deje a la familia sin componentes asociados recibiendo las entidades seleccionadas para la remoción, y produciendo un veredicto de viabilidad operativa (Capa GUI y BLL).
5. El Sistema elimina de forma permanente las relaciones estructurales seleccionadas recibiendo la confirmación de la validación de negocio aprobada, y produciendo la baja de los registros en las tablas intermedias (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación recibiendo la información del evento de desasignación y los datos del Administrador logueado en la bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso y actualiza la vista estructural (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU15 - QUITAR PERMISOS A FAMILIA

Descripción:	El sistema permite al administrador eliminar permisos (ya sean patentes o subfamilias) de una familia existente en el sistema.
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. La familia seleccionada debe tener al menos un permiso asignado en la base de datos.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El administrador selecciona los permisos que desea quitar de una familia y presiona el botón "Quitar Permisos a Familia".
4	El sistema consulta la base de datos y verifica que la familia no quede completamente vacía tras la remoción de los elementos seleccionados.
5	El sistema modifica las tablas afectadas de la base de datos.
6	El sistema registra la acción en la bitácora.
7	El sistema informa que los permisos fueron removidos correctamente de la familia.
Flujo alternativo:	
4.1.1	El sistema detecta que la remoción de los permisos seleccionados dejaría a la familia sin ningún componente asociado (vacía).
4.1.2	El sistema bloquea la operación e informa el error.
Poscondición:	La familia queda actualizada en la base de datos habiendo desvinculado los permisos seleccionados y la acción queda registrada con éxito en la bitácora del sistema.

Diagrama de Secuencia

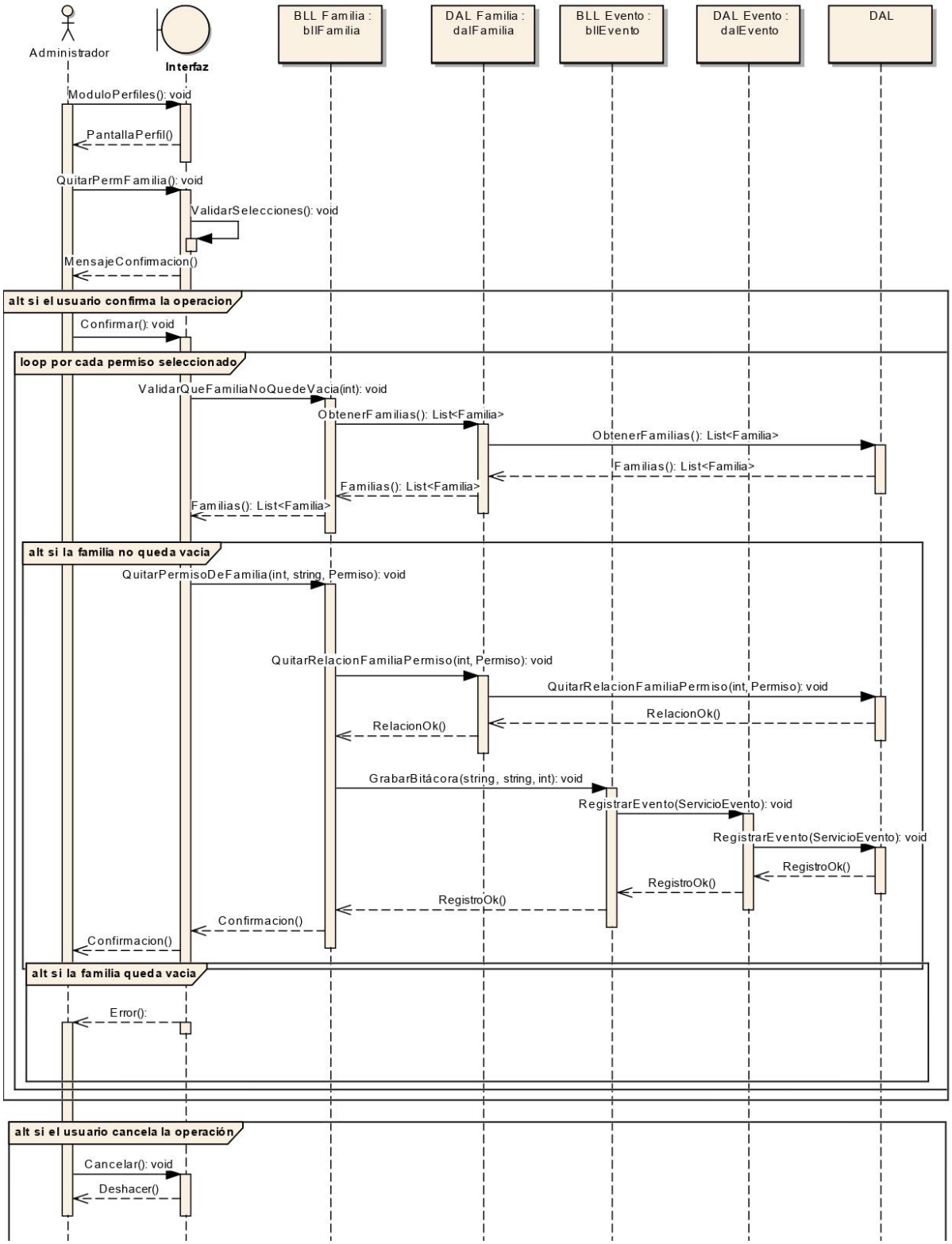
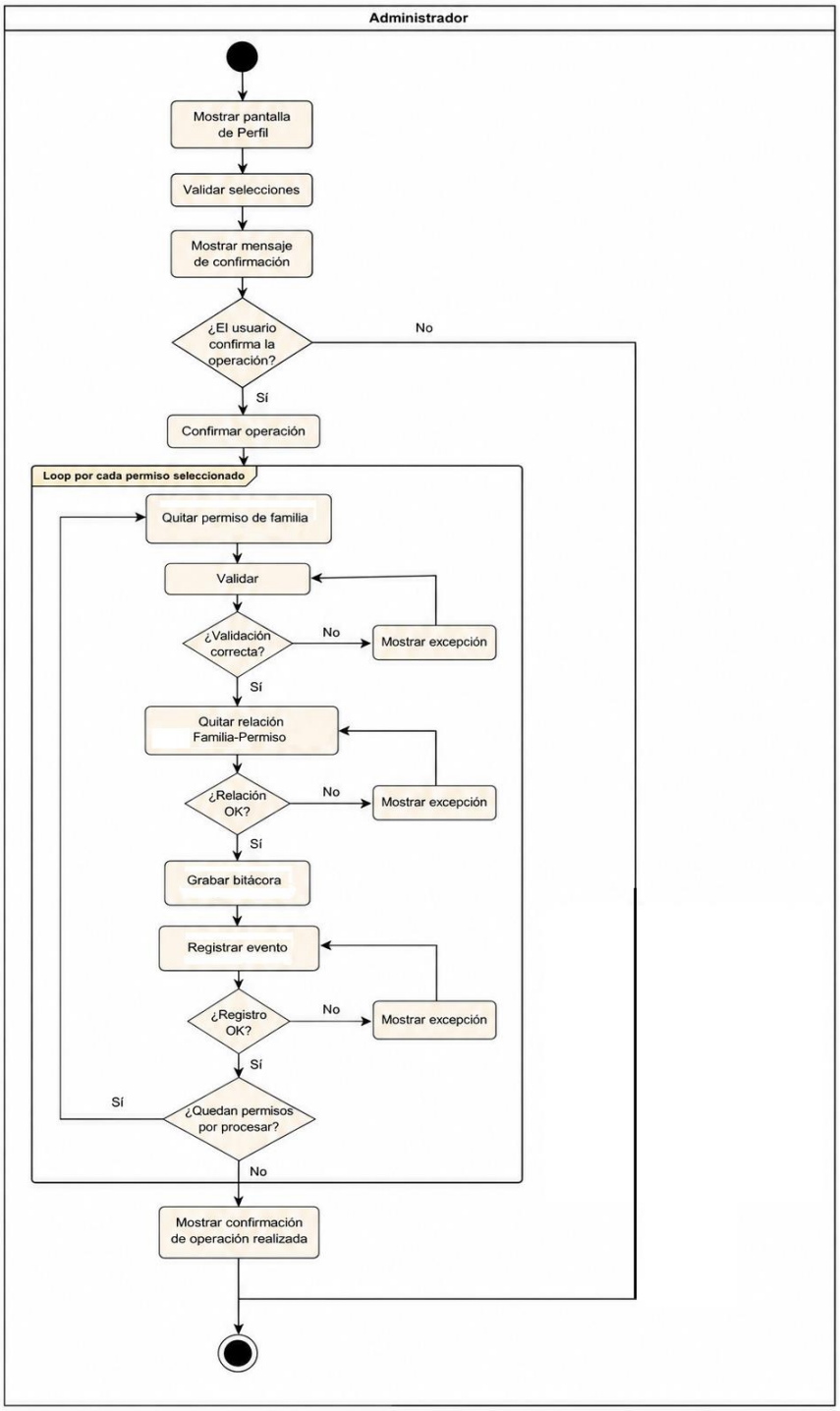
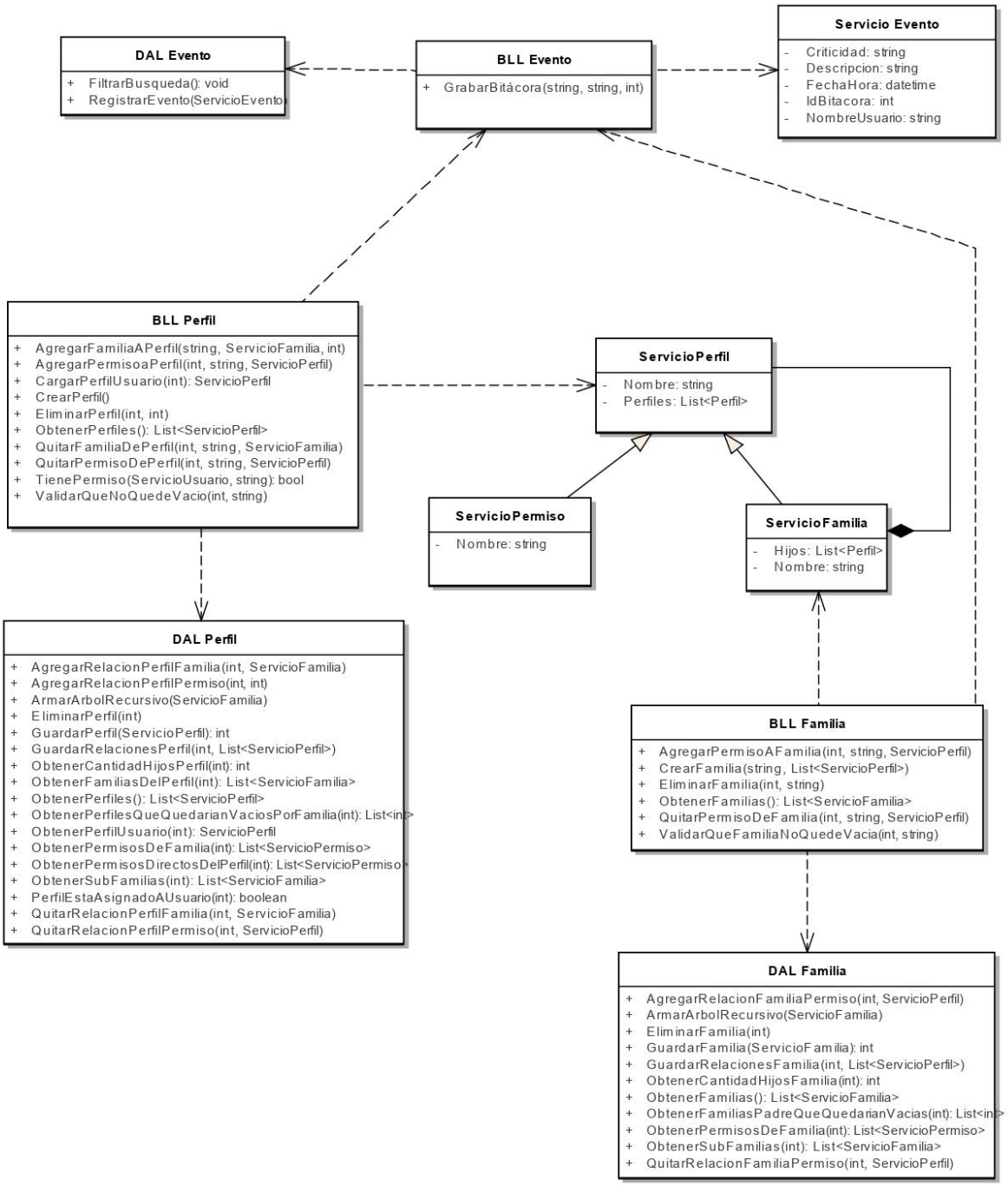

Administrador
Interfaz
BLL Familia : bllFamilia
DAL Familia : dalFamilia
BLL Evento : bllEvento
DAL Evento : dalEvento
DAL
ModuloPerfiles(): void
PantallaPerfil()
QuitarPermFamilia(): void
ValidarSelecciones(): void
MensajeConfirmacion()
alt si el usuario confirma la operacion
Confirmar(): void
loop por cada permiso seleccionado
ValidarQueFamiliaNoQuedeVacia(int): void
ObtenerFamilias(): List<Familia>
ObtenerFamilias(): List<Familia>
Familias(): List<Familia>
Familias(): List<Familia>
Familias(): List<Familia>
alt si la familia no queda vacia
QuitarPermisoDeFamilia(int, string, Permiso): void
QuitarRelacionFamiliaPermiso(int, Permiso): void
QuitarRelacionFamiliaPermiso(int, Permiso): void
RelacionOk()
RelacionOk()
GrabarBitácora(string, string, int): void
RegistrarEvento(ServicioEvento): void
RegistrarEvento(ServicioEvento): void
RegistroOk()
RegistroOk()
RegistroOk()
Confirmacion()
Confirmacion()
alt si la familia queda vacia
Error():
alt si el usuario cancela la operación
Cancelar(): void
Deshacer()

Diagrama de Actividad



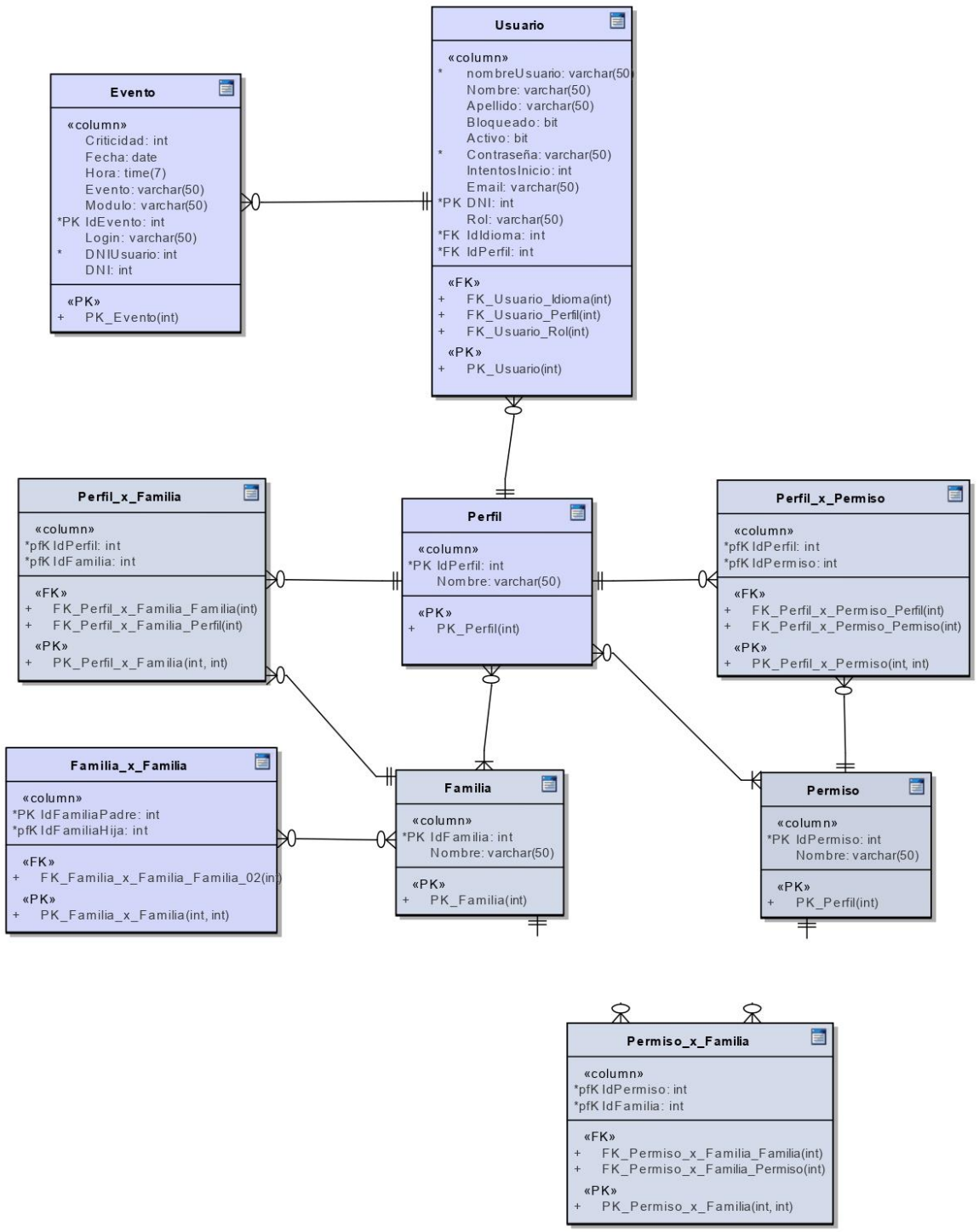
UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Diagrama de Clases



UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Diagrama Entidad Relación

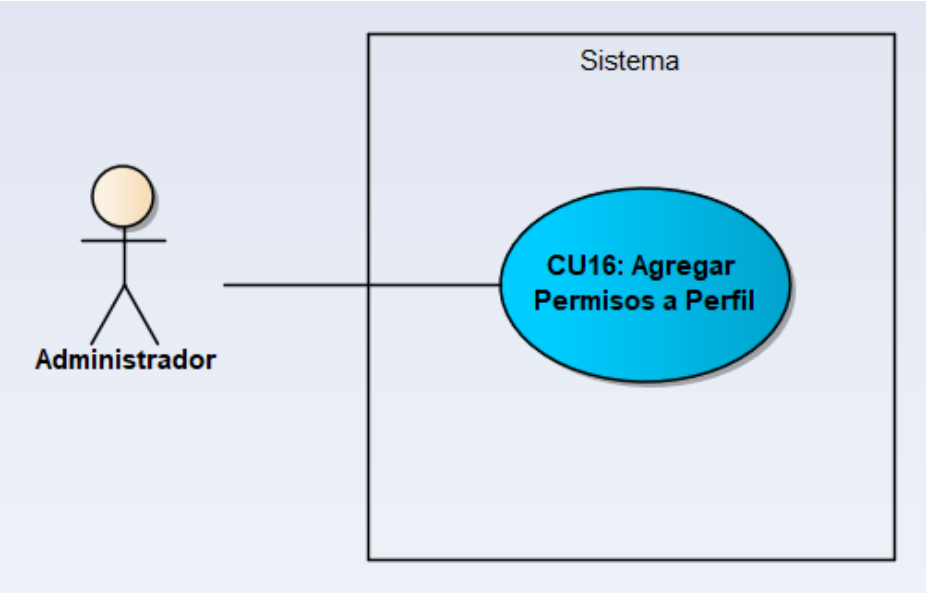


7. Agregar Permisos a Perfil

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad recibiendo la selección del apartado "Administración" y la opción "Perfiles" por parte del Administrador, y produciendo la petición de visualización en la Capa GUI.
2. El Sistema recupera y despliega la totalidad de los componentes configurados recibiendo la solicitud de consulta de patentes, familias y perfiles, y produciendo las colecciones de objetos correspondientes desde la Capa DAL.
3. El Sistema procesa el intento de asociación de elementos recibiendo la selección del perfil y los permisos a agregar junto con la pulsación del botón "Agregar Permisos a Perfil", y produciendo el envío de los datos hacia el negocio (Capa GUI y BLL).
4. El Sistema valida la inexistencia previa de los vínculos seleccionados recibiendo las entidades seleccionadas para la asignación, y produciendo un veredicto de disponibilidad que filtra los duplicados para procesar solo los nuevos registros (Capa GUI y BLL).
5. El Sistema inserta de forma permanente las nuevas relaciones estructurales recibiendo la confirmación de la validación de negocio y los elementos no duplicados, y produciendo la inserción de los registros en las tablas correspondientes (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación recibiendo la información del evento de asignación y los datos del Administrador logueado en la bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso y actualiza la vista estructural (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU16 - AGREGAR PERMISOS A PERFIL

Descripción:	El sistema permite al administrador agregar un permiso individual a un perfil existente en el sistema.
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. Deben existir permisos y perfiles creados previamente en el sistema.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El administrador selecciona un perfil y los permisos que desea agregar a ese perfil y presiona el botón "Agregar Permisos a Perfil".
4	El sistema consulta la base de datos para verificar que las asociaciones seleccionadas no existan en ese perfil.
5	El sistema inserta las nuevas relaciones en la base de datos.
6	El sistema registra la acción en la bitácora.
7	El sistema informa que los permisos fueron agregados correctamente al perfil.
Flujo alternativo:	
4.1.1	El sistema detecta que algunos de los permisos seleccionados ya se encuentran asignados al perfil.
4.1.2	El sistema omite los duplicados, inserta sólo los perfiles nuevos y notifica los errores al usuario.
Poscondición:	El perfil queda actualizado en la base de datos con sus nuevos perfiles asociados y la acción queda registrada con éxito en la bitácora del sistema.

Diagrama de Secuencia

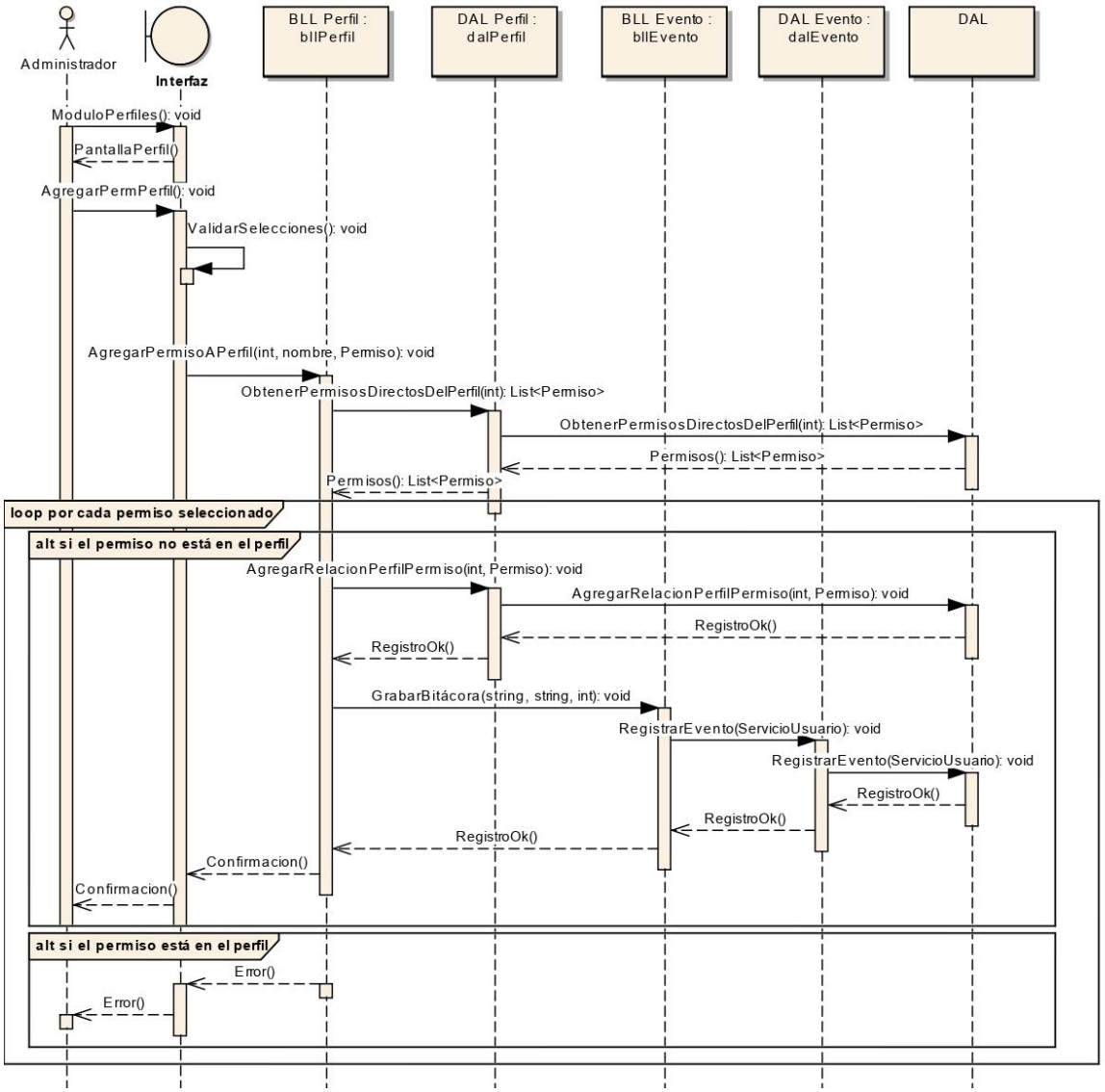


Diagrama de Actividad

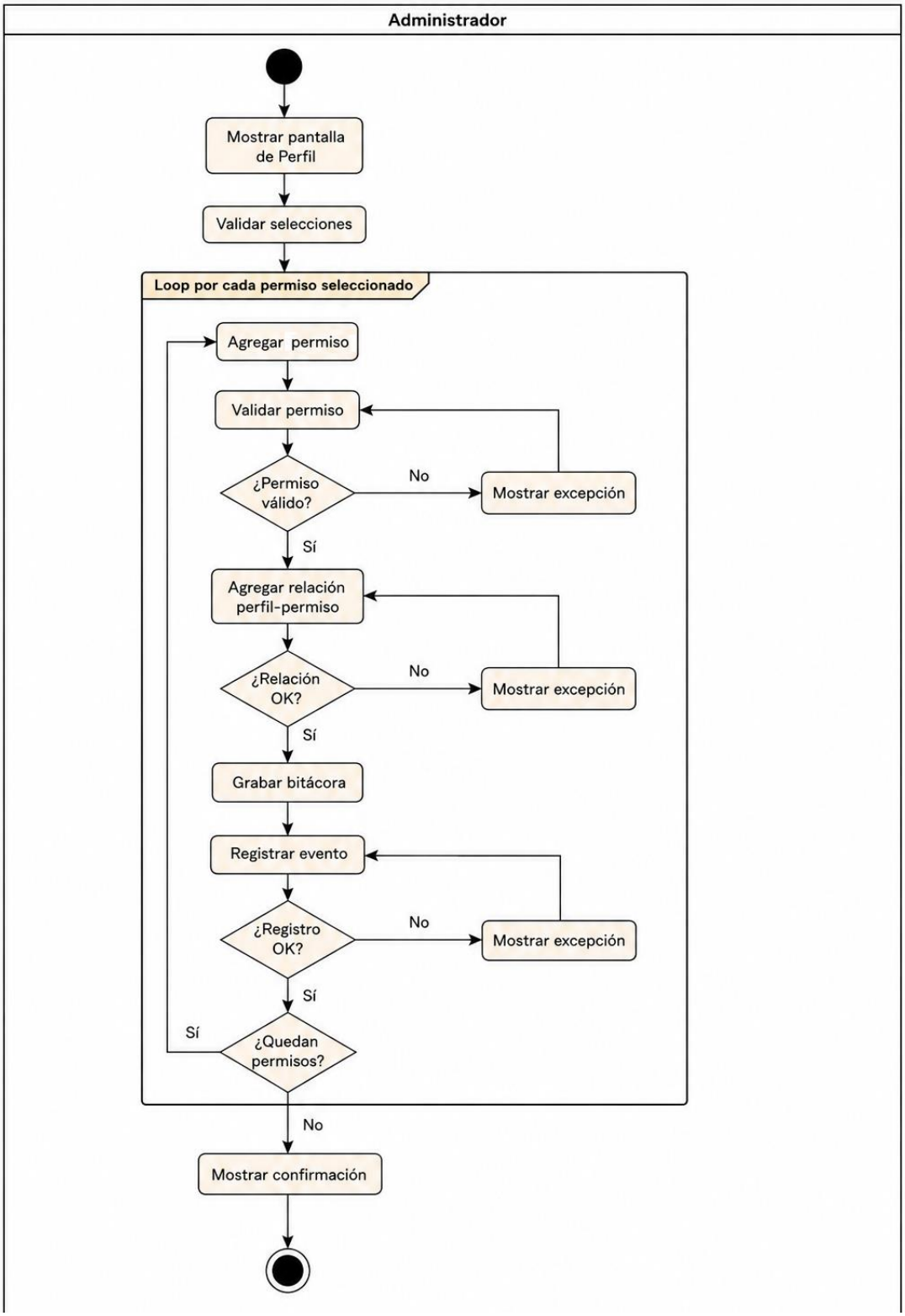
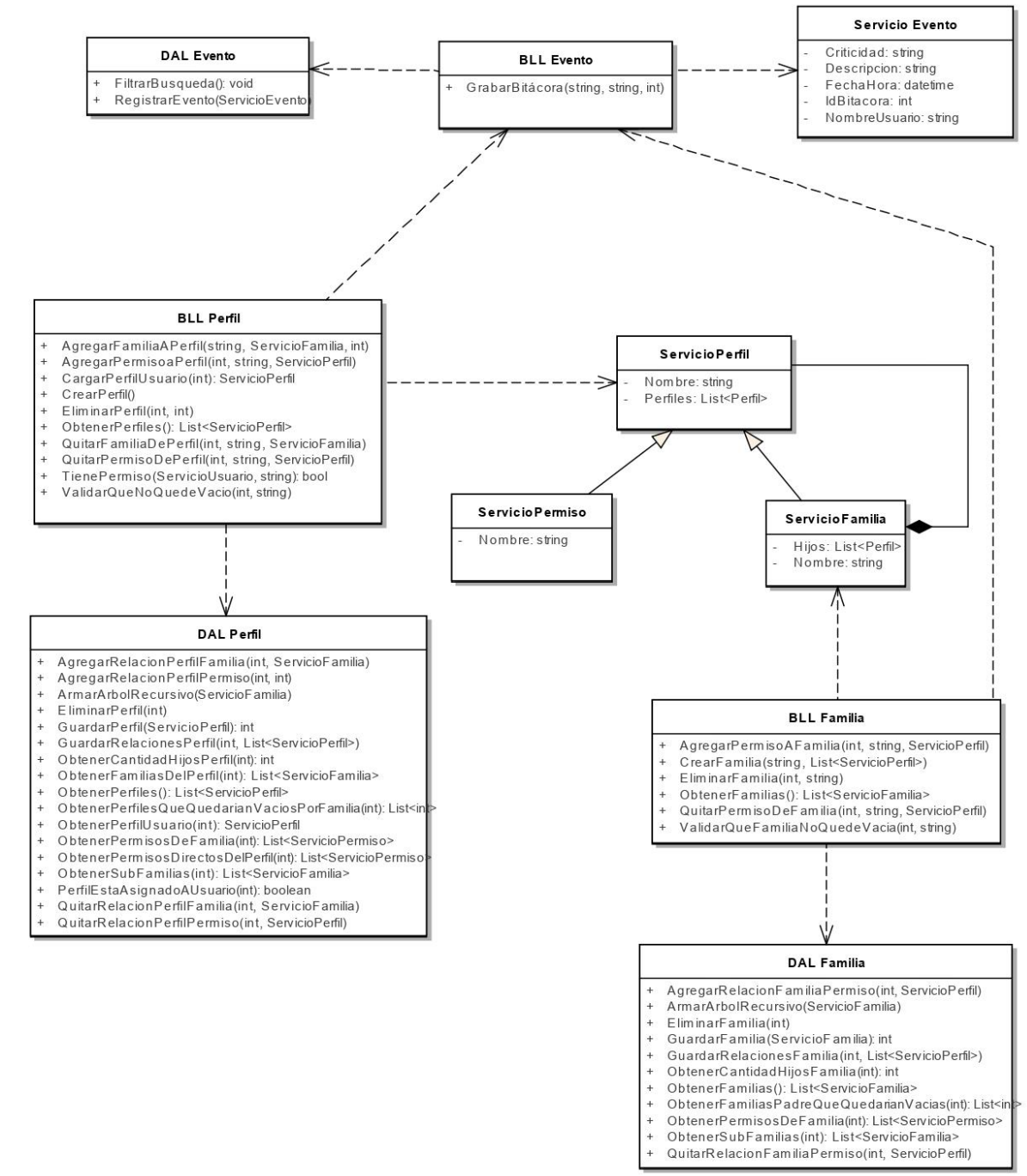
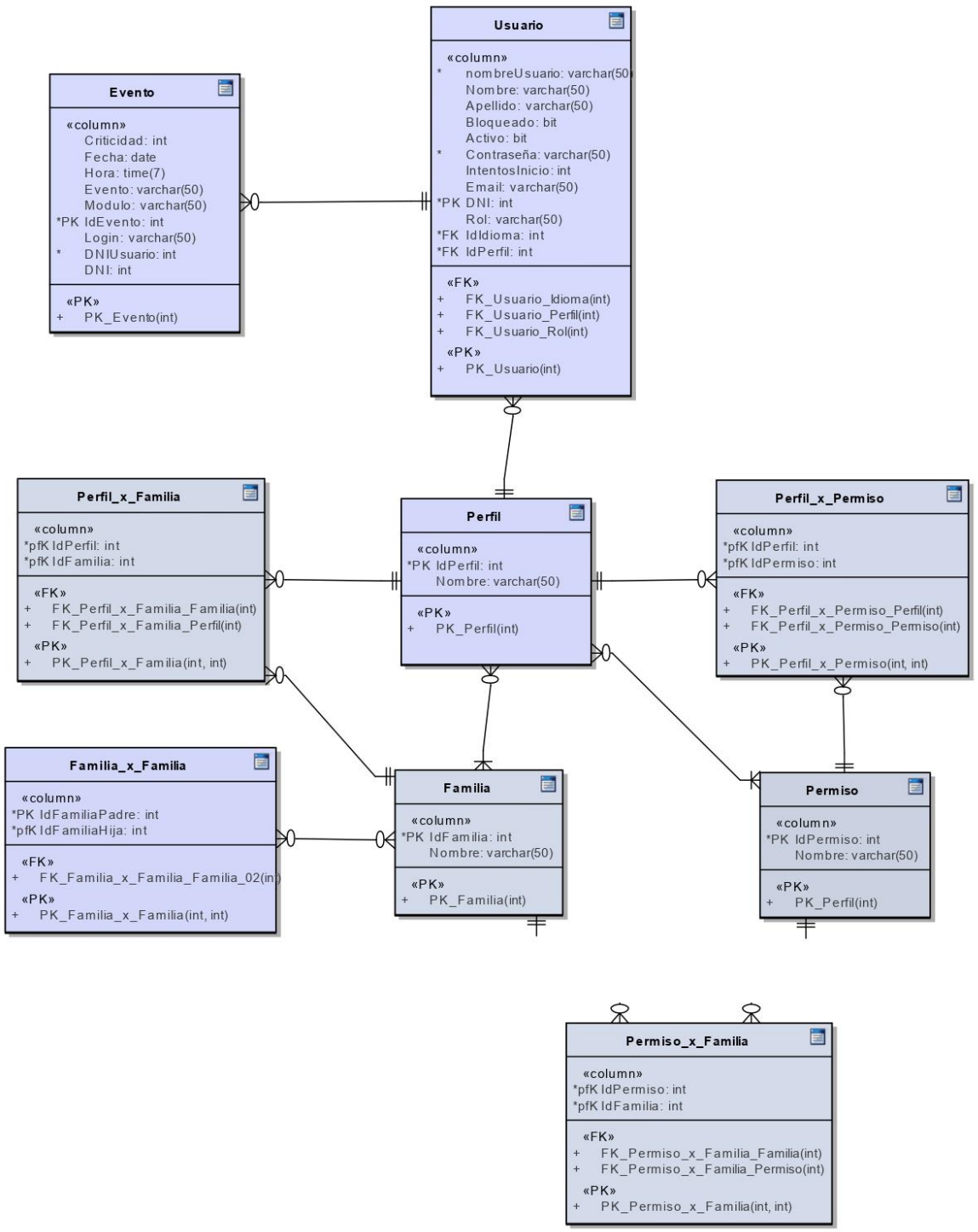

Administrador
Mostrar pantalla de Perfil
Validar selecciones
Loop por cada permiso seleccionado
Agregar permiso
Validar permiso
¿Permiso válido?
No
Mostrar excepción
Sí
Agregar relación perfil-permiso
¿Relación OK?
No
Mostrar excepción
Sí
Grabar bitácora
Registrar evento
¿Registro OK?
No
Mostrar excepción
Sí
Sí
¿Quedan permisos?
No
Mostrar confirmación

Diagrama de Clases



UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Diagrama Entidad Relación

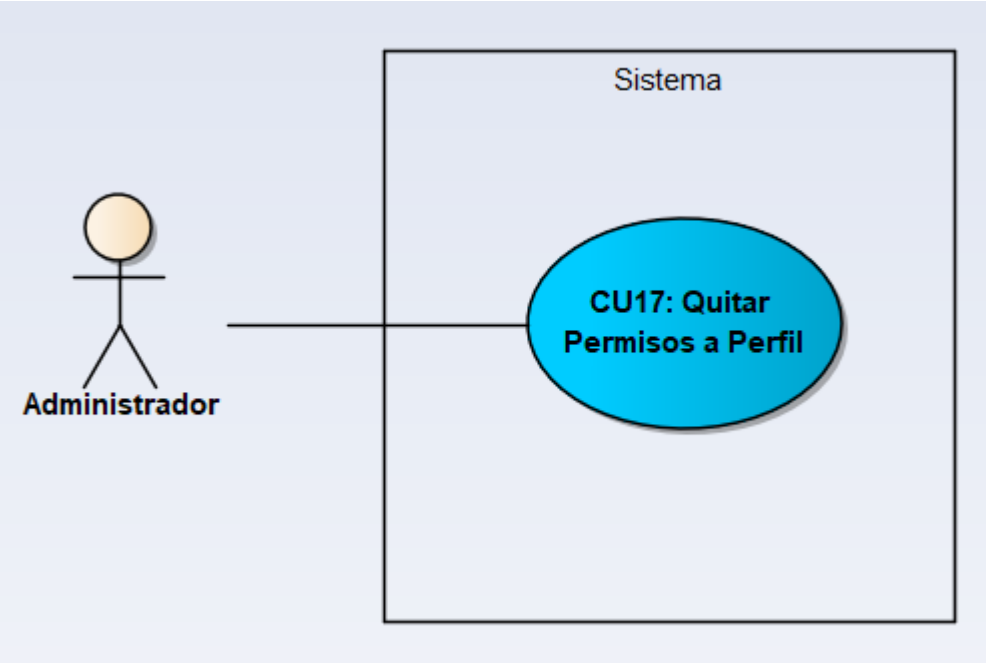


8. Quitar Permisos a Perfil

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad recibiendo la selección del apartado "Administración" y la opción "Perfiles" por parte del Administrador, y produciendo la petición de visualización en la Capa GUI.
2. El Sistema recupera y despliega la totalidad de los componentes configurados recibiendo la solicitud de consulta de patentes, familias y perfiles, y produciendo las colecciones de objetos correspondientes desde la Capa DAL.
3. El Sistema procesa el intento de des asociación de elementos recibiendo la selección del perfil y los permisos a remover junto con la pulsación del botón "Quitar Permisos a Perfil", y produciendo el envío de los datos hacia el negocio (Capa GUI y BLL).
4. El Sistema valida que la estructura resultante no deje al perfil sin componentes asociados recibiendo las entidades seleccionadas para la remoción, y produciendo un veredicto de viabilidad operativa que bloquea la acción en caso de vaciado (Capa GUI y BLL).
5. El Sistema elimina de forma permanente las relaciones estructurales seleccionadas recibiendo la confirmación de la validación de negocio aprobada, y produciendo la baja de los registros en las tablas correspondientes (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación recibiendo la información del evento de des asignación y los datos del Administrador logueado en la bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso y actualiza la vista estructural (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU17 - QUITAR PERMISOS A PERFIL

Descripción:	El sistema permite al administrador remover un permiso individual (patente) de un perfil existente en el sistema.
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. El perfil seleccionado debe tener al menos una patente asignada.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El administrador selecciona los permisos que desea quitar a un perfil y presiona el botón "Quitar Permisos a Perfil".
4	El sistema consulta la base de datos para verificar que el perfil no quede vacío tras la remoción.
5	El sistema elimina las relaciones correspondientes en la base de datos.
6	El sistema registra la acción en la bitácora.
7	El sistema informa que los permisos fueron removidos correctamente del perfil.
Flujo alternativo:	
4.1.1	El sistema detecta que la remoción de las patentes seleccionadas dejaría al perfil sin componentes asociados.
4.1.2	El sistema bloquea la operación e informa el error.
Poscondición:	El perfil queda actualizado en la base de datos y la acción queda registrada con éxito en la bitácora del sistema.

Diagrama de Secuencia

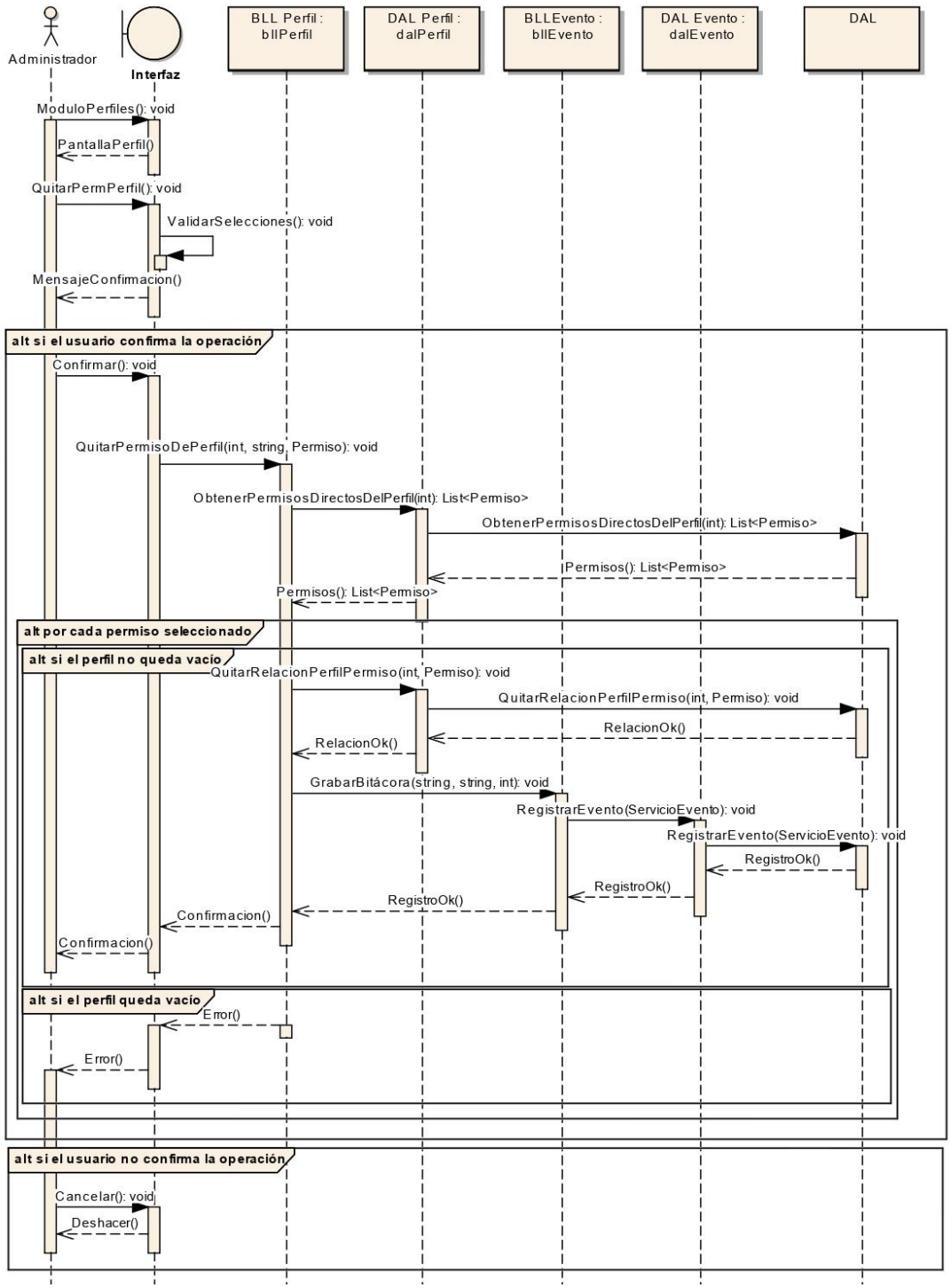


Diagrama de Actividad

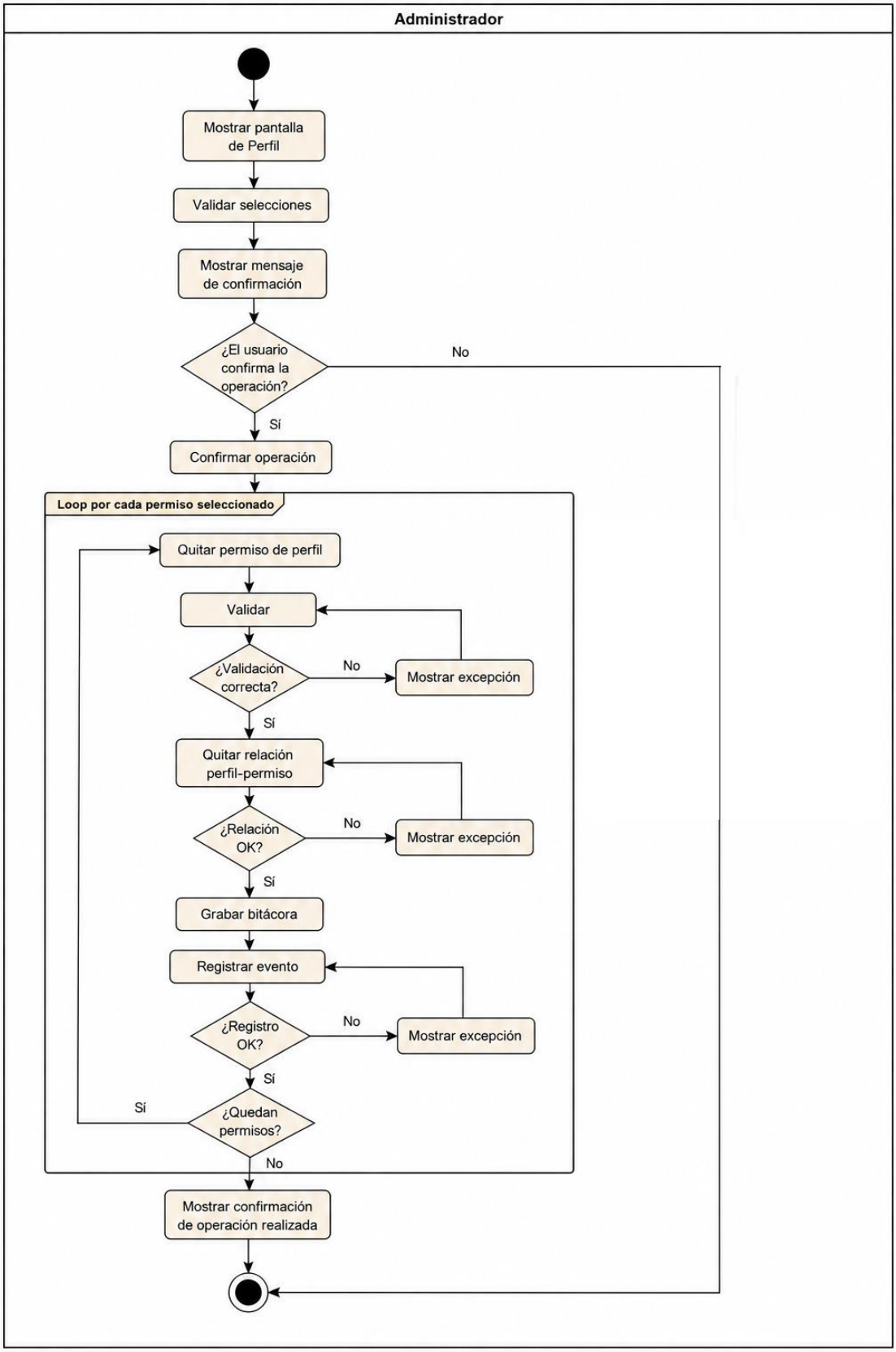

Administrador
Mostrar pantalla de Perfil
Validar selecciones
Mostrar mensaje de confirmación
¿El usuario confirma la operación?
No
Sí
Confirmar operación
Loop por cada permiso seleccionado
Quitar permiso de perfil
Validar
¿Validación correcta?
No
Mostrar excepción
Sí
Quitar relación perfil-permiso
¿Relación OK?
No
Mostrar excepción
Sí
Grabar bitácora
Registrar evento
¿Registro OK?
No
Mostrar excepción
Sí
¿Quedan permisos?
Sí
No
Mostrar confirmación de operación realizada

Diagrama de Clases

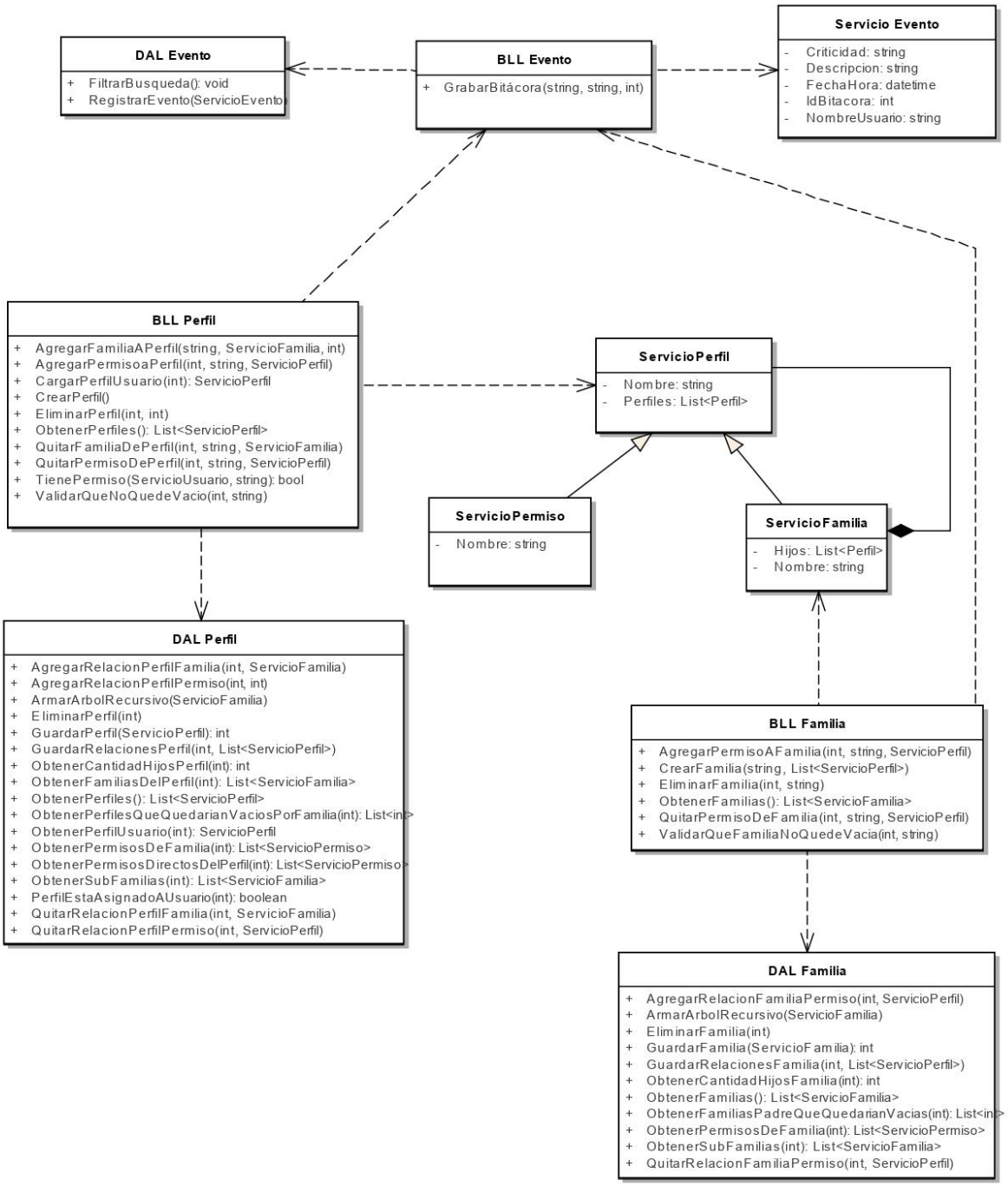
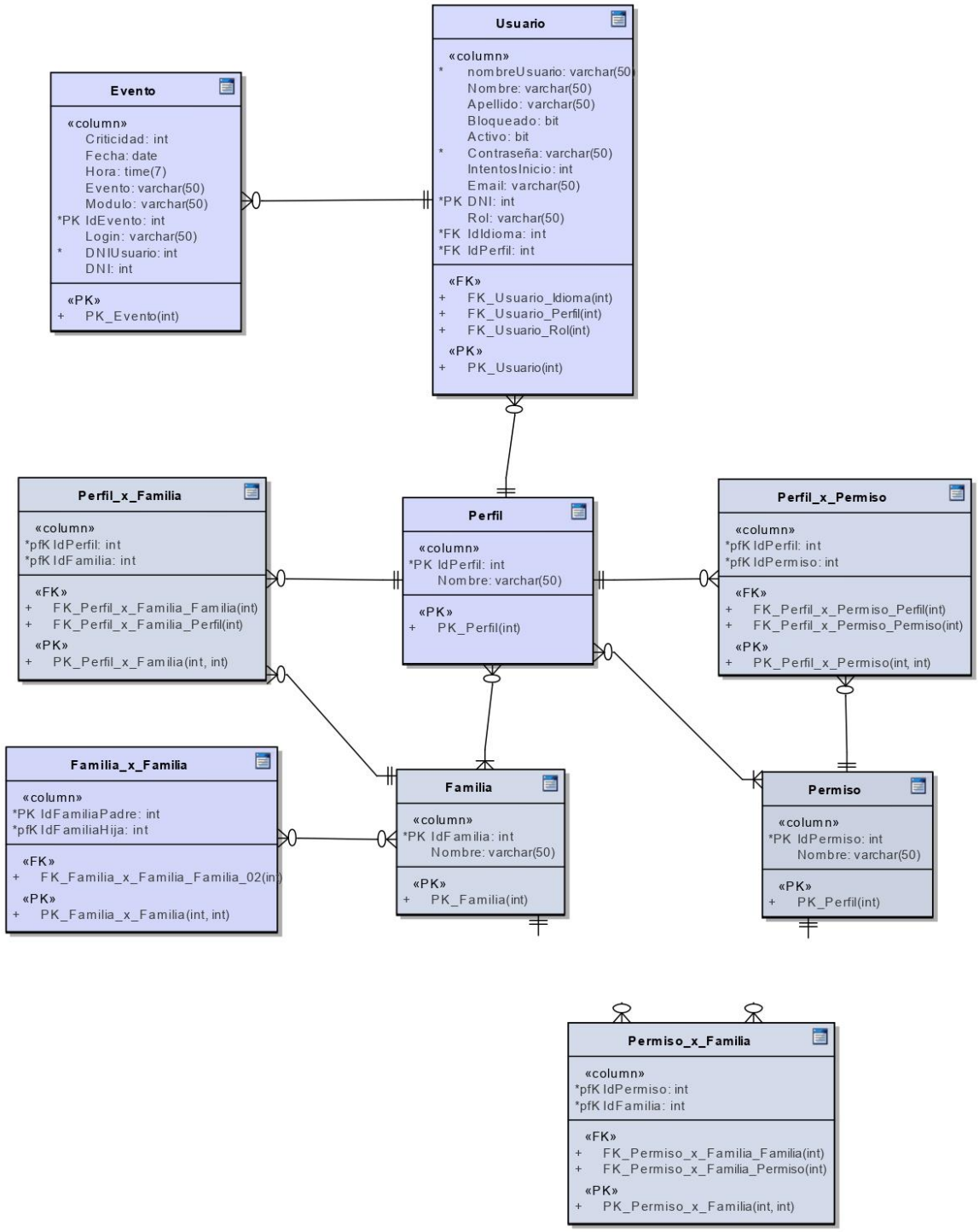


Diagrama Entidad Relación

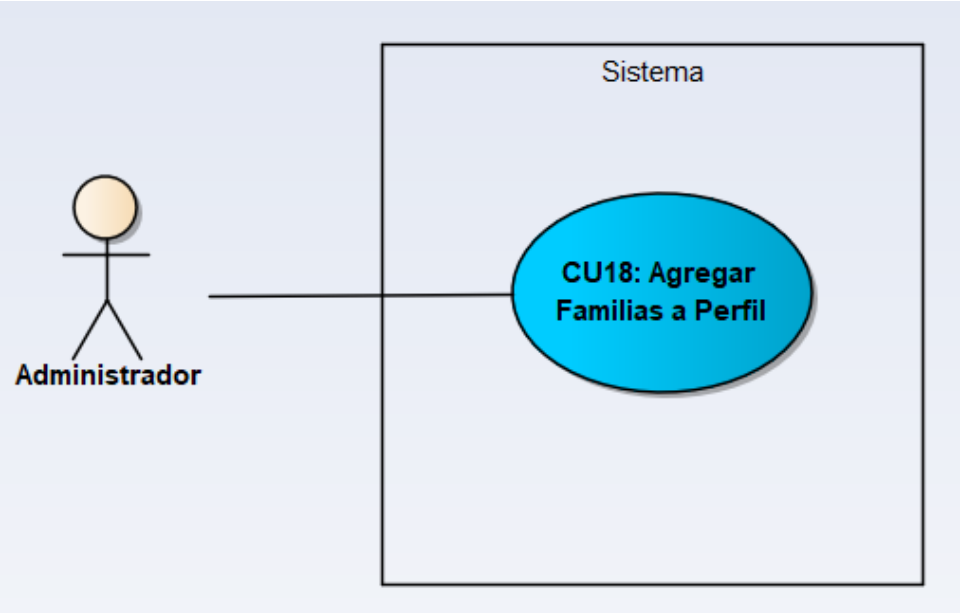

Usuario
«column»
* nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
* Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
*PK DNI: int
Rol: varchar(50)
*FK IdIdioma: int
*FK IdPerfil: int
«FK»
+ FK_Usuario_Idioma(int)
+ FK_Usuario_Perfil(int)
+ FK_Usuario_Rol(int)
«PK»
+ PK_Usuario(int)
Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
* DNIUsuario: int
DNI: int
«PK»
+ PK_Evento(int)
Perfil_x_Familia
«column»
*pfK IdPerfil: int
*pfK IdFamilia: int
«FK»
+ FK_Perfil_x_Familia_Familia(int)
+ FK_Perfil_x_Familia_Perfil(int)
«PK»
+ PK_Perfil_x_Familia(int, int)
Perfil
«column»
*PK IdPerfil: int
Nombre: varchar(50)
«PK»
+ PK_Perfil(int)
Perfil_x_Permiso
«column»
*pfK IdPerfil: int
*pfK IdPermiso: int
«FK»
+ FK_Perfil_x_Permiso_Perfil(int)
+ FK_Perfil_x_Permiso_Permiso(int)
«PK»
+ PK_Perfil_x_Permiso(int, int)
Familia_x_Familia
«column»
*PK IdFamiliaPadre: int
*pfK IdFamiliaHija: int
«FK»
+ FK_Familia_x_Familia_Familia_02(int)
«PK»
+ PK_Familia_x_Familia(int, int)
Familia
«column»
*PK IdFamilia: int
Nombre: varchar(50)
«PK»
+ PK_Familia(int)
Permiso
«column»
*PK IdPermiso: int
Nombre: varchar(50)
«PK»
+ PK_Perfil(int)
Permiso_x_Familia
«column»
*pfK IdPermiso: int
*pfK IdFamilia: int
«FK»
+ FK_Permiso_x_Familia_Familia(int)
+ FK_Permiso_x_Familia_Permiso(int)
«PK»
+ PK_Permiso_x_Familia(int, int)

9. Agregar Familias a Perfil

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad recibiendo la selección del apartado "Administración" y la opción "Perfiles" por parte del Administrador, y produciendo la petición de visualización en la Capa GUI.
2. El Sistema recupera y despliega la totalidad de los componentes configurados recibiendo la solicitud de consulta de patentes, familias y perfiles, y produciendo las colecciones de objetos correspondientes desde la Capa DAL.
3. El Sistema procesa el intento de asociación de elementos recibiendo la selección del perfil y las familias a agregar junto con la pulsación del botón "Agregar Familias a Perfil", y produciendo el envío de los datos hacia el negocio (Capa GUI y BLL).
4. El Sistema valida la inexistencia previa de los vínculos seleccionados recibiendo las entidades seleccionadas para la asignación, y produciendo un veredicto de disponibilidad que filtra los duplicados para procesar solo los nuevos registros (Capa GUI y BLL).
5. El Sistema inserta de forma permanente las nuevas relaciones estructurales recibiendo la confirmación de la validación de negocio y los elementos no duplicados, y produciendo la inserción de los registros en las tablas correspondientes (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación recibiendo la información del evento de asignación y los datos del Administrador logueado en la bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso y actualiza la vista estructural (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU18 - AGREGAR FAMILIAS A PERFIL

Descripción:	El sistema permite al administrador agregar una familia a un perfil existente en el sistema.
Actores:	Administrador
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. Deben existir familias y perfiles creados previamente en el sistema.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El administrador selecciona un perfil y las familias que desea agregar a ese perfil presiona el botón "Agregar Familias a Perfil".
4	El sistema consulta la base de datos para verificar que las relaciones seleccionadas no existan previamente para ese perfil.
5	El sistema inserta las nuevas relaciones en la base de datos.
6	El sistema registra la acción en la bitácora.
7	El sistema informa que las familias fueron agregadas correctamente al perfil.
Flujo alternativo:	
4.1.1	El sistema detecta que alguna de las familias seleccionadas ya se encuentran asignadas al perfil elegido.
4.1.2	El sistema omite las familias duplicadas, inserta sólo las nuevas y notifica al usuario.
Poscondición:	El perfil queda actualizado en la base de datos con sus nuevas familias asociadas y la acción queda registrada con éxito en la bitácora del sistema.

Diagrama de Secuencia

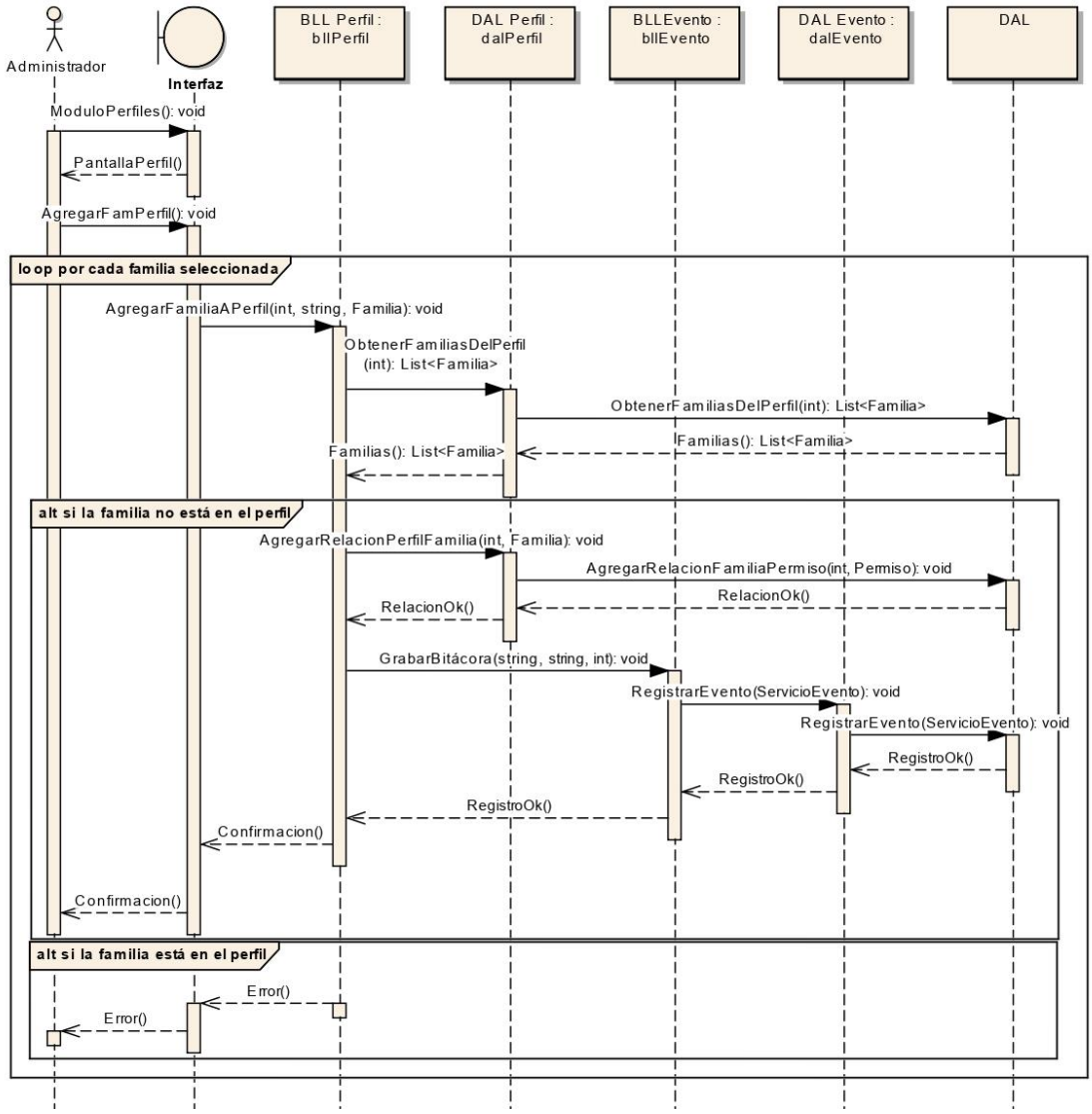
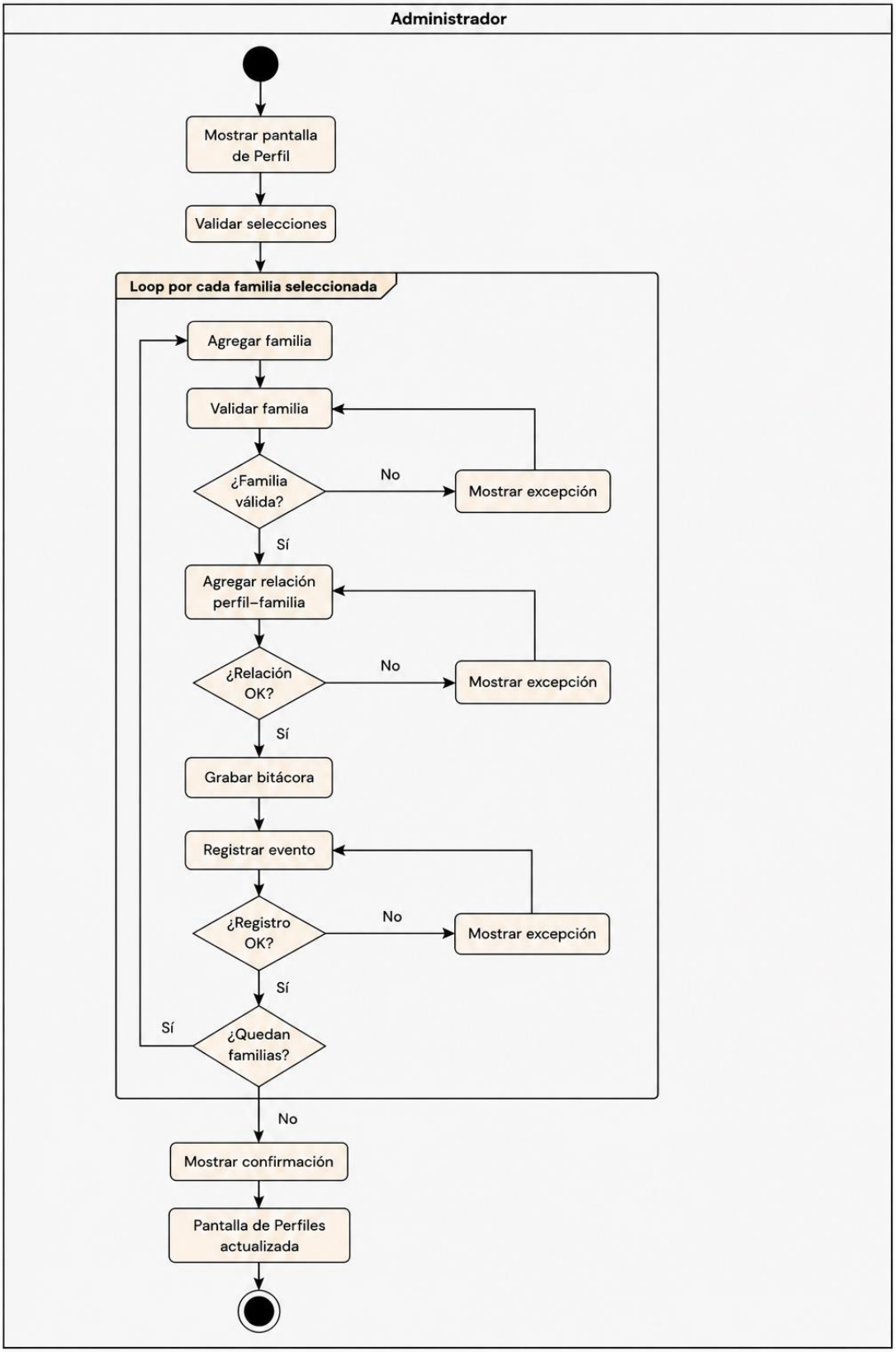


Diagrama de Actividad


Administrador
Mostrar pantalla de Perfil
Validar selecciones
Loop por cada familia seleccionada
Agregar familia
Validar familia
¿Familia válida?
No
Mostrar excepción
Sí
Agregar relación perfil–familia
¿Relación OK?
No
Mostrar excepción
Sí
Grabar bitácora
Registrar evento
¿Registro OK?
No
Mostrar excepción
Sí
¿Quedan familias?
Sí
No
Mostrar confirmación
Pantalla de Perfiles actualizada

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Diagrama de Clases

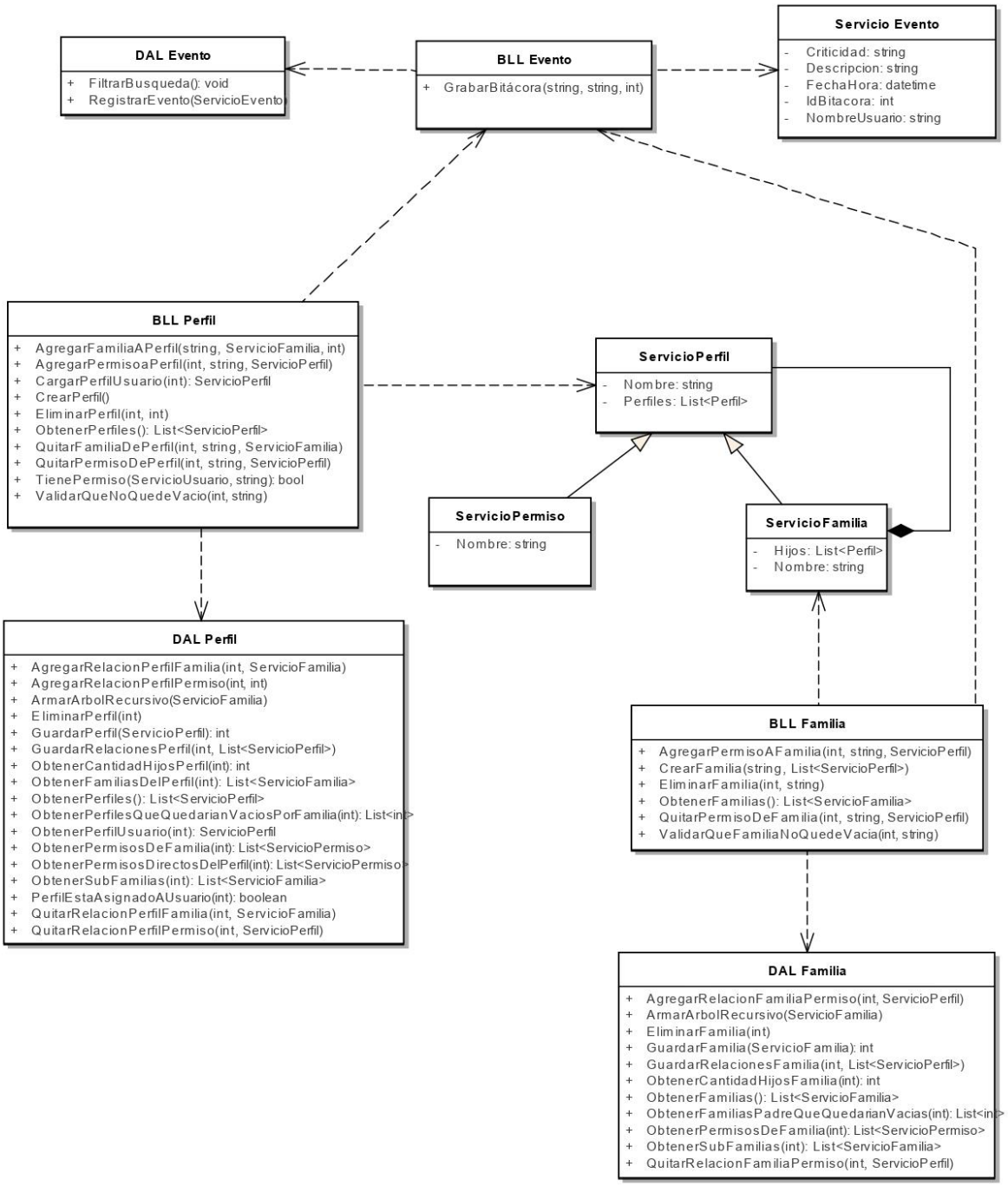
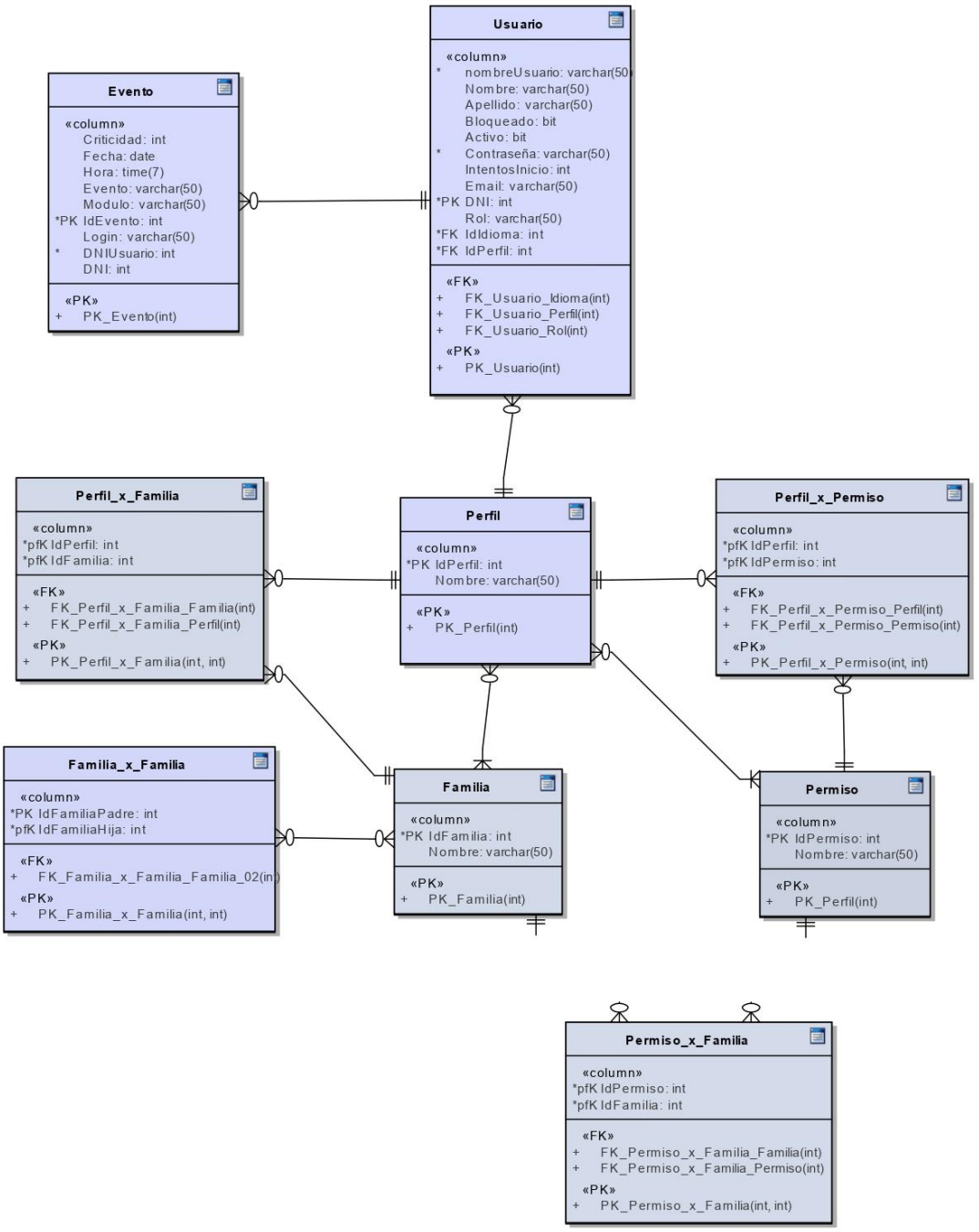


Diagrama Entidad Relación

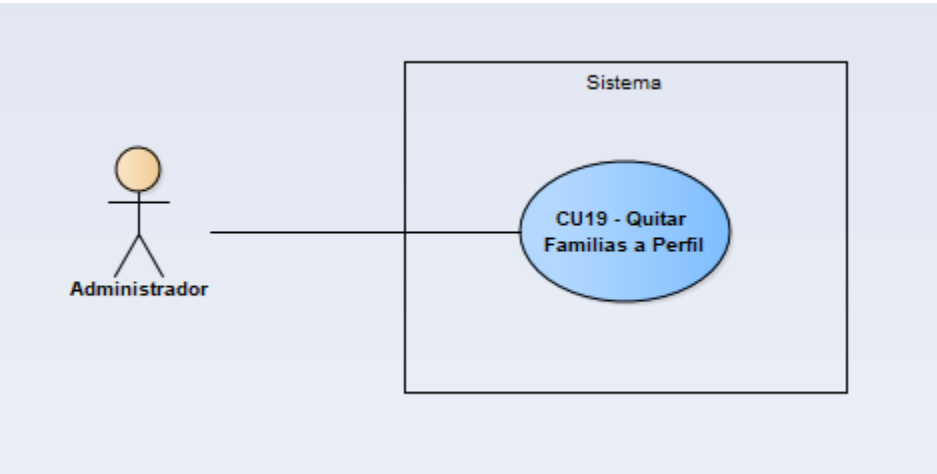


10. Quitar Familias a Perfil

Descomposición Funcional

1. El Sistema inicia la interfaz de gestión de perfiles de seguridad. Recibe la selección del apartado "Perfiles" y produce la petición de verificación y lectura hacia el componente de negocio (Capa GUI).
2. El Sistema recupera la totalidad de los componentes configurados (permisos, perfiles y familias) para popular la interfaz. Recibe la orden de lectura y produce las colecciones de objetos desde la persistencia (Capa DAL).
3. El Sistema procesa la solicitud de desasociación disparada por el usuario al seleccionar los elementos a remover. Recibe los identificadores de los componentes desde la interfaz (Capa GUI).
4. El Sistema verifica que el perfil afectado no quede completamente vacío de permisos o componentes tras la remoción. Recibe el estado actual del perfil y la lista de elementos a quitar, ejecuta la validación de integridad en el negocio y produce un veredicto booleano de viabilidad (Capa BLL y DAL).
5. El Sistema elimina de forma permanente las relaciones correspondientes entre el perfil y las familias seleccionadas. Recibe la orden de actualización de vínculos y produce la baja de los registros en la persistencia (Capa DAL).
6. El Sistema genera el registro de auditoría de la operación para el control de seguridad. Recibe la información del evento de desasociación y del usuario logueado, y produce un nuevo asiento persistido en la Bitácora (Capa DAL).
7. El Sistema notifica al usuario el resultado exitoso de la operación. Recibe la confirmación de la persistencia y produce el despliegue del mensaje de éxito en la interfaz (Capa GUI).

Diagrama de Casos de Uso



Especificación de Casos de Uso

CU19 - QUITAR FAMILIAS A PERFIL

Descripción:	El sistema permite al administrador remover una o más familias de un perfil existente en el sistema.
Actores:	Administrador
Precondiciones:	1. El administrador debe haber iniciado sesión en el sistema. 2. El sistema debe encontrarse operativo y con conexión activa a la base de datos SQL Server. 3. El perfil seleccionado debe tener al menos una familia asignada en la base de datos.
Puntos de extensión:	N/A
Flujo principal:	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Perfiles”.
2	El sistema carga y muestra todos los componentes existentes en el sistema (permisos, perfiles y familias).
3	El administrador selecciona las familias que desea remover y presiona el botón "Quitar Familia a perfil".
4	El sistema consulta la base de datos para verificar que el perfil no quede completamente vacío tras la remoción de los elementos seleccionados.
5	El sistema elimina las relaciones correspondientes en la base de datos.
6	El sistema registra la acción en la bitácora.
7	El sistema informa que las familias fueron removidas correctamente del perfil.
Flujo alternativo:	
4.1.1	El sistema detecta que la remoción de las familias seleccionadas dejaría al perfil sin ningún componente asociado.
4.1.2	El sistema bloquea la operación e informa el error.
Poscondición:	El perfil queda actualizado en la base de datos habiendo desvinculado las familias seleccionadas y la acción queda registrada con éxito en la bitácora del sistema.

Diagrama de Secuencia

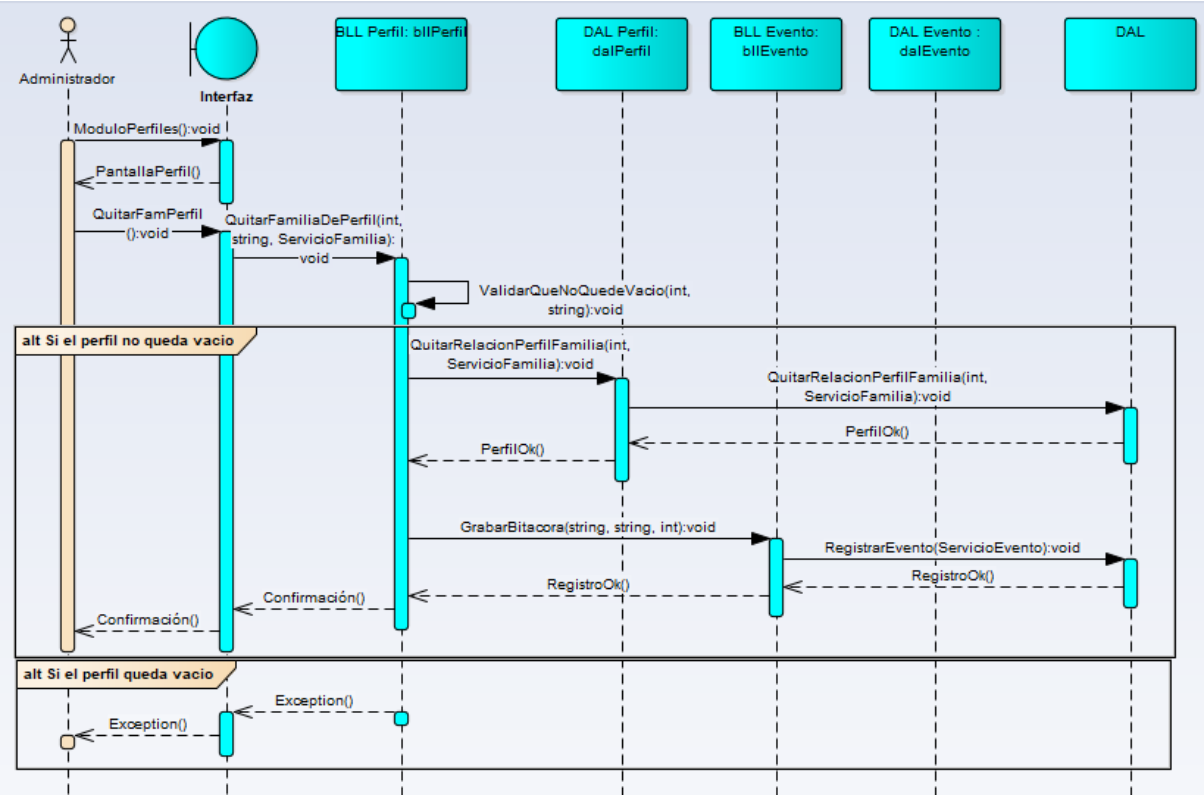


Diagrama de Actividad

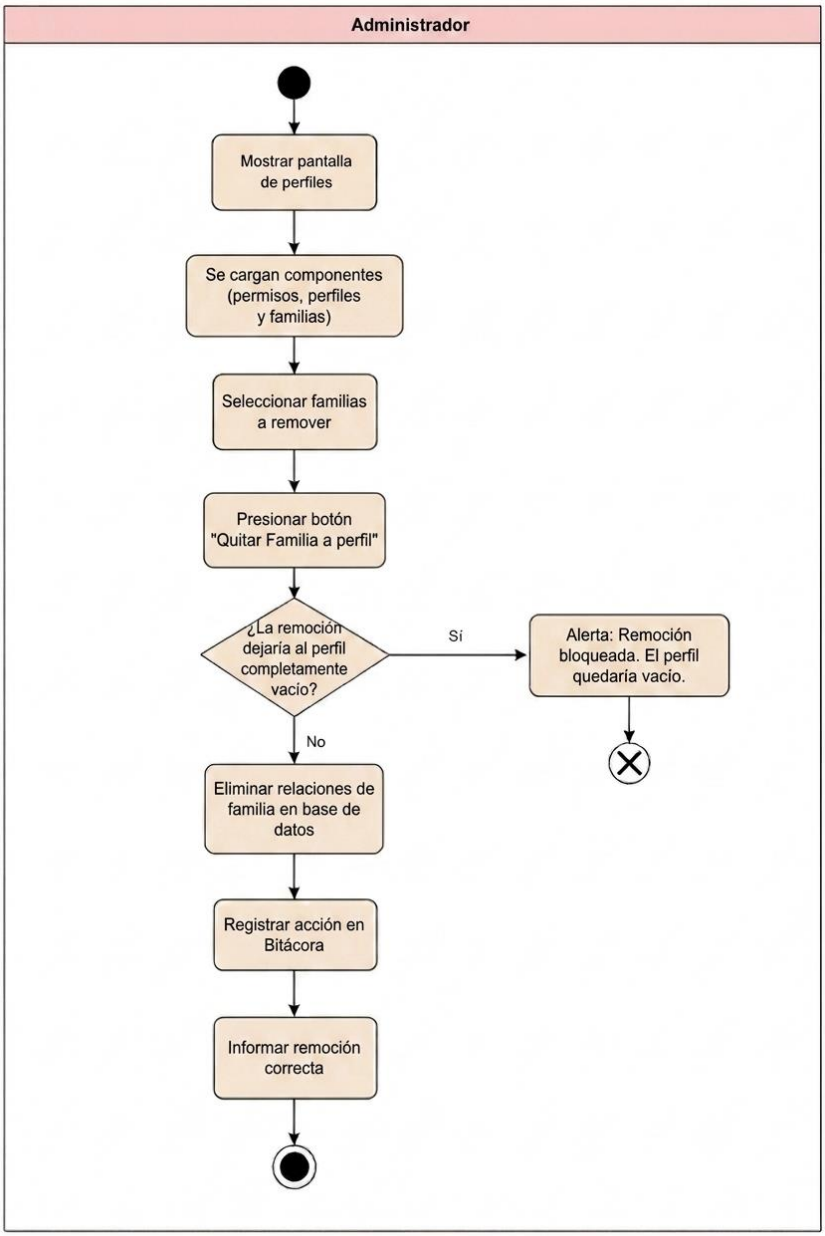
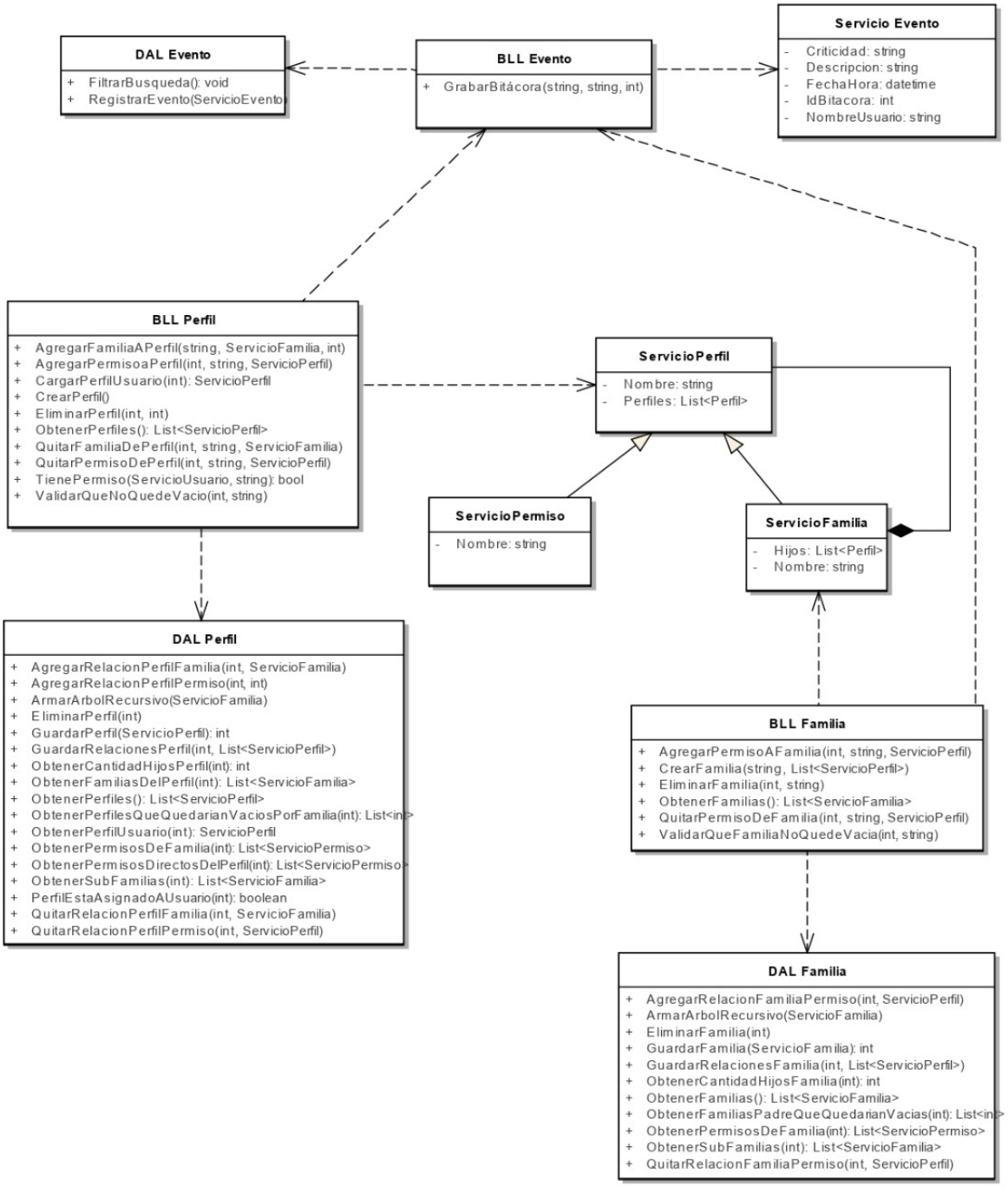


Diagrama de Clases

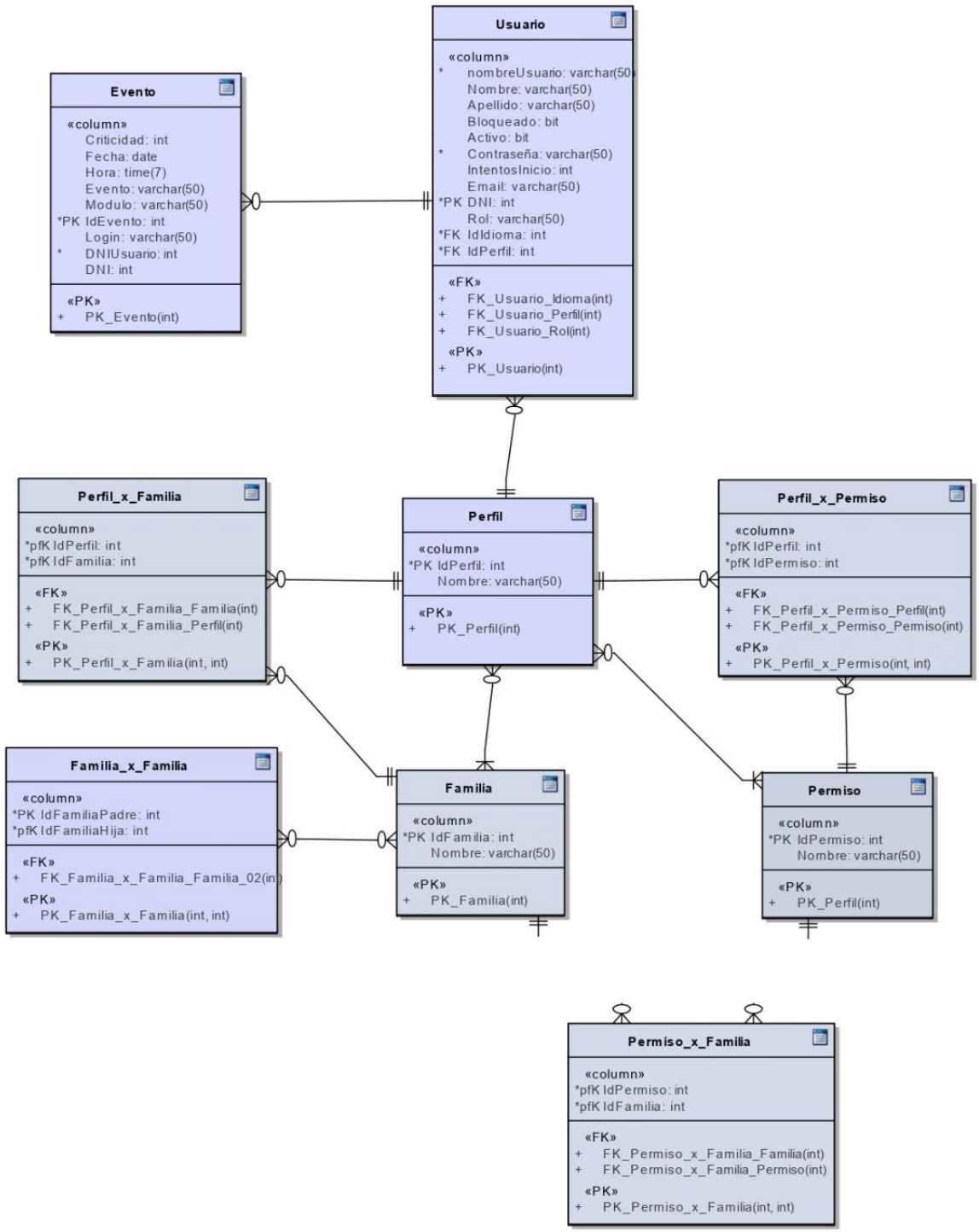
UNIVERSIDAD ABIERTA INTERAMERICANA
Facultad de Tecnología Informática

Materia: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel	
Alumno: Gesualdo Antonini, Sofía.			
Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

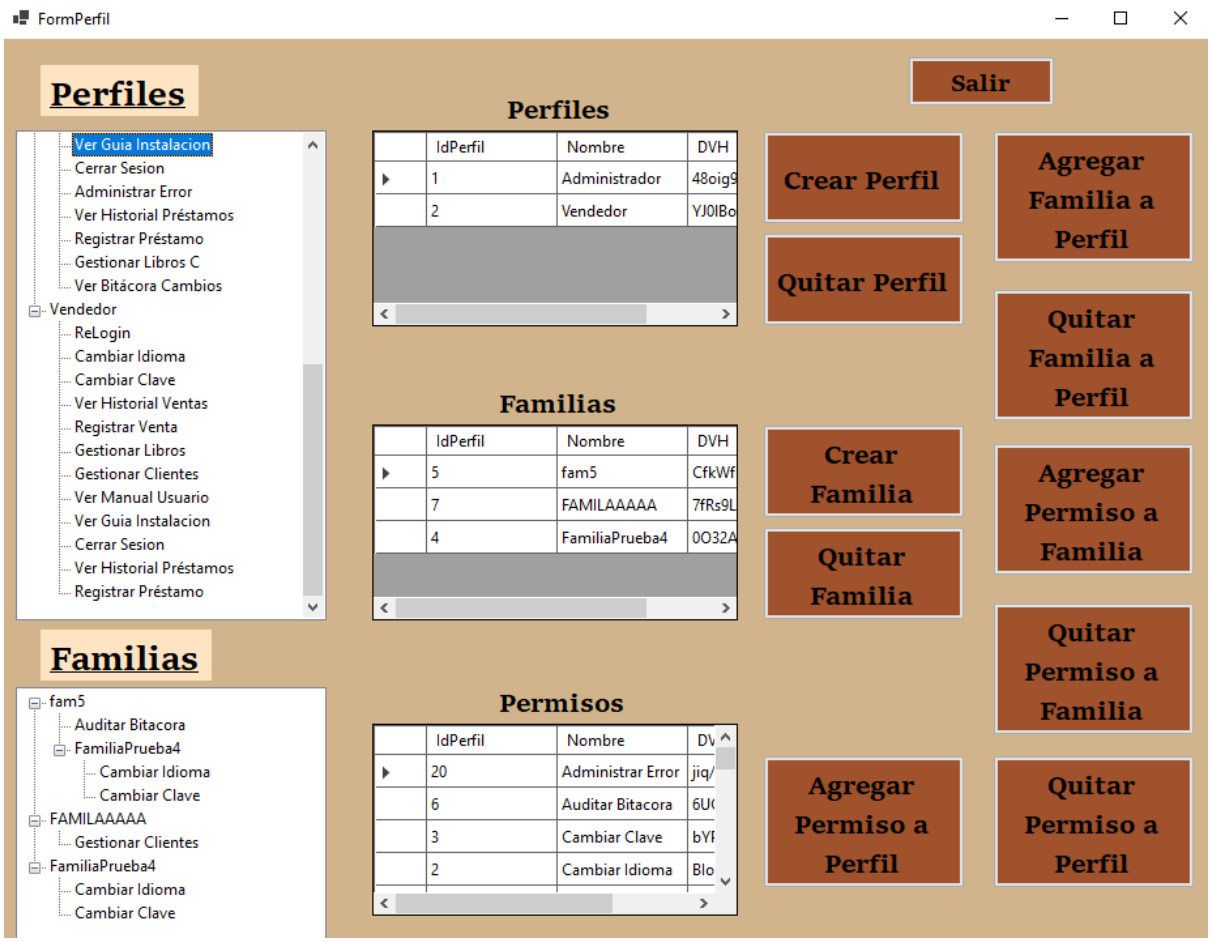


UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel	
	Alumno: Gesualdo Antonini, Sofía.			
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Diagrama Entidad Relación



GUI PERFILES



T05. Idiomas (Documentación)

Implementación del Patrón Observer

Para la funcionalidad de cambio de idioma, se ha seleccionado el patrón de diseño Observer. Esta elección responde a la necesidad de mantener la consistencia en la interfaz de usuario ante cambios globales de estado. En un sistema donde múltiples pantallas y componentes gráficos deben actualizarse de forma sincronizada tras la acción del usuario, el patrón Observer permite establecer una relación de dependencia uno-a-muchos.

De esta forma, el objeto que gestiona la sesión (Sujeto) notifica automáticamente a todos los componentes visuales (Observadores) que la cultura ha cambiado, garantizando que el refresco de los textos sea inmediato y desacoplado. Esta implementación elimina la

necesidad de que la lógica de cambio de idioma conozca explícitamente a cada formulario o control, cumpliendo con los principios de bajo acoplamiento y alta cohesión fundamentales en la arquitectura de nuestro sistema.

5.1 Descomposición Funcional CU9 “Cambiar Idioma”

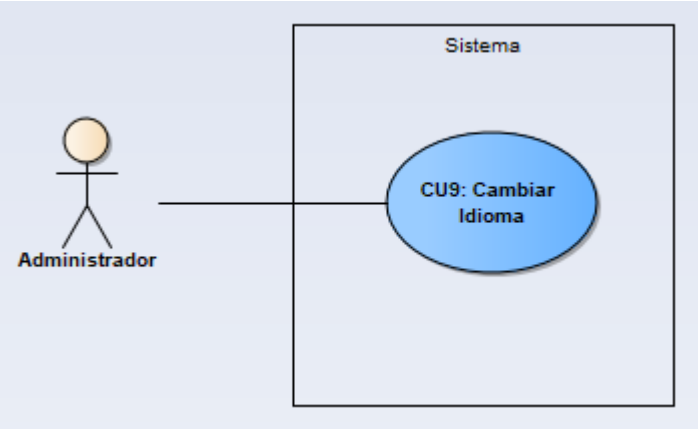
1. El Usuario selecciona el botón de “Cambiar idioma” en la barra del menú principal.
2. El Sistema lee de forma local el archivo de texto serializado (JSON) para identificar los idiomas disponibles.
3. El Sistema despliega la lista con las opciones de idioma recuperadas.
4. El Usuario selecciona el idioma deseado de la lista desplegable.
5. El Sistema captura el código de cultura asociado a la selección del usuario.
6. El Sistema actualiza la propiedad de cultura de la sesión del usuario en la memoria RAM de la aplicación.
7. El Sistema refresca los controles visuales de todas las pantallas activas mostrando los textos traducidos correspondientes.
8. El Sistema registra el evento de cambio de idioma exitoso en la Bitácora de eventos del sistema.

Nota Técnica sobre la Localización del Archivo de Idiomas:

Para garantizar que la aplicación sea 100% portable y ejecutable en cualquier entorno sin depender de configuraciones locales de disco, el archivo serializado idiomas.json se encuentra integrado directamente dentro de la estructura de carpetas de la solución.

Mediante la propiedad de Visual Studio "Copiar en el directorio de salida = Copiar si es posterior", el entorno de ejecución de .NET se encarga de replicar automáticamente el archivo JSON dentro del directorio binario de compilación. De este modo, el sistema accede al recurso utilizando la propiedad dinámica AppDomain.CurrentDomain.BaseDirectory, asegurando que el proyecto funcione de forma inmediata al ser descargado y ejecutado por los docentes sin necesidad de modificar ninguna ruta manual.

5.2 Diagrama de Caso de Uso



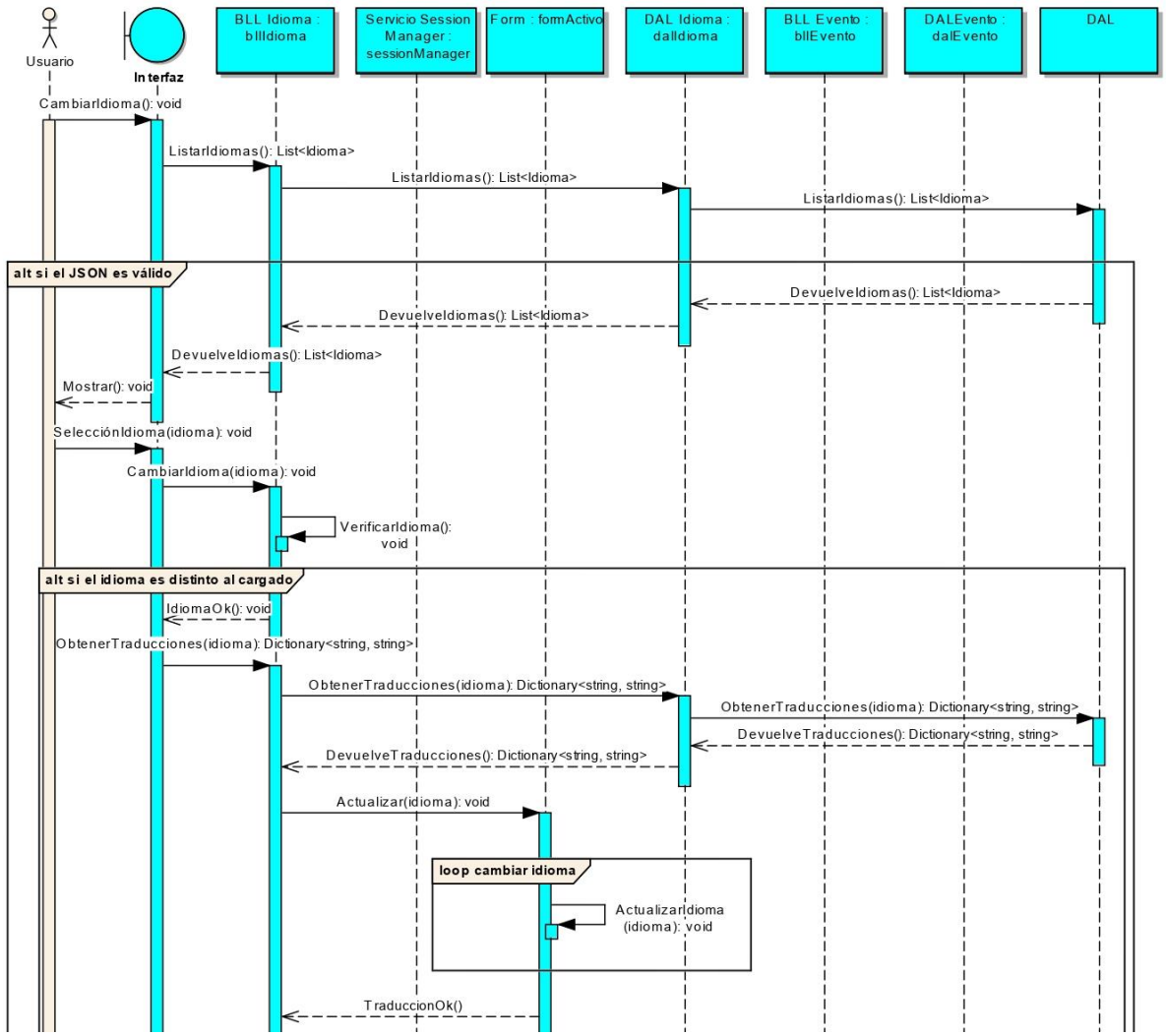
5.3 Especificación

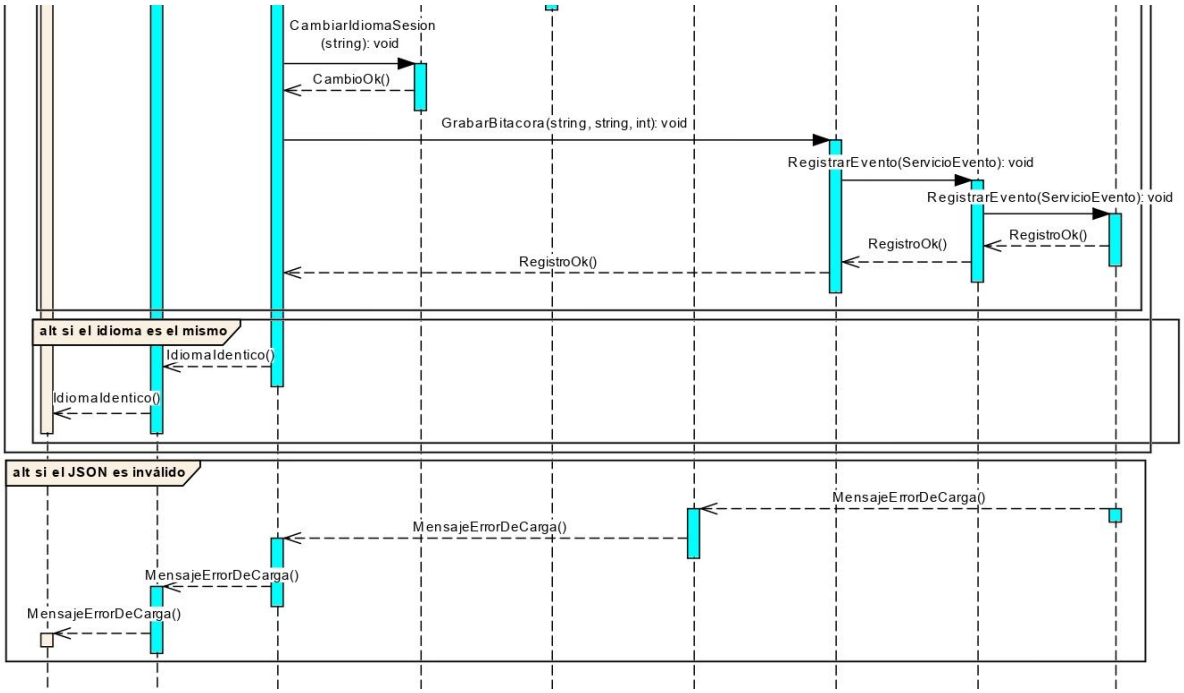
CU9 - CAMBIAR IDIOMA

Descripción:	El sistema permite al usuario modificar el idioma de la interfaz gráfica en tiempo de ejecución.
Actores:	Usuario
Precondiciones:	1. El usuario debe contar con una sesión activa registrada en el Session Manager. 2. El sistema debe encontrarse operativo y ejecutando la interfaz principal. 3. Los archivos JSON con las traducciones de los idiomas soportados deben estar accesibles en el directorio del sistema.
Puntos de extensión:	N/A
Flujo principal:	
1	El usuario selecciona el botón de “Cambiar idioma” en la barra del menú principal.
2	El sistema lee de forma local el archivo de texto serializado (JSON) para identificar los idiomas disponibles.
3	El sistema despliega la lista con las opciones de idioma recuperadas.
4	El usuario selecciona el idioma deseado de la lista desplegable.
5	El sistema captura el código de cultura asociado a la selección.
6	El sistema actualiza la propiedad de cultura de la sesión del usuario en memoria RAM.
7	El sistema refresca los controles visuales mostrando los textos traducidos.
8	El sistema registra el evento de cambio de idioma en la Bitácora.
Flujo alternativo:	
2.1.1	El sistema detecta que el formato del archivo JSON de idiomas es inválido.
2.1.2	El sistema muestra un mensaje de alerta en la interfaz informando que no se pudieron cargar los idiomas adicionales.
4.1.1	El sistema detecta que el código de cultura del idioma seleccionado por el usuario es idéntico a la propiedad de cultura actualmente activa en la sesión.
4.1.2	El sistema cancela las operaciones de actualización en

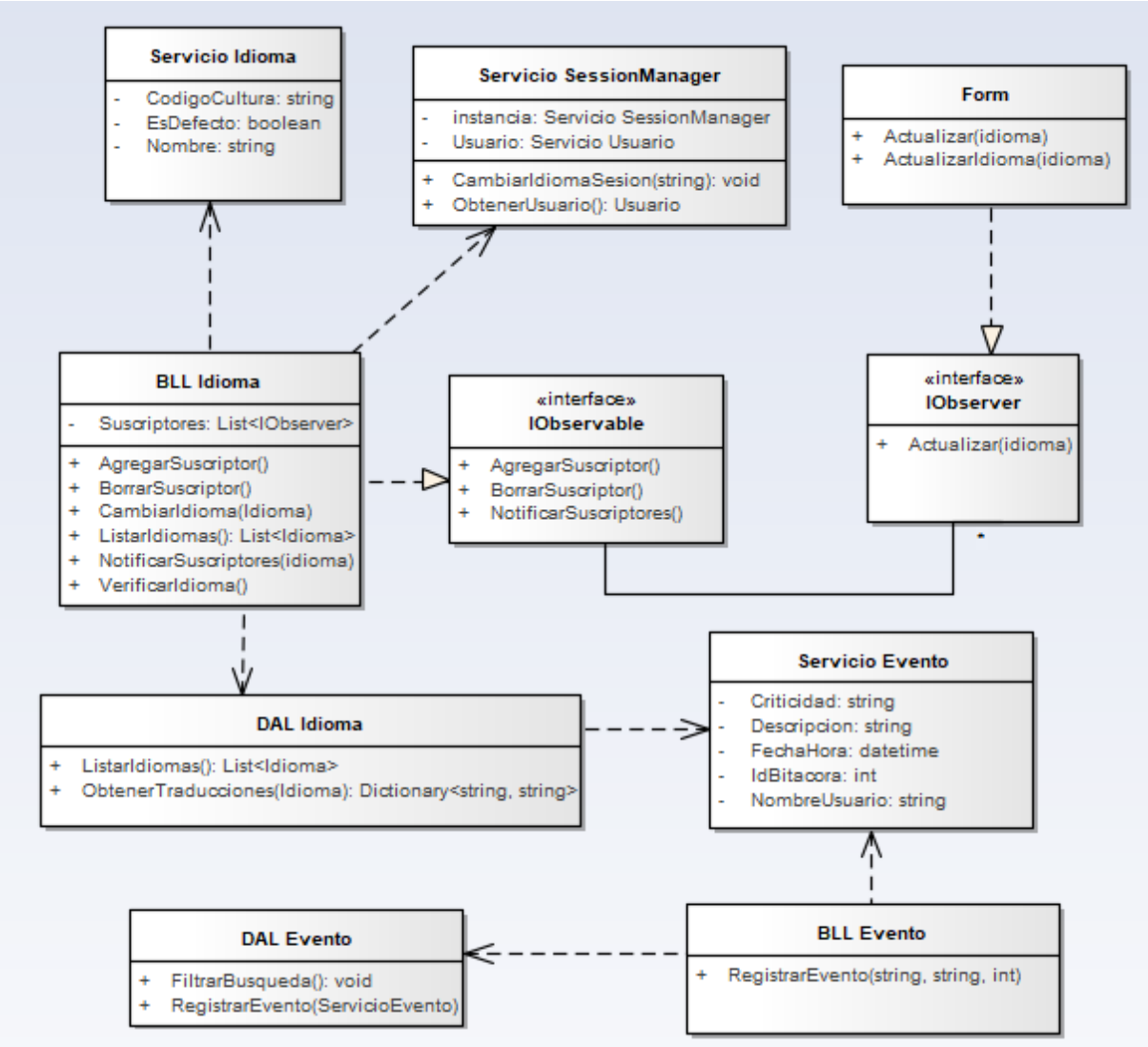
	memoria RAM y notifica la situación.
Poscondición:	El nuevo usuario queda registrado correctamente en el sistema con su contraseña encriptada, rol asignado y registro correspondiente en la bitácora.

5.4 Diagrama de Secuencia

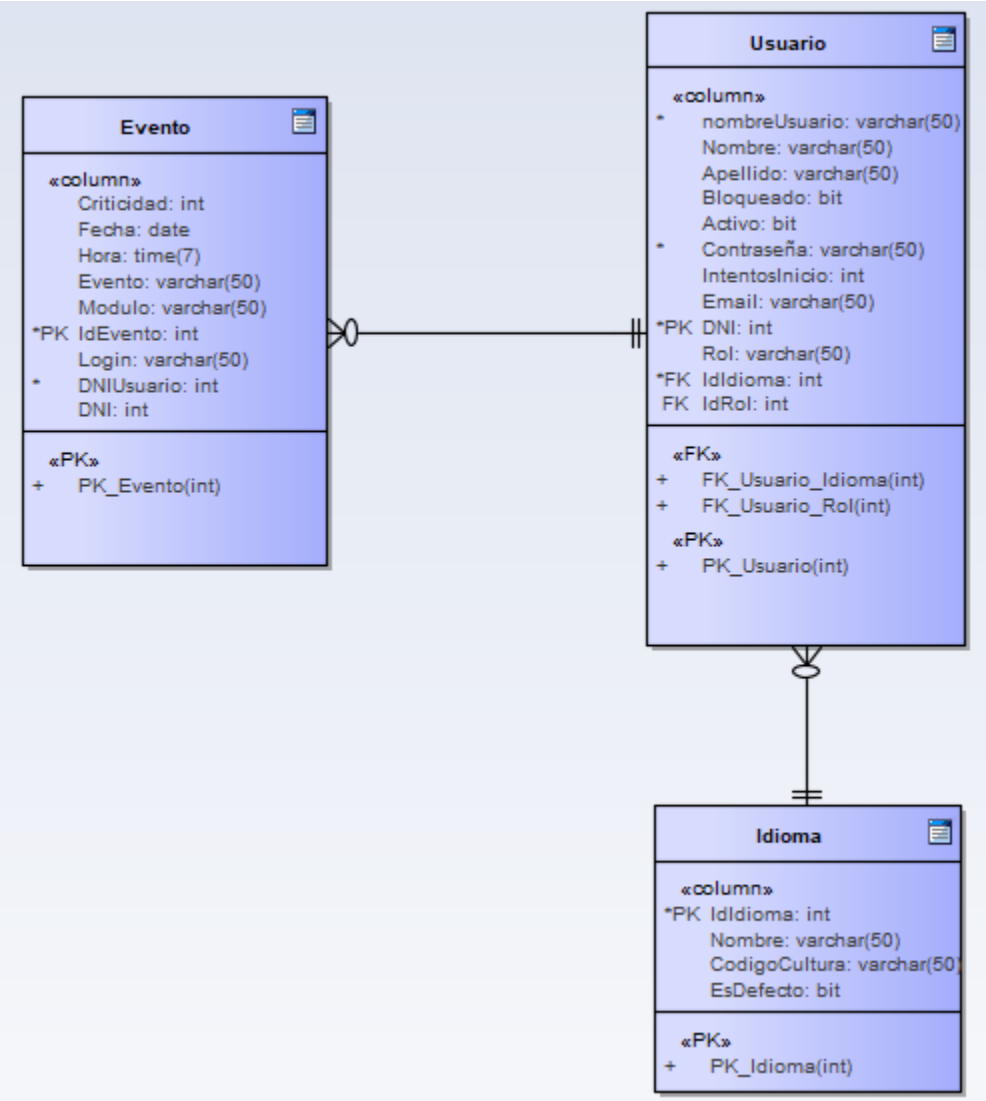




5.5 Diagrama de Clases



5.6 Diagrama Entidad Relación


Usuario
«column»
* nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
* Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
*PK DNI: int
Rol: varchar(50)
*FK IdIdioma: int
FK IdRol: int
«FK»
+ FK_Usuario_Idioma(int)
+ FK_Usuario_Rol(int)
«PK»
+ PK_Usuario(int)
Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
* DNIUsuario: int
DNI: int
«PK»
+ PK_Evento(int)
Idioma
«column»
*PK IdIdioma: int
Nombre: varchar(50)
CodigoCultura: varchar(50)
EsDefecto: bit
«PK»
+ PK_Idioma(int)

5.7 GUI



T06. Bitácora de Eventos

1. Registro de Eventos (BackEnd)

El registro de eventos en la bitácora es un mecanismo de seguridad automatizado y que atraviesa toda la aplicación. No tiene casos de uso asociados porque su ejecución es transparente e invisible para el usuario; se dispara automáticamente por el comportamiento interno del software ante transacciones que deben ser persistidas en la base de datos.

El registro de eventos es el primer momento en el ciclo de vida de la bitácora. Ocurre de forma reactiva y en tiempo de ejecución. Cada vez que una clase de la capa de lógica de negocio ejecuta una operación que altera el estado de la base de datos o compromete la seguridad, invoca de manera automática a la Bitácora de Eventos de la Capa de Servicios. Este método recibe el evento y carga una nueva fila en SQL Server de manera asincrónica, sin que el usuario intervenga en dicha acción.

Para garantizar la integridad y trazabilidad de los datos, cada registro en la bitácora es marcado con un identificador único, una fecha y el usuario que disparó la acción. Este diseño asegura que, ante cualquier auditoría, el sistema sea capaz de reconstruir la línea de

tiempo exacta de los eventos, permitiendo identificar al actor responsable, el módulo afectado y el impacto de la operación mediante los niveles de criticidad definidos.

Criterio de Criticidad

Para jerarquizar la gravedad de las acciones ocurridas en el sistema, los eventos se categorizan bajo tres niveles de criticidad:

1. **Criticidad 1 (Alta)**: Registra eventos que ponen en riesgo el control del sistema o que manejan datos sensibles, como el bloqueo automático de cuentas de usuario por superar los 3 intentos fallidos de inicio de sesión.
2. **Criticidad 2 (Media)**: Agrupa todas las alteraciones directas sobre el apartado "Maestro" de los datos del negocio en las tablas físicas, como la eliminación de un producto.
3. **Criticidad 3 (Baja)**: Registra acciones que no alteran la base de datos, pero sirven para el control de tiempos de operación, como el cambio de idioma de la aplicación.

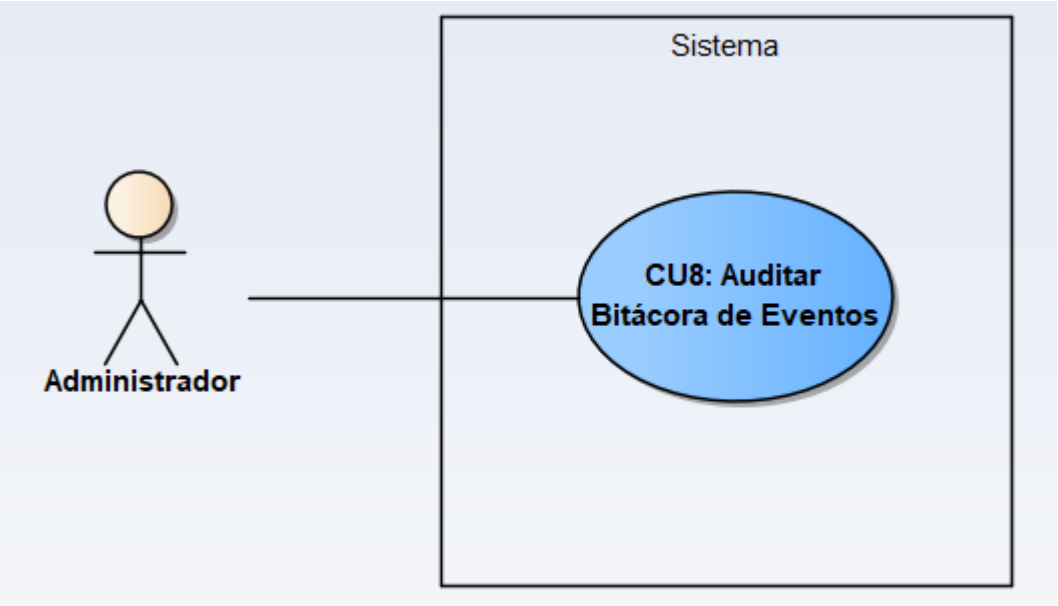
2. Auditoría de Eventos (FrontEnd)

1. Descomposición Funcional CU8 "Auditar Bitácora de Eventos"

1. El usuario administrador ingresa al sistema validando sus credenciales en la pantalla de Login.
2. El sistema muestra la interfaz gráfica del menú principal habilitando los accesos correspondientes a su perfil de privilegios.
3. El administrador selecciona el menú "Administración" y hace clic en la opción "Bitácora Eventos".
4. El sistema despliega el módulo de auditoría mostrando una grilla de datos (DataGridView) que lista de forma cronológica los registros históricos almacenados en la base de datos. Cada registro expone los atributos: NombreUsuario(texto), Fecha (datetime), Hora(datetime), Módulo, Evento y Criticidad.
5. El administrador interactúa con los controles de la interfaz gráfica pudiendo seleccionar un rango de búsqueda mediante los atributos de filtrado como, FechaInicio (fecha) y FechaFin (fecha), por Nombre y Apellido, Nombre de usuario, Módulo, Evento y Criticidad.
6. El administrador presiona el botón "Aplicar".
7. El sistema valida que las fechas ingresadas sean consistentes (FechaInicio menor o igual a FechaFin).
8. El sistema invoca a la capa de negocio (BLLBitacora) para procesar los criterios de filtrado seleccionados.

9. La capa de negocio solicita a la capa de datos (DALBitacora) la extracción física de las filas coincidentes desde SQL Server.
10. El sistema refresca la grilla visual de la pantalla mostrando únicamente los eventos que cumplen con los atributos del filtro aplicado, permitiendo al administrador auditar las operaciones ocurridas en el negocio.

2. Diagrama de Caso de Uso / Especificación Caso de Uso



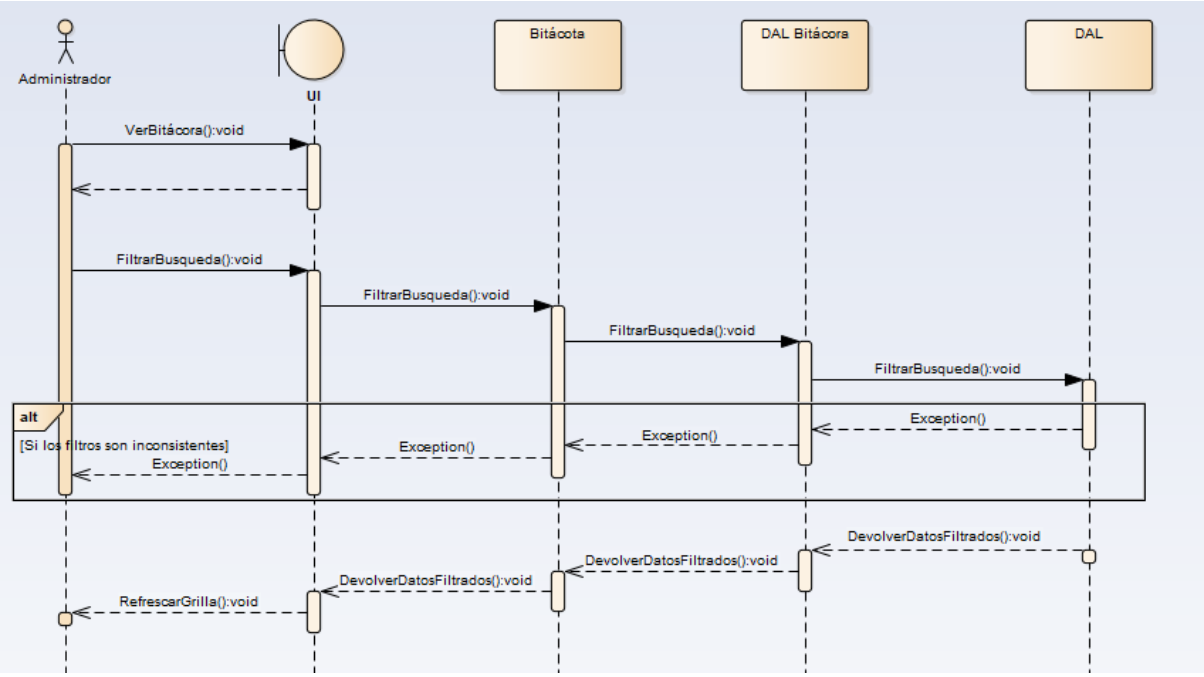
CU8 - AUDITAR BITÁCORA DE EVENTOS

Descripción:	El sistema permite al usuario administrador visualizar, buscar y filtrar el registro histórico de operaciones técnicas, de persistencia y de seguridad grabadas por el BackEnd.
Actores:	Administrador

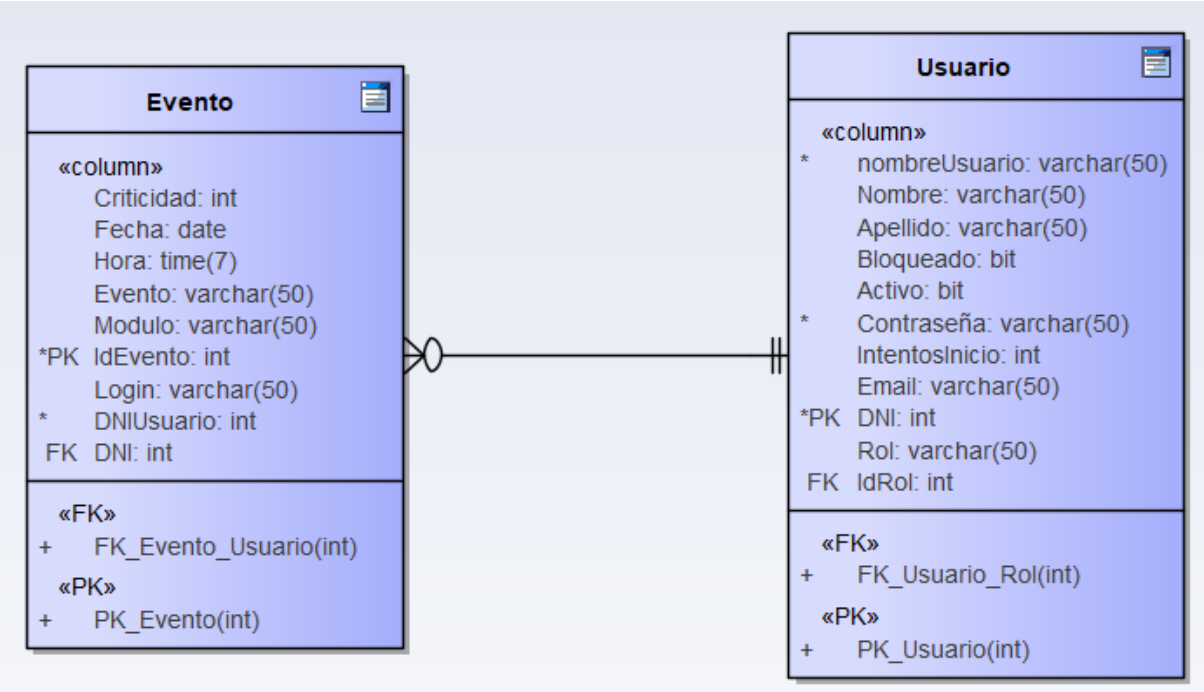
Precondiciones:	1. El usuario debe haber iniciado sesión con un rol de nivel Administrador. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server.
Puntos de extensión:	N/A
Flujo Principal	
1	El administrador selecciona el apartado de “Administración” y presiona la opción “Bitácora Eventos”.
2	El sistema despliega el módulo con todos los eventos realizados históricamente.
3	El administrador realiza la búsqueda según parámetros personalizados.
4	El sistema valida que los datos ingresados como parámetros sean válidos.
5	El sistema extrae los datos filtrados de la base de datos.

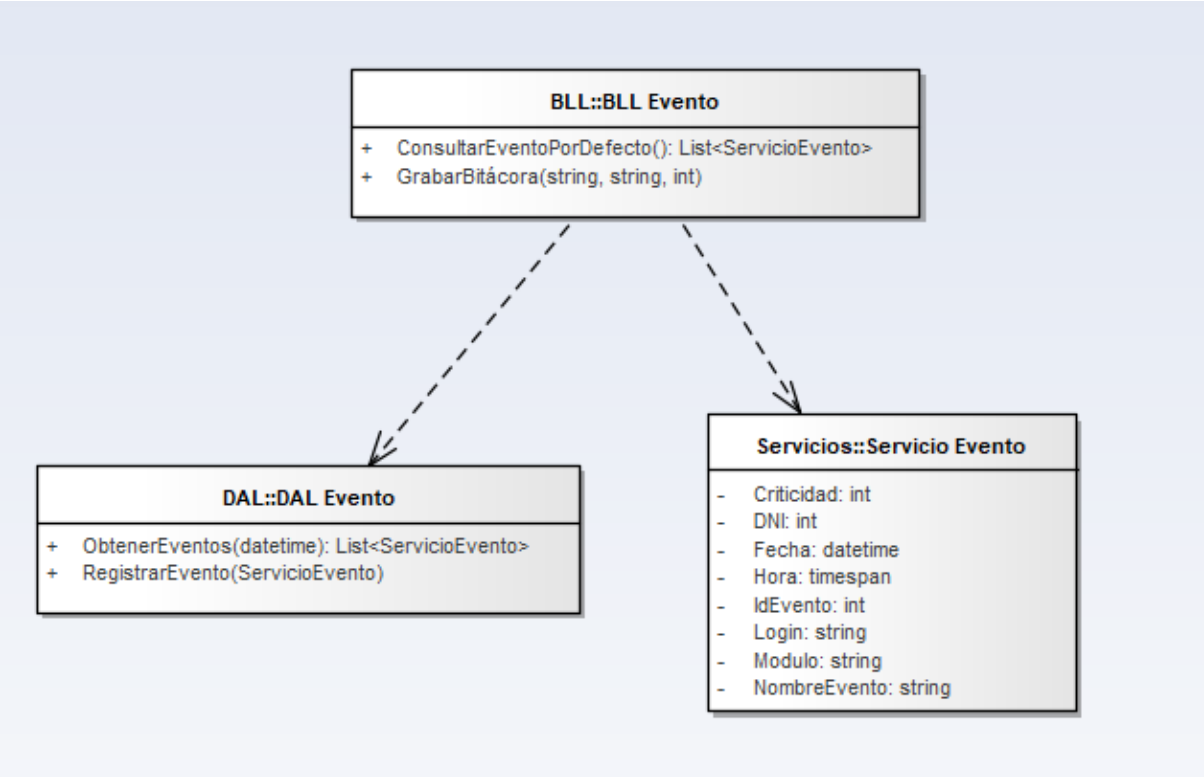
6	El sistema refresca la grilla, enseñando las filas resultantes de la auditoría.
Flujo Alternativo	
4.1.1	El administrador ingresa datos inconsistentes.
4.1.2	El sistema arroja una excepción notificando la inconsistencia.
Poscondición:	El administrador visualiza en la terminal el reporte filtrado de auditoría sin alterar el estado histórico de los registros (operación de solo lectura).

3. Diagrama de Secuencia de Sistema

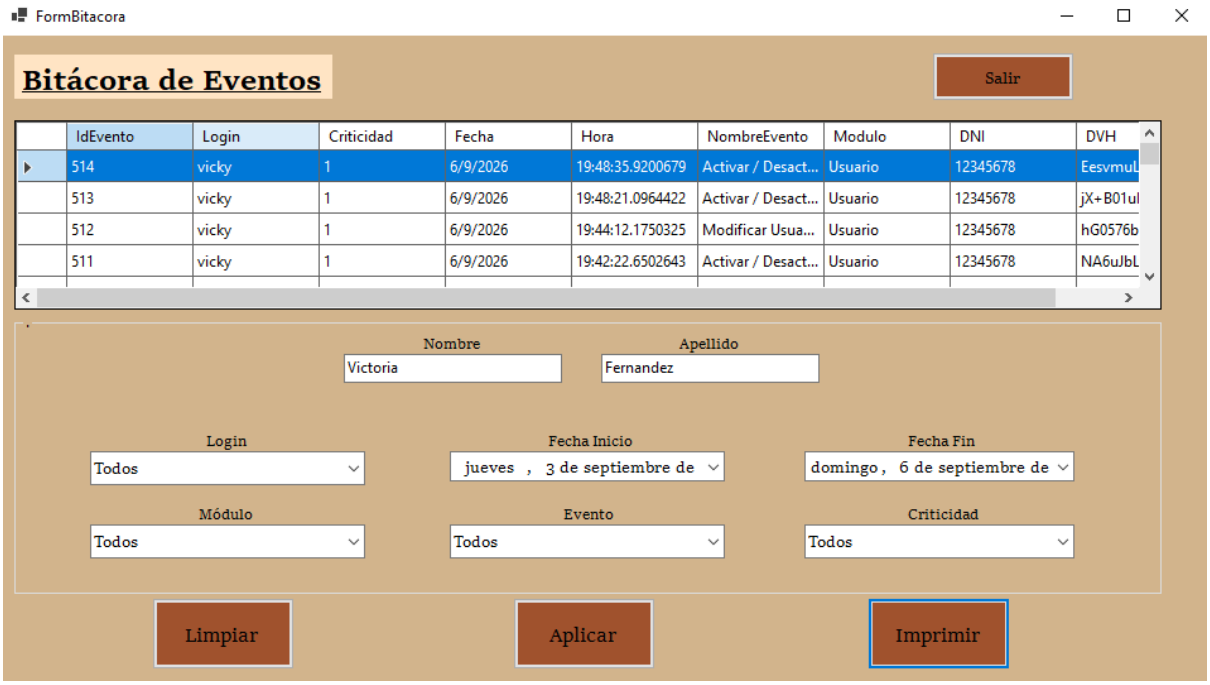


4. Diagrama Entidad Relación / Diagrama de Clases




BLL::BLL Evento
+ ConsultarEventoPorDefecto(): List<ServicioEvento>
+ GrabarBitácora(string, string, int)
DAL::DAL Evento
+ ObtenerEventos(datetime): List<ServicioEvento>
+ RegistrarEvento(ServicioEvento)
Servicios::Servicio Evento
- Criticidad: int
- DNI: int
- Fecha: datetime
- Hora: timespan
- IdEvento: int
- Login: string
- Modulo: string
- NombreEvento: string

5. GUI


FormBitacora
Bitácora de Eventos
Salir
IdEvento Login Criticidad Fecha Hora NombreEvento Modulo DNI DVH
514 vicky 1 6/9/2026 19:48:35.9200679 Activar / Desact... Usuario 12345678
513 vicky 1 6/9/2026 19:48:21.0964422 Activar / Desact... Usuario 12345678
512 vicky 1 6/9/2026 19:44:12.1750325 Modificar Usua... Usuario 12345678
511 vicky 1 6/9/2026 19:42:22.6502643 Activar / Desact... Usuario 12345678
Nombre
Victoria
Apellido
Fernandez
Login
Todos
Fecha Inicio
jueves , 3 de septiembre de
Fecha Fin
domingo , 6 de septiembre de
Módulo
Todos
Evento
Todos
Criticidad
Todos
Limpiar
Aplicar
Imprimir

T06.2 Bitácora de Cambios

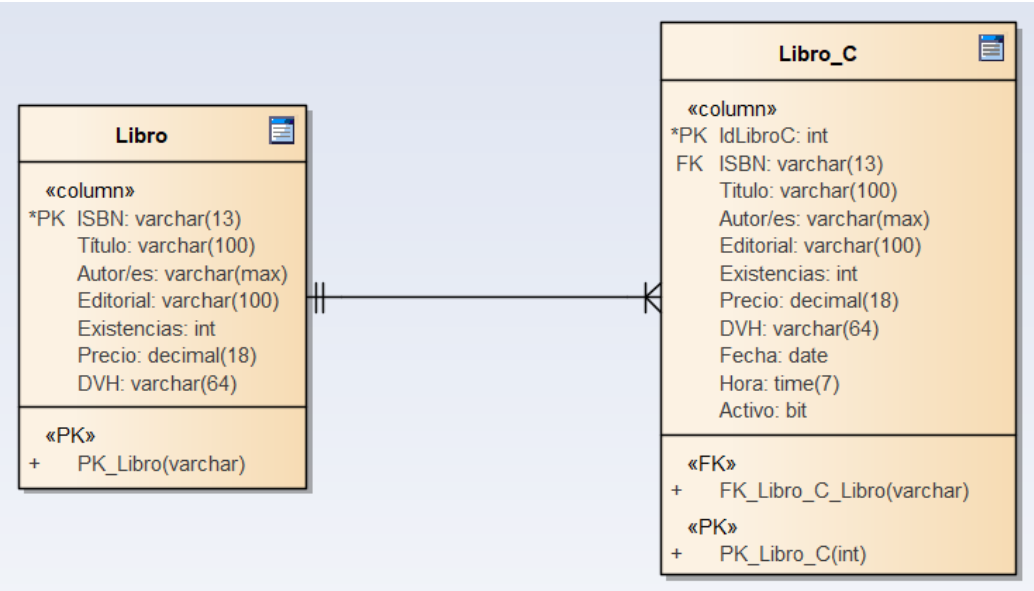
1. Registro de Cambios (BackEnd)

Descripción Funcional

La bitácora de cambios es una herramienta automatizada para persistir y controlar el historial de versiones de la entidad principal del negocio (libro). A diferencia de la Bitácora de Eventos, que únicamente registra acciones del sistema para su lectura, la Bitácora de Cambios tiene como propósito registrar cada uno de los estados que adopta un y permitir la reversión (rollback) de sus datos hacia cualquier versión histórica previa.

Este mecanismo opera de forma invisible para el usuario, ya que no depende de la interacción manual, sino que se dispara automáticamente en el motor de base de datos SQL Server cada vez que se produce una transacción que altera los datos de la tabla Libro. Para identificar y poder trazar las versiones históricas, cada una de ellas que se almacena en la tabla Libro_C incorpora un campo destinado a la fecha, otro a la hora, y un indicador de estado Act (booleano). En todos los casos, para un mismo libro, únicamente la versión que coincide con el estado actual de la tabla Libro posee un 1 como valor en el campo Act mientras que todas las versiones históricas anteriores quedan identificadas con un 0.

DER



Explicación de los Triggers

La persistencia y sincronización entre la tabla operacional Libro y la tabla histórica Libro_C se delega a disparadores (triggers) en SQL Server. Tenemos dos Triggers ubicados en la tabla Libro, que van a agregar nuevos registros en la tabla Libro_C cuando se agregue un nuevo libro o cuando se elimine (baja lógica) algún libro existente. También tenemos un trigger perteneciente a la tabla Libro_C, que va a ejecutarse cuando cambiemos el estado Act a 1 de algún registro de dicha tabla, desencadenando los cambios persistentes del libro involucrado en la tabla madre Libro.

Trigger TR_Libro_Actualizado (AFTER INSERT, UPDATE)

Se encuentra en la tabla Libro, y se dispara de manera automática inmediatamente después de que se ejecute una sentencia INSERT o UPDATE sobre cualquier registro de la tabla Libro, como el alta de un nuevo ejemplar, modificación de información, cambios en el stock, entre otros.

Trigger TR_Libro_Eliminado (AFTER DELETE)

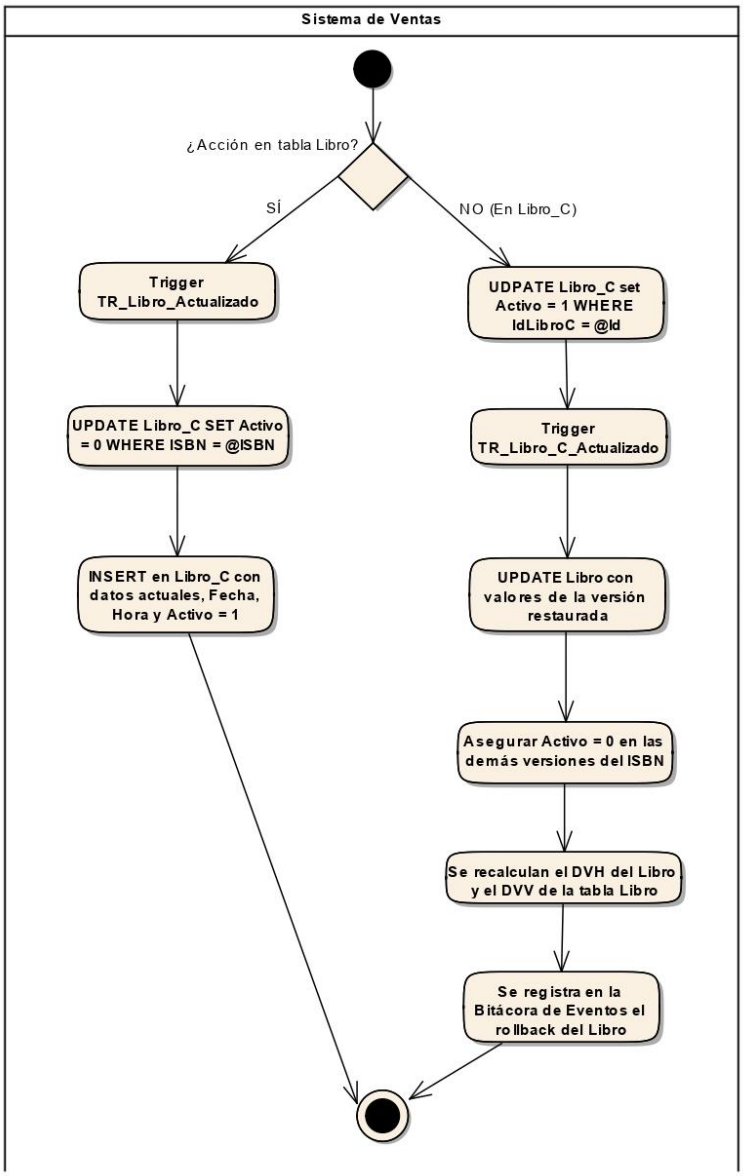
También pertenece a la tabla Libro, y se dispara tras una eliminación física (DELETE) en la tabla Libro.
En la base de datos del sistema, la tabla Libro ya cuenta con un disparador que impide que cualquier usuario o consulta ejecute una instrucción DELETE de la tabla Libro, impidiendo una eliminación física de un registro y ejecutando internamente un UPDATE en el campo Act para realizar simplemente una baja lógica. Por ende, para el motor la baja se procesa como una actualización, disparando automáticamente el Trigger TR_Libro_Actualizado.
Sin embargo, este disparador se mantiene como una medida de seguridad extrema, ya que únicamente entraría en acción si un administrador de base de datos deshabilitara explícitamente el trigger TR_Libro_BajaLogica y forzara un borrado físico. En ese escenario, este trigger previene inconsistencias marcando Act como 0 en Libro_C y preservando el historial. Para esto, identifica el ISBN de los registros eliminados y actualiza todas las versiones de dicho libro en Libro_C marcando su estado Act = 0. De esta forma, el historial completo se preserva en Libro_C con fines de auditoría, reflejando que el libro ya no existe en la tabla madre.

Trigger TR_Libro_Actualizado (AFTER UPDATE)

Este trigger se encuentra en la tabla de auditoría Libro_C. Se ejecuta cuando desde la interfaz de usuario de la bitácora de cambios se selecciona una versión histórica previa y se presiona

el botón "Activar", lo que genera una sentencia UPDATE en Libro_C, modificando el campo booleano Act del registro seleccionado. De esta manera, obtiene los atributos completos del registro seleccionado en Libro_C y sincroniza la tabla Libro realizando un UPDATE que sobre escribe los valores vigentes con los datos de la versión histórica elegida para el libro seleccionado. Por último, desactiva en Libro_C cualquier otra versión del mismo ISBN para garantizar que solo la versión recién restaurada se mantenga con Act = 1.

Diagrama de Actividad

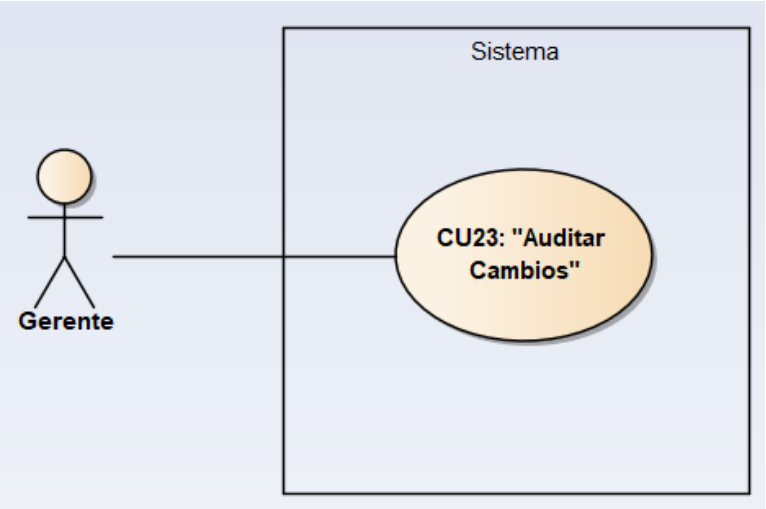


2. Auditoría de Cambios (FrontEnd)

Descomposición funcional CU23 "Auditar cambios"

1. El gerente inicia sesión en el sistema con el permiso de "gerente".
2. El gerente ingresa a la opción de "Libros C" en la sección "Maestro".
3. El sistema abre una pantalla donde se pueden visualizar todos los cambios que se hicieron en la tabla Libro dentro de una grilla.
4. El gerente puede realizar una consulta o puede realizar búsquedas con filtros, como por código de producto o por fechas.
5. El gerente carga su consulta, selecciona un registro de la grilla y presiona el botón "Aplicar".
6. El sistema le pide confirmación al gerente para realizar la acción.
7. El gerente confirma la operación.
8. El sistema realiza los cambios y actualiza la pantalla con todos los cambios aplicados.

Diagrama de caso de uso CU23 "Auditar cambios"



Especificación de caso de uso CU23 "Auditar cambios"

ID y Nombre: CU23 "Auditar cambios"
Objetivo: Permite al actor auditar el historial de cambios y modificaciones sobre los libros del negocio, y ofrecer la capacidad de revertir (rollback) el estado de un libro a una versión previa seleccionada.
Actor principal: Gerente

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

Precondiciones:
- El usuario debe haber iniciado sesión en el sistema y poseer el permiso Gestionar Libros C.
- Deben existir registros en la tabla Libro.

Punto de extensión: N/A

Escenario principal:
1. El usuario ingresa a la opción de “Libros C” en la sección “Maestro”.
2. El sistema abre una pantalla donde se puede visualizar la grilla de versiones históricas de los libros.
3. El usuario define filtros opcionales (ISBN, Título, Fecha Desde, Fecha Hasta) y presiona "Aplicar".
4. El sistema valida los filtros y muestra los registros correspondientes en la grilla.
5. El usuario selecciona una versión histórica no activa y pulsa el botón "Activar".
6. El sistema solicita confirmación mediante un diálogo.
7. El usuario confirma la acción.
8. El sistema actualiza Libro_C, restaura los datos en Libro, recalcula DVH/DVV y registra la acción en la bitácora de eventos.
9. El sistema emite un mensaje de éxito y refresca la pantalla.

Escenario alternativo:
5.1.1 El usuario presiona "Activar" sobre una fila que ya posee el campo Act = 1.
5.1.2 El sistema informa: “La versión seleccionada ya es la versión activa vigente” y cancela la operación.
7.1.1 El usuario presiona "Cancelar" en el diálogo de confirmación.
7.1.2 La operación se cancela sin modificar datos.

Postcondiciones:
- El administrador realizó una auditoría de los cambios.
- La tabla Libro quedó actualizada con los valores históricos.
- La acción quedó registrada en la Bitácora de Eventos.

Diagrama de secuencia de CU23 “Auditar cambios”

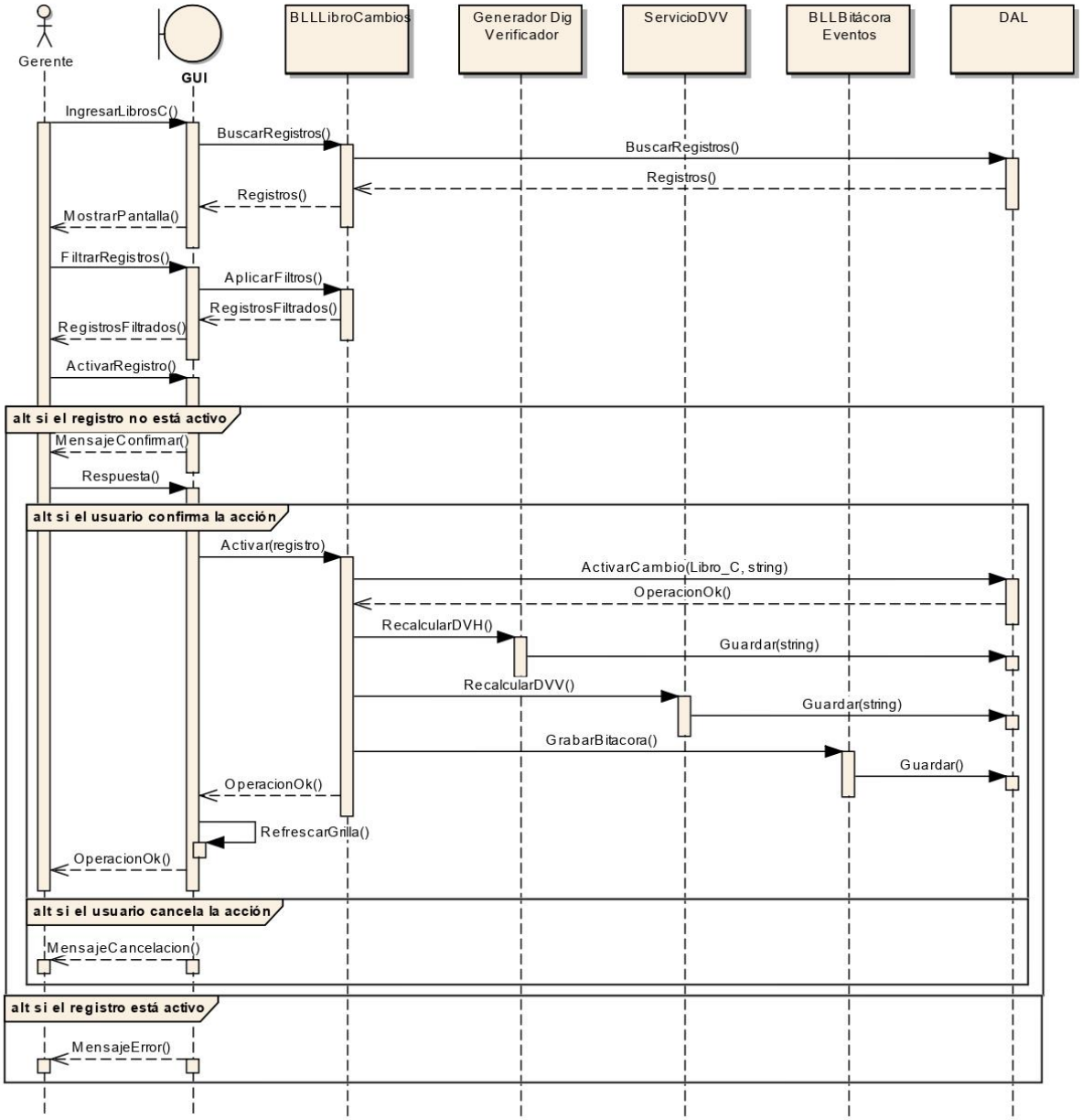

Gerente
GUI
BLLLibroCambios
Generador Dig Verificador
ServicioDVV
BLLBitácora Eventos
DAL
IngresarLibrosC()
BuscarRegistros()
BuscarRegistros()
Registros()
Registros()
MostrarPantalla()
FiltrarRegistros()
AplicarFiltros()
RegistrosFiltrados()
RegistrosFiltrados()
ActivarRegistro()
alt si el registro no está activo
MensajeConfirmar()
Respuesta()
alt si el usuario confirma la acción
Activar(registro)
ActivarCambio(Libro_C, string)
OperacionOk()
RecalcularDVH()
Guardar(string)
RecalcularDVV()
Guardar(string)
GrabarBitacora()
Guardar()
OperacionOk()
RefrescarGrilla()
OperacionOk()
alt si el usuario cancela la acción
MensajeCancelacion()
alt si el registro está activo
MensajeError()

Diagrama de clases CU23 "Auditar cambios"

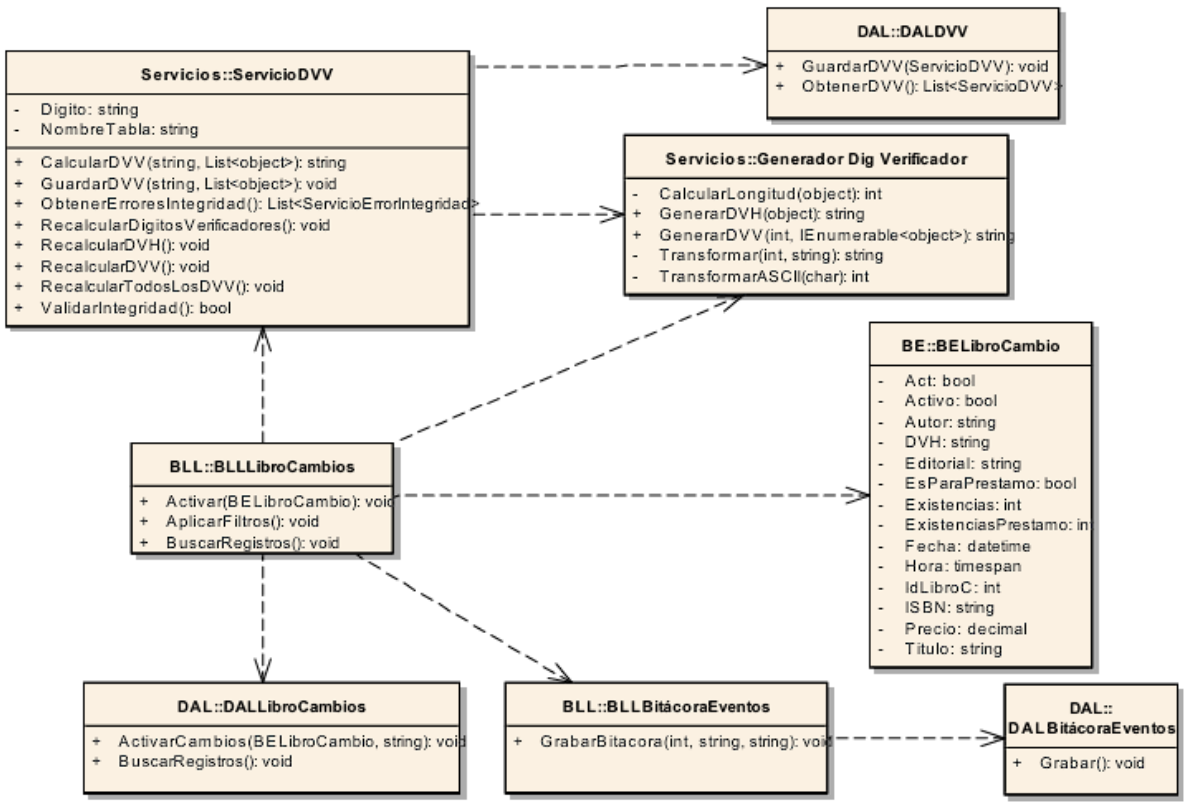
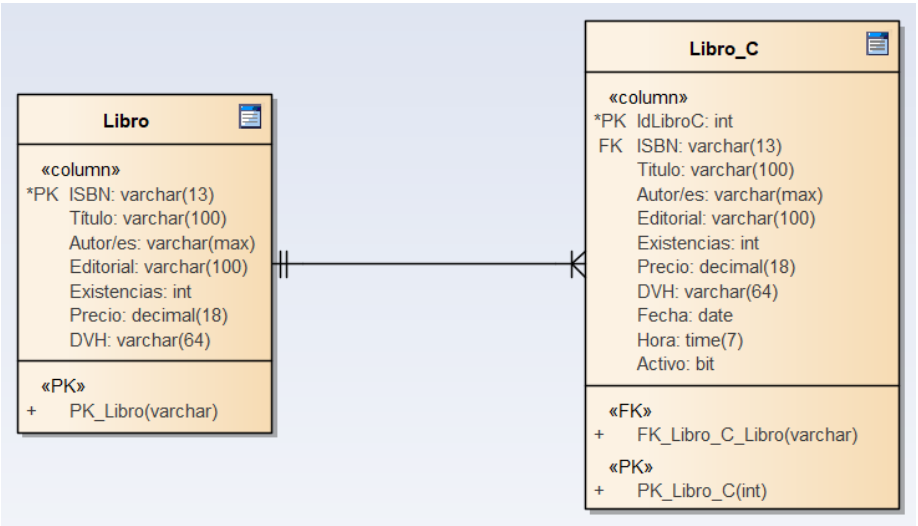


Diagrama DER CU23 "Auditar cambios"



GUI CU23 "Auditar cambios"

T07. Gestión de Respaldos

1. Descomposición Funcional

Realizar Backup

Pasos lógicos:

1. El Administrador ingresa al módulo de "Gestión de Respaldos" y selecciona la opción para realizar un nuevo backup.
2. El sistema solicita al usuario que ingrese o seleccione la ruta de destino donde se guardará el archivo generado. Atributo: Path_Destino
3. El sistema genera automáticamente el nombre del archivo de respaldo, incorporando la fecha y la hora actual para facilitar su identificación. Atributos: Nombre_Archivo, Fecha_Creación, Hora_Creación
4. El usuario confirma que desea proceder con la ejecución de la copia de seguridad.
5. El sistema valida que la ruta de destino sea accesible y realiza el volcado de la información de la base de datos al archivo correspondiente.
6. El sistema notifica al administrador si la operación finalizó con éxito o si se produjo algún error durante el proceso.
7. El sistema registra exitosamente la acción en la bitácora de eventos.

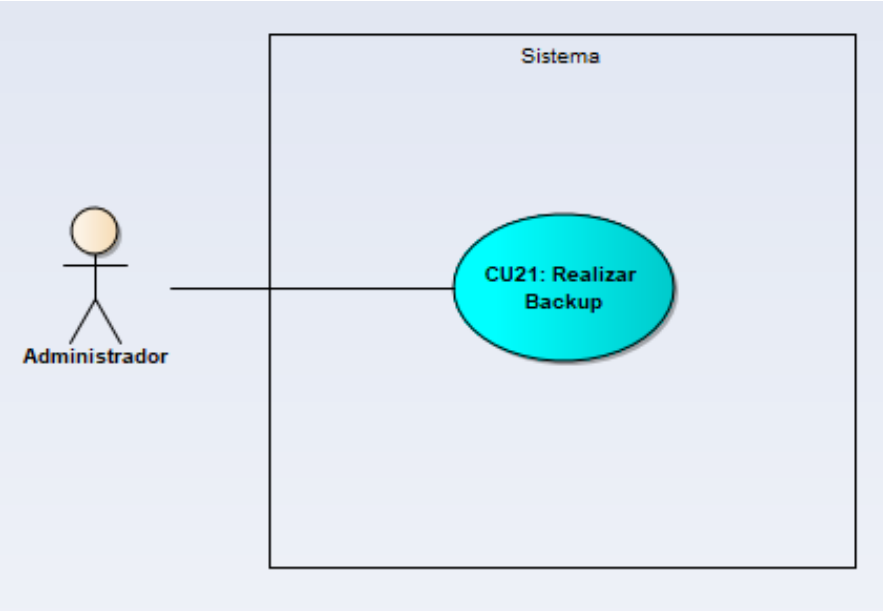
Realizar Restore

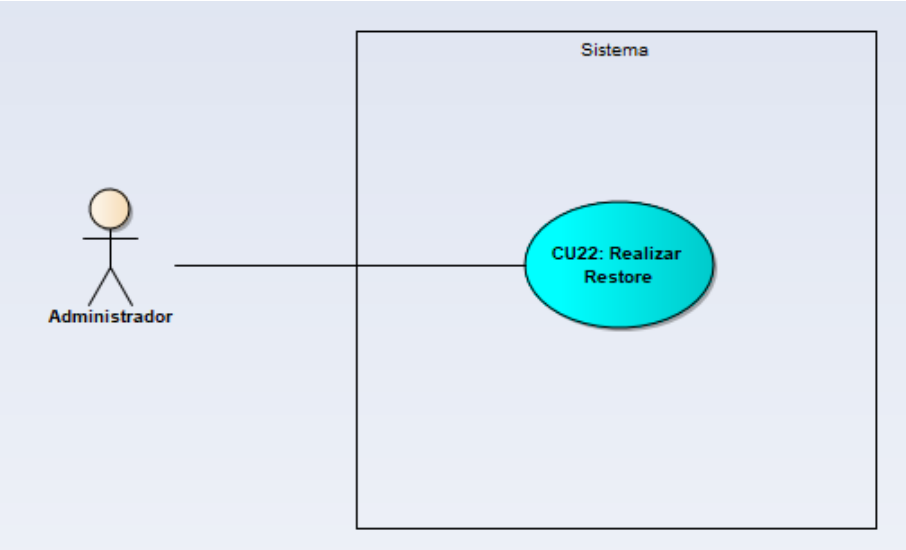
Pasos lógicos:

1. El Administrador accede al módulo de "Gestión de Respaldos" y selecciona la opción para realizar una restauración de datos.
2. El sistema solicita al usuario que busque y seleccione el archivo de backup previo que desea restaurar desde su ubicación en el equipo. Atributos: Path_Origen, Archivo_Seleccionado
3. El sistema realiza una validación técnica para asegurar que el archivo seleccionado cumpla con el formato y la estructura esperada para un backup.

4. El sistema solicita una confirmación final al administrador, advirtiendo que esta acción sobrescribirá los datos actuales almacenados en la base de datos.
5. El sistema procede a cerrar las conexiones activas, ejecuta el proceso de restauración utilizando el archivo seleccionado y restablece la conexión al finalizar.
6. El sistema informa al usuario sobre el resultado de la restauración y, por seguridad, redirige automáticamente a la pantalla de inicio de sesión.
7. El sistema registra exitosamente la acción en la bitácora de eventos.

2. Diagrama de Casos de Uso





3. Especificación de Casos de Uso

CU21 - REALIZAR BACKUP

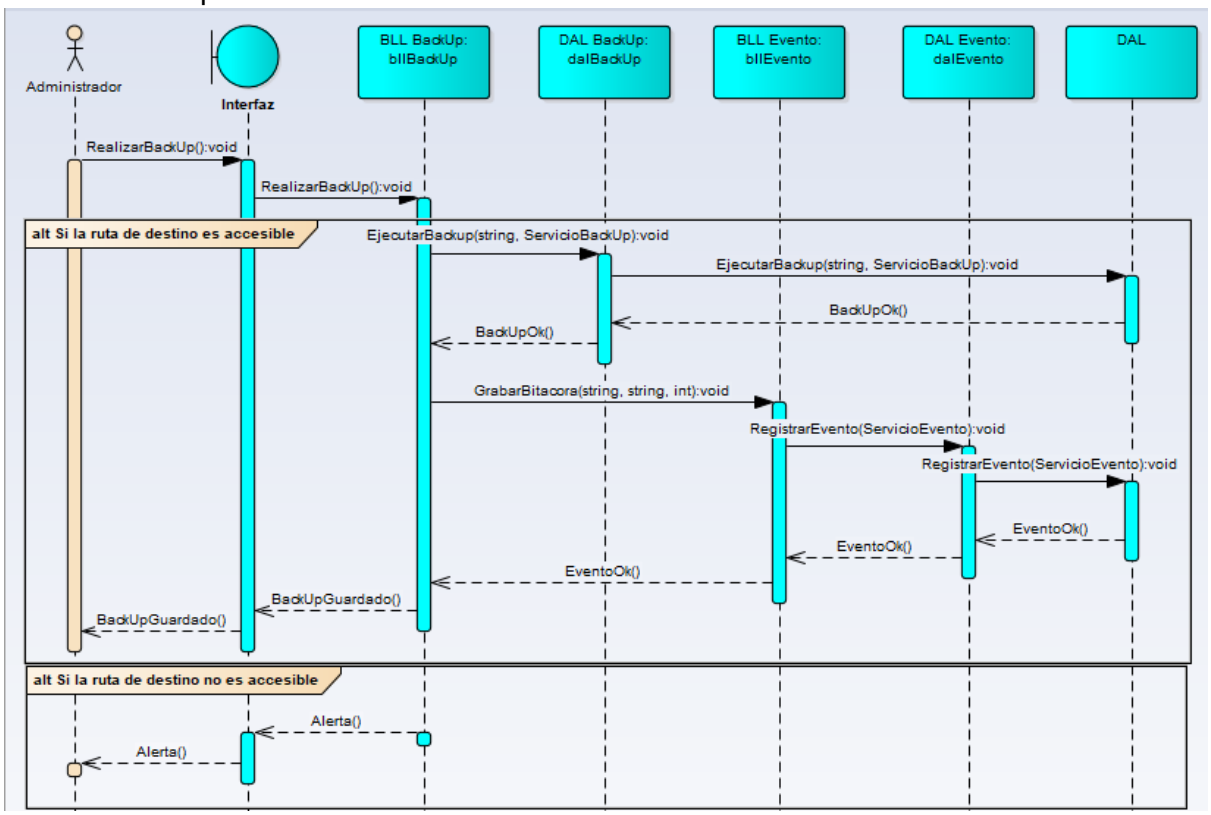
Descripción:	El sistema permite al administrador generar una copia de seguridad (backup) de la base de datos completa en un directorio local seleccionado por el usuario.
Actores:	Administrador
Precondiciones:	1. El administrador debe contar con una sesión activa en el sistema. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server
Puntos de extensión:	N/A
Flujo principal:	
1	El Administrador ingresa al módulo de "Administración" y selecciona la opción para realizar un nuevo backup.
2	El sistema solicita al usuario que ingrese o seleccione la ruta de destino donde se guardará el archivo.
3	El sistema genera automáticamente un nombre de archivo que incluye la fecha y hora actual.
4	El usuario confirma que desea proceder con la ejecución de la copia de seguridad.
5	El sistema valida que la ruta de destino sea accesible y realiza el volcado de la información al archivo correspondiente.
6	El sistema notifica al administrador que la operación finalizó con éxito.
7	El sistema registra el evento de creación de backup en la Bitácora de Eventos.
Flujo alternativo:	
5.1.1	El sistema detecta que el directorio destino no tiene permisos de escritura o no existe.
5.1.2	El sistema muestra un mensaje de alerta informando que no se pudo realizar el backup por error de acceso al directorio.
Poscondición:	Se genera un archivo de respaldo en la ruta especificada y el evento queda registrado en la bitácora del sistema.

CU22 - REALIZAR RESTORE

Descripción:	El sistema permite al administrador restaurar la base de datos a partir de un archivo de backup previamente generado, sobrescribiendo los datos actuales.
Actores:	Administrador
Precondiciones:	1. El administrador debe contar con una sesión activa en el sistema. 2. El sistema debe encontrarse operativo y con conexión activa a SQL Server. 3. Debe existir un archivo de backup válido en el directorio seleccionado.
Puntos de extensión:	N/A
Flujo principal:	
1	El Administrador accede al módulo de "Administración" y selecciona la opción para realizar una restauración de datos.
2	El sistema solicita al usuario que busque y seleccione el archivo de backup que desea restaurar.
3	El sistema realiza una validación técnica para asegurar que el archivo seleccionado cumpla con el formato de backup esperado.
4	El sistema solicita una confirmación final al administrador, advirtiendo que la acción sobrescribirá los datos actuales.
5	El sistema cierra las conexiones activas, ejecuta el proceso de restauración y restablece la conexión.
6	El sistema informa al usuario sobre el resultado exitoso y redirige a la pantalla de inicio de sesión.
7	El sistema registra el evento de restauración en la Bitácora de Eventos.
Flujo alternativo:	
3.1.1	El sistema detecta que el archivo seleccionado está corrupto o no tiene el formato correcto.
3.1.2	El sistema muestra un mensaje de error indicando que el archivo no es un backup válido y cancela la operación.
Poscondición:	La base de datos queda actualizada según el archivo seleccionado y la sesión del usuario se cierra por seguridad.

4. Diagrama de Secuencia

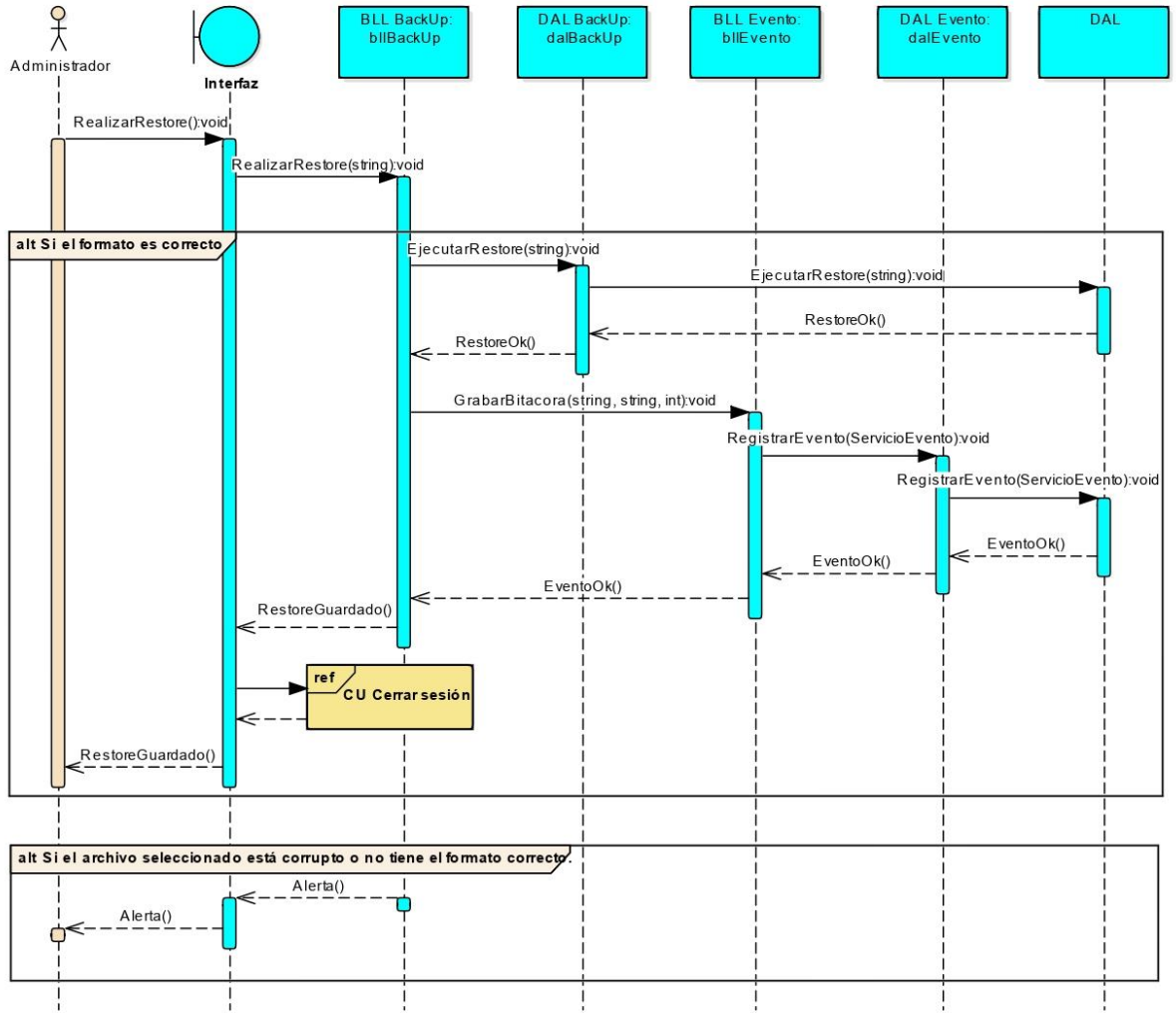
Realizar backup:



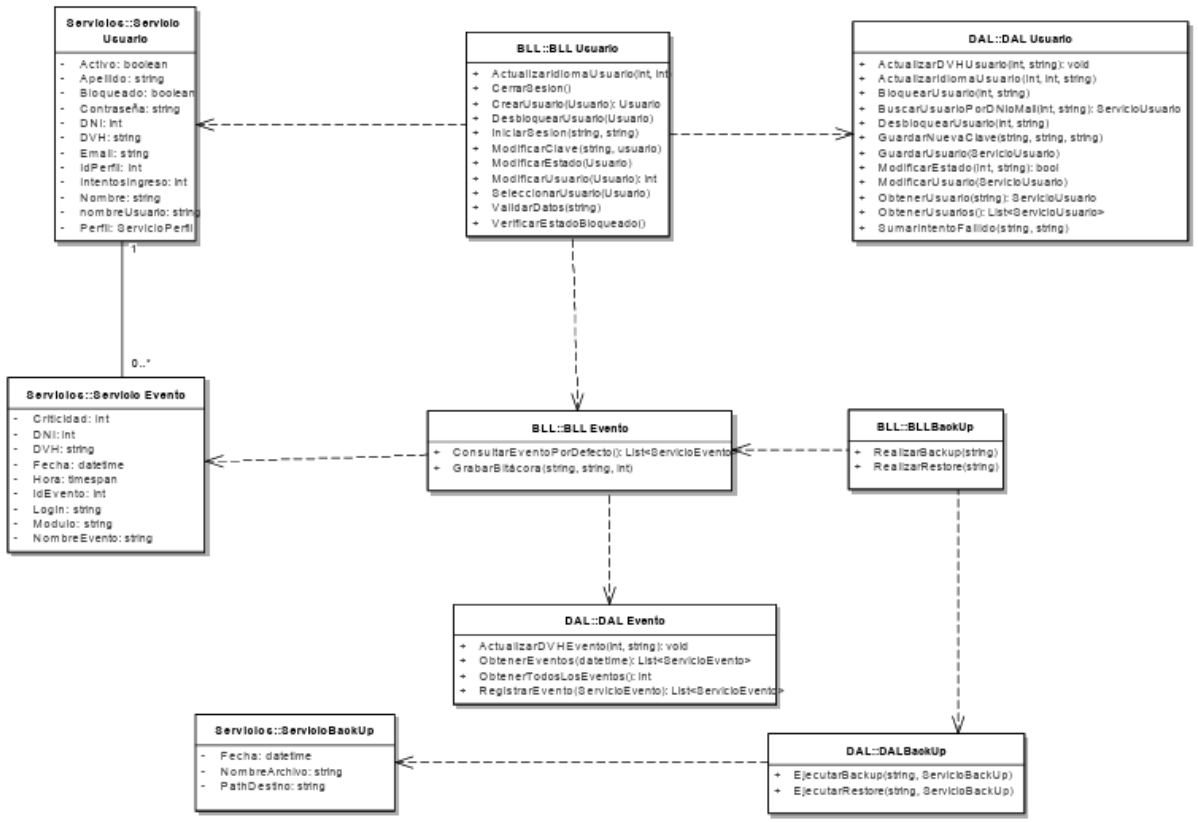
Realizar restore:

UNIVERSIDAD ABIERTA INTERAMERICANA
Facultad de Tecnología Informática

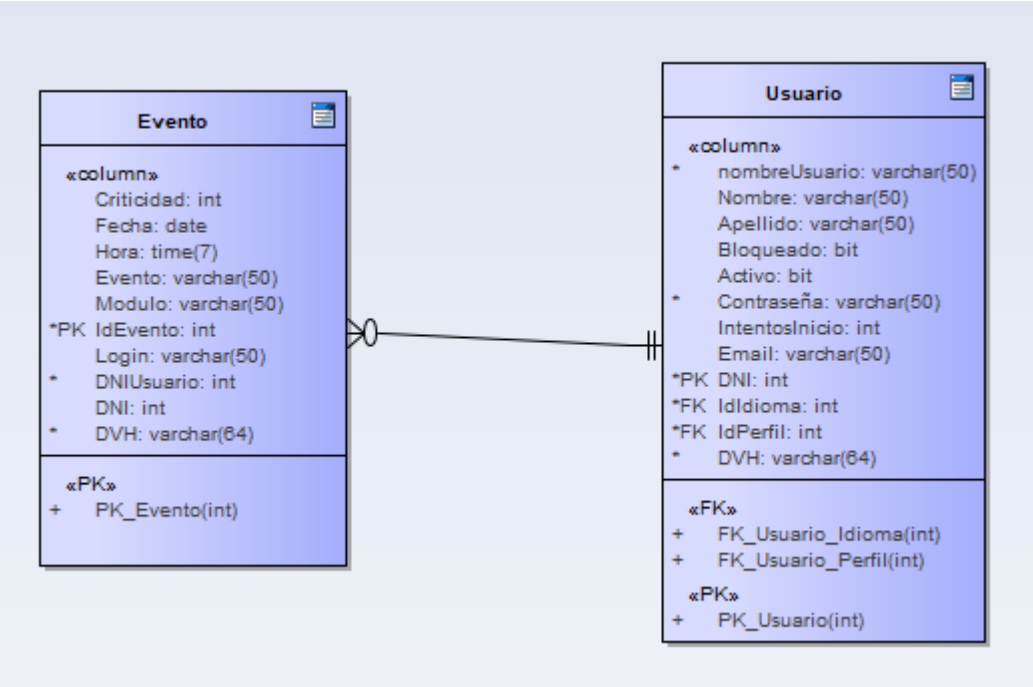
	Materia: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026



5. Diagrama de Clases



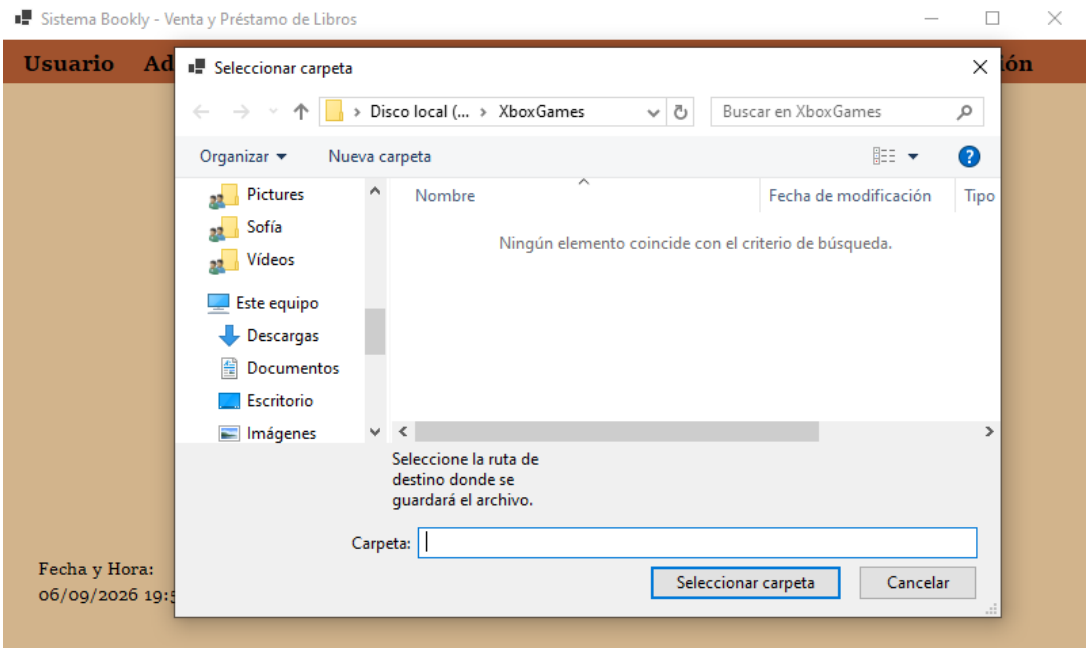
6. DER


Evento
«column»
Criticidad: int
Fecha: date
Hora: time(7)
Evento: varchar(50)
Modulo: varchar(50)
*PK IdEvento: int
Login: varchar(50)
* DNIUsuario: int
DNI: int
* DVH: varchar(64)
«PK»
+ PK_Evento(int)
Usuario
«column»
* nombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Bloqueado: bit
Activo: bit
* Contraseña: varchar(50)
IntentosInicio: int
Email: varchar(50)
*PK DNI: int
*FK IdIdioma: int
*FK IdPerfil: int
* DVH: varchar(64)
«FK»
+ FK_Usuario_Idioma(int)
+ FK_Usuario_Perfil(int)
«PK»
+ PK_Usuario(int)

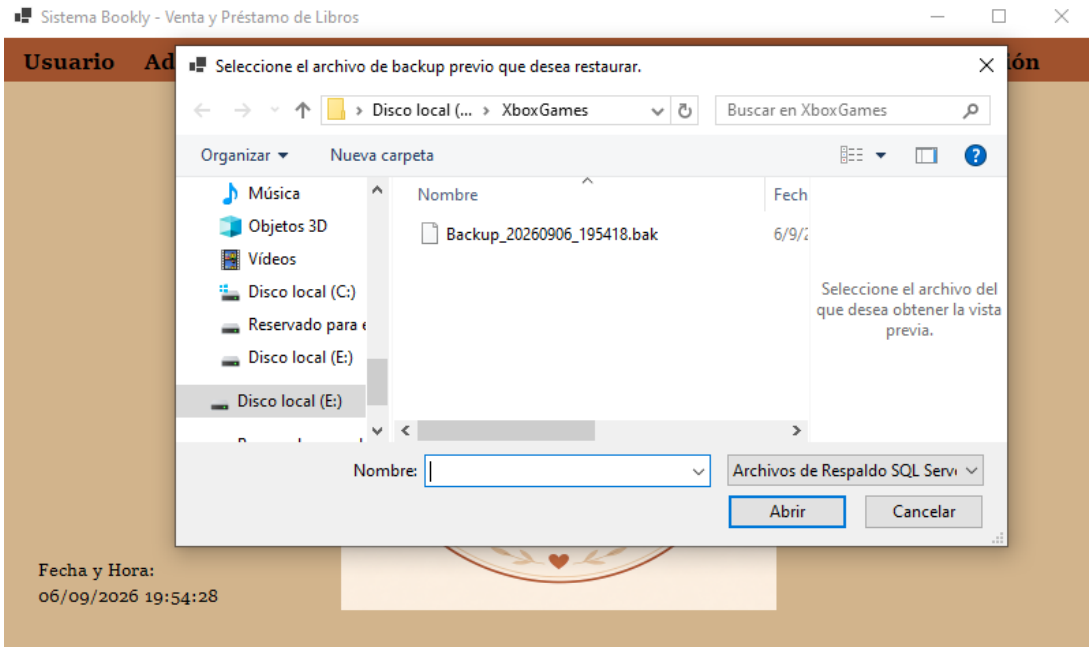
7. GUI


Sistema Bookly - Venta y Préstamo de Libros
Usuario
Administración
Operaciones
Maestro
Reportes
Ayuda
Cerrar Sesión
Usuarios
Perfiles
Bitácora Eventos
Bitácora Cambios
BackUp
Restore
o/a, Victoria Fernandez!
é desea hacer hoy?
BOOKLY
LIBROS QUE TE ACOMPAÑAN
Fecha y Hora:
06/09/2026 19:53:01

BackUp


Sistema Bookly - Venta y Préstamo de Libros
Usuario
Seleccionar carpeta
Disco local (... › XboxGames
Buscar en XboxGames
Organizar
Nueva carpeta
Pictures
Sofía
Vídeos
Este equipo
Descargas
Documentos
Escritorio
Imágenes
Nombre
Fecha de modificación
Tipo
Ningún elemento coincide con el criterio de búsqueda.
Seleccione la ruta de destino donde se guardará el archivo.
Carpeta:
Seleccionar carpeta
Cancelar
Fecha y Hora:

Restore



T08. Dígito Verificador

1. Objetivo del Uso de Dígitos Verificadores

El objetivo principal de la implementación de Dígitos Verificadores (DV) es garantizar la integridad de los datos y la seguridad del sistema en su capa de persistencia (Base de Datos).

Este mecanismo transversal, ubicado en la capa de Servicios, busca detectar alteraciones, borrados o manipulaciones no autorizadas en la base de datos (por ejemplo, si un usuario con acceso directo al motor de BD modifica un registro o añade filas "por atrás", saltándose la aplicación). Al validar los dígitos, el sistema puede asegurar que los datos no han sido corrompidos ni adulterados desde su último registro legítimo y seguro dentro de la aplicación.

Dado que todas las entidades del sistema almacenan información relevante para el funcionamiento de Everglow, se decidió implementar el mecanismo de Dígitos Verificadores sobre la totalidad de las tablas de la base de datos. De esta manera,

cualquier alteración, eliminación o incorporación no autorizada de registros podrá ser detectada, independientemente de la entidad afectada.

Esta decisión brinda una protección integral de la información, ya que no solo resguarda los datos críticos como los usuarios y sus permisos, sino también las tablas de configuración, las relaciones entre entidades y los registros históricos del sistema. Así, se garantiza que toda la información almacenada conserve su integridad y que cualquier modificación realizada por fuera de la aplicación sea identificada mediante la validación de los DVH y DVV.

2. Componentes del Mecanismo y Métodos de Cálculo

Dígito Verificador Horizontal (DVH)
El DVH es un valor numérico que se calcula e introduce de forma individual en cada fila (registro) de una tabla crítica (por ejemplo, en la tabla Usuario, cada fila tiene su propio DVH en una columna dedicada). Su función es detectar si los datos internos de ese registro en específico fueron modificados por fuera de la aplicación.

- Cálculo del DVH

El cálculo del DVH se realiza sobre las entidades definidas, tomando en cuenta todas sus propiedades a excepción del ID autogenerado de la base de datos y el propio campo de destino del DVH. El proceso técnico consta de tres etapas secuenciales:

1. Concatenación, Transformación y Ponderación por Carácter

En primera instancia, el sistema toma el nombre de la clase (entidad) en cuestión y lo une (concatena) secuencialmente con los valores en formato de texto de todas sus propiedades válidas, generando una única cadena de caracteres unificada. Luego, se procesa dicha cadena de izquierda a derecha, carácter por carácter. Por cada uno de ellos, se determina su valor equivalente en la tabla ASCII y se le suma algebraicamente el número entero que representa su posición física dentro de la palabra (comenzando desde la posición 1). El resultado de esta etapa es una secuencia de valores numéricos ponderados que destruye la posibilidad de fraude por transposición de letras.

2. Desplazamiento Proporcional (Cifrado Posicional)

Una vez obtenida la secuencia numérica del paso anterior, se aplica un algoritmo de desplazamiento posicional. Este consiste en tomar la longitud total (cantidad de caracteres) que posee el nombre de la entidad evaluada y sumarle ese valor constante a cada uno de

los números individuales calculados en la etapa de ponderación. De este modo, la estructura de los datos se altera de manera directamente proporcional a la identidad del objeto que se está resguardando.

3. Hashing y Codificación

Finalmente, para asegurar la irreversibilidad y la unicidad del dígito, la secuencia resultante se procesa mediante un algoritmo de encriptación Hash (en nuestro caso, con SHA-256), generando un bloque de bytes único. Debido a que el resultado del Hash puede contener caracteres de control ilegibles para la base de datos, este bloque se codifica en formato Base 64. Esto garantiza una cadena de texto alfanumérica limpia, estandarizada y libre de conflictos por caracteres especiales al momento de persistir en el registro correspondiente.

Dígito Verificador Vertical (DVV)

El DVV es un valor único asignado a toda una tabla en su conjunto. No se almacena en la misma tabla de negocio que se está controlando, sino en una tabla independiente de control dentro de la capa de servicios (ej: Tabla: Usuario | Registro - Valor DVV: 18450). Su función es detectar si se elimina un registro completo o si se insertó una fila nueva "fantasma" de manera externa directamente en la base de datos.

- Cálculo del DVV

El cálculo del DVV es directo y depende exclusivamente de los DVH (ya calculados y guardados en formato Base64) de la tabla correspondiente. El proceso técnico consta de los siguientes pasos secuenciales:

1. Unificación de Cadenas (Súper Concatenación)

El servicio realiza una lectura vertical de toda la columna DVH de la tabla que se quiere controlar y concatena consecutivamente cada uno de los hashes en formato de texto. A esta gran cadena resultante se le anexa el nombre de la propia entidad (clase) evaluada.

2. Sumatoria Absoluta en ASCII

Una vez consolidada la cadena unificada con todos los DVH del sistema, el algoritmo la recorre carácter por carácter, de izquierda a derecha. Por cada letra, número o símbolo especial de la cadena, se extrae su valor numérico directo en la tabla ASCII y se va acumulando en una sumatoria matemática total.

3. Registro de Control

El número entero final obtenido de la sumatoria ASCII acumulada constituye el nuevo valor del DVV, el cual se almacena o actualiza en el registro correspondiente dentro de la tabla de control de servicios asociada al nombre de la entidad en cuestión.

3. Descomposición funcional

Validar Integridad de Datos

1. El Sistema inicia la validación general de la consistencia. Recibe la orden de arranque de la aplicación y produce la petición de verificación hacia el componente de negocio vertical (Capa BLL).
2. El Sistema recupera la lista histórica de los dígitos verificadores verticales almacenados. Recibe la solicitud de lectura y produce una lista de objetos DVV desde la persistencia (Capa DAL).
3. El Sistema recupera de forma independiente la lista de los dígitos horizontales y la colección de registros de usuarios del sistema. Recibe las solicitudes de lectura y produce las colecciones de datos recuperadas (Capa DAL).
4. El Sistema recorre en un ciclo cada registro de la lista para comparar sus datos contra su dígito horizontal recalculado. Recibe los registros, ejecuta la subfunción interna de generación de DVH y produce un veredicto booleano de consistencia horizontal (Capa BLL).
5. El Sistema verifica que el dígito vertical consolidado no haya sido alterado. Recibe la lista de DVV, ejecuta la subfunción interna de generación de DVV sobre el array de datos y produce el veredicto booleano de consistencia vertical (Capa BLL).
6. El Sistema evalúa las banderas de error de los dígitos y el rol del usuario logueado. Recibe el estado de la consistencia y, en caso de éxito, produce la autorización para el inicio normal; de lo contrario, interrumpe el arranque y produce el despliegue de la pantalla de gestión si es Administrador, o el bloqueo general de la app si no lo es (Capa GUI).

Generar Dígito Verificador

1. El Sistema inicia la generación del dígito horizontal solicitando la información estructural del objeto que se quiere proteger. Recibe la entidad de datos y produce la lista de sus propiedades internas (Capa BLL).

2. El Sistema recorre en un ciclo cada propiedad del objeto para extraer su información. Recibe la lista de propiedades y produce los valores individuales de texto o número que contiene el registro (Capa BLL).
3. El Sistema convierte los caracteres de los valores obtenidos a un formato numérico estándar. Recibe cada letra o número y produce su equivalente en código ASCII (Capa BLL).
4. El Sistema procesa y cifra los códigos numéricos obtenidos para transformarlos en una clave única. Recibe la cadena de códigos ASCII y produce el texto final del nuevo dígito verificador horizontal (DVH) (Capa BLL).
5. El Sistema inicia la generación del dígito vertical solicitando el conjunto de todos los dígitos horizontales de la tabla. Recibe la lista de los DVH y produce el inicio de un ciclo de recorrido (Capa BLL).
6. El Sistema acumula la sumatoria de las posiciones de los dígitos horizontales dentro del ciclo. Recibe cada caracter de los DVH, los transforma a código ASCII y produce el valor final del nuevo dígito verificador vertical (DVV) (Capa BLL).

Diagrama de Secuencia - Generar Dígito Verificador Horizontal

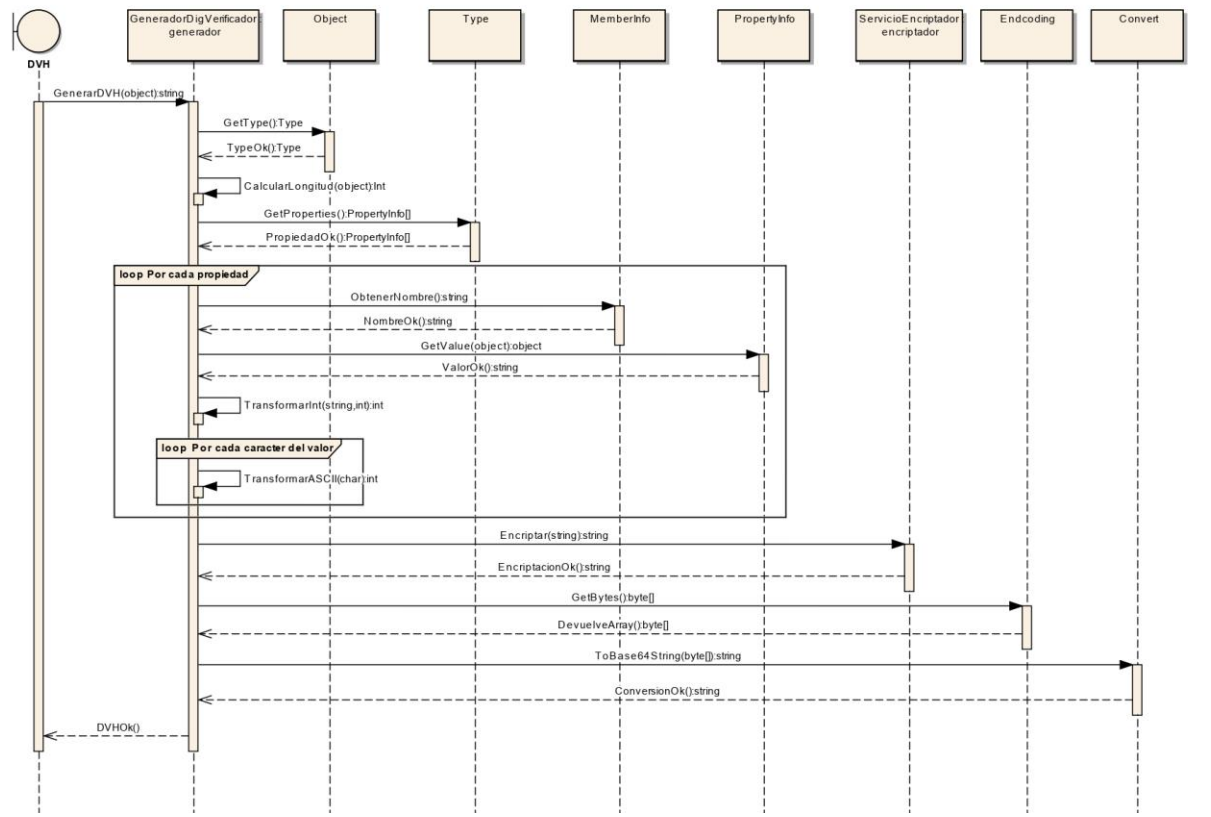


Diagrama de Secuencia - Generar Dígito Verificador Vertical

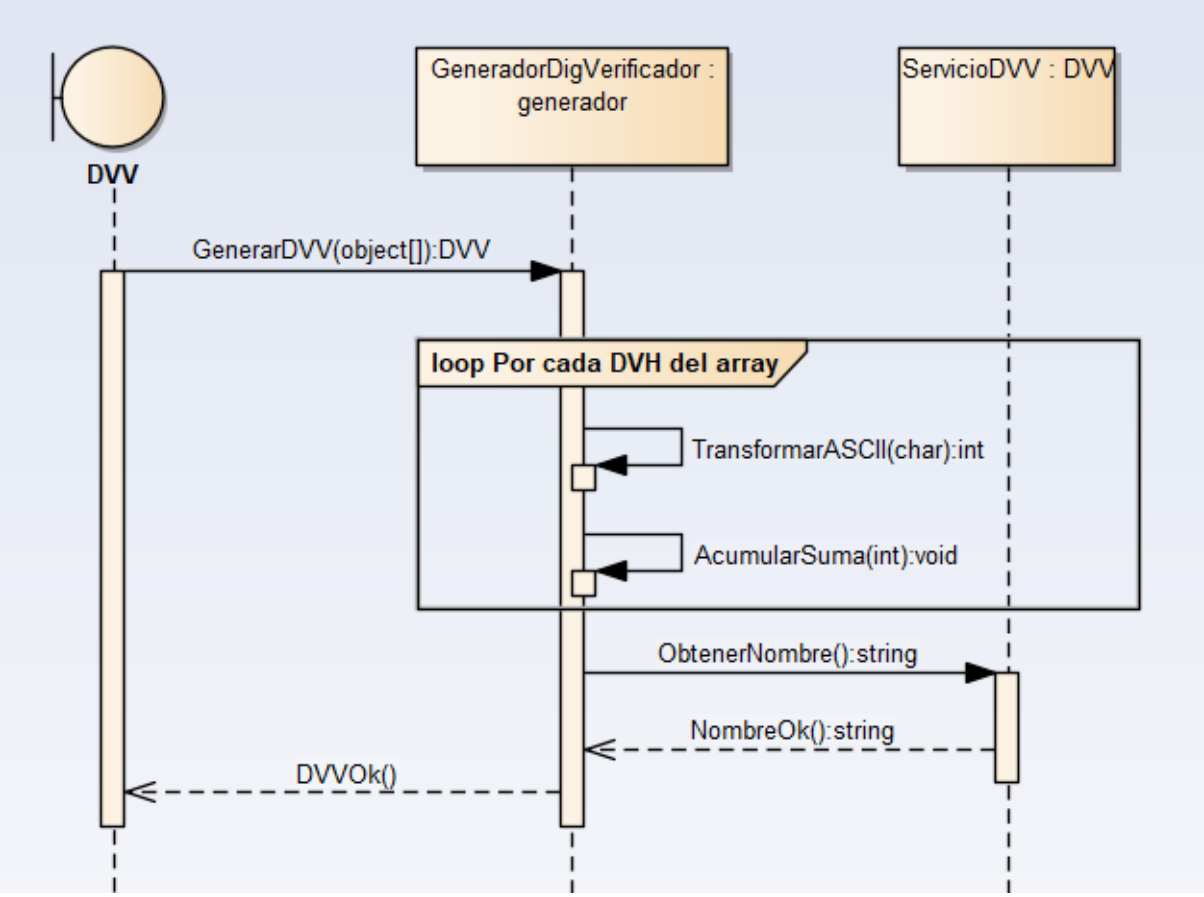


Diagrama de Secuencia - Validar Integridad de Datos

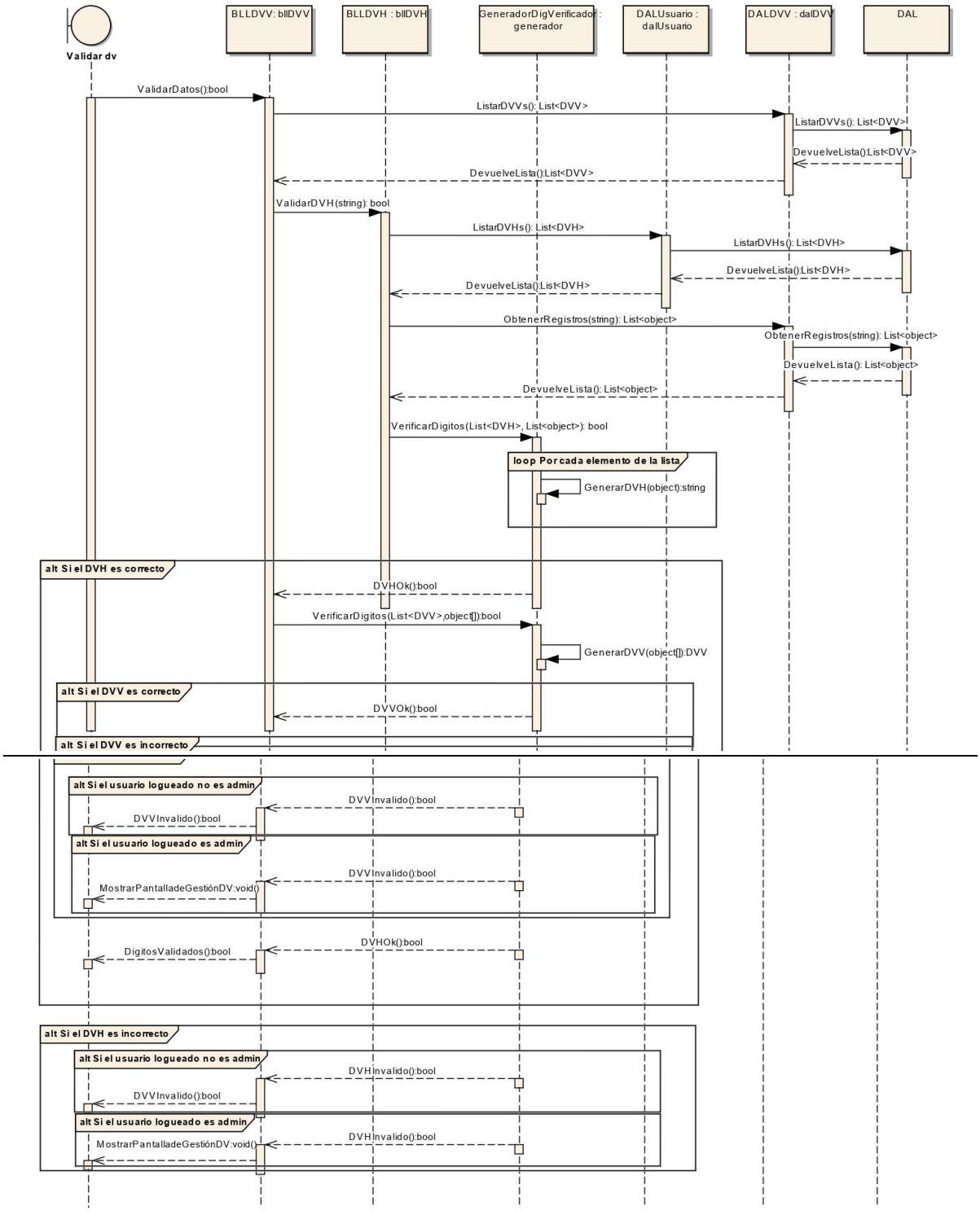


Diagrama de Clases

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática			
UAI Universidad Abierta Interamericana	*Materia*: Trabajo de Diploma		*Docente*: Gamboa Jiménez, Leonel
	Alumno: Gesualdo Antonini, Sofía.		
	Sede: Centro - 091	*Comisión*: 3B	*Turno*: Mañana	*Año*: 2026

